SolvefyTask

10/27/2026 15:37:10
https://chatgpt.com/g/g-p-6a92bbad502c8191b1067ba78b4ddb42-ahmad-abdullah/c/6adc59f5-7ad8-83ee-b9b6-446e0af5eb9c

SolvefyTask

Could you please clarify what you mean by SolvefyTask? Are you referring to a...

please guide me how to start angular project from scratch give me the terminal...

Thinking ● Searched 5 websites ✓ Worked for a couple of seconds How to Start a...

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend> ng...

Thinking ● Searched 4 websites ✓ Worked for a few seconds Your Angular CLI is...

[Image: image.png] please help me generate this into angular code ?

Yes. We can recreate your Horizon Survey Services — Equipment Tracking System...

Pasted markdown(2).md i have made a new component (login) how to use that...

Since your Angular project uses Angular 21 and standalone components, you can...

Application bundle generation complete. [0.258 seconds] - 2026-10-...

The error is clear: Angular cannot find the LoginComponent class at the path you...

[Image: image(1).png] now give me this dashboard code in angular ??

We can build this Equipment Tracking System Dashboard in your existing Angular ...

import { Component } from '@angular/core'; @Component({ selector: 'app-login',...

You can do this using Angular Router. When the user clicks Login, Angular will...

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend\solvefy-...

The error occurs because inject has not been imported from @angular/core in you...

[Image: image(2).png] please generate this in angular code ??

Let's create the Equipment List page for your existing Angular 21 Equipment...

import { Component } from '@angular/core'; @Component({ selector: 'app-...

You need to use Angular Router instead of only changing selectedMenu. When the...

when i click on equipment then dashboard sidebar closes i want to open it as it is ??

The sidebar closes because your current Equipment route loads only the...

Application bundle generation failed. [0.110 seconds] - 2026-10-24T08:18:54.334...

This error means Angular does not recognize <router-outlet> in your MainLayout...

[Image: image(3).png] i want this dashboard to use and not the upper dashboard ?

Your screenshot shows that two dashboards are appearing at the same time:The...

no i want to keep the dashboard which is in dashboard.html file and remove the...

Now I understand. You want to keep the complete sidebar and dashboard layout...

how to remove a component in angular ??

In Angular, you can remove a component by deleting its files and removing all...

 <!-- Sidebar --> <aside class="sidebar"> <div...

You can achieve this by keeping your existing dashboard.html sidebar and topbar ...

[Image: image(5).png] i want this dashboard sidebar to be fixed

To keep your Horizon sidebar fixed on the left while scrolling or switching between...

:host { display: block; min-height: 100vh; color: #202b3c; font-family: Arial,...

Your current sidebar HTML is fine. The problem is that when you click Equipment,...

[Image: image(7).png] please generate this image into angular code ??

I'll build this Check Out Equipment page for your existing Angular SolvefyTask...

[Image: image(8).png] please generate the image into angular code ??

We'll create a Check In Equipment component matching your screenshot, with the...

[Image: image(9).png] please generate the code of this in angular ??

I'll create an Approval Request (Drone) page for your SolvefyTask Angular project,...

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend\solvefy-...

The error occurs because your Approvals component is using...

[Image: image(10).png] please generate the code of this image in angular ??

I'll create the My Equipment page for your existing Angular SolvefyTask applicatio...

[Image: image(20261024-093259).png] please generate the code of thus image i...

I'll create the Damaged Reports page for your SolvefyTask Angular project,...

[Image: image(20261024-093802).png] please generate the code of this image in...

I'll create the Reports / History page for your SolvefyTask Angular application base...

SOLVEFY · AI ENGINEER HIRING FRAMEWORK 03-test-task-associate Internal ·...

I'll keep this Solvefy AI Engineer I test task as your original reference and will not...

please help me write the decisions such that what i have taken during my fronten...

Based on the frontend development we have done for SolvefyTask — Horizon...

using Microsoft.AspNetCore.Http; using Microsoft.AspNetCore.Mvc; namespace...

For your SolvefyTask — Horizon Survey Services project, a clean approach is to us...

{ "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore":...

Since your SolvefyTask backend uses SQL Server, you need to configure the...

how to apply Migration using package manager console ??

You can apply Entity Framework Core migrations through Package Manager Consol...

please write me an API to insert the equipment ??

For your SolvefyTask ASP.NET Core Web API, we'll create a POST API using the sam...

now make an service in angular such that it calls the above 2 apis and perform th...

We'll create an Angular EquipmentService that calls both of your ASP.NET Core...

import { Component, OnInit } from '@angular/core'; import { FormsModule } from...

Below is the complete updated equipment.ts for your Angular Equipment page. It...

X [ERROR] TS2769: No overload matches this call. Overload 1 of 2,...

The error occurs because your EquipmentService.getAllEquipment() method is...

import { HttpClient } from '@angular/common/http'; import { inject, Injectable }...

The issue is that you're using EquipmentService as the data type. EquipmentServi...

 3. Equipment...

The error occurs because your HTML calls saveEquipment(), but your updated...

Application bundle generation failed. [0.308 seconds] - 2026-10-24T14:23:35.459...

This error means your equipment.html is passing 4 arguments to addEquipment(),...

Application bundle generation failed. [3.394 seconds] - 2026-10-24T14:26:51.830...

The error is coming from line 12, where your top Add Equipment button still calls...

 3. Equipment...

I can see the main flow in your code. Your form calls addEquipment() and that...

Angular is running in development mode. equipment:1 Access to XMLHttpRequest...

Your console clearly identifies the problem: your ASP.NET Core API at...

Angular is running in development mode. equipment:1 Access to XMLHttpRequest...

The CORS error is still happening, which means the browser is still not receiving th...

using Microsoft.EntityFrameworkCore; using SolvefyTask.Data; using...

I found the issue in your Program.cs: you registered the CORS policy but never...

 3. Equipment...

Your modal can close automatically after the equipment is successfully saved. You...

import { Component, OnInit, inject } from '@angular/core'; import {...

Your TypeScript code already sets this.showAddForm = false correctly. The likely...

\- Node.js (LTS version). Check with `node -v` \- Angular CLI: `npm install -g...

Here is a README-ready section you can add to your SolvefyTask README.md. It...

please make it small only relavant details ??

Frontend Setup and Backend Integration Run the Frontend cd...

Combine the two run sections

Setup and Run the Application Prerequisites: Node.js (LTS), npm, Angular CLI, .NET...

import { Component } from '@angular/core'; import { FormsModule } from...

We can implement your Check Out API using the same architecture as your...

using Microsoft.AspNetCore.Http; using Microsoft.AspNetCore.Mvc; using...

You need two GET APIs in your CheckoutController:GET /api/Checkout/equipment ...

please also provide repository and then get the assign to using equipment id ??

We'll update your checkout flow to use the Equipment ID instead of the equipmen...

import { Component, inject } from '@angular/core'; import { FormsModule } from...

Yes. You need to integrate both GET APIs into your existing Angular Checkout...

 4. Check Out...

Below is the complete updated code for your Angular Checkout page, keeping you...

X [ERROR] TS2339: Property 'minReturnDate' does not exist on type 'Checkout'....

This error means minReturnDate is missing from your Checkout class in checkout....

in check out when i change the employee the assignee must have to load it and g...

Yes, exactly. In your Check Out Equipment form, when you change the selected...

[Image: image(20261024-170900).png] assignee is not loading from the database...

I can see the problem from your code. The page is successfully loading the...

in network tab the api is getting the data perfectly but it is now showing it on the...

If the API returns the assignee correctly in the Network tab, your backend is...

import { HttpClient } from '@angular/common/http'; import { inject, Injectable }...

I reviewed all three files you shared. The most important thing is that your Angula...

[Image: image(20261024-171944).png]

Now we have identified the issue more clearly. Your Swagger screenshot confirms...

i want to use this.cdr.detectChanges() how to apply it ??

You can use this.cdr.detectChanges() in your Checkout component by injecting...

when i click on the submit checkout but it is not saving into the database ??

Let's trace the checkout submission from Angular to SQL Server. Since your...

{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1", "title": "One or more...

The error confirms the problem: your Angular frontend sends EquipmentId, but yo...

i want to send equipment name not equipment ID ??

Understood. You want to send equipmentName instead of equipmentId from...

there is a problem with my code it is showing dashboard at the end on every page...

This usually happens in Angular when the Dashboard component or Dashboard...

 <!-- Sidebar --> <aside class="sidebar"> <div...

I found the problem in your HTML. Your Dashboard layout contains <router-outlet...

 <!-- Sidebar --> <aside class="sidebar"> <div...

The issue is that your Dashboard HTML contains both the shared layout and the...

[Image: image(20261025-065120).png] the dashboard layout has been disturbed...

25/10/2026, 11:51:52

> What does the dashboard home page need fixed first? Restore the original...

The dashboard screenshot shows that the original styling has been lost: the...

\ .dashboard-home {\   display: flex;\   flex-direction: column;\ &#x2...

I found the cause. Your original dashboard.css contains the styling that makes the...

import { Component } from '@angular/core'; import { FormsModule } from...

I'll implement the Check In Equipment APIs in your existing .NET backend first,...

import { Component } from '@angular/core'; import { FormsModule } from...

We'll connect your existing Checkin component to the three .NET APIs without...

 <section class="checkin-card"> <!-- Header -->...

Below is the complete implementation of your Check In Equipment page, preservi...

import { Component } from '@angular/core'; import { FormsModule } from...

We'll implement the Approvals feature entirely in the .NET backend for your existi...

import { Component } from '@angular/core'; import { FormsModule } from...

Thinking ● Implementing the Angular screen I'm building the service, model,...

i am facing no approval requests were found how to apply for approval ??

You are seeing "No approval requests were found" because the ApprovalRequests...

i want when i click on the checkout then it goes to the approval can i achieve this ??

Yes, absolutely. In your SolvefyTask application, when the user clicks Check Out, y...

import { ChangeDetectorRef, Component, inject, OnInit } from '@angular/core';...

You need to write the drone approval logic inside your existing requestCheckOut()...

import { ChangeDetectorRef, Component, inject, OnInit } from '@angular/core';...

The problem is likely in the way your checkout and approval workflows are...

how to delete from tables with referenced by foreign key and the id becomes 1 in...

In your SQL Server database, you can delete records from tables that have foreign...

import { ChangeDetectorRef, Component, inject, OnInit } from '@angular/core';...

The issue is probably not in the drone condition itself. In your requestCheckOut()...

Request URL\ https\://localhost:7135/api/Approvals\ Request method\ POST\ Statu...

Your Network details confirm that the approval request is being saved successfully...

import { Component } from '@angular/core'; import { Router } from...

Yes. We'll implement only the .NET backend APIs for your My Equipment page, usi...

please send the repositories and service layers like you have send for the previuo...

Absolutely. We'll follow the same Repository → Service → Controller architecture...

import { Component } from '@angular/core'; import { Router } from...

We'll connect your existing Myequipment component to the backend APIs while...

can you tell me how to assign equipment because it is showing No Equipment...

Your My Equipment page is showing "No equipment assigned" because the API is...

please make a myequipment table you have not made previous and then...

Yes. Since your My Equipment page needs its own database records, we can creat...

import { Component } from '@angular/core'; import { FormsModule } from...

For your Damage Reports page, we'll implement only the .NET backend, following...

import { Component } from '@angular/core'; import { FormsModule } from...

We'll now connect your existing Damagereports TypeScript component to the .NET...

import { Component } from '@angular/core'; import { FormsModule } from...

For your Reports / History page, we'll implement only the .NET backend using your...

import { Component } from '@angular/core'; import { FormsModule } from...

We'll connect your existing Reports TypeScript component to the .NET...

can u guide me how to populate the equipment history means which functionality...

Yes. In your equipment tracking system, the Equipment History table should be...

using Microsoft.AspNetCore.Http; using Microsoft.AspNetCore.Mvc; using...

Your MyEquipmentController is already calling _service.ReturnEquipmentAsync(id)...

using Microsoft.AspNetCore.Http; using Microsoft.AspNetCore.Mvc; using...

آپ کے ApprovalsController میں EquipmentHistory کی entry یہ طریقہ بہتر کے لیے ڈالنے...

in english ??

In your ApprovalsController, you do not need to insert EquipmentHistory directly....

using SolvefyTask.DTOs; using SolvefyTask.Models; using SolvefyTask.Repositories...

In your current ApprovalService, the DecideAsync() method delegates the approv...

using SolvefyTask.DTOs; using SolvefyTask.Models; using SolvefyTask.Repositories...

Your DamageReportService is already structured correctly. If you want to...

[Image: image(20261025-125332).png] please generate the typescript and html...

Below is the Angular implementation of your Team Members UI, based on the...

_debug_node-chunk.mjs:11046 Angular is running in development mode....

The error is related to Angular component compilation, not your CSS or the team-...

import { Component, inject } from '@angular/core'; import { Router, RouterOutlet...

Your app.routes.ts looks correct: TeamMembersComponent is registered under...

import { CommonModule } from '@angular/common'; import { Component } from...

We can connect your Team Members Angular component to your existing ASP.NET...

i want to implement the backend dotnet first please give me the code of apis etc ??

We'll implement the Team Members backend first using your existing ASP.NET Cor...

SolvefyTask

24/10/2026, 12:10:57

Could you please clarify what you mean by **SolvefyTask**? Are you referring to a software project, a task-management tool, or a specific coding task in your Ahmad Abdullah project?

24/10/2026, 12:10:57 · gpt-6

please guide me how to start angular project from scratch give me the terminal commands how to start ??

24/10/2026, 12:13:18

Thinking

- **Searched 5 websites**

✓ Worked for a couple of seconds

How to Start an Angular Project from Scratch

I'll guide you step by step, from checking the required software to running your first Angular application in the browser.

Step 1: Install the required software

You need these two tools:

- **Node.js (LTS):** Runs Angular's development tools.
- **Visual Studio Code (VS Code):** Code editor.

Download them from their official websites:

- nodejs.org
- code.visualstudio.com

Step 2: Check installation in the terminal

Open **VS Code → Terminal → New Terminal** or open Command Prompt, then run:

```
node -v
npm -v
```

Both commands should display version numbers. If either command is not recognized, install or repair Node.js and reopen the terminal.

Step 3: Install Angular CLI

Run this command in the terminal:

```
npm install -g @angular/cli
```

After installation, verify it:

```
ng version
```

Angular CLI helps you create, run, and manage Angular applications. [Installation; Local set-up]

Step 4: Create a new Angular project

First, navigate to the folder where you want to keep your project. For example, to create it on your Desktop in Windows:

```
cd "$HOME\Desktop"
```

Now create your project:

```
ng new solvefy-task
```

Angular will ask you some configuration questions. For a beginner, I recommend:

- **Stylesheet:** CSS
- **Routing:** Yes, if you plan to have multiple pages
- **Other options:** Accept the defaults unless you have a specific requirement.

Wait for Angular to finish installing the packages.

Step 5: Open the project in VS Code

Run these commands one by one:

```
cd solvefy-task
code .
```

The first command enters your project folder. The second opens that folder in VS Code.

Step 6: Run your Angular project

In the VS Code terminal, run:

```
ng serve --open
```

Your browser should open at:

localhost

You should see your new Angular application's welcome page. Keep the terminal running while you work. Angular will automatically rebuild the application when you save code changes. [Local set-up; Serving Angular apps for development]

All commands in one place

If Node.js and Angular CLI are installed, these are the main commands you need:

```
# 1. Go to Desktop
cd "$HOME\Desktop"

# 2. Create Angular project
ng new solvefy-task

# 3. Enter project folder
cd solvefy-task

# 4. Open VS Code
code .

# 5. Run the application
ng serve --open
```

Step 7: Understand the main files

`src/app/`

Your main Angular application code, components and templates.

`src/app/app.ts`

The root component in the default modern Angular naming style.

`src/app/app.html`

The HTML template displayed by the root component.

`src/app/app.css`

Styles for your root component.

`angular.json` **and** `package.json`

Project configuration, scripts and dependencies.

These filenames can differ if you select a different naming convention during project creation. [ng new; Angular CLI • Overview]

One important question: Are you starting a completely new Angular project for learning, or do you want to start your existing **Angular + ABP project**? The commands and setup are different

for ABP.

24/10/2026, 12:13:18 · gpt-6

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend> ng version



Angular CLI : 21.2.5
Node.js : 22.19.0
Package Manager : npm 10.9.3
Operating System : win32 x64

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend> ng new solvefy-task
✔ Which stylesheet system would you like to use? CSS [https://developer.mozilla.org/docs/Web/CSS]
✔ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✔ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE solvefy-task/.prettierrc (173 bytes)
CREATE solvefy-task/angular.json (1994 bytes)
CREATE solvefy-task/package.json (787 bytes)
CREATE solvefy-task/README.md (1523 bytes)
CREATE solvefy-task/tsconfig.json (990 bytes)
CREATE solvefy-task/.editorconfig (331 bytes)
CREATE solvefy-task/.gitignore (666 bytes)
CREATE solvefy-task/tsconfig.app.json (444 bytes)
CREATE solvefy-task/tsconfig.spec.json (456 bytes)
CREATE solvefy-task/.vscode/extensions.json (134 bytes)
CREATE solvefy-task/.vscode/launch.json (490 bytes)
CREATE solvefy-task/.vscode/mcp.json (188 bytes)
CREATE solvefy-task/.vscode/tasks.json (1020 bytes)
CREATE solvefy-task/src/main.ts (228 bytes)
CREATE solvefy-task/src/index.html (310 bytes)
CREATE solvefy-task/src/styles.css (81 bytes)
CREATE solvefy-task/src/app/app.spec.ts (702 bytes)

CREATE solvefy-task/src/app/app.ts (306 bytes)
CREATE solvefy-task/src/app/app.css (0 bytes)
CREATE solvefy-task/src/app/app.html (20488 bytes)
CREATE solvefy-task/src/app/app.config.ts (324 bytes)
CREATE solvefy-task/src/app/app.routes.ts (80 bytes)
CREATE solvefy-task/public/favicon.ico (15086 bytes)
npm error Cannot read properties of null (reading 'edgesOut')
npm error A complete log of this run can be found in:
C:\Users\Ahmad Abdullah\AppData\Local\npm-cache_logs\2026-10-24T07_14_47_855Z-debug-0.log
✖ Package install failed, see above.
The Schematic workflow failed. See above.
PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend>

24/10/2026, 12:15:42

Thinking

- **Searched 4 websites**

✓ Worked for a few seconds

Your Angular CLI is installed correctly. The problem happened during **npm package installation**, not during Angular project creation.

From your terminal output, Angular 21.2.5 created the project files successfully, but npm failed with:

`Cannot read properties of null (reading 'edgesOut')`

Let's fix this without creating the project again unnecessarily.

Step 1: Enter the project folder

You are currently in:

`C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend`

Run:

```
cd solvefy-task
```

Then check that the project files exist:

```
dir
```

You should see `package.json`, `angular.json`, and the `src` folder.

Step 2: Try installing the packages again

Your Node.js `22.19.0` is compatible with Angular 21, so don't uninstall it yet. [Version compatibility]

First, run:

```
npm install
```

If it succeeds, skip to Step 4.

Step 3: If the same error appears again

This error is associated with npm's dependency-resolution process, and similar reports have suggested retrying with legacy peer-dependency resolution. [npm error Cannot read properties of null (reading 'edgesOut') · Issue #9960 · npm/cli]

Try:

```
npm install --legacy-peer-deps
```

If that also fails, don't delete anything else yet. Please share the new error output so we can identify the exact dependency causing the problem.

Step 4: Start Angular

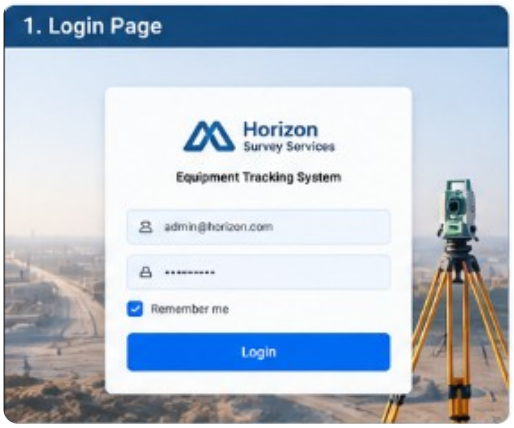
Once installation succeeds, run:

```
ng serve --open
```

Your app should open at `http://localhost:4200/`.

Important: You created this project inside `BMADSolvefy\src\AngularFrontend`. That's fine if you want a separate practice app, but if `AngularFrontend` already contains your actual Solvefy application, don't replace or modify its existing files.

24/10/2026, 12:15:43 · gpt-6



please help me generate this into angular code ?

24/10/2026, 12:30:05

Yes. We can recreate your **Horizon Survey Services — Equipment Tracking System login page** in Angular using HTML and CSS, matching the screenshot as closely as possible.

We'll build a responsive login card with the company logo, email and password fields, a “Remember me” checkbox, a blue Login button, and a background image featuring a surveying instrument.

Since you created the Angular project named `solvefy-task`, let's use that project.

Step 1: Open your Angular project

In the VS Code terminal:

```
cd "C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend\solvefy-task"
```

Make sure the dependencies are installed first:

```
npm install
```

Step 2: Replace `src/app/app.html`

Open `src/app/app.html`, remove its existing content, and paste this code:

```
<div class="login-page">
  <div class="page-heading">1. Login Page</div>

  <div class="login-card">
    <div class="brand">
      <div class="brand-logo">
        <svg viewBox="0 0 52 34" aria-label="Horizon logo">
```

```
        <path d="M4 26 L15 6 L23 20 L31 6 L45 26"
              fill="none" stroke="#17619a"
              stroke-width="5" stroke-linejoin="round"
              stroke-linecap="round"/>
        <path d="M18 26 L25 15 L32 26"
              fill="none" stroke="#17619a"
              stroke-width="4" stroke-linejoin="round"/>
      </svg>
    </div>

    <div class="brand-name">
      <h2>Horizon</h2>
      <span>Survey Services</span>
    </div>
  </div>

  <h3>Equipment Tracking System</h3>

  <form (submit)="login($event)">
    <label class="input-field">
      <span class="field-icon">♙</span>
      <input
        type="email"
        placeholder="admin@horizon.com"
        value="admin@horizon.com"
        required
        autocomplete="username"
      />
    </label>

    <label class="input-field">
      <span class="field-icon">♙</span>
      <input
        type="password"
        placeholder="Password"
        value="password123"
        required
        autocomplete="current-password"
      />
    </label>

    <label class="remember-row">
      <input type="checkbox" checked />
      <span>Remember me</span>
    </label>
```

```
      <button type="submit">Login</button>
    </form>
  </div>
</div>
```

Step 3: Replace `src/app/app.css`

Paste this CSS into `src/app/app.css`:

```
:host {
  display: block;
  min-height: 100vh;
  font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

.login-page {
  min-height: 100vh;
  position: relative;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 55px 20px 20px;
  overflow: hidden;

  background:
    linear-gradient(
      90deg,
      rgba(218, 232, 244, 0.40),
      rgba(224, 239, 250, 0.18)
    ),
    url('/survey-bg.jpg') center center / cover no-repeat,
    linear-gradient(135deg, #d9e8f5, #9bbbd3);
}

.page-heading {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  min-height: 27px;
  padding: 6px 12px;
  color: #fff;
```

```
  background: #20557d;
  font-size: 13px;
  font-weight: 700;
  z-index: 1;
}

.login-card {
  width: 100%;
  max-width: 340px;
  padding: 25px 24px 22px;
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid rgba(255, 255, 255, 0.8);
  border-radius: 7px;
  box-shadow: 0 5px 22px rgba(23, 55, 80, 0.18);
}

.brand {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 8px;
  margin-bottom: 18px;
}

.brand-logo {
  width: 43px;
  flex-shrink: 0;
}

.brand-logo svg {
  display: block;
  width: 100%;
}

.brand-name h2 {
  margin: 0;
  color: #185b91;
  font-size: 23px;
  line-height: 1;
  font-weight: 700;
}

.brand-name span {
  display: block;
  margin-top: 3px;
  color: #3c4a55;
```

```
  font-size: 11px;
  font-weight: 600;
}

.login-card h3 {
  margin: 0 0 19px;
  text-align: center;
  font-size: 12px;
  font-weight: 700;
  color: #222;
}

.input-field {
  display: flex;
  align-items: center;
  gap: 8px;
  height: 37px;
  margin-bottom: 9px;
  padding: 0 10px;
  border: 1px solid #d8e2ee;
  border-radius: 5px;
  background: #f0f5fb;
  transition: border-color 0.2s, box-shadow 0.2s;
}

.input-field:focus-within {
  border-color: #0875e1;
  box-shadow: 0 0 0 2px rgba(8, 117, 225, 0.12);
}

.field-icon {
  font-size: 15px;
  color: #34495e;
}

.input-field input {
  width: 100%;
  min-width: 0;
  border: none;
  outline: none;
  background: transparent;
  color: #273747;
  font-size: 11px;
}

.input-field input::placeholder {
```

```
  color: #65778a;
}

.remember-row {
  display: flex;
  align-items: center;
  gap: 6px;
  margin: 12px 0 16px;
  color: #333;
  font-size: 11px;
  cursor: pointer;
}

.remember-row input {
  width: 13px;
  height: 13px;
  margin: 0;
  accent-color: #0875e1;
  cursor: pointer;
}

button {
  display: block;
  width: 100%;
  height: 39px;
  border: none;
  border-radius: 5px;
  background: #0868e8;
  color: #fff;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s, transform 0.2s;
}

button:hover {
  background: #0756c4;
}

button:active {
  transform: scale(0.99);
}

@media (max-width: 480px) {
  .login-page {
    padding: 45px 16px 20px;
```

```
  }

  .login-card {
    max-width: 340px;
    padding: 24px 20px;
  }
}
```

Step 4: Update `src/app/app.ts`

The form needs a submit handler so Angular recognizes the login action. Replace the contents of `src/app/app.ts` with:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
  login(event: Event): void {
    event.preventDefault();

    // UI demonstration only.
    // Connect your authentication API here.
    alert('Login form submitted. Authentication is not connected yet.');
  }
}
```

Important: This creates the login-page design, not a working authentication system. Real login functionality requires connecting the form to your backend API.

Step 5: Add the background image

Your screenshot has a surveying landscape behind the login card. To reproduce that effect:

1. Open the `public` folder in your Angular project.
2. Add a suitable surveying-site photograph.
3. Name it `survey-bg.jpg`.

Your folder structure should look like this:

```
solvefy-task/
├── public/
│   ├── favicon.ico
│   └── survey-bg.jpg
├── src/
│   └── app/
│       ├── app.ts
│       ├── app.html
│       └── app.css
├── angular.json
└── package.json
```

If you don't add the photograph yet, the fallback blue gradient will still appear.

Step 6: Run the project

In the terminal:

```
ng serve --open
```

Open `http://localhost:4200/` if the browser doesn't open automatically.

Next step

Once this design is running, we can improve it further by matching the screenshot's exact dimensions and spacing, adding a proper Horizon logo, and connecting the login form to your actual backend.

24/10/2026, 12:30:16 · gpt-6

Pasted markdown(2).md

i have made a new component (login) how to use that component in angular app.html file such that when i start the project that page loads first ??

24/10/2026, 12:37:52

Since your Angular project uses Angular 21 and standalone components, you can display your `LoginComponent` directly in `app.html` by importing it into the root component.

Step 1: Check your login component

Suppose your component is located at:

```
src/app/login/
├── login.ts
├── login.html
└── login.css
```

Open `login.ts` and check its component selector. For example:

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [],
  templateUrl: './login.html',
  styleUrl: './login.css'
})
export class LoginComponent {}
```

The important part is `selector: 'app-login'`. Your selector might be different, so use the actual one from your file.

Step 2: Import `LoginComponent` into `app.ts`

Open `src/app/app.ts` and update it as follows. Adjust the import path if your login component is in a different folder.

```
import { Component } from '@angular/core';
import { LoginComponent } from './login/login';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [LoginComponent],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {}
```

Important: The import path `./login/login` assumes that your component file is `src/app/login/login.ts`.

Step 3: Display the component in `app.html`

Open `src/app/app.html`. Your uploaded file contains Angular's default welcome-page template, so replace its contents with just:

```
<app-login></app-login>
```

This renders your login component instead of the default Angular welcome page.

Step 4: Run the project

In your terminal, from the Angular project root, run:

```
ng serve --open
```

Open `http://localhost:4200/` if it doesn't open automatically.

One important detail

Your uploaded `app.html` also contains `<router-outlet />` at the bottom. You don't need it for this direct-component approach. If you want to use Angular routing instead, you can keep the router outlet and configure the login page as the default route.

My recommendation: Use the direct-component approach first because it is the simplest way to make your login page load immediately. Later, you can configure routing for the dashboard and other pages.

24/10/2026, 12:37:51 · gpt-6

Application bundle generation complete. [0.258 seconds] - 2026-10-24T07:35:55.008Z

Component update sent to client(s).
Application bundle generation failed. [0.144 seconds] - 2026-10-24T07:38:46.603Z

X [ERROR] NG1010: 'imports' must be an array of components, directives, pipes, or NgModules.
  Value could not be determined statically. [plugin angular-compiler]

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵   ~~~~~~~~~~~~~~~

  Unknown reference.

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵   ~~~~~~~~~~~~~~~

X [ERROR] TS2304: Cannot find name 'LoginComponent'. [plugin angular-compiler]

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵   ~~~~~~~~~~~~~~~

Application bundle generation failed. [0.172 seconds] - 2026-10-24T07:39:01.806Z

X [ERROR] TS2307: Cannot find module './login/login.component' or its corresponding type declarations. [plugin angular-compiler]

    src/app/app.ts:3:31:
      3 │ import { LoginComponent } from './login/login.component';
        ╵
~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~

```
X [ERROR] NG1010: 'imports' must be an array of components,
directives, pipes, or NgModules.
  Value could not be determined statically. [plugin angular-
compiler]

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~

  Unknown reference.

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~


X [ERROR] Could not resolve "./login/login.component"

    src/app/app.ts:3:31:
      3 │ import { LoginComponent } from
'./login/login.component';
        ╵
~~~~~~~~~~~~~~~~~~~~~~~~~~~~


Application bundle generation failed. [0.154 seconds] - 2026-
10-24T07:39:25.156Z

X [ERROR] TS2305: Module '"./components/login/login"' has no
exported member 'LoginComponent'. [plugin angular-compiler]

    src/app/app.ts:3:9:
      3 │ import { LoginComponent } from
'./components/login/login';
        ╵         ~~~~~~~~~~~~~~~


X [ERROR] NG1010: 'imports' must be an array of components,
directives, pipes, or NgModules.
  Value could not be determined statically. [plugin angular-
compiler]

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~
```

```
  Unknown reference.

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~


Application bundle generation failed. [0.126 seconds] - 2026-10-24T07:39:44.920Z

X [ERROR] TS2305: Module '"./components/login/login"' has no exported member 'LoginComponent'. [plugin angular-compiler]

    src/app/app.ts:3:9:
      3 │ import { LoginComponent } from './components/login/login';
        ╵         ~~~~~~~~~~~~~~


Application bundle generation failed. [0.071 seconds] - 2026-10-24T07:40:41.449Z

X [ERROR] TS2305: Module '"./components/login/login"' has no exported member 'LoginComponent'. [plugin angular-compiler]

    src/app/app.ts:3:9:
      3 │ import { LoginComponent } from './components/login/login';
        ╵         ~~~~~~~~~~~~~~


X [ERROR] NG1010: 'imports' must be an array of components, directives, pipes, or NgModules.
  Value could not be determined statically. [plugin angular-compiler]

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~

  Unknown reference.

    src/app/app.ts:6:12:
      6 │   imports: [LoginComponent],
        ╵            ~~~~~~~~~~~~~~~
```

The error is clear: **Angular cannot find the `LoginComponent` class at the path you are importing, or your login file exports the component under a different class name.**

Your latest error shows that you are using this path:

```
import { LoginComponent } from './components/login/login';
```

But the file at that path does not export a class named `LoginComponent`.

Let's fix it step by step.

Step 1: Check your folder structure

In VS Code, expand `src/app/components/login/`.

You may have files such as:

```
src/
└── app/
    ├── app.ts
    ├── app.html
    └── components/
        └── login/
            ├── login.ts
            ├── login.html
            └── login.css
```

Because Angular 21 uses concise filenames by default, your file may be `login.ts`, not `login.component.ts`.

Step 2: Check `login.ts`

Open `src/app/components/login/login.ts`. Make sure the component class is exported. For example:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [],
  templateUrl: './login.html',
  styleUrl: './login.css'
```

```
})
export class LoginComponent {}
```

Important: Keep your existing HTML and CSS filenames if they differ. The key part is:

```
export class LoginComponent {}
```

If your class is named `Login` instead, you must import `Login`, not `LoginComponent`.

Step 3: Correct `app.ts`

If your file is `src/app/components/login/login.ts` and it exports `LoginComponent`, use:

```
import { Component } from '@angular/core';
import { LoginComponent } from './components/login/login';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [LoginComponent],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {}
```

The import path is relative to `src/app/app.ts`.

Step 4: Update `app.html`

Replace the existing Angular welcome-page content with:

```
<app-login></app-login>
```

The selector must match `selector: 'app-login'` in your login component.

Step 5: Save and check

Save all three files. Angular should automatically rebuild. If needed, stop the server with `Ctrl + C` and restart:

```
ng serve --open
```

If the error still appears

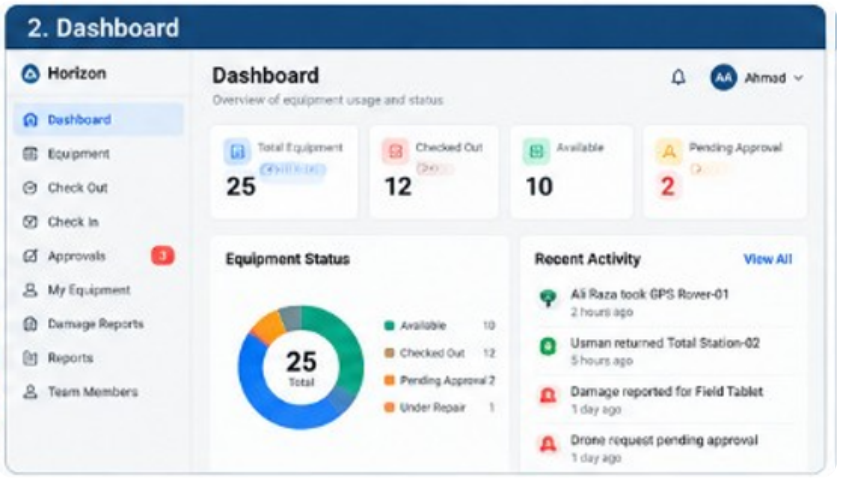
Please share the contents of these two files:

1. `src/app/components/login/login.ts`

2. `src/app/app.ts`

I can then give you the exact corrected code using your actual filenames and exported class name.

24/10/2026, 12:40:56 · gpt-6



now give me this dashboard code in angular ??

24/10/2026, 12:50:32

We can build this **Equipment Tracking System Dashboard** in your existing Angular 21 project, matching your screenshot with a sidebar, top header, summary cards, equipment-status donut chart, and recent activity panel.

I'll use a standalone Angular component, HTML, and CSS so you can run it directly. The dashboard will use sample data for now; later, we can connect it to your backend API.

1. Create the dashboard component

Run this command from your Angular project root:

```
ng generate component components/dashboard
```

Angular will generate the component files. With your Angular CLI version, they may be named `dashboard.ts`, `dashboard.html`, and `dashboard.css`.

2. Add the dashboard logic — `dashboard.ts`

Open `src/app/components/dashboard/dashboard.ts` and replace its contents with:

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css'
```

```
})
export class DashboardComponent {
  equipmentStats = [
    {
      title: 'Total Equipment',
      value: 25,
      icon: '▣',
      color: 'blue',
      detail: 'Registered assets'
    },
    {
      title: 'Checked Out',
      value: 12,
      icon: '▤',
      color: 'red',
      detail: 'Currently in use'
    },
    {
      title: 'Available',
      value: 10,
      icon: '▦',
      color: 'green',
      detail: 'Ready to use'
    },
    {
      title: 'Pending Approval',
      value: 2,
      icon: '♧',
      color: 'yellow',
      detail: 'Awaiting review'
    }
  ];

  equipmentStatus = [
    { label: 'Available', value: 10, color: '#119c78' },
    { label: 'Checked Out', value: 12, color: '#b98d61' },
    { label: 'Pending Approval', value: 2, color: '#f4bd45' },
    { label: 'Under Repair', value: 1, color: '#f28b47' }
  ];

  activities = [
    {
      type: 'success',
      title: 'Ali Raza took GPS Rover-01',
      time: '2 hours ago',
      icon: '↗'
```

```
    },
    {
      type: 'success',
      title: 'Usman returned Total Station-02',
      time: '5 hours ago',
      icon: '↙'
    },
    {
      type: 'danger',
      title: 'Damage reported for Field Tablet',
      time: '1 day ago',
      icon: '!'
    },
    {
      type: 'danger',
      title: 'Drone request pending approval',
      time: '1 day ago',
      icon: '!'
    }
  ];

  menuItems = [
    { label: 'Dashboard', icon: '⌂', active: true },
    { label: 'Equipment', icon: '▦', active: false },
    { label: 'Check Out', icon: '↗', active: false },
    { label: 'Check In', icon: '↙', active: false },
    { label: 'Approvals', icon: '☑', active: false },
    { label: 'My Equipment', icon: '♙', active: false },
    { label: 'Damage Reports', icon: '⚒', active: false },
    { label: 'Reports', icon: '▤', active: false },
    { label: 'Team Members', icon: '♧', active: false }
  ];

  selectedMenu = 'Dashboard';
  showAllActivities = false;
  showProfileMenu = false;

  get visibleActivities() {
    return this.showAllActivities
      ? this.activities
      : this.activities.slice(0, 4);
  }

  selectMenu(label: string): void {
    this.selectedMenu = label;
    this.menuItems.forEach(item => {
```

```
      item.active = item.label === label;
    });
  }

  toggleActivities(): void {
    this.showAllActivities = !this.showAllActivities;
  }

  toggleProfile(): void {
    this.showProfileMenu = !this.showProfileMenu;
  }
}
```

3. Build the dashboard layout — `dashboard.html`

Open `src/app/components/dashboard/dashboard.html` and paste:

```
<div class="dashboard-shell">

  <!-- Sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="selectMenu(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>

          @if (item.label === 'Approvals') {
            <span class="nav-badge">3</span>
          }
        </button>
      }
    </nav>

    <div class="sidebar-footer">
      <span class="status-dot"></span>
```

```
      Equipment Tracking System
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- Header -->
    <header class="topbar">
      <div class="mobile-brand">
        <div class="brand-mark">H</div>
        <span>Horizon</span>
      </div>

      <div class="topbar-spacer"></div>

      <button
        class="notification-button"
        type="button"
        aria-label="Notifications"
        title="Notifications"
      >
        ♧
        <span class="notification-dot"></span>
      </button>

      <div class="profile-container">
        <button
          type="button"
          class="profile-button"
          (click)="toggleProfile()"
        >
          <span class="avatar">AA</span>
          <span>Ahmad</span>
          <span class="chevron">⌄</span>
        </button>

        @if (showProfileMenu) {
          <div class="profile-menu">
            <div class="profile-menu-name">Ahmad</div>
            <button type="button" (click)="toggleProfile()">
              Close menu
            </button>
          </div>
        }
      </div>
```

```
</header>

<!-- Page Heading -->
<section class="page-heading">
  <div>
    <h1>{{ selectedMenu }}</h1>
    <p>Overview of equipment usage and status</p>
  </div>
</section>

<!-- Summary Cards -->
<section class="stats-grid">
  @for (stat of equipmentStats; track stat.title) {
    <article class="stat-card">
      <div class="stat-top">
        <span class="stat-icon" [class]=" 'stat-icon ' + stat.color">
          {{ stat.icon }}
        </span>
        <span class="stat-title">{{ stat.title }}</span>
      </div>

      <div class="stat-value">{{ stat.value }}</div>
      <div class="stat-detail">{{ stat.detail }}</div>
    </article>
  }
</section>

<!-- Dashboard Panels -->
<section class="dashboard-grid">

  <!-- Equipment Status -->
  <article class="panel status-panel">
    <div class="panel-heading">
      <h2>Equipment Status</h2>
      <button type="button" class="more-button" title="Equipment status">
        …
      </button>
    </div>

    <div class="status-content">
      <div class="donut-chart">
        <div class="donut-center">
          <strong>25</strong>
          <span>Total</span>
        </div>
      </div>
```

```
    <div class="legend">
      @for (status of equipmentStatus; track status.label) {
        <div class="legend-row">
          <span
            class="legend-dot"
            [style.background]="status.color"
          ></span>

          <span class="legend-label">{{ status.label }}</span>
          <span class="legend-value">{{ status.value }}</span>
        </div>
      }
    </div>
  </div>
</article>

<!-- Recent Activity -->
<article class="panel activity-panel">
  <div class="panel-heading">
    <h2>Recent Activity</h2>
    <button
      type="button"
      class="view-all"
      (click)="toggleActivities()"
    >
      {{ showAllActivities ? 'Show less' : 'View All' }}
    </button>
  </div>

  <div class="activity-list">
    @for (activity of visibleActivities; track activity.title) {
      <div class="activity-item">
        <div
          class="activity-icon"
          [class.success]="activity.type === 'success'"
          [class.danger]="activity.type === 'danger'"
        >
          {{ activity.icon }}
        </div>

        <div class="activity-info">
          <p>{{ activity.title }}</p>
          <span>{{ activity.time }}</span>
        </div>
      </div>
```

```
          }
        </div>
      </article>

    </section>

    <footer class="dashboard-footer">
      Horizon Survey Services
      <span>Equipment Tracking System</span>
    </footer>

  </main>
</div>
```

4. Style the dashboard — `dashboard.css`

Paste the following into `src/app/components/dashboard/dashboard.css`. This creates the sidebar, cards, colors, chart, and responsive layout.

```
:host {
  display: block;
  min-height: 100vh;
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 13px;
}

* {
  box-sizing: border-box;
}

button {
  font: inherit;
}

.dashboard-shell {
  display: flex;
  min-height: 100vh;
  background: #f6f8fb;
}

/* Sidebar */

.sidebar {
  width: 225px;
  flex-shrink: 0;
```

```
  display: flex;
  flex-direction: column;
  background: #fff;
  border-right: 1px solid #e5eaf1;
  padding: 18px 12px;
}

.brand {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 0 7px 28px;
  color: #27364b;
  font-size: 16px;
  font-weight: 700;
}

.brand-mark {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 24px;
  height: 24px;
  border-radius: 50%;
  background: #e8f2ff;
  color: #1766b2;
  font-size: 14px;
  font-weight: 800;
}

.navigation {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  width: 100%;
  min-height: 38px;
  padding: 9px 10px;
  border: 0;
  border-radius: 5px;
  background: transparent;
```

```
  color: #435064;
  text-align: left;
  font-size: 12px;
  cursor: pointer;
  transition: background 0.2s, color 0.2s;
}

.nav-item:hover {
  background: #f0f5fc;
}

.nav-item.active {
  background: #eaf2ff;
  color: #1769c2;
  font-weight: 700;
}

.nav-icon {
  width: 15px;
  flex-shrink: 0;
  font-size: 15px;
  text-align: center;
}

.nav-badge {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 17px;
  height: 17px;
  margin-left: auto;
  border-radius: 5px;
  background: #ed4d54;
  color: white;
  font-size: 10px;
}

.sidebar-footer {
  display: flex;
  align-items: center;
  gap: 7px;
  margin-top: auto;
  padding: 18px 7px 0;
  color: #8792a3;
  font-size: 10px;
}
```

```css
.status-dot,
.notification-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: #13a879;
}

/* Main area */

.main-content {
  flex: 1;
  min-width: 0;
  padding: 0 24px 18px;
}

.topbar {
  position: relative;
  display: flex;
  align-items: center;
  gap: 15px;
  min-height: 62px;
  border-bottom: 1px solid #e9edf3;
}

.topbar-spacer {
  flex: 1;
}

.mobile-brand {
  display: none;
  align-items: center;
  gap: 7px;
  font-weight: 700;
}

.notification-button {
  position: relative;
  width: 34px;
  height: 34px;
  border: 0;
  background: transparent;
  color: #59677b;
  font-size: 19px;
  cursor: pointer;
```

```
}

.notification-dot {
  position: absolute;
  top: 6px;
  right: 5px;
  width: 6px;
  height: 6px;
  background: #e84b54;
}

.profile-container {
  position: relative;
}

.profile-button {
  display: flex;
  align-items: center;
  gap: 8px;
  border: 0;
  background: transparent;
  color: #344155;
  font-size: 12px;
  cursor: pointer;
}

.avatar {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 31px;
  height: 31px;
  border-radius: 50%;
  background: #1d5b94;
  color: white;
  font-size: 10px;
  font-weight: 700;
}

.chevron {
  color: #7b8798;
}

.profile-menu {
  position: absolute;
  z-index: 5;
```

```
  top: 42px;
  right: 0;
  min-width: 145px;
  padding: 12px;
  border: 1px solid #e5eaf1;
  border-radius: 7px;
  background: white;
  box-shadow: 0 6px 20px #1b35501a;
}

.profile-menu-name {
  margin-bottom: 10px;
  font-weight: 700;
}

.profile-menu button {
  border: 0;
  background: transparent;
  color: #1769c2;
  cursor: pointer;
}

.page-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 0 18px;
}

.page-heading h1 {
  margin: 0 0 6px;
  color: #202a39;
  font-size: 24px;
  font-weight: 700;
}

.page-heading p {
  margin: 0;
  color: #8994a4;
  font-size: 12px;
}

/* Summary cards */

.stats-grid {
  display: grid;
```

```
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 20px;
}

.stat-card {
  min-width: 0;
  padding: 15px 14px 13px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.stat-top {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 10px;
}

.stat-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 6px;
  font-size: 15px;
  font-weight: 700;
}

.stat-icon.blue {
  background: #eaf2ff;
  color: #2675dd;
}

.stat-icon.red {
  background: #fff0ee;
  color: #e86661;
}

.stat-icon.green {
  background: #e6f8f1;
  color: #169c76;
```

```
}

.stat-icon.yellow {
  background: #fff6df;
  color: #d99c18;
}

.stat-title {
  color: #68758a;
  font-size: 11px;
  line-height: 1.4;
}

.stat-value {
  margin-bottom: 4px;
  color: #202a39;
  font-size: 27px;
  font-weight: 700;
  line-height: 1.1;
}

.stat-detail {
  color: #a0a9b7;
  font-size: 10px;
}

/* Panels */

.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  align-items: stretch;
  gap: 16px;
}

.panel {
  min-width: 0;
  padding: 18px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.panel-heading {
  display: flex;
```

```
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 23px;
}

.panel-heading h2 {
  margin: 0;
  color: #283447;
  font-size: 14px;
  font-weight: 700;
}

.more-button {
  border: 0;
  background: transparent;
  color: #8591a2;
  font-size: 20px;
  cursor: pointer;
}

.status-content {
  display: flex;
  align-items: center;
  justify-content: space-around;
  gap: 16px;
  padding: 5px 0 14px;
}

.donut-chart {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 155px;
  height: 155px;
  flex-shrink: 0;
  border-radius: 50%;
  background: conic-gradient(
    #119c78 0% 40%,
    #b98d61 40% 88%,
    #f4bd45 88% 96%,
    #f28b47 96% 100%
  );
  transform: rotate(-90deg);
}
```

```css
.donut-center {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  width: 100px;
  height: 100px;
  border-radius: 50%;
  background: white;
  transform: rotate(90deg);
}

.donut-center strong {
  color: #263346;
  font-size: 27px;
  line-height: 1.2;
}

.donut-center span {
  margin-top: 3px;
  color: #929cac;
  font-size: 11px;
}

.legend {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 14px;
  min-width: 0;
}

.legend-row {
  display: grid;
  grid-template-columns: 8px minmax(0, 1fr) auto;
  align-items: center;
  gap: 7px;
  color: #5f6b7c;
  font-size: 11px;
}

.legend-dot {
  width: 8px;
  height: 8px;
  border-radius: 2px;
}
```

```css
.legend-label {
  line-height: 1.4;
}

.legend-value {
  color: #748093;
}

/* Recent activity */

.view-all {
  border: 0;
  background: transparent;
  color: #2675dd;
  font-size: 12px;
  font-weight: 600;
  white-space: nowrap;
  cursor: pointer;
}

.activity-list {
  display: flex;
  flex-direction: column;
}

.activity-item {
  display: flex;
  align-items: flex-start;
  gap: 11px;
  padding: 0 0 19px;
}

.activity-item:last-child {
  padding-bottom: 0;
}

.activity-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 7px;
  font-size: 14px;
```

```
  font-weight: 700;
}

.activity-icon.success {
  background: #e4f7ed;
  color: #139968;
}

.activity-icon.danger {
  background: #fff0ef;
  color: #e45454;
}

.activity-info {
  min-width: 0;
  padding-top: 1px;
}

.activity-info p {
  margin: 0 0 5px;
  color: #3b4657;
  font-size: 11px;
  line-height: 1.45;
  overflow-wrap: anywhere;
}

.activity-info span {
  color: #9aa4b2;
  font-size: 10px;
}

.dashboard-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  padding-top: 22px;
  color: #8a95a4;
  font-size: 10px;
}

.dashboard-footer span {
  text-align: right;
}

/* Responsive layout */
```

```css
@media (max-width: 1100px) {
  .sidebar {
    width: 190px;
  }

  .main-content {
    padding-right: 16px;
    padding-left: 16px;
  }

  .stats-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 650px) {
  .sidebar {
    display: none;
  }

  .main-content {
    padding: 0 13px 16px;
  }

  .topbar {
    min-height: 56px;
  }

  .mobile-brand {
    display: flex;
  }

  .page-heading {
    padding: 18px 0;
  }

  .page-heading h1 {
    font-size: 22px;
  }

  .stats-grid {
    gap: 10px;
```

```
  }

  .stat-card {
    padding: 12px 10px;
  }

  .stat-title {
    font-size: 10px;
  }

  .stat-value {
    font-size: 24px;
  }

  .panel {
    padding: 14px;
  }

  .status-content {
    flex-wrap: wrap;
  }

  .donut-chart {
    width: 135px;
    height: 135px;
  }

  .donut-center {
    width: 88px;
    height: 88px;
  }

  .legend {
    min-width: 135px;
  }
}
```

5. Load the dashboard when Angular starts

Now connect the new component to your root application.

Open `src/app/app.ts` and use:

```
import { Component } from '@angular/core';
import { DashboardComponent } from './components/dashboard/dashboard';

@Component({
```

```
  selector: 'app-root',
  standalone: true,
  imports: [DashboardComponent],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {}
```

This assumes the generated file is `src/app/components/dashboard/dashboard.ts` and exports `DashboardComponent`.

Now replace all the contents of `src/app/app.html` with:

```
<app-dashboard></app-dashboard>
```

One small detail: in `dashboard.html`, keep the summary icon's class binding like this for compatibility:

```
<span
  class="stat-icon"
  [class.blue]="stat.color === 'blue'"
  [class.red]="stat.color === 'red'"
  [class.green]="stat.color === 'green'"
  [class.yellow]="stat.color === 'yellow'"
>
  {{ stat.icon }}
</span>
```

This applies the correct color class to each card.

6. Run the dashboard

In the terminal, from your project root:

```
ng serve --open
```

Then open `http://localhost:4200/`.

You should see the dashboard with:

- Sidebar navigation for Dashboard, Equipment, Check Out, Check In, Approvals, and other sections.
- Four summary cards showing equipment counts.
- A donut chart with status categories.
- Recent activity with a working “View All” toggle.

- A profile dropdown and responsive layout for smaller screens.

Note: The menu currently changes the selected heading, while the chart, cards, and activities use sample data. The navigation destinations and real login/session data are not connected yet. We can add those next, along with API integration for your actual Equipment Tracking System.

24/10/2026, 12:51:04 · gpt-6

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [],
  templateUrl: './login.html',
  styleUrl: './login.css',
})
export class Login {
  login(event: Event): void {
    event.preventDefault();

    // UI demonstration only.
    // Connect your authentication API here.
    alert('Login form submitted. Authentication is not connected yet.');
  }
}
```

i want after login then dashboard page opens ?

24/10/2026, 12:54:03

You can do this using **Angular Router**. When the user clicks Login, Angular will navigate to the Dashboard page.

Since your login component class is named `Login`, we can keep that name.

Step 1: Configure the routes

Open `src/app/app.routes.ts` and replace its contents with:

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { DashboardComponent } from './components/dashboard/dashboard';

export const routes: Routes = [
  { path: 'login', component: Login },
  { path: 'dashboard', component: DashboardComponent },
  { path: '', redirectTo: 'login', pathMatch: 'full' },
```

```
  { path: '**', redirectTo: 'login' }
];
```

Adjust the import paths if your files are located elsewhere.

Step 2: Update `app.ts`

Remove the direct component imports and use `RouterOutlet` instead.

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {}
```

Step 3: Update `app.html`

Replace its contents with:

```
<router-outlet></router-outlet>
```

Angular will display the component matching the current URL.

Step 4: Update `app.config.ts`

Open `src/app/app.config.ts` and ensure the router is provided:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

If you already have other providers in this file, keep them and add `provideRouter(routes)` rather than replacing them.

Step 5: Update your login component

In `src/app/components/login/login.ts`, import `Router` and inject it into your component.

```ts
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [],
  templateUrl: './login.html',
  styleUrl: './login.css'
})
export class Login {
  private router = inject(Router);

  login(event: Event): void {
    event.preventDefault();

    // Demo navigation only — no credentials are verified.
    this.router.navigate(['/dashboard']);
  }
}
```

Step 6: Check your login form

In `src/app/components/login/login.html`, ensure your form calls the `login()` method:

```html
<form (submit)="login($event)">
  <!-- Your existing email and password fields -->

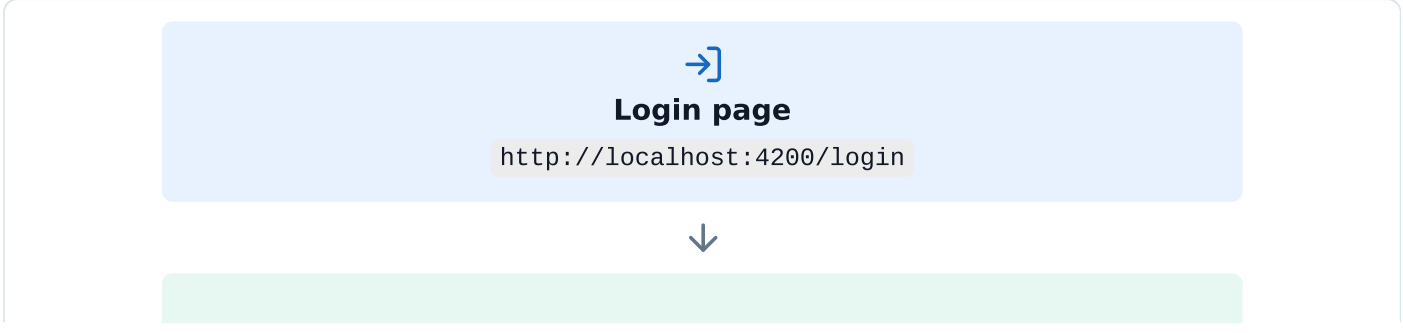
  <button type="submit">Login</button>
</form>
```

Keep your existing fields and styling; just ensure the form has the submit handler and button.

Step 7: Run the application

```
ng serve --open
```

Your flow will now be:



Dashboard page

`http://localhost:4200/dashboard`

Important: This currently navigates to the dashboard whenever Login is clicked. It does not authenticate the user. For a real Equipment Tracking System, you should validate the credentials through your backend API before navigating to the dashboard.

24/10/2026, 12:54:02 · gpt-6

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend\solvefy-task> ng serve --open
Application bundle generation failed. [2.362 seconds] - 2026-10-24T07:58:51.267Z

X [ERROR] TS2304: Cannot find name 'inject'. [plugin angular-compiler]

    src/app/components/login/login.ts:12:20:
      12 │   private router = inject(Router);
         ╵   ~~~~~~

Watch mode enabled. Watching for file changes...

24/10/2026, 12:59:03

The error occurs because `inject` has not been imported from `@angular/core` in your `login.ts` file.

Fix: Update `login.ts`

At the top of `src/app/components/login/login.ts`, make sure you have **both imports**:

```
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';
```

Your complete file should look like this:

```
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [],
  templateUrl: './login.html',
  styleUrl: './login.css'
})
export class Login {
  private router = inject(Router);
```

```
  login(event: Event): void {
    event.preventDefault();

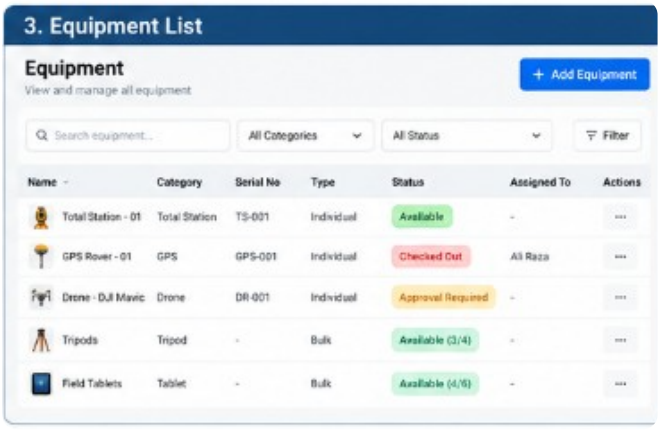
    // Demo navigation only; authentication is not connected.
    this.router.navigate(['/dashboard']);
  }
}
```

Then check your application

1. Save the file with `Ctrl + S`.
2. Angular should automatically rebuild because watch mode is enabled.
3. Open `http://localhost:4200/login`.
4. Click **Login**. It should navigate to `/dashboard`.

If another error appears, share the complete error message and we'll fix it next.

24/10/2026, 12:59:03 · gpt-6



please generate this in angular code ??

24/10/2026, 13:08:39

Let's create the **Equipment List page** for your existing Angular 21 Equipment Tracking System, matching your screenshot.

We'll build a page with the sidebar navigation, a blue “Add Equipment” button, search field, category and status dropdowns, a Filter button, and an equipment table with status badges and action menus.

We'll use your existing Angular project and keep the page as a standalone component, just like your Login and Dashboard components.

Step 1: Generate the component

Open your terminal in the `solvefy-task` project folder and run:

```
ng generate component components/equipment-list
```

This should create the component under `src/app/components/equipment-list/`.

Step 2: Add the TypeScript logic

Open `src/app/components/equipment-list/equipment-list.ts` and replace its contents with this code.

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface Equipment {
  id: number;
  name: string;
  category: string;
```

```
  serialNo: string;
  type: string;
  status: string;
  assignedTo: string;
}

@Component({
  selector: 'app-equipment-list',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './equipment-list.html',
  styleUrl: './equipment-list.css'
})
export class EquipmentList {
  searchText = '';
  selectedCategory = 'All Categories';
  selectedStatus = 'All Status';
  showFilters = false;
  showAddForm = false;
  selectedEquipment: number | null = null;

  categories = ['All Categories', 'Total Station', 'GPS', 'Drone', 'Tripod', 'Tablet'];

  statuses = [
    'All Status',
    'Available',
    'Checked Out',
    'Approval Required',
    'Under Repair'
  ];

  equipment: Equipment[] = [
    {
      id: 1,
      name: 'Total Station - 01',
      category: 'Total Station',
      serialNo: 'TS-001',
      type: 'Individual',
      status: 'Available',
      assignedTo: '-'
    },
    {
      id: 2,
      name: 'GPS Rover - 01',
      category: 'GPS',
      serialNo: 'GPS-001',
```

```
      type: 'Individual',
      status: 'Checked Out',
      assignedTo: 'Ali Raza'
    },
    {
      id: 3,
      name: 'Drone - DJI Mavic',
      category: 'Drone',
      serialNo: 'DR-001',
      type: 'Individual',
      status: 'Approval Required',
      assignedTo: '-'
    },
    {
      id: 4,
      name: 'Tripods',
      category: 'Tripod',
      serialNo: '-',
      type: 'Bulk',
      status: 'Available (3/4)',
      assignedTo: '-'
    },
    {
      id: 5,
      name: 'Field Tablets',
      category: 'Tablet',
      serialNo: '-',
      type: 'Bulk',
      status: 'Available (4/6)',
      assignedTo: '-'
    }
  ];

  get filteredEquipment(): Equipment[] {
    const search = this.searchText.trim().toLowerCase();

    return this.equipment.filter(item => {
      const matchesSearch =
        !search ||
        item.name.toLowerCase().includes(search) ||
        item.category.toLowerCase().includes(search) ||
        item.serialNo.toLowerCase().includes(search) ||
        item.assignedTo.toLowerCase().includes(search);

      const matchesCategory =
        this.selectedCategory === 'All Categories' ||
```

```
      item.category === this.selectedCategory;

    const matchesStatus =
      this.selectedStatus === 'All Status' ||
      (this.selectedStatus === 'Available'
        ? item.status.startsWith('Available')
        : item.status === this.selectedStatus);

    return matchesSearch && matchesCategory && matchesStatus;
  });
}

addEquipment(): void {
  this.showAddForm = true;
}

saveEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    return;
  }

  this.equipment = [
    ...this.equipment,
    {
      id: Math.max(0, ...this.equipment.map(item => item.id)) + 1,
      name: name.trim(),
      category,
      serialNo: serialNo.trim() || '-',
      type,
      status: 'Available',
      assignedTo: '-'
    }
  ];

  this.showAddForm = false;
}

toggleFilters(): void {
  this.showFilters = !this.showFilters;
}
```

```
  toggleActions(id: number): void {
    this.selectedEquipment =
      this.selectedEquipment === id ? null : id;
  }

  clearFilters(): void {
    this.searchText = '';
    this.selectedCategory = 'All Categories';
    this.selectedStatus = 'All Status';
  }

  trackById(_index: number, item: Equipment): number {
    return item.id;
  }
}
```

This provides sample equipment records, live search, category and status filtering, an Add Equipment form, and row action menus.

Step 3: Create the HTML — `equipment-list.html`

Replace the contents of `src/app/components/equipment-list/equipment-list.html` with:

```
<div class="equipment-page">
  <div class="page-banner">3. Equipment List</div>

  <main class="page-content">
    <header class="page-heading">
      <div>
        <h1>Equipment</h1>
        <p>View and manage all equipment</p>
      </div>

      <button class="primary-button" (click)="addEquipment()">
        <span>+</span> Add Equipment
      </button>
    </header>

    <section class="toolbar">
      <label class="search-box">
        <span class="search-icon">⌕</span>
        <input
          type="search"
          placeholder="Search equipment..."
          [(ngModel)]="searchText"
        />
      </label>
```

```
  <select
    aria-label="Filter by category"
    [(ngModel)]="selectedCategory"
  >
    @for (category of categories; track category) {
      <option [value]="category">{{ category }}</option>
    }
  </select>

  <select
    aria-label="Filter by status"
    [(ngModel)]="selectedStatus"
  >
    @for (status of statuses; track status) {
      <option [value]="status">{{ status }}</option>
    }
  </select>

  <button class="filter-button" (click)="toggleFilters()">
    <span>☷</span> Filter
  </button>
</section>

@if (showFilters) {
  <section class="filter-panel">
    <div>
      <strong>Equipment filters</strong>
      <p>Filter the list by category and current status.</p>
    </div>
    <button class="clear-button" (click)="clearFilters()">
      Clear filters
    </button>
  </section>
}

<section class="table-card">
  <div class="table-scroll">
    <table>
      <thead>
        <tr>
          <th>Name <span class="sort-icon">↕</span></th>
          <th>Category</th>
          <th>Serial No.</th>
          <th>Type</th>
          <th>Status</th>
```

```
        <th>Assigned To</th>
        <th class="actions-column">Actions</th>
      </tr>
    </thead>

    <tbody>
      @for (item of filteredEquipment; track item.id) {
        <tr>
          <td>
            <div class="equipment-name">
              <span class="equipment-icon">
                @switch (item.category) {
                  @case ('Total Station') { ⊙ }
                  @case ('GPS') { ◉ }
                  @case ('Drone') { ✣ }
                  @case ('Tripod') { ⁄ }
                  @case ('Tablet') { ▣ }
                  @default { ▦ }
                }
              </span>
              <span>{{ item.name }}</span>
            </div>
          </td>

          <td>{{ item.category }}</td>
          <td>{{ item.serialNo }}</td>
          <td>{{ item.type }}</td>

          <td>
            <span
              class="status-badge"
              [class.available]="item.status.startsWith('Available')"
              [class.checked-out]="item.status === 'Checked Out'"
              [class.approval]="item.status === 'Approval Required'"
              [class.repair]="item.status === 'Under Repair'"
            >
              {{ item.status }}
            </span>
          </td>

          <td>{{ item.assignedTo }}</td>

          <td class="actions-column">
            <button
              class="action-button"
              [attr.aria-label]="'Actions for ' + item.name"
```

```
                    (click)="toggleActions(item.id)"
                  >
                    …
                  </button>

                  @if (selectedEquipment === item.id) {
                    <div class="action-menu">
                      <button (click)="toggleActions(item.id)">
                        View Details
                      </button>
                      <button (click)="toggleActions(item.id)">
                        Edit Equipment
                      </button>
                    </div>
                  }
                </td>
              </tr>
            } @empty {
              <tr>
                <td colspan="7" class="empty-state">
                  <span class="empty-icon">⌕</span>
                  <strong>No equipment found</strong>
                  <p>Try changing your search or filters.</p>
                  <button class="clear-button" (click)="clearFilters()">
                    Clear filters
                  </button>
                </td>
              </tr>
            }
          </tbody>
        </table>
      </div>

      <footer class="table-footer">
        <span>
          Showing {{ filteredEquipment.length }} of {{ equipment.length }} items
        </span>
        <span class="records-label">Equipment records</span>
      </footer>
    </section>
  </main>
</div>

@if (showAddForm) {
  <div class="modal-backdrop" (click)="showAddForm = false">
    <section
```

```
      class="equipment-modal"
      role="dialog"
      aria-modal="true"
      aria-labelledby="add-equipment-title"
      (click)="$event.stopPropagation()"
    >
      <header class="modal-heading">
        <div>
          <h2 id="add-equipment-title">Add Equipment</h2>
          <p>Enter the details of the new equipment.</p>
        </div>
        <button
          type="button"
          class="close-button"
          aria-label="Close"
          (click)="showAddForm = false"
        >×</button>
      </header>

      <form
        #equipmentForm="ngForm"
        (ngSubmit)="saveEquipment(
          newName.value,
          newCategory.value,
          newSerial.value,
          newType.value
        )"
      >
        <label class="form-field">
          Equipment Name *
          <input #newName required placeholder="e.g. Total Station - 02" />
        </label>

        <label class="form-field">
          Category *
          <select #newCategory required>
            <option value="">Select category</option>
            @for (category of categories.slice(1); track category) {
              <option [value]="category">{{ category }}</option>
            }
          </select>
        </label>

        <label class="form-field">
          Serial Number
          <input #newSerial placeholder="e.g. TS-002" />
```

```
        </label>

        <label class="form-field">
          Equipment Type *
          <select #newType required>
            <option value="">Select type</option>
            <option value="Individual">Individual</option>
            <option value="Bulk">Bulk</option>
          </select>
        </label>

        <div class="modal-actions">
          <button
            type="button"
            class="cancel-button"
            (click)="showAddForm = false"
          >
            Cancel
          </button>
          <button
            type="submit"
            class="primary-button"
            [disabled]="!equipmentForm.valid"
          >
            Save Equipment
          </button>
        </div>
      </form>
    </section>
  </div>
}
```

Step 4: Add the CSS — `equipment-list.css`

Paste this into `src/app/components/equipment-list/equipment-list.css`:

```
:host {
  display: block;
  min-height: 100vh;
  color: #263445;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 12px;
}

* {
  box-sizing: border-box;
```

```
}

button,
input,
select {
  font: inherit;
}

button {
  cursor: pointer;
}

.equipment-page {
  min-height: 100vh;
  background: #f6f8fb;
}

.page-banner {
  height: 34px;
  padding: 9px 15px;
  background: #205780;
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}

.page-content {
  padding: 17px 20px 24px;
}

.page-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 14px;
  margin-bottom: 18px;
}

.page-heading h1 {
  margin: 0 0 4px;
  color: #202b3a;
  font-size: 23px;
  line-height: 1.2;
}

.page-heading p {
```

```
  margin: 0;
  color: #8994a4;
  font-size: 11px;
}

.primary-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  min-height: 34px;
  padding: 0 13px;
  border: 1px solid #0868e8;
  border-radius: 5px;
  background: #0868e8;
  color: #fff;
  font-weight: 600;
  white-space: nowrap;
  transition: background 0.2s;
}

.primary-button:hover {
  background: #0756c4;
}

.primary-button:disabled {
  cursor: not-allowed;
  opacity: 0.5;
}

.primary-button span {
  font-size: 17px;
  line-height: 1;
}

.toolbar {
  display: grid;
  grid-template-columns: minmax(170px, 1.4fr) minmax(120px, 1fr)
    minmax(120px, 1fr) auto;
  gap: 9px;
  margin-bottom: 13px;
}

.search-box {
  display: flex;
  align-items: center;
```

```
  gap: 7px;
  min-width: 0;
  height: 35px;
  padding: 0 10px;
  border: 1px solid #dfe5ed;
  border-radius: 5px;
  background: #fff;
}

.search-icon {
  color: #758397;
  font-size: 19px;
}

.search-box input {
  width: 100%;
  min-width: 0;
  border: 0;
  outline: 0;
  background: transparent;
  color: #263445;
  font-size: 11px;
}

.search-box:focus-within {
  border-color: #3983e9;
  box-shadow: 0 0 0 2px #3983e91a;
}

.toolbar select {
  width: 100%;
  min-width: 0;
  height: 35px;
  padding: 0 10px;
  border: 1px solid #dfe5ed;
  border-radius: 5px;
  background: #fff;
  color: #354154;
  outline: none;
  font-size: 11px;
}

.toolbar select:focus {
  border-color: #3983e9;
}
```

```css
.filter-button {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  height: 35px;
  padding: 0 12px;
  border: 1px solid #dfe5ed;
  border-radius: 5px;
  background: #fff;
  color: #354154;
}

.filter-button:hover,
.cancel-button:hover {
  background: #f0f4f9;
}

.filter-panel {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 12px;
  padding: 12px 14px;
  border: 1px solid #dfe7f0;
  border-radius: 6px;
  background: #fff;
}

.filter-panel strong {
  font-size: 12px;
}

.filter-panel p {
  margin: 4px 0 0;
  color: #8994a4;
  font-size: 11px;
}

.clear-button {
  padding: 6px 9px;
  border: 1px solid #dfe5ed;
  border-radius: 4px;
  background: white;
  color: #1769c2;
```

```
  white-space: nowrap;
}

.table-card {
  overflow: visible;
  border: 1px solid #e4e9f0;
  border-radius: 7px;
  background: #fff;
  box-shadow: 0 2px 6px #213a5206;
}

.table-scroll {
  width: 100%;
  overflow-x: auto;
  border-radius: 7px 7px 0 0;
}

table {
  width: 100%;
  min-width: 760px;
  border-collapse: collapse;
  text-align: left;
}

thead {
  background: #f7f9fc;
}

th {
  height: 36px;
  padding: 0 11px;
  border-bottom: 1px solid #e7ecf2;
  color: #354154;
  font-size: 10px;
  font-weight: 700;
  white-space: nowrap;
}

.sort-icon {
  margin-left: 3px;
  color: #99a4b3;
}

td {
  height: 37px;
  padding: 5px 11px;
```

```
  border-bottom: 1px solid #edf0f5;
  color: #475367;
  font-size: 10px;
  white-space: nowrap;
}

tbody tr:last-child td {
  border-bottom: 0;
}

tbody tr:hover {
  background: #fafcff;
}

.equipment-name {
  display: flex;
  align-items: center;
  gap: 8px;
  color: #293648;
  font-weight: 500;
}

.equipment-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 20px;
  height: 22px;
  flex-shrink: 0;
  border-radius: 4px;
  background: #f0f4f9;
  color: #3e6489;
  font-size: 15px;
  font-weight: 700;
}

.status-badge {
  display: inline-flex;
  align-items: center;
  min-height: 21px;
  padding: 3px 8px;
  border-radius: 12px;
  font-size: 9px;
  line-height: 1.2;
}
```

```css
.status-badge.available {
  background: #d8f5e5;
  color: #137849;
}

.status-badge.checked-out {
  background: #ffe0e0;
  color: #bd3039;
}

.status-badge.approval {
  background: #fff0c9;
  color: #9b6b00;
}

.status-badge.repair {
  background: #ffeadc;
  color: #b6531c;
}

.actions-column {
  position: relative;
  width: 50px;
  text-align: center;
}

.action-button {
  width: 25px;
  height: 24px;
  border: 1px solid #e4e9f0;
  border-radius: 5px;
  background: #fff;
  color: #536176;
  font-size: 17px;
  line-height: 1;
}

.action-button:hover {
  background: #eef4fc;
}

.action-menu {
  position: absolute;
  z-index: 4;
  top: 31px;
  right: 8px;
```

```
  display: flex;
  flex-direction: column;
  min-width: 140px;
  padding: 5px;
  border: 1px solid #e4e9f0;
  border-radius: 6px;
  background: #fff;
  box-shadow: 0 5px 18px #23384d1a;
}

.action-menu button {
  padding: 9px;
  border: 0;
  border-radius: 4px;
  background: #fff;
  color: #354154;
  text-align: left;
  font-size: 11px;
}

.action-menu button:hover {
  background: #eef4fc;
}

.table-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  padding: 12px;
  border-top: 1px solid #edf0f5;
  color: #8994a4;
  font-size: 10px;
}

.empty-state {
  height: 155px;
  text-align: center;
  white-space: normal;
}

.empty-state strong {
  display: block;
  margin-top: 7px;
  color: #354154;
  font-size: 12px;
}
```

```css
.empty-state p {
  color: #8994a4;
}

.empty-icon {
  display: block;
  color: #8a9bb0;
  font-size: 24px;
}

.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 10;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
  background: #15263d80;
}

.equipment-modal {
  width: 100%;
  max-width: 440px;
  max-height: 90vh;
  overflow-y: auto;
  padding: 22px;
  border-radius: 9px;
  background: #fff;
  box-shadow: 0 16px 50px #13243d33;
}

.modal-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 20px;
}

.modal-heading h2 {
  margin: 0 0 5px;
  color: #202b3a;
  font-size: 20px;
}
```

```css
.modal-heading p {
  margin: 0;
  color: #8994a4;
  font-size: 11px;
}

.close-button {
  width: 29px;
  height: 29px;
  border: 0;
  border-radius: 5px;
  background: #f0f4f9;
  color: #354154;
  font-size: 20px;
}

.form-field {
  display: flex;
  flex-direction: column;
  gap: 7px;
  margin-bottom: 15px;
  color: #354154;
  font-size: 12px;
  font-weight: 600;
}

.form-field input,
.form-field select {
  width: 100%;
  height: 38px;
  padding: 0 10px;
  border: 1px solid #dfe5ed;
  border-radius: 5px;
  outline: none;
  background: #fff;
  color: #354154;
  font-weight: 400;
}

.form-field input:focus,
.form-field select:focus {
  border-color: #3983e9;
  box-shadow: 0 0 0 2px #3983e91a;
}
```

```css
.modal-actions {
  display: flex;
  justify-content: flex-end;
  gap: 9px;
  margin-top: 22px;
}

.cancel-button {
  min-height: 34px;
  padding: 0 13px;
  border: 1px solid #dfe5ed;
  border-radius: 5px;
  background: white;
  color: #354154;
}

@media (max-width: 700px) {
  .page-content {
    padding: 14px 12px;
  }

  .page-heading h1 {
    font-size: 20px;
  }

  .toolbar {
    grid-template-columns: 1fr 1fr;
  }

  .search-box {
    grid-column: 1 / -1;
  }

  .filter-button {
    width: 100%;
  }

  .records-label {
    display: none;
  }
}

@media (max-width: 420px) {
  .page-heading {
    align-items: flex-start;
  }
```

```
  .primary-button {
    padding: 0 9px;
    font-size: 11px;
  }

  .toolbar {
    gap: 7px;
  }
}
```

Step 5: Add the Equipment List route

Open `src/app/app.routes.ts` and add the new component to your existing routes. Keep your Login and Dashboard routes.

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { DashboardComponent } from './components/dashboard/dashboard';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },
  { path: 'dashboard', component: DashboardComponent },
  { path: 'equipment', component: EquipmentList },
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: '**', redirectTo: 'login' }
];
```

If your actual component class or file names differ, use the names shown in your project.

Your `app.html` should still contain:

```
<router-outlet></router-outlet>
```

And `app.ts` should import `RouterOutlet` and include it in the component's `imports` array, as configured previously.

Step 6: Run and test

```
ng serve --open
```

After logging in, you can open the Equipment List directly at:

localhost

The page currently uses sample records and stores newly added equipment in memory. A page refresh resets those additions. The **View Details** and **Edit Equipment** actions are placeholders; they don't yet open detail or editing screens.

Next, we can connect the Dashboard's **Equipment** sidebar item to this route so clicking it navigates to the Equipment List instead of only changing the heading.

24/10/2026, 13:09:04 · gpt-6

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  imports: [],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
```

i want in this when i click on the equipment then equipment page opens ??

```
export class Dashboard {


  equipmentStats = [
    {
      title: 'Total Equipment',
      value: 25,
      icon: '▣',
      color: 'blue',
      detail: 'Registered assets'
    },
    {
      title: 'Checked Out',
      value: 12,
      icon: '▤',
      color: 'red',
      detail: 'Currently in use'
    },
    {
      title: 'Available',
      value: 10,
      icon: '▦',
      color: 'green',
      detail: 'Ready to use'
    },
    {
      title: 'Pending Approval',
      value: 2,
      icon: '♧',
      color: 'yellow',
      detail: 'Awaiting review'
    }
  ];
```

```
equipmentStatus = [
  { label: 'Available', value: 10, color: '#119c78' },
  { label: 'Checked Out', value: 12, color: '#b98d61' },
  { label: 'Pending Approval', value: 2, color: '#f4bd45' },
  { label: 'Under Repair', value: 1, color: '#f28b47' }
];

activities = [
  {
    type: 'success',
    title: 'Ali Raza took GPS Rover-01',
    time: '2 hours ago',
    icon: '↗'
  },
  {
    type: 'success',
    title: 'Usman returned Total Station-02',
    time: '5 hours ago',
    icon: '↙'
  },
  {
    type: 'danger',
    title: 'Damage reported for Field Tablet',
    time: '1 day ago',
    icon: '!'
  },
  {
    type: 'danger',
    title: 'Drone request pending approval',
    time: '1 day ago',
    icon: '!'
  }
];

menuItems = [
  { label: 'Dashboard', icon: '⌂', active: true },
  { label: 'Equipment', icon: '▦', active: false },
  { label: 'Check Out', icon: '↗', active: false },
  { label: 'Check In', icon: '↙', active: false },
  { label: 'Approvals', icon: '☑', active: false },
  { label: 'My Equipment', icon: '♙', active: false },
  { label: 'Damage Reports', icon: '⚒', active: false },
  { label: 'Reports', icon: '▤', active: false },
  { label: 'Team Members', icon: '♧', active: false }
];
```

```
  selectedMenu = 'Dashboard';
  showAllActivities = false;
  showProfileMenu = false;

  get visibleActivities() {
    return this.showAllActivities
      ? this.activities
      : this.activities.slice(0, 4);
  }

  selectMenu(label: string): void {
    this.selectedMenu = label;
    this.menuItems.forEach(item => {
      item.active = item.label === label;
    });
  }

  toggleActivities(): void {
    this.showAllActivities = !this.showAllActivities;
  }

  toggleProfile(): void {
    this.showProfileMenu = !this.showProfileMenu;
  }
}
```

24/10/2026, 13:13:39

You need to use **Angular Router** instead of only changing `selectedMenu`. When the user clicks **Equipment**, Angular should navigate to `/equipment` and load your `EquipmentList` component.

Follow these three steps.

Step 1: Update `dashboard.ts`

Add `Router` to your imports and inject it into your `Dashboard` class.

Change the top of your file to:

```
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
```

```
  imports: [],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
export class Dashboard {
  private router = inject(Router);

  // Keep all your existing arrays and properties here.
```

Then replace your existing `selectMenu()` method with:

```
selectMenu(label: string): void {
  this.selectedMenu = label;

  this.menuItems.forEach(item => {
    item.active = item.label === label;
  });

  if (label === 'Equipment') {
    this.router.navigate(['/equipment']);
  }
}
```

Keep all your other existing methods unchanged.

Step 2: Check `dashboard.html`

Find your sidebar navigation loop. It should call `selectMenu(item.label)` when a menu item is clicked:

```
@for (item of menuItems; track item.label) {
  <button
    type="button"
    class="nav-item"
    [class.active]="selectedMenu === item.label"
    (click)="selectMenu(item.label)"
  >
    <span class="nav-icon">{{ item.icon }}</span>
    <span>{{ item.label }}</span>
  </button>
}
```

If you already have this code, you don't need to change it.

Step 3: Check `app.routes.ts`

Make sure the Equipment route exists:

```ts
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { Dashboard } from './components/dashboard/dashboard';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },
  { path: 'dashboard', component: Dashboard },
  { path: 'equipment', component: EquipmentList },
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: '**', redirectTo: 'login' }
];
```

Use your actual component class names if they differ.

Now run:

```
ng serve --open
```

Open `/dashboard` and click **Equipment**. The URL should change to `/equipment`, and your Equipment List page should appear.

Note: This change makes the Equipment menu functional. Other menu items will still only update the selected menu until you assign routes to them.

24/10/2026, 13:13:38 · gpt-6

when i click on equipment then dashboard sidebar closes i want to open it as it is ??

24/10/2026, 13:15:42

The sidebar closes because your current **Equipment route loads only the `EquipmentList` component**, replacing the entire Dashboard component, including its sidebar.

The correct solution is to create a **shared layout** containing the sidebar and a `<router-outlet>`. Then Angular will change only the page content while keeping the sidebar visible.

Step 1: Create a layout component

Run this command:

```
ng generate component components/main-layout
```

Step 2: Move the sidebar into `main-layout.html`

Open `src/app/components/main-layout/main-layout.html` and use this structure:

```
<div class="dashboard-shell">

  <!-- Keep your existing sidebar here -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="navigateTo(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>
        </button>
      }
    </nav>
  </aside>
```

```
  <!-- Only the page content changes -->
  <main class="main-content">
    <router-outlet></router-outlet>
  </main>

</div>
```

Copy your existing sidebar CSS into `main-layout.css` as well.

Step 3: Add the layout logic

In `main-layout.ts`:

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-main-layout',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './main-layout.html',
  styleUrl: './main-layout.css'
})
export class MainLayout {
  private router = inject(Router);

  menuItems = [
    { label: 'Dashboard', icon: '⌂', path: '/dashboard' },
    { label: 'Equipment', icon: '▦', path: '/equipment' },
    { label: 'Check Out', icon: '↗', path: '/check-out' },
    { label: 'Check In', icon: '↙', path: '/check-in' },
    { label: 'Approvals', icon: '☑', path: '/approvals' },
    { label: 'My Equipment', icon: '♙', path: '/my-equipment' },
    { label: 'Damage Reports', icon: '⚒', path: '/damage-reports' },
    { label: 'Reports', icon: '▤', path: '/reports' },
    { label: 'Team Members', icon: '♧', path: '/team-members' }
  ];

  get selectedMenu(): string {
    const currentUrl = this.router.url.split('?')[0];
    return this.menuItems.find(item => item.path === currentUrl)?.label
      ?? 'Dashboard';
  }

  navigateTo(label: string): void {
    const item = this.menuItems.find(menu => menu.label === label);
```

```
    if (item) {
      this.router.navigate([item.path]);
    }
  }
}
```

Step 4: Update your routes

Replace the Dashboard and Equipment routes with child routes inside `MainLayout` in `app.routes.ts`:

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { MainLayout } from './components/main-layout/main-layout';
import { Dashboard } from './components/dashboard/dashboard';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },

  {
    path: '',
    component: MainLayout,
    children: [
      { path: 'dashboard', component: Dashboard },
      { path: 'equipment', component: EquipmentList },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },

  { path: '**', redirectTo: 'login' }
];
```

The other sidebar paths in `main-layout.ts` are placeholders until you create their components and routes.

Step 5: Simplify `dashboard.html`

Remove the sidebar and outer layout from `dashboard.html`. Keep only the Dashboard page content: heading, summary cards, equipment status, and recent activity.

Likewise, `equipment-list.html` should contain only the Equipment page content. Remove its own outer sidebar/layout wrapper so the shared layout controls the overall screen.

Result: Clicking Equipment changes the URL to `/equipment`, the equipment table appears, and the sidebar stays in place. Clicking Dashboard returns to `/dashboard` without recreating the

whole application layout.

24/10/2026, 13:15:41 · gpt-6

Application bundle generation failed. [0.110 seconds] - 2026-10-24T08:18:54.334Z

X [ERROR] NG8001: 'router-outlet' is not a known element:
1. If 'router-outlet' is an Angular component, then verify that it is included in the '@Component.imports' of this component.
2. If 'router-outlet' is a Web Component then add 'CUSTOM_ELEMENTS_SCHEMA' to the '@Component.schemas' of this component to suppress this message. [plugin angular-compiler]

src/app/components/main-layout/main-layout.html:27:4:
27 │ <router-outlet></router-outlet>
╵ ~~~~~~~~~~~~~~~~

Error occurs in the template of component MainLayout.

src/app/components/main-layout/main-layout.ts:7:15:
7 │ templateUrl: './main-layout.html',
╵ ~~~~~~~~~~~~~~~~~~~~~~

24/10/2026, 13:19:27

This error means Angular does not recognize `<router-outlet>` in your `MainLayout` component. You need to import `RouterOutlet` from `@angular/router`.

Fix: Update `main-layout.ts`

Open `src/app/components/main-layout/main-layout.ts` and make sure it looks like this:

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-main-layout',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './main-layout.html',
  styleUrl: './main-layout.css'
})
export class MainLayout {
  private router = inject(Router);
```

```
  menuItems = [
    { label: 'Dashboard', icon: '⌂', path: '/dashboard' },
    { label: 'Equipment', icon: '▦', path: '/equipment' },
    { label: 'Check Out', icon: '↗', path: '/check-out' },
    { label: 'Check In', icon: '↙', path: '/check-in' },
    { label: 'Approvals', icon: '☑', path: '/approvals' },
    { label: 'My Equipment', icon: '♙', path: '/my-equipment' },
    { label: 'Damage Reports', icon: '⚒', path: '/damage-reports' },
    { label: 'Reports', icon: '▤', path: '/reports' },
    { label: 'Team Members', icon: '♧', path: '/team-members' }
  ];

  get selectedMenu(): string {
    const currentUrl = this.router.url.split('?')[0];

    return this.menuItems.find(item => item.path === currentUrl)?.label
      ?? 'Dashboard';
  }

  navigateTo(label: string): void {
    const item = this.menuItems.find(menu => menu.label === label);

    if (item) {
      this.router.navigate([item.path]);
    }
  }
}
```

The important change is:

```
import { Router, RouterOutlet } from '@angular/router';

@Component({
  // ...
  imports: [RouterOutlet]
})
```

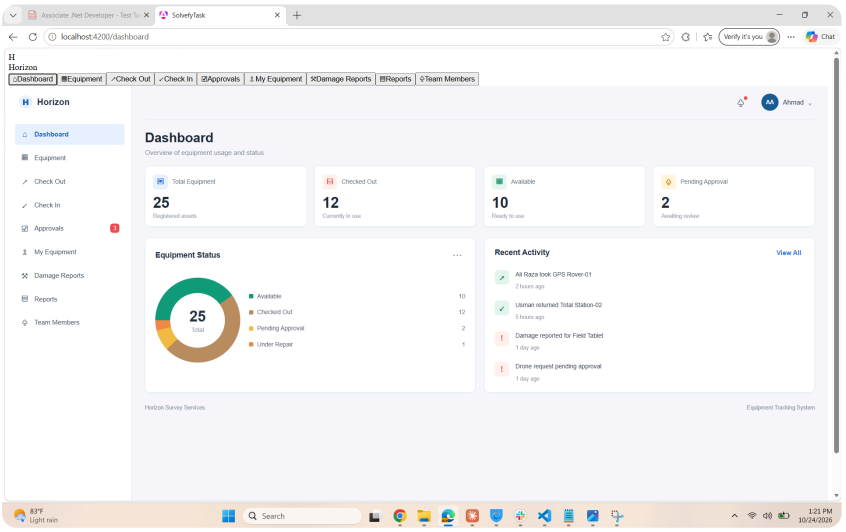
Then test it

1. Save `main-layout.ts` using `Ctrl + S`.
2. Angular should rebuild automatically.
3. Open `http://localhost:4200/dashboard`.
4. Click **Equipment**.

Your equipment page should load inside the shared layout, keeping the sidebar visible.

If another error appears, send me the exact terminal output. We'll fix it without changing unrelated code.

24/10/2026, 13:19:26 · gpt-6



i want this dashboard to use and not the upper dashboard ?

24/10/2026, 13:22:39

Your screenshot shows that **two dashboards are appearing at the same time**:

1. The upper part with plain buttons (Dashboard, Equipment, Check Out, etc.) is the extra sidebar we added in `main-layout.html`.
2. The lower part is your original, properly styled Dashboard, which still contains its own sidebar.

You should keep the **original styled dashboard** and remove the duplicate sidebar and layout.

Step 1: Update `main-layout.html`

Open `src/app/components/main-layout/main-layout.html` and replace its entire content with:

```
<div class="dashboard-shell">

  <!-- Shared sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
```

```
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="navigateTo(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>
        </button>
      }
    </nav>
  </aside>

  <!-- Dashboard or Equipment page loads here -->
  <main class="main-content">
    <router-outlet></router-outlet>
  </main>

</div>
```

Step 2: Remove the duplicate sidebar from `dashboard.html`

Open `src/app/components/dashboard/dashboard.html`.

Your Dashboard currently contains its own:

```
<div class="dashboard-shell">
  <aside class="sidebar">
```

Remove the outer layout and sidebar from this file. Keep only the dashboard's **topbar, page heading, summary cards, equipment status chart, recent activity, and footer**.

The dashboard template should contain only the content that belongs inside the main area.

Step 3: Remove the duplicate layout from `equipment-list.html`

Open `src/app/components/equipment-list/equipment-list.html`.

Remove its outer page wrapper and banner if those are duplicating the shared layout. Keep the Equipment heading, Add Equipment button, filters, table, and modal.

Step 4: Check `app.html`

It should contain only:

```
<router-outlet></router-outlet>
```

Step 5: Fix the styling

The shared sidebar and main content should use the CSS from your original dashboard. Move the relevant `.dashboard-shell`, `.sidebar`, `.navigation`, `.nav-item`, and `.main-content` rules into

`main-layout.css`.

Expected result: One styled sidebar on the left, with the Dashboard or Equipment page displayed to its right. Clicking Equipment changes only the main content; the sidebar stays in place.

24/10/2026, 13:22:40 · gpt-6

no i want to keep the dashboard which is in dashboard.html file and remove the upper dashboard and when i click on equipment or any other component the dashboard sidebar which is in file dashboard.html keep it as it is ??

24/10/2026, 13:26:04

Now I understand. You want to **keep the complete sidebar and dashboard layout already written in** `dashboard.html` and remove the extra sidebar created in `main-layout.html`. When you click Equipment or another menu item, the same sidebar should remain visible while only the page content changes.

The cleanest solution is to make your existing `Dashboard` component the shared layout.

Step 1: Update `dashboard.html`

Keep your existing sidebar, topbar, and styling exactly as they are. Find the main content area where you currently display the dashboard cards, chart, and recent activity.

Replace only that dashboard-specific content with:

```
<router-outlet></router-outlet>
```

Keep the sidebar and topbar outside the router outlet.

Your structure should look like this:

```
<div class="dashboard-shell">

  <!-- KEEP your existing sidebar -->
  <aside class="sidebar">
    <!-- Your existing Horizon logo and menu -->
  </aside>

  <main class="main-content">

    <!-- KEEP your existing topbar -->
    <header class="topbar">
      <!-- Your existing notifications and profile -->
    </header>

    <!-- Page content changes here -->
    <router-outlet></router-outlet>
```

```
  </main>
</div>
```

This is a structural example. Preserve your existing HTML and CSS instead of replacing the whole file with these placeholders.

Step 2: Update `dashboard.ts`

Import `RouterOutlet` and keep your existing menu logic.

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
export class Dashboard {
  private router = inject(Router);

  // Keep your existing properties and methods.
}
```

Your `selectMenu()` method should navigate to the selected route:

```
selectMenu(label: string): void {
  this.selectedMenu = label;

  this.menuItems.forEach(item => {
    item.active = item.label === label;
  });

  const routes: Record<string, string> = {
    Dashboard: '/dashboard',
    Equipment: '/equipment',
    'Check Out': '/check-out',
    'Check In': '/check-in',
    Approvals: '/approvals',
    'My Equipment': '/my-equipment',
    'Damage Reports': '/damage-reports',
    Reports: '/reports',
    'Team Members': '/team-members'
  };
```

```
  const path = routes[label];

  if (path) {
    this.router.navigate([path]);
  }
}
```

Step 3: Update `app.routes.ts`

Make `Dashboard` the parent layout for all your application pages:

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { Dashboard } from './components/dashboard/dashboard';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },

  {
    path: '',
    component: Dashboard,
    children: [
      { path: 'dashboard', component: DashboardHome },
      { path: 'equipment', component: EquipmentList },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },

  { path: '**', redirectTo: 'login' }
];
```

Important: `DashboardHome` here represents a separate component for your existing dashboard cards, chart, and activity content. You should create that component and move only that content into it. The outer `Dashboard` component retains the sidebar and topbar. This avoids nesting the Dashboard component inside itself.

Your `app.html` remains:

```
<router-outlet></router-outlet>
```

Step 4: Remove the extra layout

You no longer need the separate sidebar from `main-layout.html`. Your existing `dashboard.html` layout will be the single shared layout for the Dashboard and Equipment pages.

The result will be:

- **Dashboard:** your existing sidebar + dashboard cards, chart, and activity.
- **Equipment:** the exact same sidebar + equipment table.
- **Other pages:** the same sidebar + their respective page content.

One key detail: Angular cannot use the same `Dashboard` component as both the parent layout and its own child page. Separating the dashboard's inner content into a `DashboardHome` component is necessary to keep the layout stable.

24/10/2026, 13:26:04 · gpt-6

how to remove a component in angular ??

24/10/2026, 13:28:46

In Angular, you can remove a component by deleting its files and removing all references to it.

For your **SolvefyTask Angular project**, follow these steps.

1. Delete the component files

In VS Code Explorer, navigate to:

```
src/app/components/
```

Find the component folder you want to remove, for example:

```
main-layout/
├── main-layout.ts
├── main-layout.html
├── main-layout.css
└── main-layout.spec.ts
```

Right-click the `main-layout` folder and select **Delete**.

Alternatively, use PowerShell from your project root:

```
Remove-Item -Recurse -Force .\src\app\components\main-layout
```

Replace `main-layout` with the actual component folder name.

2. Remove references from `app.routes.ts`

If you are deleting `MainLayout`, remove its import:

```
import { MainLayout } from './components/main-layout/main-layout';
```

Also remove any route that uses it:

```
{
  path: '',
  component: MainLayout,
  children: [
    // ...
```

```
  ]
}
```

Important: If other routes depend on this layout, configure those routes to use your intended replacement before deleting it.

3. Remove references from other files

Press `Ctrl + Shift + F` in VS Code and search for:

- `MainLayout`
- `app-main-layout`
- `main-layout`

Remove references that belong to the deleted component, including imports, selectors, and route definitions.

4. Save and run Angular

```
ng serve
```

Angular should rebuild the application. If errors appear, check whether any imports or routes still reference the deleted component.

For your current project: If you want to remove the extra sidebar component we created earlier, delete `main-layout` only after moving any required routing and layout functionality elsewhere. Your original Dashboard sidebar will not be deleted when you remove the separate `main-layout` component.

24/10/2026, 13:28:45 · gpt-6

```
<div class="dashboard-shell">

  <!-- Sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="selectMenu(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>

          @if (item.label === 'Approvals') {
            <span class="nav-badge">3</span>
          }
        </button>
      }
    </nav>

    <div class="sidebar-footer">
      <span class="status-dot"></span>
      Equipment Tracking System
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- Header -->
    <header class="topbar">
      <div class="mobile-brand">
        <div class="brand-mark">H</div>
        <span>Horizon</span>
      </div>

      <div class="topbar-spacer"></div>
```

```
  <button
    class="notification-button"
    type="button"
    aria-label="Notifications"
    title="Notifications"
  >
    ♧
    <span class="notification-dot"></span>
  </button>

  <div class="profile-container">
    <button
      type="button"
      class="profile-button"
      (click)="toggleProfile()"
    >
      <span class="avatar">AA</span>
      <span>Ahmad</span>
      <span class="chevron">⌄</span>
    </button>

    @if (showProfileMenu) {
      <div class="profile-menu">
        <div class="profile-menu-name">Ahmad</div>
        <button type="button" (click)="toggleProfile()">
          Close menu
        </button>
      </div>
    }
  </div>
</header>

<!-- Page Heading -->
<section class="page-heading">
  <div>
    <h1>{{ selectedMenu }}</h1>
    <p>Overview of equipment usage and status</p>
  </div>
</section>

<!-- Summary Cards -->
<section class="stats-grid">
  @for (stat of equipmentStats; track stat.title) {
    <article class="stat-card">
      <div class="stat-top">
```

```
          <span class="stat-icon" [class]=" 'stat-icon ' +
stat.color">
            {{ stat.icon }}
          </span>
          <span class="stat-title">{{ stat.title }}</span>
        </div>

        <div class="stat-value">{{ stat.value }}</div>
        <div class="stat-detail">{{ stat.detail }}</div>
      </article>
    }
  </section>

  <!-- Dashboard Panels -->
  <section class="dashboard-grid">

    <!-- Equipment Status -->
    <article class="panel status-panel">
      <div class="panel-heading">
        <h2>Equipment Status</h2>
        <button type="button" class="more-button"
title="Equipment status">
          …
        </button>
      </div>

      <div class="status-content">
        <div class="donut-chart">
          <div class="donut-center">
            <strong>25</strong>
            <span>Total</span>
          </div>
        </div>

        <div class="legend">
          @for (status of equipmentStatus; track status.label) {
            <div class="legend-row">
              <span
                class="legend-dot"
                [style.background]="status.color"
              ></span>

              <span class="legend-label">{{ status.label }}
</span>
              <span class="legend-value">{{ status.value }}
</span>
```

```
          </div>
        }
      </div>
    </div>
  </article>

  <!-- Recent Activity -->
  <article class="panel activity-panel">
    <div class="panel-heading">
      <h2>Recent Activity</h2>
      <button
        type="button"
        class="view-all"
        (click)="toggleActivities()"
      >
        {{ showAllActivities ? 'Show less' : 'View All' }}
      </button>
    </div>

    <div class="activity-list">
      @for (activity of visibleActivities; track activity.title) {
        <div class="activity-item">
          <div
            class="activity-icon"
            [class.success]="activity.type === 'success'"
            [class.danger]="activity.type === 'danger'"
          >
            {{ activity.icon }}
          </div>

          <div class="activity-info">
            <p>{{ activity.title }}</p>
            <span>{{ activity.time }}</span>
          </div>
        </div>
      }
    </div>
  </article>

</section>

<footer class="dashboard-footer">
  Horizon Survey Services
  <span>Equipment Tracking System</span>
</footer>
```

</main>

i want the sidebar to remian as it is and when i click on the component for example equipment the sidebar remain as it is ??

24/10/2026, 13:32:11

You can achieve this by keeping your existing `dashboard.html` sidebar and topbar as a **shared layout**. When you click Equipment, Angular will replace only the page content inside `<router-outlet>`. Your sidebar will remain unchanged.

You do **not** need to redesign your sidebar or CSS.

1. Update `dashboard.html`

In your existing HTML, keep the sidebar and topbar exactly as they are. Replace the dashboard-specific content — from `<!-- Page Heading -->` through the dashboard footer — with a router outlet.

Your structure should look like this:

```
<div class="dashboard-shell">

  <!-- KEEP YOUR EXISTING SIDEBAR UNCHANGED -->
  <aside class="sidebar">
    <!-- Keep your existing Horizon logo, navigation,
         menu buttons and sidebar footer here -->
  </aside>

  <main class="main-content">

    <!-- KEEP YOUR EXISTING TOPBAR UNCHANGED -->
    <header class="topbar">
      <!-- Keep your existing notifications and profile here -->
    </header>

    <!-- Only this area changes between pages -->
    <router-outlet></router-outlet>

  </main>
</div>
```

Important: This is a structural example. Keep the actual sidebar and topbar markup you shared; do not replace them with the comments above.

2. Update `dashboard.ts`

Import `RouterOutlet` so Angular recognizes it:

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css'
})
export class Dashboard {
  private router = inject(Router);

  // Keep your existing menuItems, equipmentStats,
  // equipmentStatus, activities and profile properties.

  selectMenu(label: string): void {
    const routes: Record<string, string> = {
      Dashboard: '/dashboard',
      Equipment: '/equipment',
      'Check Out': '/check-out',
      'Check In': '/check-in',
      Approvals: '/approvals',
      'My Equipment': '/my-equipment',
      'Damage Reports': '/damage-reports',
      Reports: '/reports',
      'Team Members': '/team-members'
    };

    const path = routes[label];

    if (path) {
      this.router.navigate([path]);
    }
  }
}
```

Keep your existing `toggleProfile()` and `toggleActivities()` methods as well. Your dashboard data and sidebar menu can remain in this class.

3. Create a separate component for dashboard content

Because `Dashboard` now owns the shared sidebar, its dashboard cards and chart need to be moved into a separate component, such as `DashboardHome`.

Run:

```
ng generate component components/dashboard-home
```

Move the following sections from your current `dashboard.html` into `dashboard-home.html`:

- Page Heading
- Summary Cards
- Equipment Status
- Recent Activity
- Dashboard Footer

Move the related data arrays and methods (`equipmentStats`, `equipmentStatus`, `activities`, and `toggleActivities()`) into `DashboardHome`.

This prevents the Dashboard component from trying to load itself inside its own router outlet.

4. Configure `app.routes.ts`

Use your existing Login component and configure Dashboard as the parent layout:

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { Dashboard } from './components/dashboard/dashboard';
import { DashboardHome } from './components/dashboard-home/dashboard-home';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },

  {
    path: '',
    component: Dashboard,
    children: [
      { path: 'dashboard', component: DashboardHome },
      { path: 'equipment', component: EquipmentList },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },

  { path: '**', redirectTo: 'login' }
];
```

Use the actual exported class names and file paths generated in your project.

Your `app.html` should contain only:

```
<router-outlet></router-outlet>
```

And `app.ts` must import `RouterOutlet` and register it in the root component's `imports`.

Result

Permanent sidebar

Your existing Horizon sidebar and topbar stay in the Dashboard layout.

Click Dashboard

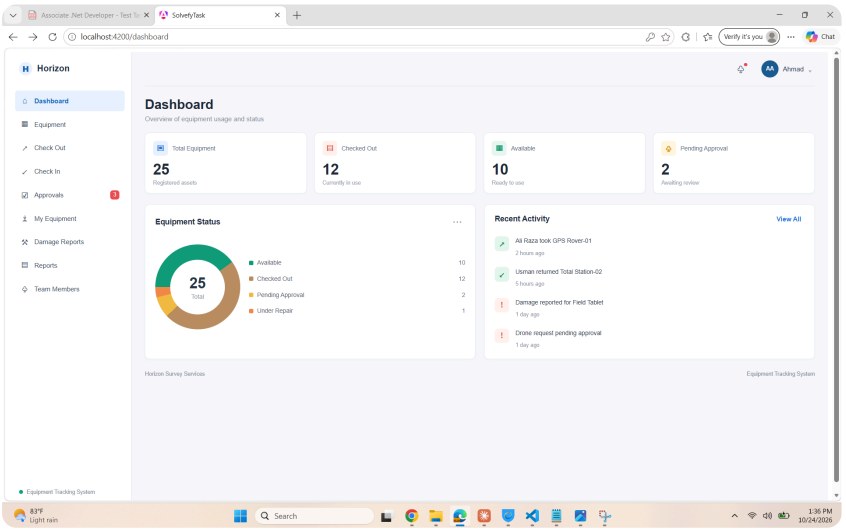
The dashboard cards, chart, and recent activity appear on the right.

Click Equipment

The equipment table replaces the dashboard content; the same sidebar remains visible.

The key is that **Dashboard becomes the parent layout, not the dashboard content page**. This lets Angular switch between child pages without removing your sidebar.

24/10/2026, 13:32:10 · gpt-6



i want this dashboard sidebar to be fixed

24/10/2026, 13:38:12

To keep your Horizon sidebar **fixed on the left while scrolling or switching between Dashboard and Equipment**, you only need to update the CSS. You don't need to change your Angular routing or TypeScript.

1. Update `dashboard.css`

Find your existing `.sidebar` CSS and replace it with:

```
.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  bottom: 0;

  width: 225px;
  height: 100vh;

  display: flex;
  flex-direction: column;

  background: #fff;
  border-right: 1px solid #e5eaf1;

  padding: 18px 12px;
  overflow-y: auto;
```

```
  z-index: 1000;
}
```

2. Adjust the main content

Add or update `.main-content` in the same CSS file:

```
.main-content {
  margin-left: 225px;
  width: calc(100% - 225px);
  min-width: 0;
  min-height: 100vh;
  padding: 0 24px 18px;
}
```

The `margin-left: 225px` ensures that your dashboard content starts to the right of the fixed sidebar instead of appearing underneath it.

3. Keep the sidebar footer at the bottom

Your existing sidebar footer can remain at the bottom with:

```
.sidebar-footer {
  margin-top: auto;
}
```

4. Make it responsive

If you already have a mobile media query, update it to:

```
@media (max-width: 650px) {
  .sidebar {
    width: 180px;
  }

  .main-content {
    margin-left: 180px;
    width: calc(100% - 180px);
    padding: 0 12px 16px;
  }
}
```

If you want the sidebar hidden on mobile instead, tell me and I'll adjust the responsive behavior.

Result: Your sidebar stays fixed in the same position, the main content scrolls independently as needed, and your existing Dashboard design remains unchanged.

24/10/2026, 13:38:13 · gpt-6

```
:host {
  display: block;
  min-height: 100vh;
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 13px;
}

\* {
  box-sizing: border-box;
}

button {
  font: inherit;
}

.dashboard-shell {
  display: flex;
  min-height: 100vh;
  background: #f6f8fb;
}

/\* Sidebar \*/

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  bottom: 0;

  width: 225px;
  height: 100vh;

  display: flex;
  flex-direction: column;

  background: #fff;
  border-right: 1px solid #e5eaf1;

  padding: 18px 12px;
  overflow-y: auto;

  z-index: 1000;
}
```

```
.brand {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 0 7px 28px;
  color: #27364b;
  font-size: 16px;
  font-weight: 700;
}

.brand-mark {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 24px;
  height: 24px;
  border-radius: 50%;
  background: #e8f2ff;
  color: #1766b2;
  font-size: 14px;
  font-weight: 800;
}

.navigation {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  width: 100%;
  min-height: 38px;
  padding: 9px 10px;
  border: 0;
  border-radius: 5px;
  background: transparent;
  color: #435064;
  text-align: left;
  font-size: 12px;
  cursor: pointer;
  transition: *background* 0.2s, color 0.2s;
}
```

```
.nav-item:hover {
  background: #f0f5fc;
}

.nav-item.active {
  background: #eaf2ff;
  color: #1769c2;
  font-weight: 700;
}

.nav-icon {
  width: 15px;
  flex-shrink: 0;
  font-size: 15px;
  text-align: center;
}

.nav-badge {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 17px;
  height: 17px;
  margin-left: auto;
  border-radius: 5px;
  background: #ed4d54;
  color: white;
  font-size: 10px;
}

.sidebar-footer {
  display: flex;
  align-items: center;
  gap: 7px;
  margin-top: auto;
  padding: 18px 7px 0;
  color: #8792a3;
  font-size: 10px;
}

.status-dot,
.notification-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
```

```
  background: #13a879;
}

/\* Main area \*/

.main-content {
  margin-left: 225px;
  width: calc(100% - 225px);
  min-width: 0;
  min-height: 100vh;
  padding: 0 24px 18px;
}

.topbar {
  position: relative;
  display: flex;
  align-items: center;
  gap: 15px;
  min-height: 62px;
  border-bottom: 1px solid #e9edf3;
}

.topbar-spacer {
  flex: 1;
}

.mobile-brand {
  display: none;
  align-items: center;
  gap: 7px;
  font-weight: 700;
}

.notification-button {
  position: relative;
  width: 34px;
  height: 34px;
  border: 0;
  background: transparent;
  color: #59677b;
  font-size: 19px;
  cursor: pointer;
}

.notification-dot {
  position: absolute;
```

```
  top: 6px;
  right: 5px;
  width: 6px;
  height: 6px;
  background: #e84b54;
}

.profile-container {
  position: relative;
}

.profile-button {
  display: flex;
  align-items: center;
  gap: 8px;
  border: 0;
  background: transparent;
  color: #344155;
  font-size: 12px;
  cursor: pointer;
}

.avatar {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 31px;
  height: 31px;
  border-radius: 50%;
  background: #1d5b94;
  color: white;
  font-size: 10px;
  font-weight: 700;
}

.chevron {
  color: #7b8798;
}

.profile-menu {
  position: absolute;
  z-index: 5;
  top: 42px;
  right: 0;
  min-width: 145px;
  padding: 12px;
```

```
  border: 1px solid #e5eaf1;
  border-radius: 7px;
  background: white;
  box-shadow: 0 6px 20px #1b35501a;
}

.profile-menu-name {
  margin-bottom: 10px;
  font-weight: 700;
}

.profile-menu button {
  border: 0;
  background: transparent;
  color: #1769c2;
  cursor: pointer;
}

.page-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 0 18px;
}

.page-heading h1 {
  margin: 0 0 6px;
  color: #202a39;
  font-size: 24px;
  font-weight: 700;
}

.page-heading p {
  margin: 0;
  color: #8994a4;
  font-size: 12px;
}

/\* Summary cards \*/

.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 20px;
}
```

```
.stat-card {
  min-width: 0;
  padding: 15px 14px 13px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.stat-top {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 10px;
}

.stat-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 6px;
  font-size: 15px;
  font-weight: 700;
}

.stat-icon.blue {
  background: #eaf2ff;
  color: #2675dd;
}

.stat-icon.red {
  background: #fff0ee;
  color: #e86661;
}

.stat-icon.green {
  background: #e6f8f1;
  color: #169c76;
}

.stat-icon.yellow {
  background: #fff6df;
```

```
  color: #d99c18;
}

.stat-title {
  color: #68758a;
  font-size: 11px;
  line-height: 1.4;
}

.stat-value {
  margin-bottom: 4px;
  color: #202a39;
  font-size: 27px;
  font-weight: 700;
  line-height: 1.1;
}

.stat-detail {
  color: #a0a9b7;
  font-size: 10px;
}

/\* Panels \*/

.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  align-items: stretch;
  gap: 16px;
}

.panel {
  min-width: 0;
  padding: 18px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.panel-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 23px;
```

```
}

.panel-heading h2 {
  margin: 0;
  color: #283447;
  font-size: 14px;
  font-weight: 700;
}

.more-button {
  border: 0;
  background: transparent;
  color: #8591a2;
  font-size: 20px;
  cursor: pointer;
}

.status-content {
  display: flex;
  align-items: center;
  justify-content: space-around;
  gap: 16px;
  padding: 5px 0 14px;
}

.donut-chart {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 155px;
  height: 155px;
  flex-shrink: 0;
  border-radius: 50%;
  background: conic-gradient(
    #119c78 0% 40%,
    #b98d61 40% 88%,
    #f4bd45 88% 96%,
    #f28b47 96% 100%
  );
  transform: rotate(-90deg);
}

.donut-center {
  display: flex;
  flex-direction: column;
  align-items: center;
```

```
  justify-content: center;
  width: 100px;
  height: 100px;
  border-radius: 50%;
  background: white;
  transform: rotate(90deg);
}

.donut-center strong {
  color: #263346;
  font-size: 27px;
  line-height: 1.2;
}

.donut-center span {
  margin-top: 3px;
  color: #929cac;
  font-size: 11px;
}

.legend {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 14px;
  min-width: 0;
}

.legend-row {
  display: grid;
  grid-template-columns: 8px minmax(0, 1fr) auto;
  align-items: center;
  gap: 7px;
  color: #5f6b7c;
  font-size: 11px;
}

.legend-dot {
  width: 8px;
  height: 8px;
  border-radius: 2px;
}

.legend-label {
  line-height: 1.4;
}
```

```
.legend-value {
  color: #748093;
}

/\* Recent activity \*/

.view-all {
  border: 0;
  background: transparent;
  color: #2675dd;
  font-size: 12px;
  font-weight: 600;
  white-space: nowrap;
  cursor: pointer;
}

.activity-list {
  display: flex;
  flex-direction: column;
}

.activity-item {
  display: flex;
  align-items: flex-start;
  gap: 11px;
  padding: 0 0 19px;
}

.activity-item:last-child {
  padding-bottom: 0;
}

.activity-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 7px;
  font-size: 14px;
  font-weight: 700;
}

.activity-icon.success {
```

```
  background: #e4f7ed;
  color: #139968;
}

.activity-icon.danger {
  background: #fff0ef;
  color: #e45454;
}

.activity-info {
  min-width: 0;
  padding-top: 1px;
}

.activity-info p {
  margin: 0 0 5px;
  color: #3b4657;
  font-size: 11px;
  line-height: 1.45;
  overflow-wrap: anywhere;
}

.activity-info span {
  color: #9aa4b2;
  font-size: 10px;
}

.dashboard-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  padding-top: 22px;
  color: #8a95a4;
  font-size: 10px;
}

.dashboard-footer span {
  text-align: right;
}

/\* Responsive layout \*/

@media (max-width: 1100px) {
  .sidebar {
    width: 190px;
  }
```

```
  .main-content {
    padding-right: 16px;
    padding-left: 16px;
  }

  .stats-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 650px) {
  .sidebar {
    display: none;
  }

  .main-content {
    padding: 0 13px 16px;
  }

  .topbar {
    min-height: 56px;
  }

  .mobile-brand {
    display: flex;
  }

  .page-heading {
    padding: 18px 0;
  }

  .page-heading h1 {
    font-size: 22px;
  }

  .stats-grid {
    gap: 10px;
  }

  .stat-card {
    padding: 12px 10px;
```

```
  }

  .stat-title {
    font-size: 10px;
  }

  .stat-value {
    font-size: 24px;
  }

  .panel {
    padding: 14px;
  }

  .status-content {
    flex-wrap: wrap;
  }

  .donut-chart {
    width: 135px;
    height: 135px;
  }

  .donut-center {
    width: 88px;
    height: 88px;
  }

  .legend {
    min-width: 135px;
  }
}



\
\<div class="dashboard-shell">\
\
&#x20; \<!-- Sidebar -->\
&#x20; \<aside class="sidebar">\
&#x20;   \<div class="brand">\
&#x20;     \<div class="brand-mark">H\</div>\
&#x20;     \<span>Horizon\</span>\
&#x20;   \</div>\
\
&#x20;   \<nav class="navigation">\
&#x20;     @for (item of menuItems; track item.label) {\
```

```
&#x20;      \<button\
&#x20;        type="button"\
&#x20;        class="nav-item"\
&#x20;        [class.active]="selectedMenu === item.label"\
&#x20;        (click)="selectMenu(item.label)"\
&#x20;      \>\
&#x20;        \<span class="nav-icon">{{ item.icon }}\
</span>\
&#x20;        \<span>{{ item.label }}\</span>\
\
&#x20;        @if (item.label === 'Approvals') {\
&#x20;          \<span class="nav-badge">3\</span>\
&#x20;        }\
&#x20;      \</button>\
&#x20;    }\
&#x20;  \</nav>\
\
&#x20;  \<div class="sidebar-footer">\
&#x20;    \<span class="status-dot">\</span>\
&#x20;    Equipment Tracking System\
&#x20;  \</div>\
&#x20; \</aside>\
\
&#x20; \<!-- Main Content -->\
&#x20; \<main class="main-content">\
\
&#x20;  \<!-- Header -->\
&#x20;  \<header class="topbar">\
&#x20;    \<div class="mobile-brand">\
&#x20;      \<div class="brand-mark">H\</div>\
&#x20;      \<span>Horizon\</span>\
&#x20;    \</div>\
\
&#x20;    \<div class="topbar-spacer">\</div>\
\
&#x20;    \<button\
&#x20;      class="notification-button"\
&#x20;      type="button"\
&#x20;      aria-label="Notifications"\
&#x20;      title="Notifications"\
&#x20;    \>\
&#x20;      ♧\
&#x20;      \<span class="notification-dot">\</span>\
&#x20;    \</button>\
\
&#x20;    \<div class="profile-container">\
```

```
&#x20;      \<button\
&#x20;        type="button"\
&#x20;        class="profile-button"\
&#x20;        (click)="toggleProfile()"\
&#x20;      \>\
&#x20;        \<span class="avatar">AA\</span>\
&#x20;        \<span>Ahmad\</span>\
&#x20;        \<span class="chevron">⌄\</span>\
&#x20;      \</button>\
\
&#x20;      @if (showProfileMenu) {\
&#x20;        \<div class="profile-menu">\
&#x20;          \<div class="profile-menu-name">Ahmad\
</div>\
&#x20;          \<button type="button"
(click)="toggleProfile()">\
&#x20;            Close menu\
&#x20;          \</button>\
&#x20;        \</div>\
&#x20;      }\
&#x20;    \</div>\
&#x20;  \</header>\
\
&#x20;  \<!-- Page Heading -->\
&#x20;  \<section class="page-heading">\
&#x20;    \<div>\
&#x20;      \<h1>{{ selectedMenu }}\</h1>\
&#x20;      \<p>Overview of equipment usage and status\
</p>\
&#x20;    \</div>\
&#x20;  \</section>\
\
&#x20;  \<!-- Summary Cards -->\
&#x20;  \<section class="stats-grid">\
&#x20;    @for (stat of equipmentStats; track stat.title) {\
&#x20;      \<article class="stat-card">\
&#x20;        \<div class="stat-top">\
&#x20;          \<span class="stat-icon" [class]=" 'stat-icon ' +
stat.color">\
&#x20;            {{ stat.icon }}\
&#x20;          \</span>\
&#x20;          \<span class="stat-title">{{ stat.title }}\
</span>\
&#x20;        \</div>\
\
&#x20;        \<div class="stat-value">{{ stat.value }}\
```

```
</div>\
&#x20;       \<div class="stat-detail">{{ stat.detail }}\
</div>\
&#x20;     \</article>\
&#x20;   }\
&#x20;  \</section>\
\
&#x20;  \<!-- Dashboard Panels -->\
&#x20;  \<section class="dashboard-grid">\
\
&#x20;    \<!-- Equipment Status -->\
&#x20;    \<article class="panel status-panel">\
&#x20;      \<div class="panel-heading">\
&#x20;        \<h2>Equipment Status\</h2>\
&#x20;        \<button type="button" class="more-button"
title="Equipment status">\
&#x20;          ⋯\
&#x20;        \</button>\
&#x20;      \</div>\
\
&#x20;      \<div class="status-content">\
&#x20;        \<div class="donut-chart">\
&#x20;          \<div class="donut-center">\
&#x20;            \<strong>25\</strong>\
&#x20;            \<span>Total\</span>\
&#x20;          \</div>\
&#x20;        \</div>\
\
&#x20;        \<div class="legend">\
&#x20;          @for (status of equipmentStatus; track
status.label) {\
&#x20;            \<div class="legend-row">\
&#x20;              \<span\
&#x20;                class="legend-dot"\
&#x20;                [style.background]="status.color"\
&#x20;              \>\</span>\
\
&#x20;              \<span class="legend-label">{{ status.label
}}\</span>\
&#x20;              \<span class="legend-value">{{ status.value
}}\</span>\
&#x20;            \</div>\
&#x20;          }\
&#x20;        \</div>\
&#x20;      \</div>\
&#x20;    \</article>\
```

```
\
&#x20;    \<!-- Recent Activity -->\
&#x20;    \<article class="panel activity-panel">\
&#x20;      \<div class="panel-heading">\
&#x20;        \<h2>Recent Activity\</h2>\
&#x20;        \<button\
&#x20;          type="button"\
&#x20;          class="view-all"\
&#x20;          (click)="toggleActivities()"\
&#x20;        \>\
&#x20;          {{ showAllActivities ? 'Show less' : 'View All' }}\
&#x20;        \</button>\
&#x20;      \</div>\
\
&#x20;      \<div class="activity-list">\
&#x20;        @for (activity of visibleActivities; track
activity.title) {\
&#x20;          \<div class="activity-item">\
&#x20;            \<div\
&#x20;              class="activity-icon"\
&#x20;              [class.success]="activity.type ===
'success'"\
&#x20;              [class.danger]="activity.type === 'danger'"\
&#x20;            \>\
&#x20;              {{ activity.icon }}\
&#x20;            \</div>\
\
&#x20;            \<div class="activity-info">\
&#x20;              \<p>{{ activity.title }}\</p>\
&#x20;              \<span>{{ activity.time }}\</span>\
&#x20;            \</div>\
&#x20;          \</div>\
&#x20;        }\
&#x20;      \</div>\
&#x20;    \</article>\
\
&#x20;  \</section>\
\
&#x20;  \<footer class="dashboard-footer">\
&#x20;    Horizon Survey Services\
&#x20;    \<span>Equipment Tracking System\</span>\
&#x20;  \</footer>\
\
&#x20; \</main>\
\</div>\
```

```
import { Component, inject } from '@angular/core';\
import { Router } from '@angular/router';\
\
@Component({\
&#x20; selector: 'app-dashboard',\
&#x20; imports: [],\
&#x20; templateUrl: './dashboard.html',\
&#x20; styleUrl: './dashboard.css',\
})\
export class Dashboard {\
private router = inject(Router);\
\
&#x20; equipmentStats = [\
&#x20;   {\
&#x20;     title: 'Total Equipment',\
&#x20;     value: 25,\
&#x20;     icon: '▣',\
&#x20;     color: 'blue',\
&#x20;     detail: 'Registered assets'\
&#x20;   },\
&#x20;   {\
&#x20;     title: 'Checked Out',\
&#x20;     value: 12,\
&#x20;     icon: '▤',\
&#x20;     color: 'red',\
&#x20;     detail: 'Currently in use'\
&#x20;   },\
&#x20;   {\
&#x20;     title: 'Available',\
&#x20;     value: 10,\
&#x20;     icon: '▦',\
&#x20;     color: 'green',\
&#x20;     detail: 'Ready to use'\
&#x20;   },\
&#x20;   {\
&#x20;     title: 'Pending Approval',\
&#x20;     value: 2,\
&#x20;     icon: '♧',\
&#x20;     color: 'yellow',\
&#x20;     detail: 'Awaiting review'\
&#x20;   }\
&#x20; ];\
\
&#x20; equipmentStatus = [\
&#x20;   { label: 'Available', value: 10, color: '#119c78' },\
&#x20;   { label: 'Checked Out', value: 12, color: '#b98d61' },\
```

```
&#x20;   { label: 'Pending Approval', value: 2, color: '#f4bd45'
},\
&#x20;   { label: 'Under Repair', value: 1, color: '#f28b47' }\
&#x20; ];\
\
&#x20; activities = [\
&#x20;   {\
&#x20;     type: 'success',\
&#x20;     title: 'Ali Raza took GPS Rover-01',\
&#x20;     time: '2 hours ago',\
&#x20;     icon: '↗'\
&#x20;   },\
&#x20;   {\
&#x20;     type: 'success',\
&#x20;     title: 'Usman returned Total Station-02',\
&#x20;     time: '5 hours ago',\
&#x20;     icon: '↙'\
&#x20;   },\
&#x20;   {\
&#x20;     type: 'danger',\
&#x20;     title: 'Damage reported for Field Tablet',\
&#x20;     time: '1 day ago',\
&#x20;     icon: '!'\
&#x20;   },\
&#x20;   {\
&#x20;     type: 'danger',\
&#x20;     title: 'Drone request pending approval',\
&#x20;     time: '1 day ago',\
&#x20;     icon: '!'\
&#x20;   }\
&#x20; ];\
\
&#x20; menuItems = [\
&#x20;   { label: 'Dashboard', icon: '⌂', active: true },\
&#x20;   { label: 'Equipment', icon: '▦', active: false },\
&#x20;   { label: 'Check Out', icon: '↗', active: false },\
&#x20;   { label: 'Check In', icon: '↙', active: false },\
&#x20;   { label: 'Approvals', icon: '☑', active: false },\
&#x20;   { label: 'My Equipment', icon: '♙', active: false },\
&#x20;   { label: 'Damage Reports', icon: '⚒', active: false },\
&#x20;   { label: 'Reports', icon: '▤', active: false },\
&#x20;   { label: 'Team Members', icon: '♧', active: false }\
&#x20; ];\
\
&#x20; selectedMenu = 'Dashboard';\
&#x20; showAllActivities = false;\
```

  showProfileMenu = false;\
\
  get visibleActivities() {\
   return this.showAllActivities\
   ? this.activities\
   : this.activities.slice(0, 4);\
  }\
\
  selectMenu(label: string): void {\
  this.selectedMenu = label;\
\
  this.menuItems.forEach(item => {\
   item.active = item.label === label;\
  });\
\
  if (label === 'Equipment') {\
   this.router.navigate(['/equipment']);\
  }\
}\
\
  toggleActivities(): void {\
   this.showAllActivities = !this.showAllActivities;\
  }\
\
  toggleProfile(): void {\
   this.showProfileMenu = !this.showProfileMenu;\
  }\
}\
i want this dashboard sidebar to be fixed when i click on equipment sidebar remains same ??

24/10/2026, 13:41:44

Your current sidebar HTML is fine. The problem is that when you click **Equipment**, Angular navigates to another component, which replaces the entire `Dashboard` component. To keep your existing sidebar exactly as it is, make `Dashboard` the parent layout and load Equipment inside its `<router-outlet>`.

Follow these changes in your existing code.

1. Update `dashboard.html`

Keep your existing sidebar and topbar HTML unchanged.

Find this section:

```
<!-- Page Heading -->
<section class="page-heading">
```

Replace everything from `<!-- Page Heading -->` through the dashboard footer with:

```
<!-- Child page content appears here -->
<router-outlet></router-outlet>
```

Your sidebar, topbar, and outer `.dashboard-shell` must remain in `dashboard.html`.

2. Update `dashboard.ts`

Import `RouterOutlet` and keep your existing dashboard properties and methods.

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
export class Dashboard {
  private router = inject(Router);

  // Keep your existing menuItems and other properties here.

  selectMenu(label: string): void {
    const routes: Record<string, string> = {
      Dashboard: '/dashboard',
      Equipment: '/equipment',
      'Check Out': '/check-out',
      'Check In': '/check-in',
      Approvals: '/approvals',
      'My Equipment': '/my-equipment',
      'Damage Reports': '/damage-reports',
      Reports: '/reports',
      'Team Members': '/team-members'
    };

    const path = routes[label];

    if (path) {
      this.router.navigate([path]);
```

```
    }
  }

  // Keep your existing toggleActivities() and toggleProfile() methods.
}
```

Keep your existing arrays (`equipmentStats`, `equipmentStatus`, and `activities`) if you still need them. Move the dashboard-specific content and its related data into a separate `DashboardHome` component, as explained below.

3. Create a separate Dashboard Home component

Your Dashboard component cannot load itself as its own child page. Create a component for the dashboard cards, chart, and recent activity:

```
ng generate component components/dashboard-home
```

Move the dashboard-specific HTML sections you removed in Step 1 into `dashboard-home.html`, and move the corresponding data arrays and methods into `dashboard-home.ts`.

4. Update `app.routes.ts`

Configure `Dashboard` as the parent of the Dashboard Home and Equipment pages:

```
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { Dashboard } from './components/dashboard/dashboard';
import { DashboardHome } from './components/dashboard-home/dashboard-home';
import { EquipmentList } from './components/equipment-list/equipment-list';

export const routes: Routes = [
  { path: 'login', component: Login },
  {
    path: '',
    component: Dashboard,
    children: [
      { path: 'dashboard', component: DashboardHome },
      { path: 'equipment', component: EquipmentList },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },
  { path: '**', redirectTo: 'login' }
];
```

Keep `app.html` as:

```
<router-outlet></router-outlet>
```

Ensure `app.ts` imports `RouterOutlet`.

5. Fix your sidebar CSS

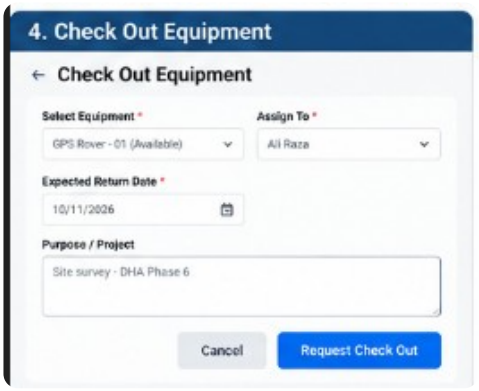
Your existing fixed sidebar CSS can stay in `dashboard.css`. Make sure the main content leaves space for it:

```
.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  bottom: 0;
  width: 225px;
  height: 100vh;
  overflow-y: auto;
  z-index: 1000;
}

.main-content {
  margin-left: 225px;
  width: calc(100% - 225px);
  min-width: 0;
  min-height: 100vh;
}
```

Expected result: When you click Equipment, the URL changes to `/equipment`, the Equipment List opens on the right, and your original Horizon sidebar remains fixed on the left. Clicking Dashboard returns to your dashboard cards without recreating the sidebar.

24/10/2026, 13:41:44 · gpt-6



please generate this image into angular code ??

24/10/2026, 13:58:58

I'll build this **Check Out Equipment** page for your existing Angular SolvefyTask project, matching the screenshot and keeping your fixed Horizon sidebar intact.

The page will include:

- A **Check Out Equipment** heading with a back arrow.
- A required equipment dropdown with availability.
- An **Assign To** dropdown.
- A required expected return date.
- A purpose/project text area.
- **Cancel** and **Request Check Out** buttons.

We'll use a standalone Angular component, with form validation and sample data to start.

1. Generate the component

Run this command from your Angular project folder:

```
ng generate component components/check-out
```

2. TypeScript — `check-out.ts`

Replace the contents of `src/app/components/check-out/check-out.ts` with:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
```

```ts
@Component({
  selector: 'app-check-out',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './check-out.html',
  styleUrl: './check-out.css'
})
export class CheckOut {
  constructor(private router: Router) {}

  selectedEquipment = 'GPS Rover - 01';
  assignedTo = 'Ali Raza';
  expectedReturnDate = '2026-11-10';
  purpose = '';

  submitted = false;

  equipmentOptions = [
    { name: 'GPS Rover - 01', availability: 'Available' },
    { name: 'Total Station - 01', availability: 'Available' },
    { name: 'Drone - DJI Mavic', availability: 'Available' },
    { name: 'Field Tablet - 01', availability: 'Available' }
  ];

  teamMembers = [
    'Ali Raza',
    'Ahmad',
    'Usman',
    'Muhammad Ahmed'
  ];

  requestCheckOut(): void {
    this.submitted = true;

    if (
      !this.selectedEquipment ||
      !this.assignedTo ||
      !this.expectedReturnDate
    ) {
      return;
    }

    // Demo only: connect your checkout API here.
    alert(
      `Check-out request submitted for ${this.selectedEquipment}.`
    );
```

```
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

The dropdowns currently use sample data. Later, you can load equipment and team members from your backend API.

3. HTML — `check-out.html`

Replace the contents of `src/app/components/check-out/check-out.html` with:

```
<div class="checkout-page">
  <div class="page-banner">4. Check Out Equipment</div>

  <main class="checkout-content">
    <div class="page-title">
      <button
        type="button"
        class="back-button"
        aria-label="Go back"
        (click)="goBack()"
      >
        ←
      </button>

      <h1>Check Out Equipment</h1>
    </div>

    <form
      #checkoutForm="ngForm"
      class="checkout-form"
      (ngSubmit)="requestCheckOut()"
    >
      <div class="form-grid">
        <!-- Select Equipment -->
        <div class="form-field">
          <label for="equipment">
            Select Equipment <span class="required">*</span>
          </label>
```

```
      <select
        id="equipment"
        name="equipment"
        [(ngModel)]="selectedEquipment"
        required
        #equipmentModel="ngModel"
        [class.invalid]="submitted && equipmentModel.invalid"
      >
        <option value="" disabled>Select equipment</option>

        @for (item of equipmentOptions; track item.name) {
          <option
            [value]="item.name"
            [disabled]="item.availability !== 'Available'"
          >
            {{ item.name }} ({{ item.availability }})
          </option>
        }
      </select>

      @if (submitted && equipmentModel.invalid) {
        <small class="error-message">Please select equipment.</small>
      }
    </div>

    <!-- Assign To -->
    <div class="form-field">
      <label for="assignedTo">
        Assign To <span class="required">*</span>
      </label>

      <select
        id="assignedTo"
        name="assignedTo"
        [(ngModel)]="assignedTo"
        required
        #assignedModel="ngModel"
        [class.invalid]="submitted && assignedModel.invalid"
      >
        <option value="" disabled>Select team member</option>

        @for (member of teamMembers; track member) {
          <option [value]="member">{{ member }}</option>
        }
      </select>
```

```
        @if (submitted && assignedModel.invalid) {
          <small class="error-message">Please select a team member.</small>
        }
      </div>

      <!-- Expected Return Date -->
      <div class="form-field">
        <label for="returnDate">
          Expected Return Date <span class="required">*</span>
        </label>

        <input
          id="returnDate"
          name="returnDate"
          type="date"
          [(ngModel)]="expectedReturnDate"
          [min]="'2026-10-24'"
          required
          #dateModel="ngModel"
          [class.invalid]="submitted && dateModel.invalid"
        />

        @if (submitted && dateModel.invalid) {
          <small class="error-message">
            Please select a valid return date.
          </small>
        }
      </div>

      <!-- Purpose / Project -->
      <div class="form-field full-width">
        <label for="purpose">Purpose / Project</label>

        <textarea
          id="purpose"
          name="purpose"
          [(ngModel)]="purpose"
          rows="3"
          placeholder="Site survey - DHA Phase 6"
        ></textarea>
      </div>
    </div>

    <div class="form-actions">
      <button
        type="button"
```

```
          class="cancel-button"
          (click)="cancel()"
        >
          Cancel
        </button>

        <button type="submit" class="submit-button">
          Request Check Out
        </button>
      </div>
    </form>
  </main>
</div>
```

4. CSS — `check-out.css`

Paste this into `src/app/components/check-out/check-out.css`. The design uses the same light background, blue heading bar, compact fields, and buttons as your screenshot.

```
:host {
  display: block;
  min-height: 100%;
  color: #263445;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 12px;
}

* {
  box-sizing: border-box;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  cursor: pointer;
}

.checkout-page {
  min-height: calc(100vh - 62px);
  background: #f6f8fb;
}
```

```css
.page-banner {
  margin: 0 -24px;
  padding: 9px 15px;
  min-height: 34px;
  background: #205780;
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}

.checkout-content {
  max-width: 850px;
  padding: 20px 0 30px;
}

.page-title {
  display: flex;
  align-items: center;
  gap: 9px;
  margin-bottom: 17px;
}

.page-title h1 {
  margin: 0;
  color: #202b3a;
  font-size: 20px;
  font-weight: 700;
}

.back-button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 25px;
  height: 28px;
  padding: 0;
  border: 0;
  background: transparent;
  color: #344155;
  font-size: 20px;
}

.checkout-form {
  padding: 18px;
  border: 1px solid #e7ecf2;
```

```
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 6px #213a5206;
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 17px 15px;
}

.form-field {
  display: flex;
  flex-direction: column;
  min-width: 0;
  gap: 7px;
}

.form-field label {
  color: #303b4b;
  font-size: 11px;
  font-weight: 600;
}

.required {
  color: #e34d52;
}

.form-field input,
.form-field select,
.form-field textarea {
  width: 100%;
  min-height: 36px;
  padding: 8px 10px;
  border: 1px solid #d9e0e9;
  border-radius: 5px;
  outline: none;
  background: #fff;
  color: #455267;
  font-size: 11px;
  transition: border-color 0.2s, box-shadow 0.2s;
}

.form-field select {
  padding-right: 25px;
}
```

```css
.form-field textarea {
  min-height: 78px;
  resize: vertical;
  line-height: 1.5;
}

.form-field input:focus,
.form-field select:focus,
.form-field textarea:focus {
  border-color: #2878df;
  box-shadow: 0 0 0 2px #2878df18;
}

.form-field .invalid {
  border-color: #e34d52;
}

.form-field input::placeholder,
.form-field textarea::placeholder {
  color: #8a95a5;
}

.full-width {
  grid-column: 1 / -1;
}

.error-message {
  color: #d43e46;
  font-size: 10px;
}

.form-actions {
  display: flex;
  justify-content: flex-end;
  align-items: center;
  gap: 9px;
  margin-top: 18px;
}

.cancel-button,
.submit-button {
  min-height: 36px;
  padding: 0 17px;
  border-radius: 5px;
  font-size: 11px;
```

```
  font-weight: 600;
  transition: background 0.2s;
}

.cancel-button {
  border: 1px solid #dce3ec;
  background: #f0f3f7;
  color: #354154;
}

.cancel-button:hover {
  background: #e5eaf1;
}

.submit-button {
  border: 1px solid #0868e8;
  background: #0868e8;
  color: #fff;
}

.submit-button:hover {
  background: #0756c4;
}

@media (max-width: 650px) {
  .page-banner {
    margin-right: -13px;
    margin-left: -13px;
  }

  .checkout-content {
    padding-top: 16px;
  }

  .checkout-form {
    padding: 14px;
  }

  .form-grid {
    grid-template-columns: 1fr;
    gap: 15px;
  }

  .full-width {
    grid-column: auto;
  }
```

```
  .form-actions {
    flex-wrap: wrap;
  }
}
```

5. Add the route in `app.routes.ts`

Because you want the Horizon sidebar to remain visible, add the Check Out component as a **child route of your existing Dashboard layout**.

Import the component:

```
import { CheckOut } from './components/check-out/check-out';
```

Then add this route inside your existing `children` array:

```
{ path: 'check-out', component: CheckOut },
```

For example:

```
children: [
  { path: 'dashboard', component: DashboardHome },
  { path: 'equipment', component: EquipmentList },
  { path: 'check-out', component: CheckOut },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
]
```

Keep your other existing routes.

6. Connect the sidebar button

In your Dashboard component's `selectMenu()` method, add the Check Out route:

```
if (label === 'Check Out') {
  this.router.navigate(['/check-out']);
}
```

If you already use a route mapping object, simply add:

```
'Check Out': '/check-out',
```

7. Run and test

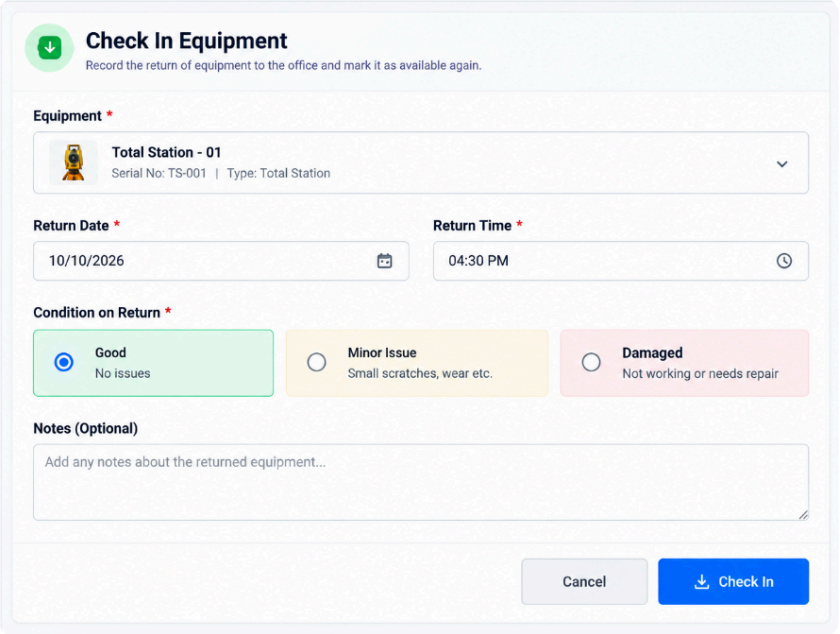
```
ng serve --open
```

Open `http://localhost:4200/dashboard`, then click **Check Out** in your sidebar.

You should see the new form while your existing Horizon sidebar stays fixed.

Note: This is a front-end demonstration. The request button currently displays a confirmation alert; it does not yet submit a checkout request to your backend. The sample equipment and team-member lists will also need to be connected to your API.

24/10/2026, 13:59:11 · gpt-6



please generate the image into angular code ??

24/10/2026, 14:08:07

We'll create a **Check In Equipment** component matching your screenshot, with the same fixed Horizon sidebar and your existing Angular routing.

1. Generate the component

Run this command from your Angular project directory:

```
ng generate component components/check-in
```

2. `check-in.ts`

Replace `src/app/components/check-in/check-in.ts` with:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-check-in',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './check-in.html',
  styleUrl: './check-in.css'
})
```

```
export class CheckIn {
  constructor(private router: Router) {}

  selectedEquipment = 'TS-001';
  returnDate = '2026-10-10';
  returnTime = '16:30';
  condition = 'good';
  notes = '';
  submitted = false;

  equipmentOptions = [
    {
      id: 'TS-001',
      name: 'Total Station - 01',
      serial: 'TS-001',
      type: 'Total Station'
    },
    {
      id: 'GPS-001',
      name: 'GPS Rover - 01',
      serial: 'GPS-001',
      type: 'GPS'
    },
    {
      id: 'DR-001',
      name: 'Drone - DJI Mavic',
      serial: 'DR-001',
      type: 'Drone'
    },
    {
      id: 'FT-001',
      name: 'Field Tablet',
      serial: 'FT-001',
      type: 'Tablet'
    }
  ];

  get selectedItem() {
    return this.equipmentOptions.find(
      item => item.id === this.selectedEquipment
    );
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }
```

```
  checkIn(): void {
    this.submitted = true;

    if (
      !this.selectedEquipment ||
      !this.returnDate ||
      !this.returnTime ||
      !this.condition
    ) {
      return;
    }

    // Demo only: connect your check-in API here.
    alert(
      `Check-in recorded for ${this.selectedItem?.name}.\n` +
      `Condition: ${this.condition}`
    );
  }
}
```

3. `check-in.html`

Replace `src/app/components/check-in/check-in.html` with:

```
<div class="checkin-page">
  <section class="checkin-card">

    <!-- Header -->
    <header class="checkin-header">
      <div class="header-icon">
        <span>↓</span>
      </div>

      <div>
        <h1>Check In Equipment</h1>
        <p>Record the return of equipment to the office and mark it as available again.
</p>
      </div>
    </header>

    <form #checkInForm="ngForm" (ngSubmit)="checkIn()">

      <div class="checkin-body">

        <!-- Equipment -->
```

```
        <div class="form-field equipment-field">
          <label for="equipment">
            Equipment <span class="required">*</span>
          </label>

          <div class="equipment-select">
            <div class="equipment-thumbnail">🛩️</div>

            <div class="equipment-info">
              <strong>{{ selectedItem?.name }}</strong>
              <span>
                Serial No: {{ selectedItem?.serial }}
                  |  
                Type: {{ selectedItem?.type }}
              </span>
            </div>

            <select
              id="equipment"
              name="equipment"
              [(ngModel)]="selectedEquipment"
              required
              aria-label="Select returned equipment"
            >
              @for (item of equipmentOptions; track item.id) {
                <option [value]="item.id">{{ item.name }}</option>
              }
            </select>

            <span class="select-chevron">⌄</span>
          </div>

          @if (submitted && !selectedEquipment) {
            <small class="error">Please select equipment.</small>
          }
        </div>

        <!-- Return date and time -->
        <div class="date-time-grid">

          <div class="form-field">
            <label for="returnDate">
              Return Date <span class="required">*</span>
            </label>

            <input
```

```
            id="returnDate"
            name="returnDate"
            type="date"
            [(ngModel)]="returnDate"
            required
          />
        </div>

        <div class="form-field">
          <label for="returnTime">
            Return Time <span class="required">*</span>
          </label>

          <input
            id="returnTime"
            name="returnTime"
            type="time"
            [(ngModel)]="returnTime"
            required
          />
        </div>

      </div>

      <!-- Condition -->
      <fieldset class="condition-field">
        <legend>
          Condition on Return <span class="required">*</span>
        </legend>

        <div class="condition-grid">

          <label class="condition-option good"
            [class.selected]="condition === 'good'">
            <input
              type="radio"
              name="condition"
              value="good"
              [(ngModel)]="condition"
              required
            />

            <span class="radio-visual"></span>

            <span class="condition-copy">
              <strong>Good</strong>
```

```
            <span>No issues</span>
          </span>
        </label>

        <label class="condition-option minor"
          [class.selected]="condition === 'minor'">
          <input
            type="radio"
            name="condition"
            value="minor"
            [(ngModel)]="condition"
            required
          />

          <span class="radio-visual"></span>

          <span class="condition-copy">
            <strong>Minor Issue</strong>
            <span>Small scratches, wear etc.</span>
          </span>
        </label>

        <label class="condition-option damaged"
          [class.selected]="condition === 'damaged'">
          <input
            type="radio"
            name="condition"
            value="damaged"
            [(ngModel)]="condition"
            required
          />

          <span class="radio-visual"></span>

          <span class="condition-copy">
            <strong>Damaged</strong>
            <span>Not working or needs repair</span>
          </span>
        </label>

      </div>
    </fieldset>

    <!-- Notes -->
    <div class="form-field notes-field">
      <label for="notes">Notes (Optional)</label>
```

```
          <textarea
            id="notes"
            name="notes"
            [(ngModel)]="notes"
            rows="3"
            placeholder="Add any notes about the returned equipment..."
          ></textarea>
        </div>

      </div>

      <!-- Actions -->
      <footer class="checkin-actions">
        <button type="button" class="cancel-button" (click)="cancel()">
          Cancel
        </button>

        <button type="submit" class="checkin-button">
          <span>↓</span>
          Check In
        </button>
      </footer>

    </form>
  </section>
</div>
```

4. `check-in.css`

Replace `src/app/components/check-in/check-in.css` with:

```
:host {
  display: block;
  color: #101d35;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 14px;
}

* {
  box-sizing: border-box;
}

button,
input,
select,
```

```css
textarea {
  font: inherit;
}

.checkin-page {
  min-height: calc(100vh - 62px);
  padding: 14px 0;
  background: #f6f8fb;
}

.checkin-card {
  overflow: hidden;
  border: 1px solid #e6ebf2;
  border-radius: 9px;
  background: #fff;
  box-shadow: 0 2px 8px #172b4d08;
}

.checkin-header {
  display: flex;
  align-items: center;
  gap: 14px;
  padding: 20px 22px;
  border-bottom: 1px solid #e9edf3;
}

.header-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 52px;
  height: 52px;
  flex-shrink: 0;
  border-radius: 50%;
  background: #d6f7e4;
}

.header-icon span {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  border-radius: 7px;
  background: #0da64f;
  color: #fff;
```

```
  font-size: 23px;
  font-weight: 700;
}

.checkin-header h1 {
  margin: 0 0 5px;
  font-size: 23px;
  line-height: 1.25;
  font-weight: 700;
}

.checkin-header p {
  margin: 0;
  color: #53658a;
  font-size: 13px;
  line-height: 1.5;
}

.checkin-body {
  display: flex;
  flex-direction: column;
  gap: 24px;
  padding: 19px 22px 23px;
}

.form-field {
  display: flex;
  flex-direction: column;
  gap: 9px;
  min-width: 0;
}

.form-field label,
.condition-field legend {
  color: #18243a;
  font-size: 14px;
  font-weight: 700;
}

.required {
  color: #ef4444;
}

.equipment-select {
  position: relative;
  display: flex;
```

```
  align-items: center;
  gap: 14px;
  min-height: 67px;
  padding: 9px 16px;
  border: 1px solid #cfd8e5;
  border-radius: 7px;
  background: #fff;
}

.equipment-thumbnail {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 51px;
  height: 48px;
  flex-shrink: 0;
  border-radius: 5px;
  background: #f1f4f8;
  font-size: 28px;
}

.equipment-info {
  display: flex;
  flex-direction: column;
  gap: 5px;
  min-width: 0;
}

.equipment-info strong {
  font-size: 15px;
}

.equipment-info span {
  color: #64748b;
  font-size: 12px;
}

.equipment-select select {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  cursor: pointer;
}
```

```css
.select-chevron {
  margin-left: auto;
  color: #52627a;
  font-size: 22px;
  pointer-events: none;
}

.date-time-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 24px;
}

.form-field input[type="date"],
.form-field input[type="time"] {
  width: 100%;
  min-height: 43px;
  padding: 10px 13px;
  border: 1px solid #cfd8e5;
  border-radius: 6px;
  outline: none;
  background: #fff;
  color: #18243a;
}

.form-field input:focus,
.form-field textarea:focus {
  border-color: #2878ef;
  box-shadow: 0 0 0 3px #2878ef15;
}

.condition-field {
  min-width: 0;
  margin: 0;
  padding: 0;
  border: 0;
}

.condition-field legend {
  margin-bottom: 13px;
  padding: 0;
}

.condition-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
```

```
  gap: 13px;
}

.condition-option {
  display: flex;
  align-items: center;
  gap: 15px;
  min-height: 72px;
  padding: 13px 16px;
  border: 1px solid;
  border-radius: 7px;
  cursor: pointer;
}

.condition-option.good {
  border-color: #a8efc8;
  background: #effcf5;
}

.condition-option.minor {
  border-color: #f9e5b9;
  background: #fffaf0;
}

.condition-option.damaged {
  border-color: #ffd5da;
  background: #fff2f4;
}

.condition-option.selected.good {
  border-color: #39d98a;
}

.condition-option.selected.minor {
  border-color: #efbd4d;
}

.condition-option.selected.damaged {
  border-color: #f37a86;
}

.condition-option input {
  position: absolute;
  width: 1px;
  height: 1px;
  opacity: 0;
```

```
}

.radio-visual {
  position: relative;
  width: 20px;
  height: 20px;
  flex-shrink: 0;
  border: 2px solid #8794a6;
  border-radius: 50%;
  background: white;
}

.condition-option input:checked + .radio-visual {
  border: 2px solid #1677ff;
}

.condition-option input:checked + .radio-visual::after {
  position: absolute;
  top: 4px;
  left: 4px;
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #1677ff;
  content: "";
}

.condition-copy {
  display: flex;
  flex-direction: column;
  gap: 7px;
  min-width: 0;
}

.condition-copy strong {
  font-size: 13px;
}

.condition-copy > span {
  color: #4b5b71;
  font-size: 12px;
  line-height: 1.4;
}

.notes-field textarea {
  width: 100%;
```

```
  min-height: 81px;
  padding: 11px 12px;
  border: 1px solid #cfd8e5;
  border-radius: 6px;
  outline: none;
  resize: vertical;
  color: #18243a;
  line-height: 1.5;
}

.notes-field textarea::placeholder {
  color: #8b97a8;
}

.checkin-actions {
  display: flex;
  justify-content: flex-end;
  gap: 11px;
  padding: 16px 22px 19px;
  border-top: 1px solid #edf0f5;
}

.cancel-button,
.checkin-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  min-width: 134px;
  min-height: 49px;
  padding: 0 19px;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
}

.cancel-button {
  border: 1px solid #d0d9e5;
  background: #f2f4f7;
  color: #39465a;
}

.cancel-button:hover {
  background: #e8edf3;
}
```

```css
.checkin-button {
  border: 1px solid #0967f5;
  background: #0868f7;
  color: #fff;
}

.checkin-button:hover {
  background: #0757d4;
}

.checkin-button span {
  font-size: 22px;
}

@media (max-width: 850px) {
  .condition-grid {
    grid-template-columns: 1fr;
  }

  .condition-option {
    min-height: 65px;
  }
}

@media (max-width: 600px) {
  .checkin-header {
    align-items: flex-start;
    padding: 16px;
  }

  .checkin-header h1 {
    font-size: 19px;
  }

  .checkin-header p {
    font-size: 12px;
  }

  .checkin-body {
    gap: 20px;
    padding: 16px;
  }

  .date-time-grid {
    grid-template-columns: 1fr;
    gap: 18px;
```

```
  }

  .equipment-select {
    gap: 9px;
    padding: 8px;
  }

  .equipment-info strong {
    font-size: 13px;
  }

  .equipment-info span {
    font-size: 10px;
  }

  .checkin-actions {
    padding: 14px 16px;
  }

  .cancel-button,
  .checkin-button {
    min-width: 0;
    flex: 1;
    padding: 0 10px;
  }
}
```

5. Add the Check In route

In `src/app/app.routes.ts`, import the component:

```
import { CheckIn } from './components/check-in/check-in';
```

Add this to the existing Dashboard layout's `children` array:

```
{ path: 'check-in', component: CheckIn },
```

Your existing Dashboard parent route must remain in place so the fixed sidebar is preserved.

6. Connect the sidebar

In your Dashboard component's route mapping, ensure this entry exists:

```
'Check In': '/check-in',
```

Run the project:

```
ng serve --open
```

Then open `http://localhost:4200/check-in` or click **Check In** in the sidebar.

Note: The screenshot's equipment dropdown uses sample records. The current button demonstrates the check-in action with an alert; saving the return, updating equipment availability, and recording damage or repair status will require connecting your backend API.

24/10/2026, 14:08:09 · gpt-6



please generate the code of this in angular ??

24/10/2026, 14:16:21

I'll create an **Approval Request (Drone)** page for your SolvefyTask Angular project, matching your screenshot and keeping your existing fixed Horizon sidebar.

The page will include the pending approval badge, drone details, request information, optional comments, and **Reject** and **Approve** buttons.

1. Generate the Angular component

Run this command from your project directory:

```
ng generate component components/approval-request
```

This creates the component files for the approval screen.

2. TypeScript — `approval-request.ts`

Replace `src/app/components/approval-request/approval-request.ts` with:

```ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-approval-request',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './approval-request.html',
  styleUrl: './approval-request.css'
})
export class ApprovalRequest {
  status: 'Pending Approval' | 'Approved' | 'Rejected' =
```

```
    'Pending Approval';

  comments = '';

  request = {
    equipmentName: 'Drone - DJI Mavic',
    serialNumber: 'DR-001',
    requestedBy: 'Ali Raza',
    requestedOn: '10/10/2026, 10:30 AM',
    expectedReturn: '10/12/2026',
    purpose: 'Aerial survey for client site'
  };

  constructor(private router: Router) {}

  approve(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Approved';
    alert('Approval demonstration: request approved.');
  }

  reject(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Rejected';
    alert('Approval demonstration: request rejected.');
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

3. HTML — `approval-request.html`

Replace `src/app/components/approval-request/approval-request.html` with:

```
<div class="approval-page">

  <!-- Blue page banner -->
  <div class="page-banner">
```

```
  5. Approval Request (Drone)
</div>

<section class="approval-card">

  <!-- Card heading -->
  <header class="approval-header">
    <h1>Approval Request</h1>

    <span
      class="status-badge"
      [class.approved]="status === 'Approved'"
      [class.rejected]="status === 'Rejected'"
    >
      {{ status }}
    </span>
  </header>

  <!-- Equipment and request information -->
  <div class="request-details">

    <div class="drone-illustration" aria-label="Drone illustration">
      <svg viewBox="0 0 140 100" role="img" aria-label="Quadcopter drone">
        <g fill="none" stroke="#596777" stroke-width="2.5"
           stroke-linecap="round" stroke-linejoin="round">
          <path d="M70 42 L28 25 M70 42 L112 25 M70 42 L28 60 M70 42 L112 60"/>
          <path d="M28 25 L18 19 M28 25 L38 19 M112 25 L102 19 M112 25 L122 19"/>
          <path d="M28 60 L18 66 M28 60 L38 66 M112 60 L102 66 M112 60 L122 66"/>
          <ellipse cx="28" cy="21" rx="18" ry="4"/>
          <ellipse cx="112" cy="21" rx="18" ry="4"/>
          <ellipse cx="28" cy="64" rx="18" ry="4"/>
          <ellipse cx="112" cy="64" rx="18" ry="4"/>
          <path d="M53 37 L87 37 L94 47 L82 55 L58 55 L46 47 Z" fill="#aeb8c2"/>
          <path d="M58 55 L54 67 M82 55 L86 67"/>
          <path d="M49 68 L91 68"/>
          <rect x="64" y="68" width="12" height="9" rx="2" fill="#6e7d8c"/>
        </g>
        <circle cx="70" cy="45" r="5" fill="#758697"/>
      </svg>
    </div>

    <div class="request-info">
      <h2>{{ request.equipmentName }}</h2>

      <dl>
        <div>
```

```
          <dt>Serial No:</dt>
          <dd>{{ request.serialNumber }}</dd>
        </div>

        <div>
          <dt>Requested By:</dt>
          <dd>{{ request.requestedBy }}</dd>
        </div>

        <div>
          <dt>Requested On:</dt>
          <dd>{{ request.requestedOn }}</dd>
        </div>

        <div>
          <dt>Expected Return:</dt>
          <dd>{{ request.expectedReturn }}</dd>
        </div>

        <div>
          <dt>Purpose:</dt>
          <dd>{{ request.purpose }}</dd>
        </div>
      </dl>
    </div>

  </div>

  <!-- Optional comments -->
  <div class="comments-section">
    <label for="comments">Comments (Optional)</label>

    <textarea
      id="comments"
      name="comments"
      [(ngModel)]="comments"
      rows="3"
      placeholder="Add a comment..."
      [disabled]="status !== 'Pending Approval'"
    ></textarea>
  </div>

  <!-- Actions -->
  <footer class="approval-actions">
    <button
      type="button"
```

```
        class="reject-button"
        [disabled]="status !== 'Pending Approval'"
        (click)="reject()"
      >
        Reject
      </button>

      <button
        type="button"
        class="approve-button"
        [disabled]="status !== 'Pending Approval'"
        (click)="approve()"
      >
        Approve
      </button>
    </footer>

  </section>
</div>
```

4. CSS — `approval-request.css`

Paste this into `src/app/components/approval-request/approval-request.css`:

```
:host {
  display: block;
  min-height: 100%;
  color: #263445;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 12px;
}

* {
  box-sizing: border-box;
}

button,
textarea {
  font: inherit;
}

.approval-page {
  min-height: calc(100vh - 62px);
  padding-bottom: 24px;
  background: #f6f8fb;
}
```

```css
.page-banner {
  margin: 0 -24px 10px;
  padding: 8px 15px;
  min-height: 33px;
  border-radius: 0 0 5px 5px;
  background: #205780;
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}

.approval-card {
  overflow: hidden;
  border: 1px solid #e7ebf1;
  border-radius: 7px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355008;
}

.approval-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 13px 15px;
  border-bottom: 1px solid #edf0f4;
}

.approval-header h1 {
  margin: 0;
  color: #202b3b;
  font-size: 16px;
  font-weight: 700;
}

.status-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 5px 10px;
  border-radius: 12px;
  background: #fff0c9;
  color: #9b6700;
  font-size: 10px;
  white-space: nowrap;
```

```
}

.status-badge.approved {
  background: #d9f6e7;
  color: #087c4d;
}

.status-badge.rejected {
  background: #ffe0e2;
  color: #c22e39;
}

.request-details {
  display: grid;
  grid-template-columns: 95px minmax(0, 1fr);
  gap: 12px;
  padding: 15px;
}

.drone-illustration {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 86px;
  border-radius: 5px;
  background: #f5f7fa;
}

.drone-illustration svg {
  display: block;
  width: 100%;
  max-width: 105px;
  height: auto;
}

.request-info {
  min-width: 0;
}

.request-info h2 {
  margin: 1px 0 8px;
  color: #263344;
  font-size: 12px;
  font-weight: 700;
}
```

```css
.request-info dl {
  display: flex;
  flex-direction: column;
  gap: 5px;
  margin: 0;
}

.request-info dl > div {
  display: grid;
  grid-template-columns: 94px minmax(0, 1fr);
  gap: 5px;
}

.request-info dt {
  color: #586679;
  font-size: 10px;
}

.request-info dd {
  margin: 0;
  color: #344154;
  font-size: 10px;
  overflow-wrap: anywhere;
}

.comments-section {
  display: flex;
  flex-direction: column;
  gap: 7px;
  padding: 0 15px 13px;
}

.comments-section label {
  color: #354154;
  font-size: 11px;
  font-weight: 700;
}

.comments-section textarea {
  display: block;
  width: 100%;
  min-height: 64px;
  padding: 9px;
  border: 1px solid #e0e5ec;
  border-radius: 5px;
  outline: none;
```

```
  resize: vertical;
  background: #fff;
  color: #354154;
  font-size: 11px;
}

.comments-section textarea:focus {
  border-color: #2878df;
  box-shadow: 0 0 0 2px #2878df15;
}

.comments-section textarea::placeholder {
  color: #929cab;
}

.comments-section textarea:disabled {
  background: #f5f6f8;
  cursor: not-allowed;
}

.approval-actions {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
  padding: 0 15px 14px;
}

.reject-button,
.approve-button {
  min-width: 80px;
  min-height: 34px;
  padding: 7px 16px;
  border: 0;
  border-radius: 5px;
  color: #fff;
  font-size: 11px;
  font-weight: 600;
  cursor: pointer;
  transition: opacity 0.2s, background 0.2s;
}

.reject-button {
  background: #ef4d50;
}

.reject-button:hover:not(:disabled) {
```

```
  background: #d83c40;
}

.approve-button {
  background: #159b79;
}

.approve-button:hover:not(:disabled) {
  background: #0e8265;
}

.reject-button:disabled,
.approve-button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

@media (max-width: 600px) {
  .page-banner {
    margin-right: -13px;
    margin-left: -13px;
  }

  .request-details {
    grid-template-columns: 75px minmax(0, 1fr);
    gap: 10px;
    padding: 12px;
  }

  .request-info dl > div {
    grid-template-columns: 85px minmax(0, 1fr);
  }

  .approval-header {
    padding: 12px;
  }

  .approval-header h1 {
    font-size: 14px;
  }

  .comments-section {
    padding-right: 12px;
    padding-left: 12px;
  }
```

```
  .approval-actions {
    padding-right: 12px;
    padding-left: 12px;
  }
}
```

5. Add the route in `app.routes.ts`

Import the new component:

```
import { ApprovalRequest } from './components/approval-request/approval-request';
```

Add this route inside your existing Dashboard layout's `children` array:

```
{ path: 'approvals', component: ApprovalRequest },
```

For example:

```
children: [
  { path: 'dashboard', component: DashboardHome },
  { path: 'equipment', component: EquipmentList },
  { path: 'check-out', component: CheckOut },
  { path: 'check-in', component: CheckIn },
  { path: 'approvals', component: ApprovalRequest },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
]
```

Keep your other existing routes and imports.

6. Connect the Approvals sidebar item

In `dashboard.ts`, make sure your route mapping includes:

```
'Approvals': '/approvals',
```

Now clicking **Approvals** will open the new page while retaining your existing Horizon sidebar.

7. Run the application

```
ng serve --open
```

Open `http://localhost:4200/approvals` to test the page.

Important: Approve and Reject currently update the status locally and display a demonstration alert. They do not save decisions to your backend or notify the requester. Connect these methods to your approval API when you're ready.

24/10/2026, 14:16:35 · gpt-6

PS C:\Users\Ahmad Abdullah\Desktop\BMADSolvefy\src\AngularFrontend\solvefy-task> ng serve --open
Application bundle generation failed. [1.929 seconds] - 2026-10-24T09:23:53.903Z

X [ERROR] NG8002: Can't bind to 'ngModel' since it isn't a known property of 'textarea'. [plugin angular-compiler]

```
  src/app/components/approvals/approvals.html:87:8:
    87 │        [(ngModel)]="comments"
       ╵        ~~~~~~~~~~~~~~~~~~~~~~~~
```

Error occurs in the template of component Approvals.

```
  src/app/components/approvals/approvals.ts:7:15:
    7 │   templateUrl: './approvals.html',
      ╵              ~~~~~~~~~~~~~~~~~~~
```

Watch mode enabled. Watching for file changes...

24/10/2026, 14:24:03

The error occurs because your `Approvals` component is using `[(ngModel)]="comments"` but hasn't imported `FormsModule`.

Fix: Update `approvals.ts`

Open `src/app/components/approvals/approvals.ts` and add `FormsModule` to the imports.

Use this structure:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-approvals',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './approvals.html',
  styleUrl: './approvals.css'
})
export class Approvals {
```

```
  comments = '';

  // Keep your existing properties and methods here.
}
```

Important: If your `approvals.ts` already contains properties and methods for approving or rejecting requests, keep them. Just add the `FormsModule` import and include it in the component's `imports` array.

```
import { FormsModule } from '@angular/forms';

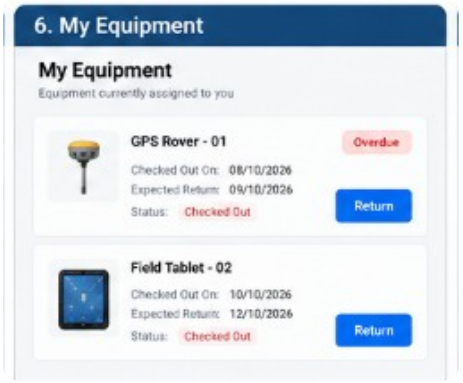
// Inside @Component:
imports: [FormsModule],
```

Then test

1. Save the file with `Ctrl + S`.
2. Angular should rebuild automatically.
3. If the error persists, check that `FormsModule` is imported in the exact component named in the error: `Approvals` in `approvals.ts`.

You don't need to change your HTML or install any additional packages.

24/10/2026, 14:24:03 · gpt-6



please generate the code of this image in angular ??

24/10/2026, 14:26:00

I'll create the **My Equipment** page for your existing Angular SolvefyTask application, matching the screenshot and preserving your fixed Horizon sidebar.

The page will show equipment assigned to the logged-in user, overdue badges, checkout and expected-return dates, status labels, and a **Return** button for each item.

1. Generate the component

Run this command in your Angular project terminal:

```
ng generate component components/my-equipment
```

This will create the component files under `src/app/components/my-equipment/`.

2. TypeScript — `my-equipment.ts`

Replace `src/app/components/my-equipment/my-equipment.ts` with:

```typescript
import { Component } from '@angular/core';
import { Router } from '@angular/router';

interface AssignedEquipment {
  id: number;
  name: string;
  serialNumber: string;
  type: string;
  checkedOutDate: string;
  expectedReturnDate: string;
  status: 'Checked Out' | 'Overdue' | 'Returned';
  icon: string;
}
```

```
@Component({
  selector: 'app-my-equipment',
  standalone: true,
  imports: [],
  templateUrl: './my-equipment.html',
  styleUrl: './my-equipment.css'
})
export class MyEquipment {
  constructor(private router: Router) {}

  equipment: AssignedEquipment[] = [
    {
      id: 1,
      name: 'GPS Rover - 01',
      serialNumber: 'GPS-001',
      type: 'GPS',
      checkedOutDate: '08/10/2026',
      expectedReturnDate: '09/10/2026',
      status: 'Overdue',
      icon: 'gps'
    },
    {
      id: 2,
      name: 'Field Tablet - 02',
      serialNumber: 'FT-002',
      type: 'Tablet',
      checkedOutDate: '10/10/2026',
      expectedReturnDate: '12/10/2026',
      status: 'Checked Out',
      icon: 'tablet'
    }
  ];

  returnEquipment(item: AssignedEquipment): void {
    if (item.status === 'Returned') {
      return;
    }

    const confirmed = window.confirm(
      `Do you want to return ${item.name}?`
    );

    if (!confirmed) {
      return;
    }
```

```
    // Demonstration only: connect your return API here.
    item.status = 'Returned';
    alert(`${item.name} has been marked as returned in this demo.`);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

3. HTML — `my-equipment.html`

Replace `src/app/components/my-equipment/my-equipment.html` with:

```
<div class="my-equipment-page">

  <div class="page-banner">
    6. My Equipment
  </div>

  <main class="page-content">

    <header class="page-heading">
      <div>
        <h1>My Equipment</h1>
        <p>Equipment currently assigned to you</p>
      </div>
    </header>

    <section class="equipment-list">

      @for (item of equipment; track item.id) {
        <article class="equipment-card">

          <!-- Equipment image/icon -->
          <div class="equipment-image" [class]=" 'equipment-image ' + item.icon">

            @if (item.icon === 'gps') {
              <svg viewBox="0 0 64 80" role="img" aria-label="GPS rover">
                <ellipse cx="32" cy="14" rx="18" ry="10" fill="#e0a52c"/>
                <rect x="17" y="12" width="30" height="13" rx="4" fill="#687786"/>
                <rect x="23" y="23" width="18" height="26" rx="4" fill="#d9dfe5"/>
                <rect x="28" y="45" width="8" height="23" rx="2" fill="#667786"/>
                <path d="M20 30 L12 39 M44 30 L52 39" stroke="#697887" stroke-width="3"/>
                <circle cx="32" cy="17" r="5" fill="#f8d35c"/>
```

```
      </svg>
    } @else {
      <svg viewBox="0 0 64 80" role="img" aria-label="Field tablet">
        <rect x="7" y="3" width="50" height="72" rx="6" fill="#253d52"/>
        <rect x="11" y="9" width="42" height="58" rx="3" fill="#397ca0"/>
        <path d="M13 45 L27 27 L37 39 L46 22 L52 31 L52 65 L13 65 Z"
          fill="#70a8c0"/>
        <circle cx="32" cy="70" r="2" fill="#c7d2dc"/>
        <circle cx="32" cy="6" r="1" fill="#bfcbd7"/>
      </svg>
    }

  </div>

  <!-- Equipment details -->
  <div class="equipment-details">

    <div class="equipment-title-row">
      <h2>{{ item.name }}</h2>

      @if (item.status === 'Overdue') {
        <span class="status-badge overdue">Overdue</span>
      } @else if (item.status === 'Returned') {
        <span class="status-badge returned">Returned</span>
      }
    </div>

    <div class="equipment-meta">
      <span>Checked Out:</span>
      <strong>{{ item.checkedOutDate }}</strong>
    </div>

    <div class="equipment-meta">
      <span>Expected Return:</span>
      <strong>{{ item.expectedReturnDate }}</strong>
    </div>

    <div class="equipment-meta">
      <span>Status:</span>
      <strong
        class="status-text"
        [class.overdue-text]="item.status === 'Overdue'"
        [class.returned-text]="item.status === 'Returned'"
      >
        {{ item.status === 'Returned' ? 'Returned' : 'Checked Out' }}
      </strong>
```

```
            </div>

          </div>

          <!-- Return action -->
          <div class="equipment-actions">
            <button
              type="button"
              class="return-button"
              [disabled]="item.status === 'Returned'"
              (click)="returnEquipment(item)"
            >
              {{ item.status === 'Returned' ? 'Returned' : 'Return' }}
            </button>
          </div>

        </article>
      } @empty {
        <div class="empty-state">
          <span class="empty-icon">✓</span>
          <h2>No equipment assigned</h2>
          <p>You currently have no equipment checked out.</p>
        </div>
      }

    </section>

    <footer class="page-footer">
      <span>{{ equipment.length }} equipment record(s)</span>
      <span>Equipment Tracking System</span>
    </footer>

  </main>
</div>
```

4. CSS — `my-equipment.css`

Replace `src/app/components/my-equipment/my-equipment.css` with:

```
:host {
  display: block;
  min-height: 100%;
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 12px;
}
```

```css
* {
  box-sizing: border-box;
}

button {
  font: inherit;
}

.my-equipment-page {
  min-height: calc(100vh - 62px);
  background: #f6f8fb;
}

.page-banner {
  margin: 0 -24px;
  padding: 9px 15px;
  min-height: 34px;
  border-radius: 0 0 5px 5px;
  background: #205780;
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}

.page-content {
  padding: 15px 0 24px;
}

.page-heading {
  margin-bottom: 13px;
}

.page-heading h1 {
  margin: 0 0 4px;
  color: #202a39;
  font-size: 20px;
  font-weight: 700;
}

.page-heading p {
  margin: 0;
  color: #8994a4;
  font-size: 11px;
}
```

```css
.equipment-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.equipment-card {
  display: grid;
  grid-template-columns: 55px minmax(0, 1fr) 62px;
  align-items: center;
  gap: 12px;
  min-height: 86px;
  padding: 10px 12px;
  border: 1px solid #e7ecf2;
  border-radius: 7px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.equipment-image {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 55px;
  height: 62px;
  overflow: hidden;
  border-radius: 5px;
  background: #f1f4f8;
}

.equipment-image svg {
  width: 43px;
  height: 56px;
}

.equipment-image.tablet svg {
  width: 40px;
  height: 56px;
}

.equipment-details {
  min-width: 0;
}

.equipment-title-row {
  display: flex;
```

```
  align-items: center;
  justify-content: space-between;
  gap: 8px;
  margin-bottom: 6px;
}

.equipment-title-row h2 {
  margin: 0;
  color: #263344;
  font-size: 12px;
  font-weight: 700;
}

.status-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 4px 8px;
  border-radius: 5px;
  font-size: 9px;
  white-space: nowrap;
}

.status-badge.overdue {
  background: #ffe1e1;
  color: #c9343b;
}

.status-badge.returned {
  background: #d9f6e7;
  color: #137a4c;
}

.equipment-meta {
  display: flex;
  flex-wrap: wrap;
  align-items: baseline;
  gap: 5px;
  margin-top: 4px;
  color: #667387;
  font-size: 10px;
  line-height: 1.3;
}

.equipment-meta strong {
  color: #475467;
```

```
  font-size: 10px;
  font-weight: 500;
}

.equipment-meta .status-text {
  color: #c33b42;
}

.equipment-meta .returned-text {
  color: #137a4c;
}

.equipment-actions {
  display: flex;
  justify-content: flex-end;
}

.return-button {
  min-width: 59px;
  min-height: 29px;
  padding: 6px 10px;
  border: 1px solid #0868e8;
  border-radius: 4px;
  background: #0868e8;
  color: #fff;
  font-size: 10px;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s;
}

.return-button:hover:not(:disabled) {
  background: #0756c4;
}

.return-button:disabled {
  border-color: #b6dec9;
  background: #e3f6eb;
  color: #137a4c;
  cursor: default;
}

.page-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
```

```
  padding-top: 18px;
  color: #8994a4;
  font-size: 10px;
}

.empty-state {
  padding: 35px 15px;
  border: 1px solid #e7ecf2;
  border-radius: 7px;
  background: #fff;
  text-align: center;
}

.empty-icon {
  color: #159b79;
  font-size: 28px;
}

.empty-state h2 {
  margin: 10px 0 5px;
  font-size: 15px;
}

.empty-state p {
  color: #8994a4;
}

@media (max-width: 650px) {
  .page-banner {
    margin-right: -13px;
    margin-left: -13px;
  }

  .equipment-card {
    grid-template-columns: 45px minmax(0, 1fr);
    gap: 10px;
    padding: 10px;
  }

  .equipment-image {
    width: 45px;
    height: 54px;
  }

  .equipment-image svg {
    width: 36px;
```

```
    height: 48px;
  }

  .equipment-title-row {
    flex-wrap: wrap;
  }

  .equipment-actions {
    grid-column: 2;
    justify-content: flex-start;
  }

  .page-footer {
    font-size: 9px;
  }
}
```

5. Add the route in `app.routes.ts`

Import your component:

```
import { MyEquipment } from './components/my-equipment/my-equipment';
```

Add this route inside the existing Dashboard layout's `children` array:

```
{ path: 'my-equipment', component: MyEquipment },
```

For example:

```
children: [
  { path: 'dashboard', component: DashboardHome },
  { path: 'equipment', component: EquipmentList },
  { path: 'check-out', component: CheckOut },
  { path: 'check-in', component: CheckIn },
  { path: 'approvals', component: Approvals },
  { path: 'my-equipment', component: MyEquipment },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
]
```

Keep your other existing routes and imports.

6. Connect the sidebar

In `dashboard.ts`, make sure your menu route mapping contains:

```
'My Equipment': '/my-equipment',
```

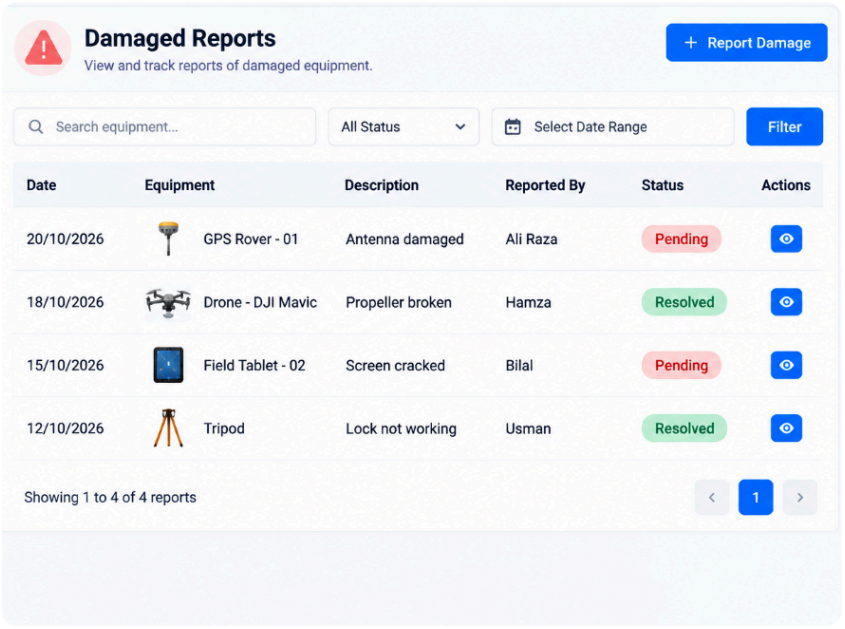
Now run:

```
ng serve --open
```

Open `http://localhost:4200/my-equipment` or click **My Equipment** in your sidebar.

Note: The page currently uses sample equipment records. The Return button demonstrates a local status change; it doesn't yet update your database or equipment availability through the backend API.

24/10/2026, 14:26:17 · gpt-6



please generate the code of thus image in angular ??

24/10/2026, 14:33:31

I'll create the **Damaged Reports** page for your SolvefyTask Angular project, matching your screenshot and keeping the existing Horizon sidebar fixed.

The page will include a damage-report table, search, status filtering, date-range filtering, a Report Damage button, status badges, View Details buttons, and pagination.

1. Generate the component

Run this command in your Angular project terminal:

```
ng generate component components/damage-reports
```

This will create the component files under `src/app/components/damage-reports/`.

2. TypeScript — `damage-reports.ts`

Replace `src/app/components/damage-reports/damage-reports.ts` with:

```ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface DamageReport {
  id: number;
  date: string;
  equipment: string;
```

```
  serial: string;
  description: string;
  reportedBy: string;
  status: 'Pending' | 'Resolved';
  icon: string;
}

@Component({
  selector: 'app-damage-reports',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './damage-reports.html',
  styleUrl: './damage-reports.css'
})
export class DamageReports {
  searchText = '';
  selectedStatus = 'All Status';
  startDate = '';
  endDate = '';
  showReportForm = false;
  currentPage = 1;
  pageSize = 4;
  selectedReport: DamageReport | null = null;

  newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };

  reports: DamageReport[] = [
    {
      id: 1,
      date: '20/10/2026',
      equipment: 'GPS Rover - 01',
      serial: 'GPS-001',
      description: 'Antenna damaged',
      reportedBy: 'Ali Raza',
      status: 'Pending',
      icon: 'gps'
    },
    {
      id: 2,
      date: '18/10/2026',
      equipment: 'Drone - DJI Mavic',
      serial: 'DR-001',
```

```
      description: 'Propeller broken',
      reportedBy: 'Hamza',
      status: 'Resolved',
      icon: 'drone'
    },
    {
      id: 3,
      date: '15/10/2026',
      equipment: 'Field Tablet - 02',
      serial: 'FT-002',
      description: 'Screen cracked',
      reportedBy: 'Bilal',
      status: 'Pending',
      icon: 'tablet'
    },
    {
      id: 4,
      date: '12/10/2026',
      equipment: 'Tripod',
      serial: 'TR-001',
      description: 'Lock not working',
      reportedBy: 'Usman',
      status: 'Resolved',
      icon: 'tripod'
    }
  ];

  get filteredReports(): DamageReport[] {
    const search = this.searchText.trim().toLowerCase();

    return this.reports.filter(report => {
      const matchesSearch =
        !search ||
        report.equipment.toLowerCase().includes(search) ||
        report.description.toLowerCase().includes(search) ||
        report.reportedBy.toLowerCase().includes(search) ||
        report.serial.toLowerCase().includes(search);

      const matchesStatus =
        this.selectedStatus === 'All Status' ||
        report.status === this.selectedStatus;

      const reportDate = this.toDate(report.date);

      const matchesStart =
        !this.startDate || reportDate >= this.startDate;
```

```
    const matchesEnd =
      !this.endDate || reportDate <= this.endDate;

    return matchesSearch && matchesStatus &&
      matchesStart && matchesEnd;
  });
}

get paginatedReports(): DamageReport[] {
  const start = (this.currentPage - 1) * this.pageSize;

  return this.filteredReports.slice(start, start + this.pageSize);
}

get totalPages(): number {
  return Math.max(1, Math.ceil(this.filteredReports.length / this.pageSize));
}

get firstVisible(): number {
  return this.filteredReports.length === 0
    ? 0
    : (this.currentPage - 1) * this.pageSize + 1;
}

get lastVisible(): number {
  return Math.min(
    this.currentPage * this.pageSize,
    this.filteredReports.length
  );
}

private toDate(date: string): string {
  const [day, month, year] = date.split('/');
  return `${year}-${month}-${day}`;
}

applyFilters(): void {
  this.currentPage = 1;
}

resetFilters(): void {
  this.searchText = '';
  this.selectedStatus = 'All Status';
  this.startDate = '';
  this.endDate = '';
```

```
    this.currentPage = 1;
  }

  changePage(page: number): void {
    if (page >= 1 && page <= this.totalPages) {
      this.currentPage = page;
    }
  }

  openReportForm(): void {
    this.newReport = {
      equipment: '',
      description: '',
      reportedBy: 'Ahmad'
    };
    this.showReportForm = true;
  }

  saveReport(): void {
    if (!this.newReport.equipment.trim() ||
        !this.newReport.description.trim()) {
      return;
    }

    const today = new Date();
    const formattedDate = [
      String(today.getDate()).padStart(2, '0'),
      String(today.getMonth() + 1).padStart(2, '0'),
      today.getFullYear()
    ].join('/');

    this.reports.unshift({
      id: Math.max(0, ...this.reports.map(r => r.id)) + 1,
      date: formattedDate,
      equipment: this.newReport.equipment.trim(),
      serial: 'N/A',
      description: this.newReport.description.trim(),
      reportedBy: this.newReport.reportedBy.trim() || 'Ahmad',
      status: 'Pending',
      icon: 'equipment'
    });

    this.showReportForm = false;
    this.resetFilters();
  }
```

```
  viewReport(report: DamageReport): void {
    this.selectedReport = report;
  }

  closeDetails(): void {
    this.selectedReport = null;
  }
}
```

3. HTML — `damage-reports.html`

Replace `src/app/components/damage-reports/damage-reports.html` with:

```
<div class="damage-page">

  <!-- Page heading -->
  <header class="damage-header">
    <div class="heading-icon">!</div>

    <div class="heading-text">
      <h1>Damaged Reports</h1>
      <p>View and track reports of damaged equipment.</p>
    </div>

    <button
      type="button"
      class="report-button"
      (click)="openReportForm()"
    >
      <span class="plus">+</span>
      Report Damage
    </button>
  </header>

  <!-- Search and filters -->
  <section class="filter-bar">
    <div class="search-box">
      <span class="search-icon">⌕</span>
      <input
        type="search"
        name="search"
        placeholder="Search equipment..."
        [(ngModel)]="searchText"
        (ngModelChange)="applyFilters()"
      />
    </div>
```

```
<select
  name="status"
  class="status-filter"
  [(ngModel)]="selectedStatus"
  (ngModelChange)="applyFilters()"
  aria-label="Filter by status"
>
  <option>All Status</option>
  <option>Pending</option>
  <option>Resolved</option>
</select>

<div class="date-range">
  <span class="calendar-icon">▦</span>
  <input
    type="date"
    aria-label="Start date"
    title="Start date"
    [(ngModel)]="startDate"
    (ngModelChange)="applyFilters()"
  />
  <span class="date-separator">–</span>
  <input
    type="date"
    aria-label="End date"
    title="End date"
    [(ngModel)]="endDate"
    (ngModelChange)="applyFilters()"
  />
</div>

<button
  type="button"
  class="filter-button"
  (click)="applyFilters()"
>
  Filter
</button>

<button
  type="button"
  class="reset-button"
  (click)="resetFilters()"
>
  Reset
```

```
    </button>
  </section>

  <!-- Reports table -->
  <section class="table-card">
    <div class="table-scroll">
      <table>
        <thead>
          <tr>
            <th>Date</th>
            <th>Equipment</th>
            <th>Description</th>
            <th>Reported By</th>
            <th>Status</th>
            <th class="actions-heading">Actions</th>
          </tr>
        </thead>

        <tbody>
          @for (report of paginatedReports; track report.id) {
            <tr>
              <td class="date-cell">{{ report.date }}</td>

              <td>
                <div class="equipment-cell">
                  <div class="equipment-icon" [class]=" 'equipment-icon ' + report.icon">

                    @if (report.icon === 'gps') {
                      <svg viewBox="0 0 48 48" aria-label="GPS rover">
                        <ellipse cx="24" cy="10" rx="11" ry="5" fill="#e5a72f"/>
                        <rect x="14" y="10" width="20" height="8" rx="3" fill="#586a7a"/>
                        <rect x="19" y="18" width="10" height="15" rx="3"
fill="#c4ccd3"/>
                        <path d="M22 32 L22 43 M26 32 L26 43" stroke="#687887" stroke-
width="2"/>
                        <circle cx="24" cy="12" r="3" fill="#ffda67"/>
                      </svg>
                    } @else if (report.icon === 'drone') {
                      <svg viewBox="0 0 48 48" aria-label="Drone">
                        <g stroke="#647383" stroke-width="2.5" stroke-linecap="round"
fill="none">
                          <path d="M24 23 L10 13 M24 23 L38 13 M24 23 L10 34 M24 23 L38
34"/>
                          <ellipse cx="10" cy="11" rx="8" ry="2.5"/>
                          <ellipse cx="38" cy="11" rx="8" ry="2.5"/>
                          <ellipse cx="10" cy="36" rx="8" ry="2.5"/>
```

```
                          <ellipse cx="38" cy="36" rx="8" ry="2.5"/>
                        </g>
                        <rect x="17" y="18" width="14" height="11" rx="4"
fill="#aeb9c5"/>
                        <circle cx="24" cy="23" r="3" fill="#647383"/>
                      </svg>
                    } @else if (report.icon === 'tablet') {
                      <svg viewBox="0 0 48 48" aria-label="Tablet">
                        <rect x="9" y="2" width="30" height="44" rx="4" fill="#233c51"/>
                        <rect x="12" y="6" width="24" height="34" rx="2" fill="#327ca5"/>
                        <path d="M13 28 L22 17 L28 24 L35 14 L35 39 L13 39 Z"
fill="#70b2d0"/>
                        <circle cx="24" cy="43" r="1" fill="#fff"/>
                      </svg>
                    } @else if (report.icon === 'tripod') {
                      <svg viewBox="0 0 48 48" aria-label="Tripod">
                        <rect x="19" y="4" width="10" height="6" rx="1" fill="#273c4e"/>
                        <path d="M21 10 L10 44 M25 10 L25 44 M28 10 L39 44"
stroke="#ce7a25" stroke-width="3" stroke-linecap="round"/>
                        <path d="M18 17 L30 17" stroke="#4c5966" stroke-width="2"/>
                      </svg>
                    } @else {
                      <span>⚒</span>
                    }

                  </div>

                  <span>{{ report.equipment }}</span>
                </div>
              </td>

              <td>{{ report.description }}</td>
              <td>{{ report.reportedBy }}</td>

              <td>
                <span
                  class="status-badge"
                  [class.pending]="report.status === 'Pending'"
                  [class.resolved]="report.status === 'Resolved'"
                >
                  {{ report.status }}
                </span>
              </td>

              <td class="actions-cell">
                <button
```

```
                type="button"
                class="view-button"
                title="View report details"
                aria-label="View report details"
                (click)="viewReport(report)"
              >
                <svg viewBox="0 0 24 24" width="19" height="19"
                     fill="none" stroke="currentColor" stroke-width="2">
                  <path d="M2 12s3.5-7 10-7 10 7 10 7-3.5 7-10 7S2 12 2 12Z"/>
                  <circle cx="12" cy="12" r="3"/>
                </svg>
              </button>
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="6" class="empty-cell">
              No damage reports match your filters.
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <!-- Pagination -->
  <footer class="table-footer">
    <span class="results-text">
      Showing {{ firstVisible }} to {{ lastVisible }}
      of {{ filteredReports.length }} reports
    </span>

    <div class="pagination">
      <button
        type="button"
        class="page-arrow"
        aria-label="Previous page"
        [disabled]="currentPage === 1"
        (click)="changePage(currentPage - 1)"
      >
        ‹
      </button>

      @for (page of [].constructor(totalPages); track $index) {
        <button
          type="button"
```

```
            class="page-number"
            [class.active]="currentPage === $index + 1"
            (click)="changePage($index + 1)"
          >
            {{ $index + 1 }}
          </button>
        }

        <button
          type="button"
          class="page-arrow"
          aria-label="Next page"
          [disabled]="currentPage === totalPages"
          (click)="changePage(currentPage + 1)"
        >
          ›
        </button>
      </div>
    </footer>
  </section>

  <!-- Report damage form -->
  @if (showReportForm) {
    <div class="modal-backdrop" (click)="showReportForm = false">
      <section class="modal-card" (click)="$event.stopPropagation()">
        <h2>Report Damage</h2>
        <p>Enter the details of the damaged equipment.</p>

        <form #damageForm="ngForm" (ngSubmit)="saveReport()">
          <label for="equipmentName">Equipment name *</label>
          <input
            id="equipmentName"
            name="equipmentName"
            [(ngModel)]="newReport.equipment"
            required
            placeholder="e.g. GPS Rover - 01"
          />

          <label for="damageDescription">Description *</label>
          <textarea
            id="damageDescription"
            name="damageDescription"
            [(ngModel)]="newReport.description"
            required
            rows="3"
            placeholder="Describe the damage..."
```

```
          ></textarea>

          <label for="reportedBy">Reported by</label>
          <input
            id="reportedBy"
            name="reportedBy"
            [(ngModel)]="newReport.reportedBy"
          />

          <div class="modal-actions">
            <button
              type="button"
              class="cancel-button"
              (click)="showReportForm = false"
            >
              Cancel
            </button>

            <button
              type="submit"
              class="report-button"
              [disabled]="damageForm.invalid"
            >
              Submit Report
            </button>
          </div>
        </form>
      </section>
    </div>
  }

  <!-- Report details -->
  @if (selectedReport; as report) {
    <div class="modal-backdrop" (click)="closeDetails()">
      <section class="modal-card details-card" (click)="$event.stopPropagation()">
        <div class="details-heading">
          <h2>Damage Report Details</h2>
          <button type="button" class="close-button" (click)="closeDetails()">×</button>
        </div>

        <p><strong>Equipment:</strong> {{ report.equipment }}</p>
        <p><strong>Serial number:</strong> {{ report.serial }}</p>
        <p><strong>Date:</strong> {{ report.date }}</p>
        <p><strong>Description:</strong> {{ report.description }}</p>
        <p><strong>Reported by:</strong> {{ report.reportedBy }}</p>
        <p><strong>Status:</strong> {{ report.status }}</p>
```

```
      </section>
    </div>
  }

</div>
```

4. CSS — `damage-reports.css`

Replace `src/app/components/damage-reports/damage-reports.css` with:

```
:host {
  display: block;
  min-width: 0;
  min-height: 100%;
  color: #17243a;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 14px;
}

* {
  box-sizing: border-box;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  cursor: pointer;
}

.damage-page {
  min-height: calc(100vh - 62px);
  background: #f7f9fc;
}

.damage-header {
  display: flex;
  align-items: center;
  gap: 14px;
  min-height: 95px;
  margin: 0 -24px;
  padding: 14px 24px;
```

```
  border-bottom: 1px solid #e7edf5;
  background: #fff;
}

.heading-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 60px;
  height: 60px;
  flex-shrink: 0;
  border-radius: 50%;
  background: #ffebed;
  color: #ed545c;
  font-size: 38px;
  font-weight: 700;
}

.heading-text {
  min-width: 0;
}

.heading-text h1 {
  margin: 0 0 5px;
  color: #111e35;
  font-size: 24px;
  font-weight: 700;
}

.heading-text p {
  margin: 0;
  color: #53658a;
  font-size: 14px;
}

.report-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 40px;
  padding: 0 17px;
  border: 1px solid #0868f7;
  border-radius: 7px;
  background: #0868f7;
  color: #fff;
```

```
  font-weight: 600;
  white-space: nowrap;
}

.report-button:hover:not(:disabled) {
  background: #0757d5;
}

.report-button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

.plus {
  font-size: 21px;
  line-height: 1;
}

.damage-header > .report-button {
  margin-left: auto;
}

.filter-bar {
  display: grid;
  grid-template-columns: minmax(150px, 1.6fr)
    minmax(110px, 0.8fr)
    minmax(180px, 1.3fr)
    auto auto;
  align-items: center;
  gap: 12px;
  padding: 16px 0;
}

.search-box,
.status-filter,
.date-range {
  display: flex;
  align-items: center;
  min-width: 0;
  height: 41px;
  border: 1px solid #dce4ef;
  border-radius: 7px;
  background: #fff;
}

.search-box {
```

```
  gap: 10px;
  padding: 0 13px;
}

.search-icon {
  color: #77869a;
  font-size: 25px;
  line-height: 1;
}

.search-box input {
  width: 100%;
  min-width: 0;
  border: 0;
  outline: none;
  background: transparent;
  color: #263448;
  font-size: 13px;
}

.search-box input::placeholder {
  color: #7b899e;
}

.status-filter {
  width: 100%;
  padding: 0 12px;
  color: #263448;
  outline: none;
  font-size: 13px;
}

.date-range {
  gap: 7px;
  padding: 0 10px;
}

.calendar-icon {
  color: #52647c;
  font-size: 19px;
}

.date-range input {
  width: 100%;
  min-width: 0;
  border: 0;
```

```
  outline: none;
  background: transparent;
  color: #35445a;
  font-size: 11px;
}

.date-separator {
  color: #94a0b0;
}

.filter-button,
.reset-button {
  height: 41px;
  padding: 0 17px;
  border-radius: 7px;
  font-size: 13px;
  font-weight: 600;
}

.filter-button {
  border: 1px solid #0868f7;
  background: #0868f7;
  color: #fff;
}

.reset-button {
  border: 1px solid #dce4ef;
  background: #fff;
  color: #46566d;
}

.table-card {
  overflow: hidden;
  border: 1px solid #edf1f6;
  border-radius: 7px;
  background: #fff;
  box-shadow: 0 2px 7px #1b355006;
}

.table-scroll {
  width: 100%;
  overflow-x: auto;
}

table {
  width: 100%;
```

```
  border-collapse: collapse;
  text-align: left;
}

thead {
  background: #f3f6fa;
}

th {
  height: 48px;
  padding: 0 13px;
  color: #152138;
  font-size: 13px;
  font-weight: 700;
  white-space: nowrap;
}

td {
  height: 67px;
  padding: 8px 13px;
  border-bottom: 1px solid #edf1f5;
  color: #17243a;
  font-size: 13px;
  white-space: nowrap;
}

tbody tr:last-child td {
  border-bottom: 0;
}

tbody tr:hover {
  background: #fafcff;
}

.date-cell {
  font-weight: 500;
}

.equipment-cell {
  display: flex;
  align-items: center;
  gap: 12px;
  min-width: 175px;
}

.equipment-icon {
```

```
  display: flex;
  align-items: center;
  justify-content: center;
  width: 45px;
  height: 45px;
  flex-shrink: 0;
  border-radius: 5px;
  background: #fff;
}

.equipment-icon svg {
  width: 37px;
  height: 40px;
}

.status-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-width: 76px;
  min-height: 29px;
  padding: 5px 12px;
  border-radius: 20px;
  font-size: 12px;
  font-weight: 600;
}

.status-badge.pending {
  background: #ffdcde;
  color: #df252c;
}

.status-badge.resolved {
  background: #c8f4df;
  color: #087c4d;
}

.actions-heading,
.actions-cell {
  text-align: center;
}

.view-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
```

```
  width: 34px;
  height: 30px;
  border: 1px solid #0868f7;
  border-radius: 6px;
  background: #0868f7;
  color: #fff;
}

.view-button:hover {
  background: #0757d5;
}

.empty-cell {
  height: 100px;
  color: #718096;
  text-align: center;
}

.table-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 14px;
  min-height: 73px;
  padding: 12px;
  border-top: 1px solid #f0f3f7;
}

.results-text {
  color: #26364d;
  font-size: 13px;
}

.pagination {
  display: flex;
  align-items: center;
  gap: 8px;
}

.page-arrow,
.page-number {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 37px;
  height: 38px;
```

```
  border: 0;
  border-radius: 7px;
  font-size: 17px;
}

.page-arrow {
  background: #f0f3f7;
  color: #687990;
}

.page-arrow:disabled {
  opacity: 0.65;
  cursor: not-allowed;
}

.page-number {
  background: #0868f7;
  color: #fff;
  font-size: 14px;
}

.page-number.active {
  background: #0868f7;
  color: #fff;
}

/* Modal forms and report details */

.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 2000;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  background: #17243a80;
}

.modal-card {
  width: 100%;
  max-width: 440px;
  max-height: 90vh;
  overflow-y: auto;
  padding: 23px;
  border: 1px solid #e5eaf1;
```

```
  border-radius: 10px;
  background: #fff;
  box-shadow: 0 15px 50px #10233a30;
}

.modal-card h2 {
  margin: 0 0 8px;
  font-size: 20px;
}

.modal-card > p {
  margin: 0 0 17px;
  color: #69788d;
  font-size: 13px;
  line-height: 1.5;
}

.modal-card form {
  display: flex;
  flex-direction: column;
  gap: 9px;
}

.modal-card form label {
  margin-top: 7px;
  font-size: 12px;
  font-weight: 600;
}

.modal-card form input,
.modal-card form textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #dce4ef;
  border-radius: 6px;
  outline: none;
  resize: vertical;
}

.modal-card form input:focus,
.modal-card form textarea:focus {
  border-color: #2878df;
}

.modal-actions {
  display: flex;
```

```
  justify-content: flex-end;
  gap: 9px;
  margin-top: 12px;
}

.cancel-button {
  padding: 10px 16px;
  border: 1px solid #dce4ef;
  border-radius: 6px;
  background: #f4f6f9;
  color: #344155;
}

.details-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 18px;
}

.details-heading h2 {
  margin: 0;
}

.close-button {
  width: 32px;
  height: 32px;
  border: 0;
  border-radius: 6px;
  background: #f0f3f7;
  color: #344155;
  font-size: 23px;
}

.details-card > p {
  margin: 12px 0;
  color: #4c5b70;
}

.details-card strong {
  color: #1d2a40;
}

@media (max-width: 1000px) {
  .filter-bar {
```

```
    grid-template-columns: minmax(0, 1fr) minmax(110px, 0.7fr);
  }

  .date-range {
    grid-column: 1;
  }

  .filter-button,
  .reset-button {
    grid-row: 2;
  }

  .filter-button {
    grid-column: 2;
  }

  .reset-button {
    grid-column: 2;
    margin-top: 46px;
  }
}

@media (max-width: 650px) {
  .damage-header {
    flex-wrap: wrap;
    margin-right: -13px;
    margin-left: -13px;
    padding: 14px;
  }

  .heading-icon {
    width: 45px;
    height: 45px;
    font-size: 29px;
  }

  .heading-text {
    flex: 1;
  }

  .heading-text h1 {
    font-size: 19px;
  }

  .heading-text p {
    font-size: 12px;
```

```
    line-height: 1.5;
  }

  .damage-header > .report-button {
    width: 100%;
    margin-left: 0;
  }

  .filter-bar {
    grid-template-columns: 1fr 1fr;
    gap: 9px;
  }

  .search-box {
    grid-column: 1 / -1;
  }

  .date-range {
    grid-column: 1 / -1;
  }

  .filter-button,
  .reset-button {
    grid-row: auto;
    grid-column: auto;
    margin-top: 0;
  }

  th,
  td {
    padding-right: 12px;
    padding-left: 12px;
  }

  .table-footer {
    align-items: flex-start;
    flex-direction: column;
    padding: 14px;
  }

  .pagination {
    align-self: flex-end;
  }
}
```

5. Make one small pagination adjustment

To ensure Angular handles pagination cleanly, add this getter inside the `DamageReports` class in `damage-reports.ts`:

```
get pageNumbers(): number[] {
  return Array.from(
    { length: this.totalPages },
    (_, index) => index + 1
  );
}
```

Then, in `damage-reports.html`, replace:

```
@for (page of [].constructor(totalPages); track $index) {
```

with:

```
@for (page of pageNumbers; track page) {
```

Inside that loop, use `[class.active]="currentPage === page"` and `(click)="changePage(page)"`, and display `{{ page }}`.

6. Add the route in `app.routes.ts`

Import the component:

```
import { DamageReports } from './components/damage-reports/damage-reports';
```

Add it to your existing Dashboard layout's `children` array:

```
{ path: 'damage-reports', component: DamageReports },
```

Also ensure your Dashboard sidebar mapping contains:

```
'Damage Reports': '/damage-reports',
```

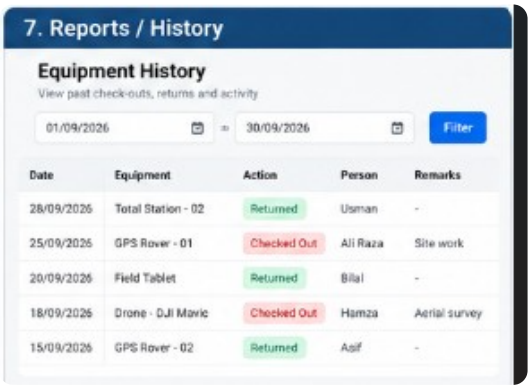
This keeps your existing Horizon sidebar visible when you navigate to Damage Reports.

7. Run the project

```
ng serve --open
```

Open `http://localhost:4200/damage-reports` or click **Damage Reports** in your sidebar.

The page will display the four sample reports from your screenshot. Search, status and date filtering, pagination, the report form, and the details modal work locally. Saving a new report to your database and retrieving actual damage records will require connecting your backend API.



please generate the code of this image in angular ??

24/10/2026, 14:38:21

I'll create the **Reports / History** page for your SolvefyTask Angular application based on the screenshot. It will match your existing Horizon design and keep the fixed sidebar visible.

The page will include an Equipment History table, start and end date filters, a Filter button, and columns for Date, Equipment, Action, Person, and Remarks.

1. Generate the component

Run this command in your Angular project terminal:

```
ng generate component components/reports-history
```

This creates the component files under `src/app/components/reports-history/`.

2. TypeScript — `reports-history.ts`

Replace `src/app/components/reports-history/reports-history.ts` with:

```ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface EquipmentHistoryRecord {
  id: number;
  date: string;
  equipment: string;
  action: 'Returned' | 'Checked Out';
  person: string;
  remarks: string;
}

@Component({
```

```
  selector: 'app-reports-history',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './reports-history.html',
  styleUrl: './reports-history.css'
})
export class ReportsHistory {
  startDate = '2026-09-01';
  endDate = '2026-09-30';

  appliedStartDate = this.startDate;
  appliedEndDate = this.endDate;

  history: EquipmentHistoryRecord[] = [
    {
      id: 1,
      date: '28/09/2026',
      equipment: 'Total Station - 02',
      action: 'Returned',
      person: 'Usman',
      remarks: ''
    },
    {
      id: 2,
      date: '25/09/2026',
      equipment: 'GPS Rover - 01',
      action: 'Checked Out',
      person: 'Ali Raza',
      remarks: 'Site work'
    },
    {
      id: 3,
      date: '20/09/2026',
      equipment: 'Field Tablet',
      action: 'Returned',
      person: 'Bilal',
      remarks: ''
    },
    {
      id: 4,
      date: '18/09/2026',
      equipment: 'Drone - DJI Mavic',
      action: 'Checked Out',
      person: 'Hamza',
      remarks: 'Aerial survey'
    },
```

```
    {
      id: 5,
      date: '15/09/2026',
      equipment: 'GPS Rover - 02',
      action: 'Returned',
      person: 'Asif',
      remarks: ''
    }
  ];

  get filteredHistory(): EquipmentHistoryRecord[] {
    return this.history.filter(record => {
      const recordDate = this.toISODate(record.date);

      const matchesStart =
        !this.appliedStartDate ||
        recordDate >= this.appliedStartDate;

      const matchesEnd =
        !this.appliedEndDate ||
        recordDate <= this.appliedEndDate;

      return matchesStart && matchesEnd;
    });
  }

  private toISODate(date: string): string {
    const [day, month, year] = date.split('/');
    return `${year}-${month}-${day}`;
  }

  applyFilter(): void {
    if (
      this.startDate &&
      this.endDate &&
      this.startDate > this.endDate
    ) {
      alert('Start date cannot be later than end date.');
      return;
    }

    this.appliedStartDate = this.startDate;
    this.appliedEndDate = this.endDate;
  }

  resetFilter(): void {
```

```
    this.startDate = '';
    this.endDate = '';
    this.appliedStartDate = '';
    this.appliedEndDate = '';
  }
}
```

3. HTML — `reports-history.html`

Replace `src/app/components/reports-history/reports-history.html` with:

```
<div class="history-page">

  <!-- Page banner -->
  <div class="page-banner">
    7. Reports / History
  </div>

  <section class="history-card">

    <!-- Heading -->
    <header class="history-header">
      <div>
        <h1>Equipment History</h1>
        <p>View past check-outs, returns and activity</p>
      </div>
    </header>

    <!-- Date filters -->
    <form class="filter-bar" (ngSubmit)="applyFilter()">

      <div class="date-control">
        <label for="startDate" class="visually-hidden">
          Start date
        </label>

        <input
          id="startDate"
          type="date"
          name="startDate"
          [(ngModel)]="startDate"
        />
      </div>

      <span class="date-separator">to</span>
```

```
      <div class="date-control">
        <label for="endDate" class="visually-hidden">
          End date
        </label>

        <input
          id="endDate"
          type="date"
          name="endDate"
          [(ngModel)]="endDate"
        />
      </div>

      <button type="submit" class="filter-button">
        Filter
      </button>

      <button
        type="button"
        class="reset-button"
        (click)="resetFilter()"
      >
        Reset
      </button>

    </form>

    <!-- History table -->
    <div class="table-scroll">
      <table>
        <thead>
          <tr>
            <th>Date</th>
            <th>Equipment</th>
            <th>Action</th>
            <th>Person</th>
            <th>Remarks</th>
          </tr>
        </thead>

        <tbody>
          @for (record of filteredHistory; track record.id) {
            <tr>
              <td class="date-cell">{{ record.date }}</td>

              <td class="equipment-cell">
```

```
              {{ record.equipment }}
            </td>

            <td>
              <span
                class="action-badge"
                [class.returned]="record.action === 'Returned'"
                [class.checked-out]="record.action === 'Checked Out'"
              >
                {{ record.action }}
              </span>
            </td>

            <td>{{ record.person }}</td>

            <td class="remarks-cell">
              {{ record.remarks || '—' }}
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="5" class="empty-state">
              No equipment history found for the selected dates.
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <!-- Footer -->
  <footer class="history-footer">
    <span>
      Showing {{ filteredHistory.length }}
      of {{ history.length }} records
    </span>

    <button
      type="button"
      class="reset-link"
      (click)="resetFilter()"
    >
      Clear filters
    </button>
  </footer>
```

```
  </section>
</div>
```

4. CSS — `reports-history.css`

Replace `src/app/components/reports-history/reports-history.css` with:

```
:host {
  display: block;
  min-width: 0;
  min-height: 100%;
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 12px;
}

* {
  box-sizing: border-box;
}

button,
input {
  font: inherit;
}

button {
  cursor: pointer;
}

.history-page {
  min-height: calc(100vh - 62px);
  background: #f7f9fc;
}

.page-banner {
  margin: 0 -24px;
  padding: 8px 14px;
  min-height: 33px;
  border-radius: 0 0 5px 5px;
  background: #205780;
  color: #fff;
  font-size: 13px;
  font-weight: 700;
}

.history-card {
```

```
  margin-top: 0;
  padding: 0 10px 10px;
  overflow: hidden;
  border: 1px solid #edf1f6;
  border-radius: 7px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.history-header {
  padding: 10px 5px 8px;
}

.history-header h1 {
  margin: 0 0 4px;
  color: #202a39;
  font-size: 16px;
  font-weight: 700;
}

.history-header p {
  margin: 0;
  color: #8994a4;
  font-size: 10px;
}

.filter-bar {
  display: flex;
  align-items: center;
  gap: 7px;
  margin-bottom: 9px;
}

.date-control {
  min-width: 0;
  width: 175px;
}

.date-control input {
  width: 100%;
  height: 29px;
  padding: 4px 7px;
  border: 1px solid #e0e6ee;
  border-radius: 5px;
  outline: none;
  background: #fff;
```

```
  color: #465469;
  font-size: 10px;
}

.date-control input:focus {
  border-color: #2878df;
}

.date-separator {
  color: #7c8797;
  font-size: 10px;
}

.filter-button {
  min-width: 52px;
  height: 29px;
  padding: 0 12px;
  border: 1px solid #0868e8;
  border-radius: 5px;
  background: #0868e8;
  color: #fff;
  font-size: 10px;
  font-weight: 600;
}

.filter-button:hover {
  background: #0756c4;
}

.reset-button {
  height: 29px;
  padding: 0 9px;
  border: 1px solid #dce4ef;
  border-radius: 5px;
  background: #fff;
  color: #526176;
  font-size: 10px;
}

.table-scroll {
  width: 100%;
  overflow-x: auto;
}

table {
  width: 100%;
```

```
  border-collapse: collapse;
  table-layout: auto;
  text-align: left;
}

thead {
  background: #f4f6fa;
}

th {
  height: 27px;
  padding: 6px 7px;
  color: #283447;
  font-size: 10px;
  font-weight: 700;
  white-space: nowrap;
}

td {
  height: 27px;
  padding: 5px 7px;
  border-bottom: 1px solid #edf1f5;
  color: #39465a;
  font-size: 10px;
  white-space: nowrap;
}

tbody tr:last-child td {
  border-bottom: 0;
}

tbody tr:hover {
  background: #f9fbfe;
}

.date-cell {
  color: #455166;
}

.equipment-cell {
  color: #29364a;
}

.action-badge {
  display: inline-flex;
  align-items: center;
```

```
  justify-content: center;
  min-height: 19px;
  padding: 3px 7px;
  border-radius: 5px;
  font-size: 9px;
  white-space: nowrap;
}

.action-badge.returned {
  background: #d8f6e6;
  color: #087c4d;
}

.action-badge.checked-out {
  background: #ffe0e0;
  color: #c82d36;
}

.remarks-cell {
  color: #4c596b;
}

.empty-state {
  height: 65px;
  color: #8793a4;
  text-align: center;
}

.history-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 10px 5px 0;
  color: #7c8797;
  font-size: 10px;
}

.reset-link {
  padding: 4px 7px;
  border: 0;
  background: transparent;
  color: #1769c2;
  font-size: 10px;
}
```

```css
.reset-link:hover {
  text-decoration: underline;
}

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  clip-path: inset(50%);
  border: 0;
}

@media (max-width: 650px) {
  .page-banner {
    margin-right: -13px;
    margin-left: -13px;
  }

  .history-card {
    padding-right: 8px;
    padding-left: 8px;
  }

  .filter-bar {
    flex-wrap: wrap;
  }

  .date-control {
    flex: 1 1 40%;
    width: auto;
  }

  .filter-button,
  .reset-button {
    flex: 1;
  }

  table {
    min-width: 520px;
  }
}
```

5. Add the route in `app.routes.ts`

Import the component:

```
import { ReportsHistory } from './components/reports-history/reports-history';
```

Add it to your existing Dashboard layout's `children` array:

```
{ path: 'reports', component: ReportsHistory },
```

Also check your `dashboard.ts` route mapping:

```
'Reports': '/reports',
```

Keep your existing Dashboard parent route so the Horizon sidebar remains visible.

6. Run the project

```
ng serve --open
```

Open `http://localhost:4200/reports` or click **Reports** in the sidebar.

The page will show the five sample records from your screenshot. Date filtering, reset, and the action badges work locally. When you're ready, you can connect the history records to your backend API so actual check-out and return activities appear automatically.

24/10/2026, 14:38:34 · gpt-6

SOLVEFY · AI ENGINEER HIRING FRAMEWORK
03-test-task-associate
Internal · October 2026
Solvefy — Test Task
AI Engineer I (Associate) · 0–1 year experience
Time to complete: 7 calendar days from the date you receive this. Submission deadline: 24 hours before your scheduled interview.
We are not going to tell you how many hours to spend. That decision is yours, and what you do with it is part of what we are looking at. All we ask is that you tell us, in your README, roughly how long you spent. There is no right answer, and a bigger submission is not automatically a better one — deciding how much to build is a real engineering judgement and we would rather see yours than give you ours.
1. Read this part first
This is not a test of whether you can write code without AI. We expect you to use AI for most of the typing. We are evaluating how you work — how you turn a vague problem into a clear one, how you decide what to build, how you direct AI, and how much you understand of what comes back.
Three things follow from that:
You must use the BMAD Method. Use it to build your requirements and your context before you build the software. BMAD — the Breakthrough Method for Agile AI-Driven Development — is an open method, and we have given you the starting points below. How far you get with it in seven days tells us something we cannot learn any other way.
Where to start with BMAD
Official masterclass video
Official channel
Documentation
Source repository
https://www.youtube.com/watch?v=LorEJPrALcg
https://www.youtube.com/@BMadCode
https://docs.bmad-method.org/
https://github.com/bmad-code-org/BMAD-METHOD
Start with the masterclass video — it is the clearest single explanation of the whole workflow. One thing to be aware of: that video is over a year old and the method has moved on since it was recorded, so some of what

you see will not match the current documentation or the current release. Work out what has changed and follow the current version. Noticing that for yourself is part of the exercise.

You must submit every AI conversation you used. Complete and unedited, including the parts that went badly. A submission without conversation logs will not be interviewed. We are not looking for perfect prompts — we are looking at how you recover when the AI gets it wrong, which is the most useful thing we can possibly see.

The requirements below are deliberately incomplete. This is what real client requirements look like. Some things are vague, some are missing, and at least one thing contradicts something else. Do not contact us for clarification — we will not answer. Make your own decisions, write them down, and be ready to defend them.

In the interview we will ask you why you chose option A over option B. "The requirements didn't say" is not an answer we accept; deciding well with incomplete information is the actual job.

2. The client

Horizon Survey Services — a land-surveying company in Lahore. About 25 people, of whom roughly 14 are field staff. They own expensive equipment: total stations, GPS rovers, tripods, drones, laptops, field tablets.

You have two pieces of input from them. That is all you get.

2.1 Transcript of a voice note from the owner, Mr Tariq

"So basically the problem is the equipment. We have a lot of equipment, it is expensive, and I never know where any of it is. The boys come in the morning, they pick up what they need, they go to site. Fine. Then somebody else needs the same thing and nobody knows who has it. Last month we bought a second GPS rover for eleven lakh because we thought ours was lost. It was in somebody's car boot for three weeks.

So I want to know who has what. That's it really. And I want it back on time, people keep things for weeks when they needed it for one day.

Anyone in the team can take anything, we don't have a system, it's all on trust, and mostly that's fine, I don't want to make it like the army. But I want to see it on a screen.

Sometimes they take the whole survey kit, sometimes just one piece. Oh and the office needs to know too, Saima in the office gets the calls asking where things are, she

is the one suffering.
One more thing — the boys are not going to use anything complicated. Half of them just use WhatsApp,
that's it. If it takes more than a few seconds they won't do it, and then we're back where we started."
2.2 Follow-up email from Mr Tariq, two days later
Subject: RE: equipment thing
Hi,
Forgot to mention a few things.
The drone is different, that needs Hamza's approval before anyone takes it, he is the only licensed operator.
Same for the new rover maybe, I will decide later.
We have 4 tripods and 6 field tablets, those are not individually tracked, we just need to know how many
are left.
Also sometimes equipment comes back damaged and nobody says anything until the next person takes it
to site and finds out. That has cost us two days of client work this year.
It would be good if the system reminds people. We mostly use WhatsApp for everything here.
And I want to be able to look back at last month and see what was out and who had it.
Thanks Tariq
3. What you have to build
A working application that addresses Horizon's problem, built with:
Backend: .NET 8 or .NET 9, C#, ASP.NET Core Web API
Data: any relational database — SQLite or SQL Server LocalDB is completely fine, and is the sensible choice
for a task this size
Frontend: your choice, including plain HTML and JavaScript. It must be usable, but we are not assessing
visual design. A beautiful UI over a confused data model scores worse than a plain UI over a sound one.
Running: one documented command, or a short, accurate list of steps in the README. If we cannot run it in
five minutes, we review it as if it does not run.
Scope guidance. Not everything mentioned above has to exist in working code. Deciding what to build first,
and being able to defend that order, is part of what we are assessing. What you deliberately left out — and why
— is as interesting to us as what you built.
4. What you must submit
A single repository (private GitHub repo with access for us, or a zip file) laid out like this:

/src
/docs
/conversations
/DECISIONS.md
/README.md
your application code
your BMAD artifacts — brief, PRD, epics, stories, architecture
every AI conversation, exported in full
your assumptions and decisions log (see below)
how to run it; what is done; what is not done; what you cut and
why; roughly how many hours you spent; which AI tools you used
4.1
DECISIONS.md
 — the most important file in your submission
For every point where the requirements did not tell you what to do, record:
Field
What to write
The gap
What I decided
The alternative
Why this one
What was unclear, missing or contradictory — quote the line
The decision you made
The other reasonable option you considered
Your reasoning
What would change my mind
What would have to be true for the other option to win
We expect somewhere between 8 and 20 of these. If you have two, you did not read carefully enough. If you
have sixty, you did not prioritise.
4.2
/conversations
 — all of it
Export every AI conversation you used for any part of this task: requirements, BMAD, design, code, debugging, documentation. Markdown, text or JSON — any readable format.
Do not clean them up. Do not delete the parts where it went wrong. The conversation where you spent an hour
going in circles is more useful to us than the one that worked first time.
5. Rules
1. Do not contact us with questions about the requirements.

Email HR only about logistics — deadline,
format, access problems.
2. Use any AI tools you like. Tell us in the README which ones you used and for what.
3. The work must be yours in the sense that you directed it and you understand it. In the interview we will ask
you to explain any part of the code and to change it live. If you cannot navigate your own submission, the
submission does not count.
4. Tell us in the README roughly how many hours you spent. We are not checking it against a target — there
isn't one. We just want your own account of it alongside the work.
Good luck.

please remember this dont change anything ??

24/10/2026, 15:29:12

I'll keep this Solvefy AI Engineer I test task as your original reference and **will not change, rewrite, or reinterpret its requirements** when we work on your project.

We'll refer back to the exact task you provided whenever we make decisions about the application, BMAD artifacts, code, or submission.

24/10/2026, 15:29:12 · gpt-6

please help me write the decisions such that what i have taken during my frontend development ??

24/10/2026, 15:30:04

Based on the frontend development we have done for **SolvefyTask — Horizon Survey Services**, we can document the decisions you actually made while building the Angular application.

I'll keep these focused on your frontend work: Angular standalone components, routing, the fixed Horizon sidebar, separate equipment pages, UI interactions, sample data, and responsive styling. I'll distinguish decisions supported by our development conversation from assumptions that still need your confirmation.

One important point: these are your **frontend decisions**, not a claim that the complete .NET backend, database, or business workflows have already been implemented.

Here is a draft you can place in your repository as `DECISIONS.md`. It records the choices reflected in your current frontend development and identifies where a decision is still provisional.

DECISIONS.md

SolvefyTask — Horizon Survey Services

Project: Equipment Tracking System
Frontend: Angular
Status: Initial frontend decisions — review and update as implementation progresses

This document records the decisions made during frontend development based on the supplied hiring task and the current Angular implementation. These decisions are limited to the frontend unless explicitly stated otherwise. UI demonstrations must not be treated as completed backend functionality.

D-01: Use Angular for the Frontend

The gap

The task specifies the backend technology but does not prescribe a frontend framework.

> *"Frontend: your choice, including plain HTML and JavaScript. It must be usable, but we are not assessing visual design."*

What I decided

I chose Angular for the frontend and created a project using Angular CLI.

The alternative

Use plain HTML, CSS, and JavaScript, or choose another frontend framework.

Why this one

Angular provides a component-based structure, reusable templates, TypeScript support, and built-in routing. These features help organize the different equipment-management screens into maintainable units.

What would change my mind

If the application became substantially simpler than expected, or the Angular setup created unnecessary complexity for the required functionality, I would reconsider the framework.

D-02: Use Standalone Components

The gap

The task does not specify whether Angular components should use standalone architecture or NgModules.

What I decided

I used Angular standalone components, with component metadata defining their imports, templates, and styles.

The alternative

Use NgModules to organize components and their dependencies.

Why this one

Standalone components reduce module boilerplate and allow individual pages to declare the Angular features they need. This fits the relatively small application and its multiple independent screens.

What would change my mind

If the application grew into a larger architecture with specific module-based integration requirements, I would reassess the organization.

D-03: Use Angular Routing for Page Navigation

The gap

The task identifies several equipment-management needs but does not prescribe how users should navigate between them.

What I decided

I used Angular Router to navigate between pages such as Dashboard, Equipment, Check Out, Check In, Approvals, My Equipment, Damage Reports, and Reports/History.

The alternative

Build a single component and manually show or hide different page sections.

Why this one

Routing separates navigation from individual page implementations and provides distinct URLs for different screens. It also makes it easier to maintain and test each page.

What would change my mind

If the application were reduced to a single screen with no meaningful navigation, a simpler component structure could be sufficient.

D-04: Keep the Sidebar in a Persistent Layout

The gap

The task requires equipment visibility for field staff and office staff but does not specify the application's layout or navigation structure.

What I decided

I placed the sidebar in a shared application layout and used a router outlet for the changing page content. The sidebar should remain visible when navigating between child routes.

The alternative

Place the sidebar separately inside each page component or recreate it whenever a page changes.

Why this one

A shared layout avoids duplicating navigation markup and keeps the navigation consistent across the application. It also addresses the requirement for a simple, consistent interface.

What would change my mind

If a particular page needed a genuinely different application layout, I would consider a separate layout for that page rather than duplicating the existing sidebar unnecessarily.

D-05: Fix the Sidebar Position

The gap

The task does not state whether the sidebar should scroll with the content or remain fixed.

What I decided

I used CSS fixed positioning for the desktop sidebar and reserved space for it in the main content area.

The alternative

Use a normal document-flow sidebar or a sticky sidebar.

Why this one

A fixed sidebar keeps the navigation available while users scroll through equipment records or reports. This supports quick access to the application's main sections.

What would change my mind

If usability testing showed that a fixed sidebar consumed too much space on smaller devices, I would introduce a collapsible or drawer-based mobile navigation.

D-06: Separate Equipment Functions into Dedicated Pages

The gap

The task describes several related workflows but does not prescribe how they should be divided into frontend screens.

What I decided

I created separate components for the main screens, including equipment listing, checkout, check-in, approvals, assigned equipment, damage reporting, and equipment history.

The alternative

Combine all functions into one large equipment-management component.

Why this one

Separate components make the interface easier to understand and allow each workflow to evolve independently. They also make it easier to connect individual screens to corresponding backend endpoints later.

What would change my mind

If two screens proved to have identical responsibilities and no meaningful independent behavior, I would consider combining them.

D-07: Use a Consistent Visual Design

The gap

The task explicitly says that visual design is not being assessed, but it still requires a usable application. It does not provide a design system.

What I decided

I used a consistent Horizon-style interface with a sidebar, page headings, cards, tables, status badges, and a restrained blue-and-white color scheme.

The alternative

Use unstyled HTML controls or introduce a comprehensive third-party UI library.

Why this one

A consistent interface makes related screens feel like one application and helps users recognize navigation, actions, and statuses. I prioritized clarity and usability over decorative complexity.

What would change my mind

If the interface became difficult to maintain or accessibility testing identified problems, I would revise the styles or adopt an appropriate component library.

D-08: Keep Page Styles Separate

The gap

The task does not prescribe how frontend styling should be organized.

What I decided

I used component-specific CSS files for the individual pages and shared layout styling for the application shell.

The alternative

Place all application styles in a single global stylesheet or adopt a CSS framework.

Why this one

Component-specific styles make it easier to identify which rules belong to which screen and reduce unintended styling changes across the application.

What would change my mind

If the same styling rules were repeatedly duplicated across many components, I would extract genuinely shared rules into reusable styles or design tokens.

D-09: Use Sample Data During Initial UI Development

The gap

The task describes real equipment, staff, checkout activity, and historical records but does not provide an existing API or dataset.

What I decided

I used representative sample records to build and demonstrate the frontend screens before connecting them to persistent backend data.

The alternative

Wait for the backend and database to be completed before developing the frontend, or introduce API integration before establishing the screen structure.

Why this one

Sample data allowed me to develop the page layouts and initial interactions independently. It also made the expected data structure visible during early development.

What would change my mind

Once the backend API and database are ready, I will replace sample records with actual API responses and test loading, empty, error, and update states. Sample records will remain only where deliberately needed for tests or demonstrations.

D-10: Implement Basic Interactions Before Backend Integration

The gap

The task requires a working application but does not specify the implementation order for individual frontend and backend features.

What I decided

I implemented initial UI interactions, including navigation, local filtering, status presentation, modal forms, and demonstration actions, before completing backend integration.

The alternative

Build the frontend only after all business logic and backend endpoints have been implemented.

Why this one

Implementing basic interactions early helps expose usability and component-structure problems. However, a local UI interaction is not evidence that the corresponding business operation has been persisted.

What would change my mind

As soon as the relevant API endpoints are available, I will connect these interactions to server-side operations and add appropriate success, failure, and validation handling.

D-11: Use Client-Side Filtering for the Initial Screens

The gap

The task asks for visibility into equipment and historical activity but does not specify whether filtering should be performed by the frontend or backend.

What I decided

For the initial sample-data implementation, I performed basic searching and date/status filtering in Angular.

The alternative

Send filter parameters to the backend and retrieve filtered results from the database.

Why this one

The local implementation is straightforward for a small set of sample records and allows the filtering interface to be developed independently.

What would change my mind

For actual records, I would evaluate server-side filtering and pagination to ensure consistent date handling, suitable performance, and access to the complete dataset. The final approach will depend on the API and expected data volume.

D-12: Use Simple Equipment and Status Representations

The gap

The task names multiple equipment types and asks for quick, uncomplicated use, but it does not specify the visual representation of equipment or its status.

What I decided

I used readable equipment names, simple icons or illustrations, and distinct status badges such as Checked Out, Returned, Pending, and Resolved.

The alternative

Use equipment photographs, detailed asset cards, or text-only tables without visual status indicators.

Why this one

Simple icons and status badges make records easier to scan without adding unnecessary complexity. The actual equipment identity and status remain visible as text rather than relying on color alone.

What would change my mind

If users found the icons ambiguous, I would replace or supplement them with clearer labels and real equipment images where useful. I would also review accessibility and contrast.

D-13: Treat Frontend Approval and Return Actions as Demonstrations Until Connected

The gap

The task requires approval for drone checkout and accountability for equipment returns, but the initial frontend work does not itself establish server-side authorization or persistence.

What I decided

I implemented UI-level actions and demonstrations where needed to develop the screens. These actions must not be considered final business operations until connected to the

backend.

The alternative

Represent the workflows as static screens without interactive actions.

Why this one

Interactive demonstrations help validate the intended user flow. However, approvals, equipment returns, and damage reports must ultimately be validated and persisted by the server.

What would change my mind

The implementation will change when the backend workflow, authorization rules, and database operations are available. In particular, the server must enforce Hamza's approval requirement for drone checkout rather than trusting the frontend.

D-14: Make the Frontend Responsive

The gap

The task does not specify the devices or screen sizes used by office and field staff.

What I decided

I added responsive CSS rules to adjust layouts for narrower screens, including changes to cards, tables, filters, and sidebar behavior.

The alternative

Optimize exclusively for desktop screens.

Why this one

Field staff may access the application on smaller devices. Responsive layouts provide a reasonable starting point for those users while preserving the desktop experience.

What would change my mind

Testing on the actual devices used by Horizon's staff may reveal the need for a more compact field interface, a collapsible sidebar, or simplified mobile workflows.

Current Limitations

The frontend decisions above describe the initial Angular implementation. They do not establish that the complete application is finished.

The following still require implementation or verification as applicable:

- Integration with the .NET Web API and relational database.
- Server-side validation and authorization.

- Persistent equipment checkout, return, approval, and damage-report operations.
- Real equipment history and reliable reporting.
- Reminder delivery and the decision about WhatsApp integration.
- Testing on the devices and workflows used by Horizon's staff.

The frontend should support the agreed business rules rather than becoming the source of truth for equipment ownership, availability, approvals, or history.

Review Note

These decisions should be revisited as the BMAD requirements, architecture, and implementation become more concrete. A decision should be changed when new evidence justifies the change, and the reason should be documented rather than silently replacing the original rationale.

Before submitting this file

Three things are especially important:

1. **Keep only decisions you can explain.** For example, you should be able to explain why Angular Router is useful and how the shared layout keeps the sidebar visible.
2. **Update decisions when the backend is implemented.** The current local demonstration behavior is temporary, not the final equipment-management design.
3. **Add the actual time spent and AI tools to `README.md`.** Do not estimate or invent those details just to complete the document.

This draft contains 14 decisions, within the task's requested range of 8–20. As your implementation develops, we should also document the deeper business and data decisions—such as quantity-based tracking for tripods and tablets, equipment kits, checkout authorization, and overdue reminders—rather than treating the frontend as the entire solution.

24/10/2026, 15:30:07 · gpt-6

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace SolvefyTask.Controllers
{
   [Route("api/[controller]")]
   [ApiController]
   public class EquipmentController : ControllerBase
   {

   }
}
```

please help me write a method that get all the equipments first it calls the service layer and then repository layer and then go to the database that gets the data ??

24/10/2026, 15:46:35

For your **SolvefyTask — Horizon Survey Services** project, a clean approach is to use three layers:

1. **Controller** — receives the HTTP request and returns the HTTP response.
2. **Service layer** — handles application logic and calls the repository.
3. **Repository layer** — queries the database and returns equipment records.

The flow will be:

```
Angular Frontend
      |
      v
EquipmentController
      |
      v
IEquipmentService
      |
      v
EquipmentService
      |
      v
IEquipmentRepository
      |
```

```
      v
EquipmentRepository
      |
      v
Database (SQLite / SQL Server)
```

We'll implement a `GET /api/equipment` endpoint using ASP.NET Core and Entity Framework Core.

1. Create the Equipment model

Create `Models/Equipment.cs`.

```
namespace SolvefyTask.Models
{
    public class Equipment
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;

        public string Category { get; set; } = string.Empty;

        public string SerialNumber { get; set; } = string.Empty;

        public string Type { get; set; } = string.Empty;

        public string Status { get; set; } = "Available";

        public string? AssignedTo { get; set; }
    }
}
```

These are example fields for your equipment-tracking application. Adjust them to match your actual database schema.

2. Configure the DbContext

Create `Data/ApplicationDbContext.cs`.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Models;

namespace SolvefyTask.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
```

```
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Equipment> Equipments => Set<Equipment>();
    }
}
```

If you already have a DbContext, add the `Equipments` property to that context instead of creating another one.

3. Create the repository layer

`Repositories/IEquipmentRepository.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface IEquipmentRepository
    {
        Task<List<Equipment>> GetAllAsync();
    }
}
```

`Repositories/EquipmentRepository.cs`

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class EquipmentRepository : IEquipmentRepository
    {
        private readonly ApplicationDbContext _context;

        public EquipmentRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<Equipment>> GetAllAsync()
        {
            return await _context.Equipments
```

```
                .AsNoTracking()
                .ToListAsync();
        }
    }
}
```

What happens here? `ToListAsync()` executes the database query and retrieves the equipment records. `AsNoTracking()` is suitable for read-only queries because Entity Framework doesn't need to track changes to the returned entities.

4. Create the service layer

`Services/IEquipmentService.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface IEquipmentService
    {
        Task<List<Equipment>> GetAllAsync();
    }
}
```

`Services/EquipmentService.cs`

```
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class EquipmentService : IEquipmentService
    {
        private readonly IEquipmentRepository _repository;

        public EquipmentService(IEquipmentRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<Equipment>> GetAllAsync()
        {
            return await _repository.GetAllAsync();
        }
```

```
    }
}
```

Later, you can put business rules in this service, such as filtering inactive equipment or applying access rules. For now, it delegates the retrieval to the repository.

5. Update your `EquipmentController.cs`

Replace your current controller with:

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EquipmentController : ControllerBase
    {
        private readonly IEquipmentService _equipmentService;

        public EquipmentController(IEquipmentService equipmentService)
        {
            _equipmentService = equipmentService;
        }

        [HttpGet]
        public async Task<ActionResult<List<Equipment>>> GetAll()
        {
            var equipments = await _equipmentService.GetAllAsync();

            return Ok(equipments);
        }
    }
}
```

Your endpoint is now:

`GET https://localhost:<your-port>/api/equipment`

If successful, it returns HTTP `200 OK` with a JSON array of equipment records. If there are no records, it returns an empty array.

6. Register the layers in `Program.cs`

Add these registrations before `var app = builder.Build();`:

```
builder.Services.AddScoped<IEquipmentRepository, EquipmentRepository>();
builder.Services.AddScoped<IEquipmentService, EquipmentService>();
```

Also ensure the database context is registered. For example, if you're using SQLite:

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

Install the SQLite EF Core provider if it isn't already installed:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

Your `appsettings.json` would contain a connection string similar to:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=solvefy.db"
  }
}
```

This assumes SQLite is your selected database; if you've already configured SQL Server, keep your existing provider and connection string.

7. Test the endpoint

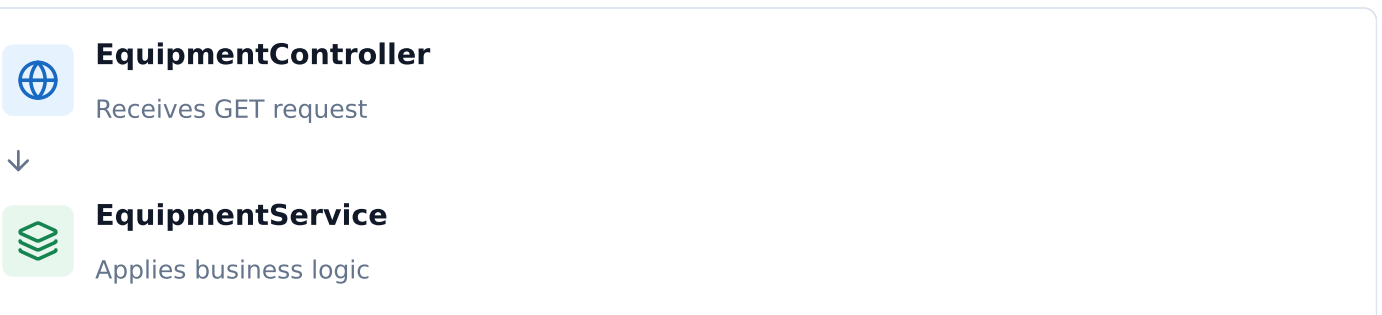
Run your backend:

```
dotnet run
```

Open Swagger, if enabled, or use Postman to send:

```
GET /api/equipment
```

The request follows this exact path:





Important for your hiring task: This implements the retrieval path, but it does not create the database tables or populate sample data automatically. If your database is new, you'll also need to configure migrations and apply them before the endpoint can return records.

24/10/2026, 15:46:37 · gpt-6

```
{
 "Logging": {
  "LogLevel": {
   "Default": "Information",
   "Microsoft.AspNetCore": "Warning"
  }
 },
 "AllowedHosts": "*"
}
```

i am using sql server please help me write appsettings.json file according to this and then register it in program.cs file ??

24/10/2026, 16:01:23

Since your SolvefyTask backend uses **SQL Server**, you need to configure the connection string in `appsettings.json` and register your Entity Framework Core `DbContext` in `Program.cs` using `UseSqlServer()` instead of `UseSqlite()`.

I'll use a SQL Server LocalDB example below. If you're using a full SQL Server instance, you can replace the server name with your actual instance.

1. Update `appsettings.json`

Replace your existing configuration with:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=
(localdb)\\MSSQLLocalDB;Database=SolvefyTaskDb;Trusted_Connection=True;MultipleActiveResu
ltSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

This configuration uses:

- **Server:** SQL Server LocalDB.
- **Database:** `SolvefyTaskDb`.
- **Authentication:** Windows authentication for the current user.
- **Connection name:** `DefaultConnection`, which we'll use in `Program.cs`.

If you're using SQL Server Express or a full SQL Server installation, the connection string will be different.

2. Install the SQL Server EF Core provider

Open the terminal in your **.NET backend project directory** and run:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

Use a package version compatible with your target framework and the EF Core version already used by your project. For a .NET 8 project, EF Core 8 is the usual matching choice; for .NET 9, use EF Core 9.

3. Register the DbContext in `Program.cs`

Assuming your context is `Data/ApplicationDbContext.cs`, add these namespaces:

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

Then register your services before `builder.Build()`:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// SQL Server database
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    ));

// Dependency injection
builder.Services.AddScoped<IEquipmentRepository, EquipmentRepository>();
builder.Services.AddScoped<IEquipmentService, EquipmentService>();
```

```
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Important: If your `Program.cs` already contains these statements, keep them and add only the missing registrations. Avoid duplicating `AddControllers()`, Swagger setup, or other existing configuration.

4. Create the database and tables

If you're using EF Core migrations and your `ApplicationDbContext` and `Equipment` model are ready, run these commands from the backend project directory:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update
```

If `dotnet-ef` is already installed, skip the first command. If you already have migrations, apply them with `dotnet ef database update` instead of creating another initial migration.

5. Verify the connection

1. Open SQL Server Object Explorer or SQL Server Management Studio.
2. Connect to `(localdb)\MSSQLLocalDB` if using LocalDB.
3. Check whether `SolvefyTaskDb` has been created.
4. Run your API and test `GET /api/equipment` through Swagger.

If you're using a regular SQL Server instance rather than LocalDB, tell me your server name—for example, `.\SQLEXPRESS`—and whether you use Windows or SQL Server authentication, and I'll tailor the connection string to your setup.

24/10/2026, 16:01:26 · gpt-6

how to apply Migration using package manager console ??

24/10/2026, 16:05:52

You can apply Entity Framework Core migrations through **Package Manager Console (PMC)** in Visual Studio for your SolvefyTask SQL Server database.

1. Open Package Manager Console

In Visual Studio, go to:

Tools → NuGet Package Manager → Package Manager Console

2. Install the required packages

Run these commands in the Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Ensure the package versions are compatible with your project's .NET and EF Core versions.

3. Select the correct project

In Package Manager Console, set **Default project** to the project containing `ApplicationDbContext` (or your actual DbContext).

If your solution has multiple projects, also ensure the correct **Startup Project** is selected in Solution Explorer.

4. Create and apply the migration

Run the following commands one by one.

Create the migration:

```
Add-Migration InitialCreate
```

Apply it to SQL Server:

```
Update-Database
```

EF Core will create the database and tables defined by your `DbContext`, provided the connection string and configuration are correct.

5. Useful commands

Purpose	PMC command
Add a new migration	`Add-Migration AddEquipment`
Apply pending migrations	`Update-Database`
List migrations	`Get-Migration`
Roll back to a previous migration	`Update-Database PreviousMigrationName`
Remove the last unapplied migration	`Remove-Migration`

If you get an error

Check these common issues:

- **No DbContext was found:** Ensure your project contains `ApplicationDbContext` and select the correct project.
- **Build failed:** Fix the compiler errors before running migrations.
- **Connection failed:** Verify `DefaultConnection` in `appsettings.json` and confirm that your SQL Server instance is running.
- **Unable to create an object of type DbContext:** Ensure `ApplicationDbContext` is registered in `Program.cs` using `AddDbContext` and `UseSqlServer`.

One important question: Are you using **Visual Studio** with Package Manager Console, or **Visual Studio Code**? PMC is built into Visual Studio; in VS Code, you would normally use the terminal with `dotnet ef` commands.

24/10/2026, 16:05:52 · gpt-6

please write me an API to insert the equipment ??

24/10/2026, 18:56:47

For your **SolvefyTask ASP.NET Core Web API**, we'll create a `POST` API using the same architecture we used for fetching equipment:

Controller → Service → Repository → SQL Server Database

1. Equipment model

Make sure your `Models/Equipment.cs` contains these properties. Keep your existing model if you already created it, and add only any missing properties.

```
namespace SolvefyTask.Models
{
    public class Equipment
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;

        public string Category { get; set; } = string.Empty;

        public string? SerialNumber { get; set; }

        public string Type { get; set; } = "Individual";

        public string Status { get; set; } = "Available";

        public string? AssignedTo { get; set; }
    }
}
```

2. Repository interface

Update `Repositories/IEquipmentRepository.cs`:

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface IEquipmentRepository
    {
```

```
        Task<List<Equipment>> GetAllAsync();

        Task<Equipment> InsertAsync(Equipment equipment);
    }
}
```

Keep your existing `GetAllAsync()` method and add `InsertAsync()`.

3. Repository implementation

Update `Repositories/EquipmentRepository.cs`:

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class EquipmentRepository : IEquipmentRepository
    {
        private readonly ApplicationDbContext _context;

        public EquipmentRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<Equipment>> GetAllAsync()
        {
            return await _context.Equipments
                .AsNoTracking()
                .ToListAsync();
        }

        public async Task<Equipment> InsertAsync(Equipment equipment)
        {
            _context.Equipments.Add(equipment);

            await _context.SaveChangesAsync();

            return equipment;
        }
    }
}
```

4. Service interface

Update `Services/IEquipmentService.cs`:

```
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface IEquipmentService
    {
        Task<List<Equipment>> GetAllAsync();

        Task<Equipment> InsertAsync(Equipment equipment);
    }
}
```

5. Service implementation

Update `Services/EquipmentService.cs`:

```
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class EquipmentService : IEquipmentService
    {
        private readonly IEquipmentRepository _repository;

        public EquipmentService(IEquipmentRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<Equipment>> GetAllAsync()
        {
            return await _repository.GetAllAsync();
        }

        public async Task<Equipment> InsertAsync(Equipment equipment)
        {
            equipment.Id = 0;
            equipment.Status = "Available";
            equipment.AssignedTo = null;

            return await _repository.InsertAsync(equipment);
        }
```

```
    }
}
```

This service sets the initial status to `Available` and clears `AssignedTo`, because newly added equipment should not already be checked out. The database generates the new ID.

6. Equipment controller

Update `Controllers/EquipmentController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EquipmentController : ControllerBase
    {
        private readonly IEquipmentService _service;

        public EquipmentController(IEquipmentService service)
        {
            _service = service;
        }

        [HttpGet]
        public async Task<ActionResult<List<Equipment>>> GetAll()
        {
            var equipment = await _service.GetAllAsync();

            return Ok(equipment);
        }

        [HttpPost]
        public async Task<ActionResult<Equipment>> Insert(
            [FromBody] Equipment equipment)
        {
            if (string.IsNullOrWhiteSpace(equipment.Name))
            {
                return BadRequest("Equipment name is required.");
            }

            if (string.IsNullOrWhiteSpace(equipment.Category))
            {
                return BadRequest("Equipment category is required.");
```

```
            }

            var createdEquipment = await _service.InsertAsync(equipment);

            return CreatedAtAction(
                nameof(GetAll),
                new { id = createdEquipment.Id },
                createdEquipment
            );
        }
    }
}
```

7. Test the API in Swagger

Run your ASP.NET Core project and open its Swagger page.

Select `POST /api/Equipment` and use this JSON request body:

```
{
  "name": "GPS Rover",
  "category": "Survey Equipment",
  "serialNumber": "GPS-001",
  "type": "Individual"
}
```

Click **Execute**.

Expected response: `201 Created`, with the newly generated ID and the saved equipment details. The item should have `Status: "Available"` and `AssignedTo: null`.

You can also insert a quantity-tracked item, such as tripods, but your current model does not yet store a quantity. We should add a `Quantity` property before implementing that requirement.

8. Check the database

After the request succeeds, verify the record in your SQL Server `Equipments` table. You do not need to run another migration for this API if the table already exists and the model has not changed.

Important: This is a basic insert API. Before final submission, add request DTOs and stronger validation so clients cannot supply fields such as `Id`, `Status`, or `AssignedTo` directly. That will make the equipment workflow safer and easier to maintain.

24/10/2026, 18:56:47 · gpt-6

now make an service in angular such that it calls the above 2 apis and perform the operation and use env to call an api ?

24/10/2026, 19:05:00

We'll create an Angular `EquipmentService` that calls both of your ASP.NET Core APIs:

- `GET /api/Equipment` — fetch all equipment.
- `POST /api/Equipment` — insert new equipment.

We'll also use Angular environment files so the API base URL is configured in one place rather than hardcoded throughout your application.

The flow will be:

`Angular Component → EquipmentService → Environment API URL → ASP.NET Core API → SQL Server`

1. Configure the environment

In your Angular project, open `src/environments/`.

If the folder or files do not exist, create them.

`src/environments/environment.ts`

```
export const environment = {
  production: false,
  apiUrl: 'https://localhost:7001/api'
};
```

Replace `7001` with the actual HTTPS port of your ASP.NET Core API. You can find it in your backend's `Properties/launchSettings.json` or the URL shown when you run the API.

For production, create or update:

`src/environments/environment.prod.ts`

```
export const environment = {
  production: true,
  apiUrl: 'https://your-production-api-domain.com/api'
};
```

Replace the production URL with your deployed API address.

2. Create the equipment model

Create `src/app/models/equipment.ts`:

```
export interface Equipment {
  id: number;
  name: string;
  category: string;
  serialNumber?: string | null;
  type: string;
  status: string;
  assignedTo?: string | null;
}
```

For inserting equipment, define a separate request type so you don't have to send the database-generated ID or status.

Add this to the same file:

```
export interface CreateEquipmentRequest {
  name: string;
  category: string;
  serialNumber?: string | null;
  type: string;
}
```

3. Enable Angular HTTP

Since your project uses standalone Angular components, open `src/app/app.config.ts` and add `provideHttpClient()` to the existing providers.

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient()
    // Keep your existing providers here.
  ]
};
```

If your file already contains providers for routing, animations, or other features, **keep them** and add `provideHttpClient()` to the existing array.

4. Create the Angular service

Run this command in your Angular project terminal:

```
ng generate service services/equipment
```

Open `src/app/services/equipment.service.ts` and add:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  Equipment,
  CreateEquipmentRequest
} from '../models/equipment';

@Injectable({
  providedIn: 'root'
})
export class EquipmentService {

  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Equipment`;

  // API 1: Get all equipment
  getAllEquipment(): Observable<Equipment[]> {
    return this.http.get<Equipment[]>(this.apiUrl);
  }

  // API 2: Insert equipment
  createEquipment(
    equipment: CreateEquipmentRequest
  ): Observable<Equipment> {
    return this.http.post<Equipment>(
      this.apiUrl,
      equipment
    );
  }
}
```

The service now uses the environment URL for both API calls. Angular's `HttpClient` returns observables, so components can subscribe to the results and handle errors.

5. Use the service in your component

For example, in your Equipment component, open `equipment.ts` (or the TypeScript file for your actual component).

```ts
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { EquipmentService } from '../../services/equipment.service';
import {
  Equipment,
  CreateEquipmentRequest
} from '../../models/equipment';

@Component({
  selector: 'app-equipment',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './equipment.html',
  styleUrl: './equipment.css'
})
export class EquipmentComponent implements OnInit {

  private equipmentService = inject(EquipmentService);

  equipmentList: Equipment[] = [];
  loading = false;
  errorMessage = '';

  ngOnInit(): void {
    this.loadEquipment();
  }

  // GET: Load all equipment
  loadEquipment(): void {
    this.loading = true;
    this.errorMessage = '';

    this.equipmentService.getAllEquipment().subscribe({
      next: (data) => {
        this.equipmentList = data;
        this.loading = false;
      },
      error: (error) => {
        console.error('Error loading equipment:', error);
        this.errorMessage = 'Unable to load equipment.';
        this.loading = false;
      }
    });
  }

  // POST: Insert new equipment
```

```
  addEquipment(): void {
    const newEquipment: CreateEquipmentRequest = {
      name: 'GPS Rover',
      category: 'Survey Equipment',
      serialNumber: 'GPS-001',
      type: 'Individual'
    };

    this.equipmentService.createEquipment(newEquipment).subscribe({
      next: (createdEquipment) => {
        console.log('Equipment created:', createdEquipment);

        // Refresh the list after successful insertion
        this.loadEquipment();
      },
      error: (error) => {
        console.error('Error creating equipment:', error);
        this.errorMessage = 'Unable to create equipment.';
      }
    });
  }
}
```

This uses your existing component structure as an example. If your component already has a constructor, imports, or other methods, integrate these members rather than replacing your whole component.

6. Display the equipment in HTML

In `equipment.html`, you can test the GET API with:

```
<div *ngIf="loading">
  Loading equipment...
</div>

<p *ngIf="errorMessage">
  {{ errorMessage }}
</p>

<table *ngIf="!loading">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Category</th>
      <th>Serial Number</th>
      <th>Type</th>
```

```
      <th>Status</th>
    </tr>
  </thead>

  <tbody>
    <tr *ngFor="let item of equipmentList">
      <td>{{ item.id }}</td>
      <td>{{ item.name }}</td>
      <td>{{ item.category }}</td>
      <td>{{ item.serialNumber }}</td>
      <td>{{ item.type }}</td>
      <td>{{ item.status }}</td>
    </tr>
  </tbody>
</table>

<button type="button" (click)="addEquipment()">
  Test Insert Equipment
</button>
```

The test button inserts a GPS Rover whenever clicked, so use it only for testing. In your actual UI, call `createEquipment()` with the values entered in your Add Equipment form.

7. Run and verify

1. Start your ASP.NET Core API.
2. Confirm the `apiUrl` in `environment.ts` matches the backend port.
3. Start Angular using `ng serve`.
4. Open the Equipment page.
5. Check the browser's Network tab for `GET /api/Equipment`.
6. Click the test insert button and check for `POST /api/Equipment` with a `201 Created` response.

If the browser reports a CORS error, configure CORS in the ASP.NET Core backend to allow your Angular development origin, commonly `http://localhost:4200`.

One important detail: This service is ready to call the two APIs we created, but the POST example is currently a test action. The next step for your SolvefyTask frontend should be to connect the POST method to your real Add Equipment form and display API validation errors to the user.

24/10/2026, 19:05:18 · gpt-6

```
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { CreateEquipmentRequest } from
'../../models/equipment';


interface Equipments {
  id: number;
  name: string;
  category: string;
  serialNo: string;
  type: string;
  status: string;
  assignedTo: string;
}
@Component({
  selector: 'app-equipment',
  imports: [FormsModule],
  templateUrl: './equipment.html',
  styleUrl: './equipment.css',
})
export class Equipment implements OnInit {

ngOnInit(): void {
    this.loadEquipment();
  }

  searchText = '';
  selectedCategory = 'All Categories';
  selectedStatus = 'All Status';
  showFilters = false;
  showAddForm = false;
  selectedEquipment: number | null = null;

  categories = ['All Categories', 'Total Station', 'GPS', 'Drone',
'Tripod', 'Tablet'];

  statuses = [
    'All Status',
    'Available',
    'Checked Out',
    'Approval Required',
    'Under Repair'
  ];
```

```
equipment: Equipments[] = [
  {
    id: 1,
    name: 'Total Station - 01',
    category: 'Total Station',
    serialNo: 'TS-001',
    type: 'Individual',
    status: 'Available',
    assignedTo: '-'
  },
  {
    id: 2,
    name: 'GPS Rover - 01',
    category: 'GPS',
    serialNo: 'GPS-001',
    type: 'Individual',
    status: 'Checked Out',
    assignedTo: 'Ali Raza'
  },
  {
    id: 3,
    name: 'Drone - DJI Mavic',
    category: 'Drone',
    serialNo: 'DR-001',
    type: 'Individual',
    status: 'Approval Required',
    assignedTo: '-'
  },
  {
    id: 4,
    name: 'Tripods',
    category: 'Tripod',
    serialNo: '-',
    type: 'Bulk',
    status: 'Available (3/4)',
    assignedTo: '-'
  },
  {
    id: 5,
    name: 'Field Tablets',
    category: 'Tablet',
    serialNo: '-',
    type: 'Bulk',
    status: 'Available (4/6)',
    assignedTo: '-'
```

```
    }
  ];

  get filteredEquipment(): Equipments[] {
    const search = this.searchText.trim().toLowerCase();

    return this.equipment.filter(item => {
      const matchesSearch =
        !search ||
        item.name.toLowerCase().includes(search) ||
        item.category.toLowerCase().includes(search) ||
        item.serialNo.toLowerCase().includes(search) ||
        item.assignedTo.toLowerCase().includes(search);

      const matchesCategory =
        this.selectedCategory === 'All Categories' ||
        item.category === this.selectedCategory;

      const matchesStatus =
        this.selectedStatus === 'All Status' ||
        (this.selectedStatus === 'Available'
          ? item.status.startsWith('Available')
          : item.status === this.selectedStatus);

      return matchesSearch && matchesCategory &&
matchesStatus;
    });
  }

  // POST: Insert new equipment
  addEquipment(): void {
    const newEquipment: CreateEquipmentRequest = {
      name: 'GPS Rover',
      category: 'Survey Equipment',
      serialNumber: 'GPS-001',
      type: 'Individual'
    };


this.equipmentService.createEquipment(newEquipment).subscrib
      next: (createdEquipment) => {
        console.log('Equipment created:', createdEquipment);

        // Refresh the list after successful insertion
        this.loadEquipment();
      },
```

```
      error: (error) => {
        console.error('Error creating equipment:', error);
        this.errorMessage = 'Unable to create equipment.';
      }
    });
  }

  saveEquipment(
    name: string,
    category: string,
    serialNo: string,
    type: string
  ): void {
    if (!name.trim() || !category || !type) {
      return;
    }

    this.equipment = [
      ...this.equipment,
      {
        id: Math.max(0, ...this.equipment.map(item => item.id))
+ 1,
        name: name.trim(),
        category,
        serialNo: serialNo.trim() || '-',
        type,
        status: 'Available',
        assignedTo: '-'
      }
    ];

    this.showAddForm = false;
  }

  toggleFilters(): void {
    this.showFilters = !this.showFilters;
  }

  toggleActions(id: number): void {
    this.selectedEquipment =
      this.selectedEquipment === id ? null : id;
  }

  clearFilters(): void {
    this.searchText = '';
    this.selectedCategory = 'All Categories';
```

```
    this.selectedStatus = 'All Status';
  }

  trackById(_index: number, item: Equipments): number {
    return item.id;
  }
}
```

please provide the complete code of this ??

24/10/2026, 19:16:40

Below is the **complete updated** `equipment.ts` for your Angular Equipment page. It keeps your existing search, category/status filters, action toggles, and add-equipment form while connecting the page to your `EquipmentService` GET and POST APIs.

Replace your current `src/app/components/equipment/equipment.ts` with this code.

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import {
  CreateEquipmentRequest,
  Equipment as EquipmentModel
} from '../../models/equipment';
import { EquipmentService } from '../../services/equipment.service';

interface Equipments {
  id: number;
  name: string;
  category: string;
  serialNo: string;
  type: string;
  status: string;
  assignedTo: string;
}

@Component({
  selector: 'app-equipment',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './equipment.html',
  styleUrl: './equipment.css',
})
export class Equipment implements OnInit {
```

```
  private equipmentService = inject(EquipmentService);

  ngOnInit(): void {
    this.loadEquipment();
  }

  searchText = '';
  selectedCategory = 'All Categories';
  selectedStatus = 'All Status';
  showFilters = false;
  showAddForm = false;
  selectedEquipment: number | null = null;

  loading = false;
  saving = false;
  errorMessage = '';
  successMessage = '';

  categories = [
    'All Categories',
    'Total Station',
    'GPS',
    'Drone',
    'Tripod',
    'Tablet'
  ];

  statuses = [
    'All Status',
    'Available',
    'Checked Out',
    'Approval Required',
    'Under Repair'
  ];

  // Bound to your Add Equipment form
  newEquipment = {
    name: '',
    category: '',
    serialNo: '',
    type: 'Individual'
  };

  equipment: Equipments[] = [];

  // GET: Fetch equipment from the ASP.NET Core API
```

```
  loadEquipment(): void {
    this.loading = true;
    this.errorMessage = '';

    this.equipmentService.getAllEquipment().subscribe({
      next: (data: EquipmentModel[]) => {
        this.equipment = data.map(item => ({
          id: item.id,
          name: item.name,
          category: item.category,
          serialNo: item.serialNumber ?? '-',
          type: item.type,
          status: item.status,
          assignedTo: item.assignedTo ?? '-'
        }));

        this.loading = false;
      },
      error: (error) => {
        console.error('Error loading equipment:', error);
        this.errorMessage = 'Unable to load equipment. Please try again.';
        this.loading = false;
      }
    });
  }

  // Search and filter equipment
  get filteredEquipment(): Equipments[] {
    const search = this.searchText.trim().toLowerCase();

    return this.equipment.filter(item => {
      const matchesSearch =
        !search ||
        item.name.toLowerCase().includes(search) ||
        item.category.toLowerCase().includes(search) ||
        item.serialNo.toLowerCase().includes(search) ||
        item.assignedTo.toLowerCase().includes(search);

      const matchesCategory =
        this.selectedCategory === 'All Categories' ||
        item.category === this.selectedCategory;

      const matchesStatus =
        this.selectedStatus === 'All Status' ||
        (this.selectedStatus === 'Available'
          ? item.status.startsWith('Available')
```

```
        : item.status === this.selectedStatus);

      return matchesSearch && matchesCategory && matchesStatus;
    });
  }

  // POST: Insert equipment using the values entered in the form
  addEquipment(): void {
    const name = this.newEquipment.name.trim();
    const category = this.newEquipment.category;
    const serialNo = this.newEquipment.serialNo.trim();
    const type = this.newEquipment.type;

    if (!name || !category || !type) {
      this.errorMessage = 'Please enter the name, category, and type.';
      return;
    }

    const request: CreateEquipmentRequest = {
      name,
      category,
      serialNumber: serialNo || null,
      type
    };

    this.saving = true;
    this.errorMessage = '';
    this.successMessage = '';

    this.equipmentService.createEquipment(request).subscribe({
      next: () => {
        this.saving = false;
        this.successMessage = 'Equipment added successfully.';

        this.newEquipment = {
          name: '',
          category: '',
          serialNo: '',
          type: 'Individual'
        };

        this.showAddForm = false;
        this.loadEquipment();
      },
      error: (error) => {
        console.error('Error creating equipment:', error);
```

```
        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
          (typeof error?.error === 'string' ? error.error : null) ||
          'Unable to add equipment. Please try again.';
      }
    });
  }

  toggleFilters(): void {
    this.showFilters = !this.showFilters;
  }

  toggleActions(id: number): void {
    this.selectedEquipment =
      this.selectedEquipment === id ? null : id;
  }

  clearFilters(): void {
    this.searchText = '';
    this.selectedCategory = 'All Categories';
    this.selectedStatus = 'All Status';
  }

  trackById(_index: number, item: Equipments): number {
    return item.id;
  }
}
```

Important HTML changes

Your existing `equipment.html` should bind the form inputs to `newEquipment`, because the POST method now uses those values instead of inserting a hardcoded GPS Rover.

For example, inside your existing Add Equipment form:

```
<input
  type="text"
  name="name"
  [(ngModel)]="newEquipment.name"
  placeholder="Equipment name"
  required
/>

<select
  name="category"
```

```
  [(ngModel)]="newEquipment.category"
  required
>
  <option value="">Select category</option>
  <option *ngFor="let category of categories.slice(1)" [value]="category">
    {{ category }}
  </option>
</select>

<input
  type="text"
  name="serialNo"
  [(ngModel)]="newEquipment.serialNo"
  placeholder="Serial number (optional)"
/>

<select name="type" [(ngModel)]="newEquipment.type">
  <option value="Individual">Individual</option>
  <option value="Bulk">Bulk</option>
</select>

<button type="button" (click)="addEquipment()" [disabled]="saving">
  {{ saving ? 'Saving...' : 'Save Equipment' }}
</button>
```

Display loading and API messages somewhere in your page:

```
<p *ngIf="loading">Loading equipment...</p>
<p *ngIf="errorMessage">{{ errorMessage }}</p>
<p *ngIf="successMessage">{{ successMessage }}</p>
```

Make sure your table uses `filteredEquipment` rather than `equipment`:

```
<tr *ngFor="let item of filteredEquipment; trackBy: trackById">
  <td>{{ item.name }}</td>
  <td>{{ item.category }}</td>
  <td>{{ item.serialNo }}</td>
  <td>{{ item.type }}</td>
  <td>{{ item.status }}</td>
  <td>{{ item.assignedTo }}</td>
</tr>
```

Final checks

- Your `EquipmentService` must have `getAllEquipment()` and `createEquipment()` methods.

- `provideHttpClient()` must be configured in `app.config.ts`.
- The API URL in `environment.ts` must match your running ASP.NET Core API.
- If your HTML already has a form with a submit event, use `(ngSubmit)="addEquipment()"` on the form and make the Save button `type="submit"` instead of using a separate click handler.

Note: Your existing page displays bulk equipment statuses such as `Available (3/4)`, but the current backend model only stores a string status and does not store total or available quantities. The GET mapping above preserves the API's status value; quantity tracking will require a backend model/API update.

24/10/2026, 19:16:40 · gpt-6

X [ERROR] TS2769: No overload matches this call.
  Overload 1 of 2, '(observerOrNext?: Partial<Observer<EquipmentService[]>> | ((value: EquipmentService[]) => void) | undefined): Subscription', gave the following error.
    Type '(data: EquipmentModel[]) => void' is not assignable to type '(value: EquipmentService[]) => void'.
      Types of parameters 'data' and 'value' are incompatible.
        Type 'EquipmentService[]' is not assignable to type 'Equipment[]'.
          Type 'EquipmentService' is missing the following properties from type 'Equipment': id, name, category, type, status
  Overload 2 of 2, '(next?: ((value: EquipmentService[]) => void) | null | undefined, error?: ((error: any) => void) | null | undefined, complete?: (() => void) | null | undefined): Subscription', gave the following error.
    Object literal may only specify known properties, and 'next' does not exist in type '(value: EquipmentService[]) => void'. [plugin angular-compiler]

    src/app/components/equipment/equipment.ts:81:6:
      81 │   next: (data: EquipmentModel[]) => {
         ╵   ~~~~

24/10/2026, 19:19:51

The error occurs because your `EquipmentService.getAllEquipment()` method is returning `EquipmentService[]` **instead of** `Equipment[]`. The service's return type is incorrect, or the wrong model type is being used.

1. Fix `equipment.service.ts`

Open `src/app/services/equipment.service.ts` and make sure the GET method returns `Observable<Equipment[]>`.

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import { Equipment } from '../models/equipment';
```

```
import { CreateEquipmentRequest } from '../models/equipment';

@Injectable({
  providedIn: 'root'
})
export class EquipmentService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Equipment`;

  // GET: Fetch all equipment
  getAllEquipment(): Observable<Equipment[]> {
    return this.http.get<Equipment[]>(this.apiUrl);
  }

  // POST: Insert equipment
  createEquipment(
    equipment: CreateEquipmentRequest
  ): Observable<Equipment> {
    return this.http.post<Equipment>(this.apiUrl, equipment);
  }
}
```

2. Check `models/equipment.ts`

Make sure your model file exports the `Equipment` interface, not a class or interface named `EquipmentService`.

```
export interface Equipment {
  id: number;
  name: string;
  category: string;
  serialNumber?: string | null;
  type: string;
  status: string;
  assignedTo?: string | null;
}

export interface CreateEquipmentRequest {
  name: string;
  category: string;
  serialNumber?: string | null;
  type: string;
}
```

3. Keep your component's GET call like this

In `equipment.ts`, your existing `loadEquipment()` method should work with the corrected service:

```
this.equipmentService.getAllEquipment().subscribe({
  next: (data: EquipmentModel[]) => {
    this.equipment = data.map(item => ({
      id: item.id,
      name: item.name,
      category: item.category,
      serialNo: item.serialNumber ?? '-',
      type: item.type,
      status: item.status,
      assignedTo: item.assignedTo ?? '-'
    }));

    this.loading = false;
  },
  error: (error) => {
    console.error('Error loading equipment:', error);
    this.errorMessage = 'Unable to load equipment.';
    this.loading = false;
  }
});
```

Also ensure the component imports the model with the alias you used:

```
import {
  CreateEquipmentRequest,
  Equipment as EquipmentModel
} from '../../models/equipment';
```

Why this fixes it: `EquipmentService` is the class that makes HTTP calls; `Equipment` is the interface describing each equipment record. The GET method must return an array of the latter. After correcting the service return type, save the files and let Angular rebuild.

24/10/2026, 19:19:52 · gpt-6

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../environments/environment';
import { Observable } from 'rxjs';
import { CreateEquipmentRequest } from
'../models/equipment';

@Injectable({
  providedIn: 'root',
})
export class EquipmentService {

  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Equipment`;

  // API 1: Get all equipment
  getAllEquipment(): Observable<EquipmentService[]> {
    return this.http.get<EquipmentService[]>(this.apiUrl);
  }

  // API 2: Insert equipment
  createEquipment(
    equipment: CreateEquipmentRequest
  ): Observable<EquipmentService> {
    return this.http.post<EquipmentService>(
      this.apiUrl,
      equipment
    );
  }
}
```

24/10/2026, 19:21:29

The issue is that you're using `EquipmentService` as the data type. `EquipmentService` is your service class, not your equipment model.

Replace your `equipment.service.ts` with this corrected code.

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../environments/environment';
import { Observable } from 'rxjs';
```

```
import {
  Equipment,
  CreateEquipmentRequest
} from '../models/equipment';

@Injectable({
  providedIn: 'root',
})
export class EquipmentService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Equipment`;

  // API 1: Get all equipment
  getAllEquipment(): Observable<Equipment[]> {
    return this.http.get<Equipment[]>(this.apiUrl);
  }

  // API 2: Insert equipment
  createEquipment(
    equipment: CreateEquipmentRequest
  ): Observable<Equipment> {
    return this.http.post<Equipment>(
      this.apiUrl,
      equipment
    );
  }
}
```

Check `models/equipment.ts`

Make sure this file contains the `Equipment` interface:

```
export interface Equipment {
  id: number;
  name: string;
  category: string;
  serialNumber?: string | null;
  type: string;
  status: string;
  assignedTo?: string | null;
}

export interface CreateEquipmentRequest {
  name: string;
```

```
  category: string;
  serialNumber?: string | null;
  type: string;
}
```

The key correction: Replace `EquipmentService[]` with `Equipment[]` in the GET method, and replace `EquipmentService` with `Equipment` in the POST method.

Your component can continue using `Equipment as EquipmentModel` in its import.

24/10/2026, 19:21:30 · gpt-6

```
<div class="equipment-page">
  <div class="page-banner">3. Equipment List</div>

  <main class="page-content">
    <header class="page-heading">
      <div>
        <h1>Equipment</h1>
        <p>View and manage all equipment</p>
      </div>

      <button class="primary-button"
(click)="addEquipment()">
        <span>+</span> Add Equipment
      </button>
    </header>

    <section class="toolbar">
      <label class="search-box">
        <span class="search-icon">⌕</span>
        <input
          type="search"
          placeholder="Search equipment..."
          [(ngModel)]="searchText"
        />
      </label>

      <select
        aria-label="Filter by category"
        [(ngModel)]="selectedCategory"
      >
        @for (category of categories; track category) {
          <option [value]="category">{{ category }}</option>
        }
      </select>

      <select
        aria-label="Filter by status"
        [(ngModel)]="selectedStatus"
      >
        @for (status of statuses; track status) {
          <option [value]="status">{{ status }}</option>
        }
      </select>
```

```
  <button class="filter-button" (click)="toggleFilters()">
    <span>☷</span> Filter
  </button>
</section>

@if (showFilters) {
  <section class="filter-panel">
    <div>
      <strong>Equipment filters</strong>
      <p>Filter the list by category and current status.</p>
    </div>
    <button class="clear-button" (click)="clearFilters()">
      Clear filters
    </button>
  </section>
}

<section class="table-card">
  <div class="table-scroll">
    <table>
      <thead>
        <tr>
          <th>Name <span class="sort-icon">↕</span></th>
          <th>Category</th>
          <th>Serial No.</th>
          <th>Type</th>
          <th>Status</th>
          <th>Assigned To</th>
          <th class="actions-column">Actions</th>
        </tr>
      </thead>

      <tbody>
        @for (item of filteredEquipment; track item.id) {
          <tr>
            <td>
              <div class="equipment-name">
                <span class="equipment-icon">
                  @switch (item.category) {
                    @case ('Total Station') { ⊙ }
                    @case ('GPS') { ◉ }
                    @case ('Drone') { ✣ }
                    @case ('Tripod') { ╱ }
                    @case ('Tablet') { ▣ }
                    @default { ▦ }
                  }
```

```
            </span>
            <span>{{ item.name }}</span>
          </div>
        </td>

        <td>{{ item.category }}</td>
        <td>{{ item.serialNo }}</td>
        <td>{{ item.type }}</td>

        <td>
          <span
            class="status-badge"

[class.available]="item.status.startsWith('Available')"
            [class.checked-out]="item.status === 'Checked
Out'"
            [class.approval]="item.status === 'Approval
Required'"
            [class.repair]="item.status === 'Under Repair'"
          >
            {{ item.status }}
          </span>
        </td>

        <td>{{ item.assignedTo }}</td>

        <td class="actions-column">
          <button
            class="action-button"
            [attr.aria-label]="'Actions for ' + item.name"
            (click)="toggleActions(item.id)"
          >
            ...
          </button>

          @if (selectedEquipment === item.id) {
            <div class="action-menu">
              <button (click)="toggleActions(item.id)">
                View Details
              </button>
              <button (click)="toggleActions(item.id)">
                Edit Equipment
              </button>
            </div>
          }
        </td>
```

```
            </tr>
          } @empty {
            <tr>
              <td colspan="7" class="empty-state">
                <span class="empty-icon">⌕</span>
                <strong>No equipment found</strong>
                <p>Try changing your search or filters.</p>
                <button class="clear-button"
(click)="clearFilters()">
                  Clear filters
                </button>
              </td>
            </tr>
          }
        </tbody>
      </table>
    </div>

    <footer class="table-footer">
      <span>
        Showing {{ filteredEquipment.length }} of {{
equipment.length }} items
      </span>
      <span class="records-label">Equipment records</span>
    </footer>
  </section>
 </main>
</div>

@if (showAddForm) {
 <div class="modal-backdrop" (click)="showAddForm =
false">
   <section
    class="equipment-modal"
    role="dialog"
    aria-modal="true"
    aria-labelledby="add-equipment-title"
    (click)="$event.stopPropagation()"
   >
    <header class="modal-heading">
      <div>
        <h2 id="add-equipment-title">Add Equipment</h2>
        <p>Enter the details of the new equipment.</p>
      </div>
      <button
        type="button"
```

```
          class="close-button"
          aria-label="Close"
          (click)="showAddForm = false"
        >×</button>
      </header>

      <form
        #equipmentForm="ngForm"
        (ngSubmit)="saveEquipment(
          newName.value,
          newCategory.value,
          newSerial.value,
          newType.value
        )"
      >
        <label class="form-field">
          Equipment Name *
          <input #newName required placeholder="e.g. Total
Station - 02" />
        </label>

        <label class="form-field">
          Category *
          <select #newCategory required>
            <option value="">Select category</option>
            @for (category of categories.slice(1); track category) {
              <option [value]="category">{{ category }}
</option>
            }
          </select>
        </label>

        <label class="form-field">
          Serial Number
          <input #newSerial placeholder="e.g. TS-002" />
        </label>

        <label class="form-field">
          Equipment Type *
          <select #newType required>
            <option value="">Select type</option>
            <option value="Individual">Individual</option>
            <option value="Bulk">Bulk</option>
          </select>
        </label>
```

```
        <div class="modal-actions">
          <button
            type="button"
            class="cancel-button"
            (click)="showAddForm = false"
          >
            Cancel
          </button>
          <button
            type="submit"
            class="primary-button"
            [disabled]="!equipmentForm.valid"
          >
            Save Equipment
          </button>
        </div>
      </form>
    </section>
  </div>
}

X [ERROR] TS2551: Property 'saveEquipment' does not exist
on type 'Equipment'. Did you mean 'equipment'? [plugin
angular-compiler]

    src/app/components/equipment/equipment.html:185:20:
      185 │         (ngSubmit)="saveEquipment(
          ╵                    ~~~~~~~~~~~~~~

  Error occurs in the template of component Equipment.

    src/app/components/equipment/equipment.ts:26:15:
      26 │   templateUrl: './equipment.html',
         ╵                ~~~~~~~~~~~~~~~~~~~
```

24/10/2026, 19:22:49

The error occurs because your HTML calls `saveEquipment()`, but your updated `equipment.ts` no longer contains that method.

Since you already have the **POST API and** `EquipmentService`, the best fix is to update the form submission to call the API instead of saving equipment only in the local array.

1. Update the form in `equipment.html`

Replace your existing form opening tag:

```
<form
  #equipmentForm="ngForm"
  (ngSubmit)="saveEquipment(
    newName.value,
    newCategory.value,
    newSerial.value,
    newType.value
  )"
>
```

with:

```
<form
  #equipmentForm="ngForm"
  (ngSubmit)="addEquipment(
    newName.value,
    newCategory.value,
    newSerial.value,
    newType.value
  )"
>
```

Also, update the Save Equipment button to show the loading state:

```
<button
  type="submit"
  class="primary-button"
  [disabled]="!equipmentForm.valid || saving"
>
  {{ saving ? 'Saving...' : 'Save Equipment' }}
</button>
```

2. Update `equipment.ts`

Replace your existing `addEquipment()` method with this version. It accepts the values from your current HTML form and calls the POST API.

```
addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please enter the name, category, and type.';
```

```
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: () => {
      this.saving = false;
      this.showAddForm = false;
      this.successMessage = 'Equipment added successfully.';

      // Reload records from SQL Server through the GET API
      this.loadEquipment();
    },
    error: (error) => {
      console.error('Error creating equipment:', error);
      this.saving = false;

      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message || 'Unable to add equipment.';
    }
  });
}
```

Your existing `loadEquipment()` method can remain unchanged.

3. Fix the Add Equipment button at the top

Your page heading currently has:

```
<button class="primary-button" (click)="addEquipment()">
  <span>+</span> Add Equipment
</button>
```

Because `addEquipment()` now accepts form values, change this button to open the modal:

```
<button
  class="primary-button"
  (click)="showAddForm = true"
>
  <span>+</span> Add Equipment
</button>
```

4. Display API messages

Add these lines somewhere inside your page content, for example below the toolbar:

```
@if (errorMessage) {
  <p class="error-message">{{ errorMessage }}</p>
}

@if (successMessage) {
  <p class="success-message">{{ successMessage }}</p>
}
```

Result: Clicking **Add Equipment** opens your modal. Submitting the form calls `POST /api/Equipment`, and after a successful insert, the page reloads the records using `GET /api/Equipment`.

One important detail: your backend currently accepts `Total Station`, `GPS`, `Drone`, `Tripod`, and `Tablet` as category values only if you send those exact values; the API code shown earlier does not enforce a fixed category list.

24/10/2026, 19:22:49 · gpt-6

Application bundle generation failed. [0.308 seconds] - 2026-10-24T14:23:35.459Z

X [ERROR] TS2554: Expected 0 arguments, but got 4. [plugin angular-compiler]

```
  src/app/components/equipment/equipment.html:186:10:
    186 │          newName.value,
        ╵          ~~~~~~~~~~~~~~

 Error occurs in the template of component Equipment.

  src/app/components/equipment/equipment.ts:26:15:
    26 │   templateUrl: './equipment.html',
       ╵                ~~~~~~~~~~~~~~~~~~
```

Application bundle generation failed. [0.291 seconds] - 2026-10-24T14:24:05.726Z

X [ERROR] TS2554: Expected 0 arguments, but got 4. [plugin angular-compiler]

```
  src/app/components/equipment/equipment.html:186:10:
    186 │          newName.value,
        ╵          ~~~~~~~~~~~~~~

 Error occurs in the template of component Equipment.

  src/app/components/equipment/equipment.ts:26:15:
    26 │   templateUrl: './equipment.html',
       ╵                ~~~~~~~~~~~~~~~~~~
```

24/10/2026, 19:24:17

This error means your `equipment.html` is passing **4 arguments** to `addEquipment()`, but your `equipment.ts` still has an `addEquipment()` method that accepts **0 arguments**.

Your HTML and TypeScript method signatures must match.

1. Fix `equipment.ts`

Find your current `addEquipment()` method and replace it with this version:

```
addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please enter the name, category, and type.';
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: () => {
      this.saving = false;
      this.showAddForm = false;
      this.successMessage = 'Equipment added successfully.';
      this.loadEquipment();
    },
    error: (error) => {
      console.error('Error creating equipment:', error);
      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message || 'Unable to add equipment.';
    }
  });
}
```

2. Check `equipment.html`

Your form should call the method with those same four arguments:

```
<form
  #equipmentForm="ngForm"
  (ngSubmit)="addEquipment(
    newName.value,
    newCategory.value,
    newSerial.value,
    newType.value
  )"
>
```

3. Fix the top Add Equipment button

Since `addEquipment()` now saves the form, the top button should open the modal instead:

```
<button
  class="primary-button"
  type="button"
  (click)="showAddForm = true"
>
  <span>+</span> Add Equipment
</button>
```

Important: Make sure there is only one `addEquipment()` method in `equipment.ts`. Save both files and let Angular rebuild. If the error persists, the method signature in the actual file is still the zero-argument version or the edits were made to a different file.

24/10/2026, 19:24:18 · gpt-6

Application bundle generation failed. [3.394 seconds] - 2026-10-24T14:26:51.830Z

X [ERROR] TS2554: Expected 4 arguments, but got 0. [plugin angular-compiler]

```
    src/app/components/equipment/equipment.html:12:46:
      12 │       <button class="primary-button"
(click)="addEquipment()">
         ╵                                       ~~~~~~~~~~~~~

  Error occurs in the template of component Equipment.

    src/app/components/equipment/equipment.ts:26:15:
      26 │   templateUrl: './equipment.html',
         ╵              ~~~~~~~~~~~~~~~~~~~~
```

Watch mode enabled. Watching for file changes...

24/10/2026, 19:27:05

The error is coming from **line 12**, where your top **Add Equipment** button still calls `addEquipment()` without arguments. Your method now requires four arguments.

You only need to change that button in `equipment.html`.

Replace this code

```
<button class="primary-button" (click)="addEquipment()">
  <span>+</span> Add Equipment
</button>
```

With this code

```
<button
  type="button"
  class="primary-button"
  (click)="showAddForm = true"
>
```

```
  <span>+</span> Add Equipment
</button>
```

Why? The top button should open the Add Equipment modal, not submit the equipment directly. The form inside the modal will call `addEquipment()` with the four values:

```
(ngSubmit)="addEquipment(
  newName.value,
  newCategory.value,
  newSerial.value,
  newType.value
)"
```

Save `equipment.html`. Angular should rebuild automatically, and this error should disappear.

24/10/2026, 19:27:05 · gpt-6

```
<div class="equipment-page">
  <div class="page-banner">3. Equipment List</div>

  <main class="page-content">
    <header class="page-heading">
      <div>
        <h1>Equipment</h1>
        <p>View and manage all equipment</p>
      </div>

      <button
          type="button"
          class="primary-button"
          (click)="showAddForm = true"
        >
          <span>+</span> Add Equipment
        </button>
    </header>

    <section class="toolbar">
      <label class="search-box">
        <span class="search-icon">⌕</span>
        <input
          type="search"
          placeholder="Search equipment..."
          [(ngModel)]="searchText"
        />
      </label>

      <select
        aria-label="Filter by category"
        [(ngModel)]="selectedCategory"
      >
        @for (category of categories; track category) {
          <option [value]="category">{{ category }}</option>
        }
      </select>

      <select
        aria-label="Filter by status"
        [(ngModel)]="selectedStatus"
      >
        @for (status of statuses; track status) {
          <option [value]="status">{{ status }}</option>
```

```
      }
    </select>

    <button class="filter-button" (click)="toggleFilters()">
      <span>☷</span> Filter
    </button>
  </section>

  @if (showFilters) {
    <section class="filter-panel">
      <div>
        <strong>Equipment filters</strong>
        <p>Filter the list by category and current status.</p>
      </div>
      <button class="clear-button" (click)="clearFilters()">
        Clear filters
      </button>
    </section>
  }

  <section class="table-card">
    <div class="table-scroll">
      <table>
        <thead>
          <tr>
            <th>Name <span class="sort-icon">↕</span></th>
            <th>Category</th>
            <th>Serial No.</th>
            <th>Type</th>
            <th>Status</th>
            <th>Assigned To</th>
            <th class="actions-column">Actions</th>
          </tr>
        </thead>

        <tbody>
          @for (item of filteredEquipment; track item.id) {
            <tr>
              <td>
                <div class="equipment-name">
                  <span class="equipment-icon">
                    @switch (item.category) {
                      @case ('Total Station') { ⊙ }
                      @case ('GPS') { ◉ }
                      @case ('Drone') { ✣ }
                      @case ('Tripod') { ╱ }
```

```
                @case ('Tablet') { ▣ }
                @default { ▦ }
              }
            </span>
            <span>{{ item.name }}</span>
          </div>
        </td>

        <td>{{ item.category }}</td>
        <td>{{ item.serialNo }}</td>
        <td>{{ item.type }}</td>

        <td>
          <span
            class="status-badge"

[class.available]="item.status.startsWith('Available')"
            [class.checked-out]="item.status === 'Checked
Out'"
            [class.approval]="item.status === 'Approval
Required'"
            [class.repair]="item.status === 'Under Repair'"
          >
            {{ item.status }}
          </span>
        </td>

        <td>{{ item.assignedTo }}</td>

        <td class="actions-column">
          <button
            class="action-button"
            [attr.aria-label]="'Actions for ' + item.name"
            (click)="toggleActions(item.id)"
          >
            ...
          </button>

          @if (selectedEquipment === item.id) {
            <div class="action-menu">
              <button (click)="toggleActions(item.id)">
                View Details
              </button>
              <button (click)="toggleActions(item.id)">
                Edit Equipment
              </button>
```

```
                </div>
              }
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="7" class="empty-state">
              <span class="empty-icon">⌕</span>
              <strong>No equipment found</strong>
              <p>Try changing your search or filters.</p>
              <button class="clear-button"
(click)="clearFilters()">
                Clear filters
              </button>
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <footer class="table-footer">
    <span>
      Showing {{ filteredEquipment.length }} of {{
equipment.length }} items
    </span>
    <span class="records-label">Equipment records</span>
  </footer>
 </section>
</main>
</div>

@if (showAddForm) {
<div class="modal-backdrop" (click)="showAddForm =
false">
 <section
  class="equipment-modal"
  role="dialog"
  aria-modal="true"
  aria-labelledby="add-equipment-title"
  (click)="$event.stopPropagation()"
 >
  <header class="modal-heading">
    <div>
      <h2 id="add-equipment-title">Add Equipment</h2>
      <p>Enter the details of the new equipment.</p>
```

```
        </div>
        <button
          type="button"
          class="close-button"
          aria-label="Close"
          (click)="showAddForm = false"
        >×</button>
      </header>

      <form
        #equipmentForm="ngForm"
        (ngSubmit)="addEquipment(
          newName.value,
          newCategory.value,
          newSerial.value,
          newType.value
        )"
      >
        <label class="form-field">
          Equipment Name *
          <input #newName required placeholder="e.g. Total
Station - 02" />
        </label>

        <label class="form-field">
          Category *
          <select #newCategory required>
            <option value="">Select category</option>
            @for (category of categories.slice(1); track category) {
              <option [value]="category">{{ category }}
</option>
            }
          </select>
        </label>

        <label class="form-field">
          Serial Number
          <input #newSerial placeholder="e.g. TS-002" />
        </label>

        <label class="form-field">
          Equipment Type *
          <select #newType required>
            <option value="">Select type</option>
            <option value="Individual">Individual</option>
            <option value="Bulk">Bulk</option>
```

```
          </select>
        </label>

        <div class="modal-actions">
          <button
            type="button"
            class="cancel-button"
            (click)="showAddForm = false"
          >
            Cancel
          </button>
          <button
            type="submit"
            class="primary-button"
            [disabled]="!equipmentForm.valid"
          >
            Save Equipment
          </button>
        </div>
      </form>
    </section>
  </div>
}

import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import {
  CreateEquipmentRequest,
  Equipment as EquipmentModel
} from '../../models/equipment';
import { EquipmentService } from '../../services/equipment';

interface Equipments {
  id: number;
  name: string;
  category: string;
  serialNo: string;
  type: string;
  status: string;
  assignedTo: string;
}

@Component({
  selector: 'app-equipment',
```

```
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './equipment.html',
  styleUrl: './equipment.css',
})
export class Equipment implements OnInit {
  private equipmentService = inject(EquipmentService);

  ngOnInit(): void {
    this.loadEquipment();
  }

  searchText = '';
  selectedCategory = 'All Categories';
  selectedStatus = 'All Status';
  showFilters = false;
  showAddForm = false;
  selectedEquipment: number | null = null;

  loading = false;
  saving = false;
  errorMessage = '';
  successMessage = '';

  categories = [
    'All Categories',
    'Total Station',
    'GPS',
    'Drone',
    'Tripod',
    'Tablet'
  ];

  statuses = [
    'All Status',
    'Available',
    'Checked Out',
    'Approval Required',
    'Under Repair'
  ];

  // Bound to your Add Equipment form
  newEquipment = {
    name: '',
    category: '',
    serialNo: '',
```

```
    type: 'Individual'
  };

  equipment: Equipments[] = [];

  // GET: Fetch equipment from the ASP.NET Core API
  loadEquipment(): void {
    this.loading = true;
    this.errorMessage = '';

    this.equipmentService.getAllEquipment().subscribe({
      next: (data: EquipmentModel[]) => {
        this.equipment = data.map(item => ({
          id: item.id,
          name: item.name,
          category: item.category,
          serialNo: item.serialNumber ?? '-',
          type: item.type,
          status: item.status,
          assignedTo: item.assignedTo ?? '-'
        }));

        this.loading = false;
      },
      error: (error) => {
        console.error('Error loading equipment:', error);
        this.errorMessage = 'Unable to load equipment. Please
try again.';
        this.loading = false;
      }
    });
  }

  // Search and filter equipment
  get filteredEquipment(): Equipments[] {
    const search = this.searchText.trim().toLowerCase();

    return this.equipment.filter(item => {
      const matchesSearch =
        !search ||
        item.name.toLowerCase().includes(search) ||
        item.category.toLowerCase().includes(search) ||
        item.serialNo.toLowerCase().includes(search) ||
        item.assignedTo.toLowerCase().includes(search);

      const matchesCategory =
```

```
        this.selectedCategory === 'All Categories' ||
        item.category === this.selectedCategory;

      const matchesStatus =
        this.selectedStatus === 'All Status' ||
        (this.selectedStatus === 'Available'
          ? item.status.startsWith('Available')
          : item.status === this.selectedStatus);

      return matchesSearch && matchesCategory &&
matchesStatus;
    });
  }

  // POST: Insert equipment using the values entered in the
form
 addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please enter the name, category, and
type.';
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: () => {
      this.saving = false;
      this.showAddForm = false;
      this.successMessage = 'Equipment added successfully.';
      this.loadEquipment();
    },
```

```
    error: (error) => {
      console.error('Error creating equipment:', error);
      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message || 'Unable to add equipment.';
    }
  });
}

  toggleFilters(): void {
    this.showFilters = !this.showFilters;
  }

  toggleActions(id: number): void {
    this.selectedEquipment =
      this.selectedEquipment === id ? null : id;
  }

  clearFilters(): void {
    this.searchText = '';
    this.selectedCategory = 'All Categories';
    this.selectedStatus = 'All Status';
  }

  trackById(_index: number, item: Equipments): number {
    return item.id;
  }
}
```

save equipment is not working ?? it is not saving to the database ??

24/10/2026, 19:30:29

I can see the main flow in your code. Your form calls `addEquipment()` and that method calls `EquipmentService.createEquipment()`, so the Angular code is intended to send a POST request. However, **the code alone does not confirm that the request is reaching SQL Server or being saved**.

Let's fix this step by step without changing your equipment page design.

1. Fix the form submission in `equipment.html`

Your form is already close to correct. Add `name` attributes to the fields and disable the button while saving.

Replace the form inside your modal with this:

```
<form
  #equipmentForm="ngForm"
  (ngSubmit)="addEquipment(
    newName.value,
    newCategory.value,
    newSerial.value,
    newType.value
  )"
>
  <label class="form-field">
    Equipment Name *
    <input
      #newName
      name="equipmentName"
      required
      placeholder="e.g. Total Station - 02"
    />
  </label>

  <label class="form-field">
    Category *
    <select #newCategory name="category" required>
      <option value="">Select category</option>
      @for (category of categories.slice(1); track category) {
        <option [value]="category">{{ category }}</option>
      }
    </select>
  </label>

  <label class="form-field">
    Serial Number
    <input
      #newSerial
      name="serialNumber"
      placeholder="e.g. TS-002"
    />
  </label>

  <label class="form-field">
    Equipment Type *
    <select #newType name="type" required>
      <option value="">Select type</option>
```

```
      <option value="Individual">Individual</option>
      <option value="Bulk">Bulk</option>
    </select>
  </label>

  @if (errorMessage) {
    <p class="error-message">{{ errorMessage }}</p>
  }

  <div class="modal-actions">
    <button
      type="button"
      class="cancel-button"
      (click)="showAddForm = false"
    >
      Cancel
    </button>

    <button
      type="submit"
      class="primary-button"
      [disabled]="!equipmentForm.valid || saving"
    >
      {{ saving ? 'Saving...' : 'Save Equipment' }}
    </button>
  </div>
</form>
```

2. Verify `equipment.ts`

Your component's `addEquipment()` method should look like this. It calls the POST API and displays the actual error if the request fails.

```
addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please fill in all required fields.';
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
```

```
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: (createdEquipment) => {
      console.log('Saved equipment:', createdEquipment);

      this.saving = false;
      this.showAddForm = false;
      this.successMessage = 'Equipment saved successfully.';

      this.loadEquipment();
    },
    error: (error) => {
      console.error('POST equipment failed:', error);

      this.saving = false;

      if (error.status === 0) {
        this.errorMessage =
          'Cannot connect to the API. Check the API URL and CORS settings.';
      } else if (error.status === 400) {
        this.errorMessage =
          typeof error.error === 'string'
            ? error.error
            : 'The API rejected the submitted equipment details.';
      } else {
        this.errorMessage =
          `Save failed (${error.status}). Check the API response and backend logs.`;
      }
    }
  });
}
```

3. Verify your Angular service

Open the service file you actually have at `src/app/services/equipment.ts`. Make sure its POST method is:

```
createEquipment(
  equipment: CreateEquipmentRequest
): Observable<Equipment> {
  return this.http.post<Equipment>(
    `${environment.apiUrl}/Equipment`,
    equipment
  );
}
```

The imports should include `Equipment` and `CreateEquipmentRequest` from your model file. Also confirm that `environment.apiUrl` points to the **running ASP.NET Core API**, not your Angular server.

For example, if Swagger opens at `https://localhost:7042/swagger`, your environment URL should be:

```
apiUrl: 'https://localhost:7042/api'
```

Use your actual port, not necessarily `7042`.

4. Check where the save is failing

This is the most important step.

1. Run your ASP.NET Core backend.
2. Open Swagger and test `POST /api/Equipment` directly with:

```
{
  "name": "Total Station - 02",
  "category": "Total Station",
  "serialNumber": "TS-002",
  "type": "Individual"
}
```

1. If Swagger succeeds, open your Angular app and press **F12 → Network**.
2. Open the Add Equipment modal, fill in the fields, and click **Save Equipment**.
3. Select the request to `POST /api/Equipment` and inspect its status code.

Result	What it usually means
`201 Created`	The API reports success; check the database and GET response.
`400 Bad Request`	The submitted data was rejected. Inspect the response body.

Result	What it usually means
`404 Not Found`	The API URL or route is incorrect.
`500 Internal Server Error`	Check the ASP.NET Core logs for a database or server exception.
`status 0` in Angular	Often a connection, HTTPS certificate, or CORS problem.

Also confirm your backend has registered the repository and service, configured `ApplicationDbContext` for SQL Server, and that the `Equipments` table exists.

Please test the POST request in Swagger first. If it works there but fails from Angular, send me the POST request's status code and response body from the browser's Network tab. If Swagger itself fails, send me its error response. That will identify the actual cause rather than guessing.

24/10/2026, 19:30:37 · gpt-6

Angular is running in development mode.
equipment:1 Access to XMLHttpRequest at 'https://localhost:7135/api/Equipment' from origin 'http://localhost:4200' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
installHook.js:1 Error loading equipment: HttpErrorResponse overrideMethod @ installHook.js:1
:7135/api/Equipment:1 Failed to load resource: net::ERR_FAILED
equipment:1 Access to XMLHttpRequest at 'https://localhost:7135/api/Equipment' from origin 'http://localhost:4200' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.
installHook.js:1 Error creating equipment: HttpErrorResponse overrideMethod @ installHook.js:1
:7135/api/Equipment:1 Failed to load resource: net::ERR_FAILED
equipment:1 Access to XMLHttpRequest at 'https://localhost:7135/api/Equipment' from origin 'http://localhost:4200' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.
installHook.js:1 Error creating equipment: HttpErrorResponse overrideMethod @ installHook.js:1
:7135/api/Equipment:1 Failed to load resource: net::ERR_FAILED
equipment:1 Access to XMLHttpRequest at 'https://localhost:7135/api/Equipment' from origin 'http://localhost:4200' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.
installHook.js:1 Error creating equipment: HttpErrorResponse {headers: _HttpHeaders, status: 0, statusText: 'Unknown Error', url: 'https://localhost:7135/api/Equipment', ok: false, …}error: ProgressEvent {isTrusted: true, lengthComputable: false, loaded: 0, total: 0, type: 'error', …}headers: _HttpHeaders {headers: Map(0), normalizedNames: Map(0), lazyInit: undefined, lazyUpdate: null}message: "Http failure

response for https://localhost:7135/api/Equipment: 0 Unknown Error"name: "HttpErrorResponse"ok: falseredirected: undefinedresponseType: undefinedstatus: 0statusText: "Unknown Error"type: undefinedurl: "https://localhost:7135/api/Equipment"[[Prototype]]: HttpResponseBase
overrideMethod @ installHook.js:1
(anonymous) @ equipment.ts:159
ConsumerObserver.prototype.error @ Subscriber.js:107
Subscriber.prototype._error @ Subscriber.js:67
Subscriber.prototype.error @ Subscriber.js:43
Subscriber.prototype._error @ Subscriber.js:67
Subscriber.prototype.error @ Subscriber.js:43
Subscriber.prototype._error @ Subscriber.js:67
Subscriber.prototype.error @ Subscriber.js:43
Subscriber.prototype._error @ Subscriber.js:67
Subscriber.prototype.error @ Subscriber.js:43
Subscriber.prototype._error @ Subscriber.js:67
Subscriber.prototype.error @ Subscriber.js:43
(anonymous) @ _module-chunk.mjs:1154
XMLHttpRequest.send
(anonymous) @ _module-chunk.mjs:1209
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: createOperatorSubscriber @ switchMap.js:14
anonymousFunction @ OperatorSubscriber.js:15
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: new Observable @ innerFrom.js:51
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ switchMap.js:10
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ finalize.js:5
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
doInnerSub @ mergeInternals.js:19

outerNext @ mergeInternals.js:14
anonymousFunction @ OperatorSubscriber.js:15
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: new Observable @ innerFrom.js:51
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
mergeInternals @ mergeInternals.js:53
anonymousCallbackTo: operate @ mergeMap.js:14
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ filter.js:6
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ map.js:6
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
addEquipment @ equipment.ts:151
(anonymous) @ equipment.ts:155
executeListenerWithErrorHandling @ _debug_node-chunk.mjs:8247
wrapListenerIn_markDirtyAndPreventDefault @ _debug_node-chunk.mjs:8234
ConsumerObserver.prototype.next @ Subscriber.js:96
Subscriber.prototype._next @ Subscriber.js:63
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: errorContext @ Subject.js:41
errorContext @ errorContext.js:19
Subject.prototype.next @ Subject.js:31
EventEmitter_.emit @ _effect-chunk2.mjs:2306
onSubmit @ forms.mjs:1823
NgForm_submit_HostBindingHandler @ forms.mjs:1852
executeListenerWithErrorHandling @ _debug_node-chunk.mjs:8247
wrapListenerIn_markDirtyAndPreventDefault @ _debug_node-chunk.mjs:8234
(anonymous) @ _dom_renderer-chunk.mjs:566
equipment.ts:151 POST https://localhost:7135/api/Equipment net::ERR_FAILED

(anonymous) @ _module-chunk.mjs:1209
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: createOperatorSubscriber @ switchMap.js:14
anonymousFunction @ OperatorSubscriber.js:15
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: new Observable @ innerFrom.js:51
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ switchMap.js:10
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ finalize.js:5
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
doInnerSub @ mergeInternals.js:19
outerNext @ mergeInternals.js:14
anonymousFunction @ OperatorSubscriber.js:15
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: new Observable @ innerFrom.js:51
Observable.prototype._trySubscribe @ Observable.js:38
anonymousCallbackTo: errorContext @ Observable.js:32
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
mergeInternals @ mergeInternals.js:53
anonymousCallbackTo: operate @ mergeMap.js:14
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ filter.js:6
anonymousCallbackTo: source.lift @ lift.js:10
anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
anonymousCallbackTo: operate @ map.js:6
anonymousCallbackTo: source.lift @ lift.js:10

anonymousCallbackTo: errorContext @ Observable.js:27
errorContext @ errorContext.js:19
Observable.prototype.subscribe @ Observable.js:23
addEquipment @ equipment.ts:151
(anonymous) @ equipment.ts:155
executeListenerWithErrorHandling @ _debug_node-chunk.mjs:8247
wrapListenerIn_markDirtyAndPreventDefault @ _debug_node-chunk.mjs:8234
ConsumerObserver.prototype.next @ Subscriber.js:96
Subscriber.prototype._next @ Subscriber.js:63
Subscriber.prototype.next @ Subscriber.js:34
anonymousCallbackTo: errorContext @ Subject.js:41
errorContext @ errorContext.js:19
Subject.prototype.next @ Subject.js:31
EventEmitter_.emit @ _effect-chunk2.mjs:2306
onSubmit @ forms.mjs:1823
NgForm_submit_HostBindingHandler @ forms.mjs:1852
executeListenerWithErrorHandling @ _debug_node-chunk.mjs:8247
wrapListenerIn_markDirtyAndPreventDefault @ _debug_node-chunk.mjs:8234
(anonymous) @ _dom_renderer-chunk.mjs:566
[NEW] Explain Console errors by using Copilot in Edge: click to explain an error. Learn more

24/10/2026, 19:32:53

Your console clearly identifies the problem: **your ASP.NET Core API at `https://localhost:7135` is not allowing requests from your Angular app at `http://localhost:4200` because CORS is not configured correctly.**

This explains why both `GET` (load equipment) and `POST` (save equipment) fail. The browser blocks the requests before Angular can receive the API response, so your equipment is not being saved through this request.

You do **not** need to change your Angular form or service to fix this. Configure CORS in the **ASP.NET Core backend**.

1. Update `Program.cs`

Add a CORS policy before `builder.Build()`, then enable it in the middleware pipeline.

Use this pattern in `Program.cs`. Keep your existing database, repository, service, and Swagger registrations.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Repositories;
using SolvefyTask.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Allow requests from the Angular development server
builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularPolicy", policy =>
    {
        policy
            .WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

// Keep your existing database registration
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    ));

// Keep your existing dependency registrations
builder.Services.AddScoped<IEquipmentRepository, EquipmentRepository>();
builder.Services.AddScoped<IEquipmentService, EquipmentService>();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

// Enable CORS before authorization and controller endpoints
app.UseCors("AngularPolicy");
```

```
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Important: If your existing `Program.cs` already registers `ApplicationDbContext`, the repository, or the service, do not duplicate those registrations. Add only the CORS configuration.

2. Restart the backend

After saving `Program.cs`:

1. Stop the ASP.NET Core API.
2. Run it again.
3. Confirm it is still listening on `https://localhost:7135`.
4. Refresh Angular at `http://localhost:4200`.
5. Open the Equipment page and try loading and saving equipment again.

3. Verify the result

Open your browser's Developer Tools → **Network**.

- `GET https://localhost:7135/api/Equipment` should return `200 OK`.
- `POST https://localhost:7135/api/Equipment` should return `201 Created` if the insert succeeds.

If the CORS error disappears but you get a different error, inspect the new status code and response body. That will help identify whether there is a separate database or API issue.

One final note: if you access Angular using a different origin, such as `http://127.0.0.1:4200`, add that exact origin to `.WithOrigins(...)` as well. CORS origins must match the browser's origin exactly.

24/10/2026, 19:33:12 · gpt-6

Angular is running in development mode.
equipment:1 Access to XMLHttpRequest at 'https://localhost:7135/api/Equipment' from origin 'http://localhost:4200' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
installHook.js:1 Error loading equipment: HttpErrorResponse overrideMethod @ installHook.js:1
:7135/api/Equipment:1 Failed to load resource: net::ERR_FAILED

24/10/2026, 19:36:44

The CORS error is still happening, which means the browser is **still not receiving the required CORS header** from `https://localhost:7135`. Let's check the backend configuration rather than changing Angular again.

1. Check your `Program.cs` middleware order

Make sure these lines are in this exact order:

```
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseRouting();

app.UseCors("AngularPolicy");

app.UseAuthorization();

app.MapControllers();

app.Run();
```

And **before** `builder.Build()`, make sure this policy is registered:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularPolicy", policy =>
    {
        policy
            .WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

Do not register a different policy name and then call `UseCors("AngularPolicy")`.

2. Verify that the correct backend is running

Your Angular app calls:

`https://localhost:7135/api/Equipment`

Open that exact address in your browser. It should reach your ASP.NET Core API, even if it displays JSON or an authorization/error response.

Also check `Properties/launchSettings.json` to confirm the API is actually running on port `7135`. Stop any old backend instance and restart the correct project after changing `Program.cs`.

3. Test the CORS response directly

Open **PowerShell** and run:

```
curl.exe -k -i -X OPTIONS "https://localhost:7135/api/Equipment" `
  -H "Origin: http://localhost:4200" `
  -H "Access-Control-Request-Method: GET" `
  -H "Access-Control-Request-Headers: content-type"
```

Look for this response header:

```
Access-Control-Allow-Origin: http://localhost:4200
```

If it is missing, the CORS policy is not being applied to that response. The status code and other headers will help narrow down why.

4. Send me these two things

To pinpoint the remaining issue, please paste:

1. Your **complete** `Program.cs`.
2. The output of the PowerShell command above.

With those, I can identify whether the problem is middleware ordering, the wrong startup project, or another response being generated before CORS middleware runs.

24/10/2026, 19:36:44 · gpt-6

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Repositories;
using SolvefyTask.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularPolicy", policy =>
    {
        policy
            .WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});



// SQL Server database
builder.Services.AddDbContext<ApplicationDbContext>
(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")
    ));




builder.Services.AddScoped<IEquipmentRepository,
EquipmentRepository>();
builder.Services.AddScoped<IEquipmentService,
EquipmentService>();
```

```
var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
   app.UseSwagger();
   app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

24/10/2026, 19:37:39

I found the issue in your `Program.cs`: **you registered the CORS policy but never enabled it in the HTTP request pipeline.**

Your code has `AddCors(...)`, but it is missing `app.UseCors("AngularPolicy")`.

Fix your `Program.cs`

Replace your existing middleware section, starting from `var app = builder.Build();`, with this:

```
var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseRouting();

// Enable CORS for Angular
app.UseCors("AngularPolicy");
```

```
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Keep your existing service registrations and CORS policy exactly as they are.

Now test it

1. Save `Program.cs`.
2. Stop the ASP.NET Core backend completely.
3. Start it again and confirm the URL is `https://localhost:7135`.
4. Refresh Angular at `http://localhost:4200`.

Your Angular app should now be able to call:

- `GET https://localhost:7135/api/Equipment`
- `POST https://localhost:7135/api/Equipment`

If the CORS error disappears but a new error appears, share that error. We can then troubleshoot the next issue, if any.

24/10/2026, 19:37:39 · gpt-6

```
<div class="equipment-page">
  <div class="page-banner">3. Equipment List</div>

  <main class="page-content">
    <header class="page-heading">
      <div>
        <h1>Equipment</h1>
        <p>View and manage all equipment</p>
      </div>

      <button
          type="button"
          class="primary-button"
          (click)="showAddForm = true"
        >
          <span>+</span> Add Equipment
        </button>
    </header>

    <section class="toolbar">
      <label class="search-box">
        <span class="search-icon">⌕</span>
        <input
          type="search"
          placeholder="Search equipment..."
          [(ngModel)]="searchText"
        />
      </label>

      <select
        aria-label="Filter by category"
        [(ngModel)]="selectedCategory"
      >
        @for (category of categories; track category) {
          <option [value]="category">{{ category }}</option>
        }
      </select>

      <select
        aria-label="Filter by status"
        [(ngModel)]="selectedStatus"
      >
        @for (status of statuses; track status) {
          <option [value]="status">{{ status }}</option>
```

```
      }
    </select>

    <button class="filter-button" (click)="toggleFilters()">
      <span>☷</span> Filter
    </button>
  </section>

  @if (showFilters) {
    <section class="filter-panel">
      <div>
        <strong>Equipment filters</strong>
        <p>Filter the list by category and current status.</p>
      </div>
      <button class="clear-button" (click)="clearFilters()">
        Clear filters
      </button>
    </section>
  }

  <section class="table-card">
    <div class="table-scroll">
      <table>
        <thead>
          <tr>
            <th>Name <span class="sort-icon">↕</span></th>
            <th>Category</th>
            <th>Serial No.</th>
            <th>Type</th>
            <th>Status</th>
            <th>Assigned To</th>
            <th class="actions-column">Actions</th>
          </tr>
        </thead>

        <tbody>
          @for (item of filteredEquipment; track item.id) {
            <tr>
              <td>
                <div class="equipment-name">
                  <span class="equipment-icon">
                    @switch (item.category) {
                      @case ('Total Station') { ⊙ }
                      @case ('GPS') { ◉ }
                      @case ('Drone') { ✣ }
                      @case ('Tripod') { ╱ }
```

```
                @case ('Tablet') { ▣ }
                @default { ▦ }
              }
            </span>
            <span>{{ item.name }}</span>
          </div>
        </td>

        <td>{{ item.category }}</td>
        <td>{{ item.serialNo }}</td>
        <td>{{ item.type }}</td>

        <td>
          <span
            class="status-badge"

[class.available]="item.status.startsWith('Available')"
            [class.checked-out]="item.status === 'Checked
Out'"
            [class.approval]="item.status === 'Approval
Required'"
            [class.repair]="item.status === 'Under Repair'"
          >
            {{ item.status }}
          </span>
        </td>

        <td>{{ item.assignedTo }}</td>

        <td class="actions-column">
          <button
            class="action-button"
            [attr.aria-label]="'Actions for ' + item.name"
            (click)="toggleActions(item.id)"
          >
            ...
          </button>

          @if (selectedEquipment === item.id) {
            <div class="action-menu">
              <button (click)="toggleActions(item.id)">
                View Details
              </button>
              <button (click)="toggleActions(item.id)">
                Edit Equipment
              </button>
```

```
                </div>
              }
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="7" class="empty-state">
              <span class="empty-icon">⌕</span>
              <strong>No equipment found</strong>
              <p>Try changing your search or filters.</p>
              <button class="clear-button"
(click)="clearFilters()">
                Clear filters
              </button>
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <footer class="table-footer">
    <span>
      Showing {{ filteredEquipment.length }} of {{
equipment.length }} items
    </span>
    <span class="records-label">Equipment records</span>
  </footer>
 </section>
</main>
</div>

@if (showAddForm) {
<div class="modal-backdrop" (click)="showAddForm =
false">
 <section
  class="equipment-modal"
  role="dialog"
  aria-modal="true"
  aria-labelledby="add-equipment-title"
  (click)="$event.stopPropagation()"
 >
  <header class="modal-heading">
    <div>
      <h2 id="add-equipment-title">Add Equipment</h2>
      <p>Enter the details of the new equipment.</p>
```

```
        </div>
        <button
          type="button"
          class="close-button"
          aria-label="Close"
          (click)="showAddForm = false"
        >×</button>
      </header>


        <form
          #equipmentForm="ngForm"
          (ngSubmit)="addEquipment(
            newName.value,
            newCategory.value,
            newSerial.value,
            newType.value
          )"
        >
          <label class="form-field">
            Equipment Name *
            <input
              #newName
              name="equipmentName"
              required
              placeholder="e.g. Total Station - 02"
            />
          </label>

          <label class="form-field">
            Category *
            <select #newCategory name="category" required>
              <option value="">Select category</option>
              @for (category of categories.slice(1); track category)
{
                <option [value]="category">{{ category }}
</option>
              }
            </select>
          </label>

          <label class="form-field">
            Serial Number
            <input
              #newSerial
              name="serialNumber"
```

```
            placeholder="e.g. TS-002"
          />
        </label>

        <label class="form-field">
          Equipment Type *
          <select #newType name="type" required>
            <option value="">Select type</option>
            <option value="Individual">Individual</option>
            <option value="Bulk">Bulk</option>
          </select>
        </label>

        @if (errorMessage) {
          <p class="error-message">{{ errorMessage }}</p>
        }

        <div class="modal-actions">
          <button
            type="button"
            class="cancel-button"
            (click)="showAddForm = false"
          >
            Cancel
          </button>

          <button
            type="submit"
            class="primary-button"
            [disabled]="!equipmentForm.valid || saving"
          >
            {{ saving ? 'Saving...' : 'Save Equipment' }}
          </button>
        </div>
      </form>

    </section>
  </div>
}
```

i want when i click on the save button it should close the add equipment modal and take me back to page ??

24/10/2026, 19:40:58

Your modal can close automatically after the equipment is successfully saved. You already have most of the required code in `equipment.ts`.

1. Update `addEquipment()` in `equipment.ts`

Inside the `next` callback, set `showAddForm = false` after the API returns a successful response.

```
this.equipmentService.createEquipment(request).subscribe({
  next: (createdEquipment) => {
    console.log('Equipment saved:', createdEquipment);

    // Close the Add Equipment modal
    this.showAddForm = false;

    // Reset form messages and loading state
    this.saving = false;
    this.successMessage = 'Equipment added successfully.';

    // Reload equipment from the database
    this.loadEquipment();
  },
  error: (error) => {
    console.error('Error creating equipment:', error);

    this.saving = false;
    this.errorMessage =
      typeof error?.error === 'string'
        ? error.error
        : error?.error?.message || 'Unable to add equipment.';
  }
});
```

2. Keep your HTML as it is

Your modal is already controlled by:

```
@if (showAddForm) {
  <div class="modal-backdrop">
    <!-- Add Equipment modal -->
  </div>
}
```

When `showAddForm` becomes `false`, Angular removes the modal from the page and returns the user to the Equipment list.

Expected behavior:

1. Click **Add Equipment** to open the modal.
2. Enter the equipment details.
3. Click **Save Equipment**.
4. The API saves the record to SQL Server.
5. On success, the modal closes and the Equipment list refreshes.

If the API request fails, the modal stays open so the user can see the error and try again.

24/10/2026, 19:40:58 · gpt-6

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import {
  CreateEquipmentRequest,
  Equipment as EquipmentModel
} from '../../models/equipment';
import { EquipmentService } from '../../services/equipment';

interface Equipments {
  id: number;
  name: string;
  category: string;
  serialNo: string;
  type: string;
  status: string;
  assignedTo: string;
}

@Component({
  selector: 'app-equipment',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './equipment.html',
  styleUrl: './equipment.css',
})
export class Equipment implements OnInit {
  private equipmentService = inject(EquipmentService);

  ngOnInit(): void {
    this.loadEquipment();
  }

  searchText = '';
  selectedCategory = 'All Categories';
  selectedStatus = 'All Status';
  showFilters = false;
  showAddForm = false;
  selectedEquipment: number | null = null;

  loading = false;
  saving = false;
  errorMessage = '';
```

```
  successMessage = '';

  categories = [
    'All Categories',
    'Total Station',
    'GPS',
    'Drone',
    'Tripod',
    'Tablet'
  ];

  statuses = [
    'All Status',
    'Available',
    'Checked Out',
    'Approval Required',
    'Under Repair'
  ];

  // Bound to your Add Equipment form
  newEquipment = {
    name: '',
    category: '',
    serialNo: '',
    type: 'Individual'
  };

  equipment: Equipments[] = [];

  // GET: Fetch equipment from the ASP.NET Core API
  loadEquipment(): void {
    this.loading = true;
    this.errorMessage = '';

    this.equipmentService.getAllEquipment().subscribe({
      next: (data: EquipmentModel[]) => {
        this.equipment = data.map(item => ({
          id: item.id,
          name: item.name,
          category: item.category,
          serialNo: item.serialNumber ?? '-',
          type: item.type,
          status: item.status,
          assignedTo: item.assignedTo ?? '-'
        }));
```

```
        this.loading = false;
      },
      error: (error) => {
        console.error('Error loading equipment:', error);
        this.errorMessage = 'Unable to load equipment. Please
try again.';
        this.loading = false;
      }
    });
  }

  // Search and filter equipment
  get filteredEquipment(): Equipments[] {
    const search = this.searchText.trim().toLowerCase();

    return this.equipment.filter(item => {
      const matchesSearch =
        !search ||
        item.name.toLowerCase().includes(search) ||
        item.category.toLowerCase().includes(search) ||
        item.serialNo.toLowerCase().includes(search) ||
        item.assignedTo.toLowerCase().includes(search);

      const matchesCategory =
        this.selectedCategory === 'All Categories' ||
        item.category === this.selectedCategory;

      const matchesStatus =
        this.selectedStatus === 'All Status' ||
        (this.selectedStatus === 'Available'
          ? item.status.startsWith('Available')
          : item.status === this.selectedStatus);

      return matchesSearch && matchesCategory &&
matchesStatus;
    });
  }

  // POST: Insert equipment using the values entered in the
form
 addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
```

```
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please enter the name, category, and
type.';
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: () => {

      this.saving = false;
      this.showAddForm = false;
      this.successMessage = 'Equipment added successfully.';
      this.loadEquipment();
    },
    error: (error) => {
      console.error('Error creating equipment:', error);
      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message || 'Unable to add equipment.';
    }
  });
}

  toggleFilters(): void {
    this.showFilters = !this.showFilters;
  }

  toggleActions(id: number): void {
    this.selectedEquipment =
      this.selectedEquipment === id ? null : id;
  }

  clearFilters(): void {
```

```
    this.searchText = '';
    this.selectedCategory = 'All Categories';
    this.selectedStatus = 'All Status';
  }

  trackById(_index: number, item: Equipments): number {
    return item.id;
  }
}
```

it is not closing on its own ??

24/10/2026, 19:43:11

Your TypeScript code already sets `this.showAddForm = false` correctly. The likely issue is that the POST request is failing, or the modal is not being controlled by the same `showAddForm` property.

Let's make the behavior clear and reliable.

1. Replace your `addEquipment()` method

Use this complete method in `equipment.ts`:

```
addEquipment(
  name: string,
  category: string,
  serialNo: string,
  type: string
): void {
  if (!name.trim() || !category || !type) {
    this.errorMessage = 'Please enter the name, category, and type.';
    return;
  }

  const request: CreateEquipmentRequest = {
    name: name.trim(),
    category,
    serialNumber: serialNo.trim() || null,
    type
  };

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.equipmentService.createEquipment(request).subscribe({
    next: (createdEquipment) => {
```

```
      console.log('Equipment saved successfully:', createdEquipment);

      // Close the modal after successful API response
      this.showAddForm = false;

      this.saving = false;
      this.successMessage = 'Equipment added successfully.';

      // Refresh the equipment list
      this.loadEquipment();
    },
    error: (error) => {
      console.error('Equipment save failed:', error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message || 'Unable to save equipment.';
    }
  });
}
```

2. Check the modal condition in `equipment.html`

Your modal should be controlled by this property:

```
@if (showAddForm) {
  <div class="modal-backdrop" (click)="showAddForm = false">
    <section
      class="equipment-modal"
      role="dialog"
      aria-modal="true"
      (click)="$event.stopPropagation()"
    >
      <!-- Your existing modal content -->
    </section>
  </div>
}
```

Keep your existing modal content inside this structure. There should not be another `showAddForm` variable in a nested component.

3. Check the Save button

```
<button
  type="submit"
  class="primary-button"
  [disabled]="!equipmentForm.valid || saving"
>
  {{ saving ? 'Saving...' : 'Save Equipment' }}
</button>
```

And make sure your form uses:

```
<form
  #equipmentForm="ngForm"
  (ngSubmit)="addEquipment(
    newName.value,
    newCategory.value,
    newSerial.value,
    newType.value
  )"
>
```

4. If it still does not close

Open the browser's Developer Tools (`F12`) → **Console** and **Network**. Submit the form and check the `POST /api/Equipment` request.

- If it returns **201 Created**, the modal should close with the code above.
- If it returns **400 or 500**, the API call failed and your modal intentionally remains open.
- If no POST request appears, the form submission is not firing.

Please check one thing first: after clicking Save Equipment, does the Network tab show `POST /api/Equipment` with status `201`, or does it show an error? That will tell us exactly what to fix next.

24/10/2026, 19:43:34 · gpt-6

\- Node.js (LTS version). Check with `node -v`
\- Angular CLI: `npm install -g @angular/cli`
1\. Open a terminal in the frontend folder:
   cd src/web
2\. Install the dependencies (first time only):
   npm install
3\. Start the app:
   ng serve
4\. Open http://localhost:4200 in your browser.
The page reloads automatically when files change. Stop the server with `Ctrl+C`.
\### Notes

\- The frontend talks to the API at `https://localhost:5001`. Start the API first (see "Running the Backend" above).

\- If `npm install` fails, delete the `node\_modules` folder and `package-lock.json`, run `npm cache clean --force`, and try again.

please write also the steps you have taken to connect frontend with backend because i have to write it in readme and also submit it ??

24/10/2026, 19:50:15

Here is a README-ready section you can add to your SolvefyTask `README.md`. It documents the frontend setup and the steps taken to connect Angular with the ASP.NET Core backend and SQL Server.

One correction before submitting: your current API URL is `https://localhost:7135/api`, based on the console logs you shared. The existing README note says port `5001`, so make sure you use the actual configured port consistently.

Frontend Setup and Backend Integration

Prerequisites

Install the following tools before running the application:

- **Node.js:** LTS version. Verify using `node -v`.
- **npm:** Installed with Node.js. Verify using `npm -v`.

- **Angular CLI:** Install using `npm install -g @angular/cli`.
- **.NET SDK:** The version required by the backend project.
- **SQL Server:** Required for storing equipment records.

Running the Frontend

1. Open a terminal in the frontend directory:

   ```
   cd src/web
   ```

   Use the actual Angular project directory if it differs from this path.

2. Install dependencies for the first run:

   ```
   npm install
   ```

3. Start the Angular development server:

   ```
   ng serve
   ```

4. Open the application in your browser:

   `http://localhost:4200`

The development server automatically rebuilds the application when source files change. Stop it using `Ctrl+C`.

Connecting Angular to the ASP.NET Core API

The Angular frontend communicates with the ASP.NET Core Web API using Angular's `HttpClient`. The API accesses SQL Server through Entity Framework Core.

The request flow is:

Angular Component → Angular EquipmentService → ASP.NET Core EquipmentController → EquipmentService → EquipmentRepository → Entity Framework Core → SQL Server

1. Configure the API base URL

The API base URL is stored in the Angular environment configuration rather than hardcoded into each component.

Example for local development:

```
export const environment = {
  production: false,
```

```
  apiUrl: 'https://localhost:7135/api'
};
```

The port must match the URL configured for the running ASP.NET Core backend. Update the environment configuration if the backend uses a different port.

2. Define the equipment data models

The Angular application defines TypeScript interfaces for equipment records and equipment creation requests.

The `Equipment` interface describes the data returned by the API, including the ID, name, category, serial number, type, status, and assigned user.

The `CreateEquipmentRequest` interface describes the fields sent when creating a new equipment record.

3. Create the Angular EquipmentService

The service uses Angular `HttpClient` to communicate with the backend. It centralizes HTTP requests so the component does not need to construct API URLs or issue requests directly.

Two operations are implemented:

- **GET** `/api/Equipment` — retrieves equipment records from the backend.
- **POST** `/api/Equipment` — submits a new equipment record for insertion.

The service returns RxJS observables, allowing the component to handle successful responses and errors.

4. Load equipment records

When the Equipment component initializes, its `ngOnInit()` lifecycle hook calls `loadEquipment()`.

The method calls `EquipmentService.getAllEquipment()`. On success, the returned records are mapped to the fields used by the frontend table and displayed in the equipment list.

The existing search, category filter, and status filter operate on the loaded records.

5. Insert equipment through the Add Equipment form

The Add Equipment button opens a modal containing fields for:

- Equipment name
- Category
- Serial number

- Equipment type

When the user submits the form, the component validates the required fields and creates a `CreateEquipmentRequest` object.

The component then calls `EquipmentService.createEquipment()`, which sends a POST request to the API.

If the API responds successfully, the modal closes and the component reloads the equipment list using the GET endpoint. If the request fails, the component displays an error message.

6. Implement the backend API

The ASP.NET Core backend uses a layered architecture:

- **EquipmentController:** Exposes the GET and POST HTTP endpoints.
- **EquipmentService:** Handles the application-level equipment operations.
- **EquipmentRepository:** Performs database operations using Entity Framework Core.
- **ApplicationDbContext:** Provides access to the equipment table in SQL Server.

The POST endpoint inserts the equipment record and saves the changes using `SaveChangesAsync()`. The GET endpoint retrieves the stored records.

7. Configure CORS

The Angular development server and the ASP.NET Core API run on different origins. To allow the frontend to call the API, a CORS policy named `AngularPolicy` is registered in the backend.

The policy allows requests from:

`http://localhost:4200`

It permits the required HTTP methods and headers. The policy is enabled in the ASP.NET Core request pipeline using `app.UseCors("AngularPolicy")`.

When the frontend origin or backend port changes, the corresponding configuration must be updated.

8. Configure SQL Server

The backend uses Entity Framework Core with SQL Server. The connection string is stored in the backend configuration under `ConnectionStrings:DefaultConnection`.

The `ApplicationDbContext` is registered with dependency injection using `UseSqlServer()`. Entity Framework Core migrations are used to create and update the database schema.

Before testing the API, ensure that SQL Server is available, the connection string is correct, and the required migrations have been applied.

Running the Full Application

1. Start SQL Server and verify the database connection string.
2. Start the ASP.NET Core backend.
3. Confirm that the API is listening on the configured HTTPS URL.
4. Start Angular using `ng serve`.
5. Open `http://localhost:4200`.
6. Navigate to the Equipment page.
7. Verify that existing equipment is loaded through the GET endpoint.
8. Submit the Add Equipment form and verify that the POST request succeeds.
9. Confirm that the newly created record appears in the list and is stored in SQL Server.

Troubleshooting

- **CORS errors:** Check the registered CORS policy, middleware order, frontend origin, and backend startup configuration.
- **Connection errors:** Verify the API URL, HTTPS certificate, and that the backend is running.
- **Database errors:** Verify the SQL Server connection string, database availability, migrations, and backend logs.
- **HTTP 400 or 500 responses:** Inspect the response body and ASP.NET Core logs to identify validation or server-side errors.
- **Missing equipment records:** Check the GET response and confirm that the API is connected to the intended database.

Current Integration Scope

The Equipment page is connected to the GET and POST equipment endpoints. The page loads records from the backend and submits new equipment records through the API.

Other equipment-management operations, such as editing records, deleting equipment, checking equipment in or out, and managing approval workflows, require their own backend endpoints and integration unless implemented separately.

Before submitting: confirm that the actual frontend directory is `src/web` or `src/AngularFrontend/solvefy-task`, and correct the README accordingly. Also ensure that the README describes only features you have actually implemented. For the SolvefyTask assessment, it is useful to list any incomplete workflows separately rather than implying the entire application is fully integrated.

24/10/2026, 19:50:15 · gpt-6

please make it small only relavant details ??

24/10/2026, 19:55:59

Frontend Setup and Backend Integration

Run the Frontend

```
cd src/AngularFrontend/solvefy-task
npm install
ng serve
```

Open `http://localhost:4200`.

Prerequisites: Node.js (LTS), npm, Angular CLI, .NET SDK, and SQL Server.

Frontend–Backend Integration

- **Environment Configuration:** Configured the API base URL in `environment.ts` to avoid hardcoding URLs in components.
- **Angular Service:** Created `EquipmentService` using Angular `HttpClient` to call the backend APIs.
- **GET Equipment:** Fetches equipment records from `GET /api/Equipment` and displays them in the equipment list.
- **POST Equipment:** Sends new equipment details to `POST /api/Equipment` through the Add Equipment form.
- **Backend Architecture:** Requests flow through the ASP.NET Core controller, service, and repository layers to SQL Server using Entity Framework Core.
- **CORS Configuration:** Configured the backend to allow requests from `http://localhost:4200`.
- **After Saving:** On successful insertion, the modal closes and the equipment list refreshes from the API.

Run the Full Application

1. Start SQL Server.
2. Run the ASP.NET Core backend.

3. Run the Angular frontend using `ng serve`.
4. Open the application and test loading and adding equipment.

Note: Ensure the API URL in `environment.ts` matches the backend's actual URL and port. Other workflows should be listed as incomplete if they have not yet been implemented.

24/10/2026, 19:55:59 · gpt-6

Combine the two run sections

24/10/2026, 19:56:18

Setup and Run the Application

Prerequisites: Node.js (LTS), npm, Angular CLI, .NET SDK, and SQL Server.

1. Start SQL Server and ensure the database connection string is configured.
2. Start the ASP.NET Core backend and confirm its API URL.
3. Open a terminal in the Angular frontend directory and run:

```
cd src/AngularFrontend/solvefy-task
npm install
ng serve
```

4. Open `http://localhost:4200` in your browser.

Frontend–Backend Integration

- **Environment Configuration:** Stored the API base URL in `environment.ts`.
- **Angular Service:** Created `EquipmentService` using `HttpClient` to call `GET /api/Equipment` and `POST /api/Equipment`.
- **Equipment List:** Fetches equipment records from the backend and displays them in the table.
- **Add Equipment:** Submits form data to the POST API; on success, the modal closes and the list refreshes.
- **Backend Architecture:** Requests pass through the ASP.NET Core controller, service, and repository layers to SQL Server using Entity Framework Core.
- **CORS:** Configured the backend to allow requests from `http://localhost:4200`.

Note: Ensure the environment API URL matches the backend's actual URL and port. Features not yet implemented should be documented separately.

24/10/2026, 19:56:19 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout {

  constructor(private router: Router) {}

  selectedEquipment = 'GPS Rover - 01';
  assignedTo = 'Ali Raza';
  expectedReturnDate = '2026-11-10';
  purpose = '';

  submitted = false;

  equipmentOptions = [
    { name: 'GPS Rover - 01', availability: 'Available' },
    { name: 'Total Station - 01', availability: 'Available' },
    { name: 'Drone - DJI Mavic', availability: 'Available' },
    { name: 'Field Tablet - 01', availability: 'Available' }
  ];

  teamMembers = [
    'Ali Raza',
    'Ahmad',
    'Usman',
    'Muhammad Ahmed'
  ];

  requestCheckOut(): void {
    this.submitted = true;

    if (
      !this.selectedEquipment ||
      !this.assignedTo ||
      !this.expectedReturnDate
    ) {
      return;
```

```
    }

    // Demo only: connect your checkout API here.
    alert(
      `Check-out request submitted for
${this.selectedEquipment}.`
    );
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

please make the api of this such that the api performs operations like insert ??

24/10/2026, 20:59:21

We can implement your **Check Out API** using the same architecture as your Equipment API:

Angular Checkout Component → CheckoutService → CheckoutController → CheckoutService (backend) → CheckoutRepository → SQL Server

The API will save the selected equipment, assigned team member, expected return date, and purpose in the database. We'll keep your existing Angular form and replace the demo `alert()` with a real API call.

1. Create the backend model

Create `Models/Checkout.cs`:

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
    public class Checkout
    {
        public int Id { get; set; }

        [Required]
        public string EquipmentName { get; set; } = string.Empty;
```

```
        [Required]
        public string AssignedTo { get; set; } = string.Empty;

        public DateTime ExpectedReturnDate { get; set; }

        public string Purpose { get; set; } = string.Empty;

        public string Status { get; set; } = "Pending";

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    }
}
```

This version stores the equipment name to match your current Angular form. Later, you can improve it by linking checkout records to equipment and team-member IDs.

2. Add the table to your DbContext

In `Data/ApplicationDbContext.cs`, add the `Checkout` DbSet while keeping your existing `Equipments` DbSet.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Models;

namespace SolvefyTask.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Equipment> Equipments => Set<Equipment>();

        public DbSet<Checkout> Checkouts => Set<Checkout>();
    }
}
```

3. Create the repository

`Repositories/ICheckoutRepository.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
```

```
{
    public interface ICheckoutRepository
    {
        Task<Checkout> InsertAsync(Checkout checkout);
    }
}
```

`Repositories/CheckoutRepository.cs`

```
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class CheckoutRepository : ICheckoutRepository
    {
        private readonly ApplicationDbContext _context;

        public CheckoutRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<Checkout> InsertAsync(Checkout checkout)
        {
            _context.Checkouts.Add(checkout);
            await _context.SaveChangesAsync();

            return checkout;
        }
    }
}
```

4. Create the backend service

`Services/ICheckoutService.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface ICheckoutService
    {
        Task<Checkout> CreateAsync(Checkout checkout);
    }
}
```

`Services/CheckoutService.cs`

```
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class CheckoutService : ICheckoutService
    {
        private readonly ICheckoutRepository _repository;

        public CheckoutService(ICheckoutRepository repository)
        {
            _repository = repository;
        }

        public async Task<Checkout> CreateAsync(Checkout checkout)
        {
            checkout.Id = 0;
            checkout.Status = "Pending";
            checkout.CreatedAt = DateTime.UtcNow;

            return await _repository.InsertAsync(checkout);
        }
    }
}
```

5. Create the API controller

Create `Controllers/CheckoutController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class CheckoutController : ControllerBase
    {
        private readonly ICheckoutService _service;

        public CheckoutController(ICheckoutService service)
        {
            _service = service;
```

```
        }

        [HttpPost]
        public async Task<ActionResult<Checkout>> Create(
            [FromBody] Checkout checkout)
        {
            if (string.IsNullOrWhiteSpace(checkout.EquipmentName) ||
                string.IsNullOrWhiteSpace(checkout.AssignedTo))
            {
                return BadRequest(
                    "Equipment and assigned team member are required.");
            }

            if (checkout.ExpectedReturnDate == default)
            {
                return BadRequest("Expected return date is required.");
            }

            if (checkout.ExpectedReturnDate.Date < DateTime.UtcNow.Date)
            {
                return BadRequest("Expected return date cannot be in the past.");
            }

            checkout.Purpose ??= string.Empty;

            var created = await _service.CreateAsync(checkout);

            return CreatedAtAction(
                nameof(Create),
                new { id = created.Id },
                created);
        }
    }
}
```

6. Register the repository and service

In `Program.cs`, add these registrations before `builder.Build()`. Keep your existing Equipment registrations too.

```
builder.Services.AddScoped<ICheckoutRepository, CheckoutRepository>();
builder.Services.AddScoped<ICheckoutService, CheckoutService>();
```

Add these `using` statements if they are not already present:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

7. Apply the database migration

In Visual Studio, open **Tools → NuGet Package Manager → Package Manager Console** and run:

```
Add-Migration AddCheckoutTable
Update-Database
```

Select the backend project as the Default project in Package Manager Console. If your solution uses a separate startup project, select the ASP.NET Core API as the Startup Project.

These commands create the `Checkouts` table in your configured SQL Server database.

8. Create the Angular model

Create `src/app/models/checkout.ts`:

```
export interface CreateCheckoutRequest {
  equipmentName: string;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
}

export interface CheckoutRecord extends CreateCheckoutRequest {
  id: number;
  status: string;
  createdAt: string;
}
```

9. Create the Angular API service

Run:

```
ng generate service services/checkout
```

Then update `src/app/services/checkout.service.ts`:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
```

```
import {
  CreateCheckoutRequest,
  CheckoutRecord
} from '../models/checkout';

@Injectable({
  providedIn: 'root'
})
export class CheckoutService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Checkout`;

  createCheckout(
    request: CreateCheckoutRequest
  ): Observable<CheckoutRecord> {
    return this.http.post<CheckoutRecord>(
      this.apiUrl,
      request
    );
  }
}
```

This uses the same API base URL and `provideHttpClient()` configuration as your Equipment service.

10. Connect your existing Checkout component

In `checkout.ts`, add the service and request-model imports, then replace your `requestCheckOut()` method. Keep your other properties, `cancel()`, and `goBack()` methods.

Add these imports:

```
import { CheckoutService } from '../../services/checkout.service';
import { CreateCheckoutRequest } from '../../models/checkout';
```

Inside your `Checkout` class, add:

```
private checkoutService = inject(CheckoutService);

saving = false;
errorMessage = '';
successMessage = '';
```

Also add `inject` to your Angular import:

```
import { Component, inject } from '@angular/core';
```

Replace `requestCheckOut()` with:

```
requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    !this.selectedEquipment ||
    !this.assignedTo ||
    !this.expectedReturnDate
  ) {
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentName: this.selectedEquipment,
    assignedTo: this.assignedTo,
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout request saved:', result);

      this.saving = false;
      this.successMessage = 'Checkout request submitted successfully.';

      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout request failed:', error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message ||
            'Unable to submit checkout request.';
    }
```

```
  });
}
```

In `checkout.html`, display the messages and disable the submit button while saving:

```
@if (errorMessage) {
  <p class="error-message">{{ errorMessage }}</p>
}

@if (successMessage) {
  <p class="success-message">{{ successMessage }}</p>
}

<button type="button" (click)="requestCheckOut()" [disabled]="saving">
  {{ saving ? 'Submitting...' : 'Request Check Out' }}
</button>
```

Use your existing submit button's styling and type if it is already inside a form.

11. Test the API

Run the backend and frontend, then submit a checkout request. You can also test `POST /api/Checkout` in Swagger with this body:

```
{
  "equipmentName": "GPS Rover - 01",
  "assignedTo": "Ali Raza",
  "expectedReturnDate": "2026-11-10",
  "purpose": "Land surveying field work"
}
```

A successful request should return `201 Created` and save a row in the `Checkouts` table.

Important for your SolvefyTask requirements: this first version records a checkout request as `Pending`. It does not yet mark the equipment as checked out, verify that it is available, or enforce Hamza's drone approval. Those need additional backend logic before the checkout workflow is complete.

24/10/2026, 20:59:32 · gpt-6

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class CheckoutController : ControllerBase
    {
        private readonly ICheckoutService _service;

        public CheckoutController(ICheckoutService service)
        {
            _service = service;
        }

        [HttpPost]
        public async Task<ActionResult<Checkout>>
Create([FromBody] Checkout checkout)
        {
            if
(string.IsNullOrWhiteSpace(checkout.EquipmentName) ||
                string.IsNullOrWhiteSpace(checkout.AssignedTo))
            {
                return BadRequest(
                    "Equipment and assigned team member are
required.");
            }

            if (checkout.ExpectedReturnDate == default)
            {
                return BadRequest("Expected return date is
required.");
            }

            if (checkout.ExpectedReturnDate.Date <
DateTime.UtcNow.Date)
            {
                return BadRequest("Expected return date cannot be
in the past.");
            }
```

```
            checkout.Purpose ??= string.Empty;

            var created = await _service.CreateAsync(checkout);

            return CreatedAtAction(
                nameof(Create),
                new { id = created.Id },
                created);
        }
    }
}
```

please make 2 apis first get the equipment name and the second is that it gets assign to according to the equipment in the database ??

24/10/2026, 21:34:10

You need two GET APIs in your `CheckoutController`:

1. **GET** `/api/Checkout/equipment` — retrieves equipment names from your `Equipments` database table.
2. **GET** `/api/Checkout/assigned-to?equipmentName=GPS%20Rover%20-%2001` — retrieves team members eligible to be assigned to the selected equipment.

One important detail: your current `Equipment` model doesn't contain an assignment-eligibility relationship between equipment and team members. The second API below uses the `AssignedTo` values already stored in your equipment table, returning the assigned team member for the selected equipment when present. If you want a list of eligible people who can check out each equipment type, we'll need to define that rule or relationship.

1. Add methods to `ICheckoutService.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface ICheckoutService
    {
        Task<Checkout> CreateAsync(Checkout checkout);

        Task<List<string>> GetEquipmentNamesAsync();

        Task<List<string>> GetAssignedToAsync(string equipmentName);
```

```
    }
}
```

2. Add methods to `CheckoutService.cs`

Keep your existing constructor and `CreateAsync()` method. Add the following dependency and methods:

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class CheckoutService : ICheckoutService
    {
        private readonly ICheckoutRepository _repository;
        private readonly ApplicationDbContext _context;

        public CheckoutService(
            ICheckoutRepository repository,
            ApplicationDbContext context)
        {
            _repository = repository;
            _context = context;
        }

        public async Task<Checkout> CreateAsync(Checkout checkout)
        {
            checkout.Id = 0;
            checkout.Status = "Pending";
            checkout.CreatedAt = DateTime.UtcNow;

            return await _repository.InsertAsync(checkout);
        }

        // API 1: Get equipment names from the database
        public async Task<List<string>> GetEquipmentNamesAsync()
        {
            return await _context.Equipments
                .AsNoTracking()
                .Where(e => !string.IsNullOrEmpty(e.Name))
                .Select(e => e.Name)
                .Distinct()
                .OrderBy(name => name)
```

```
                .ToListAsync();
        }

        // API 2: Get the assigned-to value for selected equipment
        public async Task<List<string>> GetAssignedToAsync(string equipmentName)
        {
            var assignedTo = await _context.Equipments
                .AsNoTracking()
                .Where(e => e.Name == equipmentName)
                .Select(e => e.AssignedTo)
                .Where(name => name != null && name != "")
                .Distinct()
                .ToListAsync();

            return assignedTo
                .Select(name => name!)
                .ToList();
        }
    }
}
```

3. Update `CheckoutController.cs`

Add these two GET endpoints alongside your existing POST endpoint.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class CheckoutController : ControllerBase
    {
        private readonly ICheckoutService _service;

        public CheckoutController(ICheckoutService service)
        {
            _service = service;
        }

        // GET: api/Checkout/equipment
        [HttpGet("equipment")]
        public async Task<ActionResult<List<string>>> GetEquipmentNames()
        {
```

```
            var names = await _service.GetEquipmentNamesAsync();
            return Ok(names);
        }

        // GET: api/Checkout/assigned-to?equipmentName=GPS%20Rover%20-%2001
        [HttpGet("assigned-to")]
        public async Task<ActionResult<List<string>>> GetAssignedTo(
            [FromQuery] string equipmentName)
        {
            if (string.IsNullOrWhiteSpace(equipmentName))
            {
                return BadRequest("Equipment name is required.");
            }

            var members = await _service.GetAssignedToAsync(equipmentName);
            return Ok(members);
        }

        // Keep your existing POST Create endpoint here.
    }
}
```

Keep your existing `Create()` POST method in the controller; don't delete it.

4. Test in Swagger

Method	Endpoint	Purpose
GET	`/api/Checkout/equipment`	Returns equipment names
GET	`/api/Checkout/assigned-to?equipmentName=GPS%20Rover%20-%2001`	Returns stored assignee values for that equipment
POST	`/api/Checkout`	Saves a checkout request

Note: If `AssignedTo` is `null` for available equipment, the second API returns an empty array. For your checkout form, if you intend `Assigned To` to show *all team members allowed to check out the selected equipment*, you should retrieve team members from a team-members table and apply the equipment-specific rules there.

24/10/2026, 21:34:10 · gpt-6

please also provide repository and then get the assign to using equipment id ??

24/10/2026, 21:37:11

We'll update your checkout flow to use the **Equipment ID instead of the equipment name**. We'll also add the repository layer so the database queries follow your existing architecture:

Controller → `ICheckoutService` **→** `CheckoutService` **→** `ICheckoutRepository` **→ SQL Server**

The two GET APIs will be:

- `GET /api/Checkout/equipment` — returns equipment IDs and names for the dropdown.
- `GET /api/Checkout/assigned-to/{equipmentId}` — returns the stored assignee for the selected equipment ID.

One distinction: your current `Equipment.AssignedTo` field stores an existing assignee, not necessarily everyone eligible to check out that equipment. This implementation retrieves the value stored for the selected equipment.

1. Create a DTO for the equipment dropdown

Create `DTOs/EquipmentOptionDto.cs`:

```
namespace SolvefyTask.DTOs
{
    public class EquipmentOptionDto
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
    }
}
```

2. Repository interface

Create or update `Repositories/ICheckoutRepository.cs`:

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface ICheckoutRepository
```

```
    {
        Task<Checkout> InsertAsync(Checkout checkout);

        Task<List<EquipmentOptionDto>> GetEquipmentOptionsAsync();

        Task<string?> GetAssignedToByEquipmentIdAsync(int equipmentId);
    }
}
```

3. Repository implementation

Create or update `Repositories/CheckoutRepository.cs`:

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.DTOs;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class CheckoutRepository : ICheckoutRepository
    {
        private readonly ApplicationDbContext _context;

        public CheckoutRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<Checkout> InsertAsync(Checkout checkout)
        {
            _context.Checkouts.Add(checkout);
            await _context.SaveChangesAsync();

            return checkout;
        }

        // GET: Equipment IDs and names
        public async Task<List<EquipmentOptionDto>> GetEquipmentOptionsAsync()
        {
            return await _context.Equipments
                .AsNoTracking()
                .Select(e => new EquipmentOptionDto
                {
                    Id = e.Id,
                    Name = e.Name
```

```
                })
                .OrderBy(e => e.Name)
                .ToListAsync();
        }

        // GET: Assignee for a particular equipment ID
        public async Task<string?> GetAssignedToByEquipmentIdAsync(
            int equipmentId)
        {
            return await _context.Equipments
                .AsNoTracking()
                .Where(e => e.Id == equipmentId)
                .Select(e => e.AssignedTo)
                .FirstOrDefaultAsync();
        }
    }
}
```

4. Update `ICheckoutService.cs`

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface ICheckoutService
    {
        Task<Checkout> CreateAsync(Checkout checkout);

        Task<List<EquipmentOptionDto>> GetEquipmentOptionsAsync();

        Task<string?> GetAssignedToByEquipmentIdAsync(int equipmentId);
    }
}
```

5. Update `CheckoutService.cs`

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class CheckoutService : ICheckoutService
    {
```

```
        private readonly ICheckoutRepository _repository;

        public CheckoutService(ICheckoutRepository repository)
        {
            _repository = repository;
        }

        public async Task<Checkout> CreateAsync(Checkout checkout)
        {
            checkout.Id = 0;
            checkout.Status = "Pending";
            checkout.CreatedAt = DateTime.UtcNow;

            return await _repository.InsertAsync(checkout);
        }

        public Task<List<EquipmentOptionDto>> GetEquipmentOptionsAsync()
        {
            return _repository.GetEquipmentOptionsAsync();
        }

        public Task<string?> GetAssignedToByEquipmentIdAsync(int equipmentId)
        {
            return _repository.GetAssignedToByEquipmentIdAsync(equipmentId);
        }
    }
}
```

6. Update `CheckoutController.cs`

Keep your existing POST API and add these two GET endpoints:

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class CheckoutController : ControllerBase
    {
        private readonly ICheckoutService _service;

        public CheckoutController(ICheckoutService service)
```

```
        {
            _service = service;
        }

        // GET: api/Checkout/equipment
        [HttpGet("equipment")]
        public async Task<ActionResult<List<EquipmentOptionDto>>>
            GetEquipmentOptions()
        {
            var equipment = await _service.GetEquipmentOptionsAsync();
            return Ok(equipment);
        }

        // GET: api/Checkout/assigned-to/2
        [HttpGet("assigned-to/{equipmentId:int}")]
        public async Task<ActionResult<string>> GetAssignedTo(int equipmentId)
        {
            var assignedTo =
                await _service.GetAssignedToByEquipmentIdAsync(equipmentId);

            if (assignedTo == null)
            {
                return NotFound("No assigned team member found for this equipment.");
            }

            return Ok(assignedTo);
        }

        // Keep your existing [HttpPost] Create method here.
    }
}
```

If your `Equipment.AssignedTo` is null for available equipment, this endpoint returns `404`. If you prefer it to return `200 OK` with an empty value for available equipment, that behavior can be adjusted.

7. Register the dependencies

In `Program.cs`, ensure these registrations exist before `builder.Build()`:

```
builder.Services.AddScoped<ICheckoutRepository, CheckoutRepository>();
builder.Services.AddScoped<ICheckoutService, CheckoutService>();
```

8. Test the APIs

Method	Endpoint	Response
GET	`/api/Checkout/equipment`	`[{ "id": 1, "name": "GPS Rover - 01" }]`
GET	`/api/Checkout/assigned-to/1`	`"Ali Raza"`

Important: The second endpoint returns the equipment's current `AssignedTo` value. If your checkout form should instead list team members available to receive the equipment, you will need a team-members table and a rule for determining eligibility.

24/10/2026, 21:37:13 · gpt-6

```
import { Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest } from
'../../models/checkout';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout {

  constructor(private router: Router) {}


  private checkoutService = inject(CheckoutService);


  saving = false;
  errorMessage = '';
  successMessage = '';


  selectedEquipment = 'GPS Rover - 01';
  assignedTo = 'Ali Raza';
  expectedReturnDate = '2026-11-10';
  purpose = '';

  submitted = false;

  equipmentOptions = [
    { name: 'GPS Rover - 01', availability: 'Available' },
    { name: 'Total Station - 01', availability: 'Available' },
    { name: 'Drone - DJI Mavic', availability: 'Available' },
    { name: 'Field Tablet - 01', availability: 'Available' }
  ];

  teamMembers = [
    'Ali Raza',
    'Ahmad',
    'Usman',
```

```
    'Muhammad Ahmed'
  ];


requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    !this.selectedEquipment ||
    !this.assignedTo ||
    !this.expectedReturnDate
  ) {
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentName: this.selectedEquipment,
    assignedTo: this.assignedTo,
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout request saved:', result);

      this.saving = false;
      this.successMessage = 'Checkout request submitted
successfully.';

      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout request failed:', error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message ||
            'Unable to submit checkout request.';
```

```
    }
  });
}

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

also implement the above 2 apis in this ??

24/10/2026, 21:51:46

Yes. You need to integrate both GET APIs into your existing Angular `Checkout` component:

1. **GET** `/api/Checkout/equipment` — load equipment options dynamically from the database, including their IDs.
2. **GET** `/api/Checkout/assigned-to/{equipmentId}` — fetch the stored assignee when the user selects an equipment item.
3. **POST** `/api/Checkout` — submit the checkout request using the selected equipment ID and the other form values.

One important change: your current form stores the equipment name, but the APIs use the equipment ID. The code below updates the component accordingly.

1. Update `src/app/models/checkout.ts`

```ts
export interface EquipmentOption {
  id: number;
  name: string;
}

export interface CreateCheckoutRequest {
  equipmentId: number;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
}
```

```
export interface CheckoutRecord {
  id: number;
  equipmentId: number;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
  status: string;
  createdAt: string;
}
```

This assumes you also update the backend `Checkout` model and POST endpoint to accept `EquipmentId` rather than `EquipmentName`.

2. Update `src/app/services/checkout.ts`

Keep your existing service filename and add the two GET methods alongside your POST method.

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  CheckoutRecord,
  CreateCheckoutRequest,
  EquipmentOption
} from '../models/checkout';

@Injectable({
  providedIn: 'root'
})
export class CheckoutService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Checkout`;

  // GET: /api/Checkout/equipment
  getEquipmentOptions(): Observable<EquipmentOption[]> {
    return this.http.get<EquipmentOption[]>(
      `${this.apiUrl}/equipment`
    );
  }

  // GET: /api/Checkout/assigned-to/{equipmentId}
  getAssignedToByEquipmentId(
    equipmentId: number
```

```
  ): Observable<string> {
    return this.http.get<string>(
      `${this.apiUrl}/assigned-to/${equipmentId}`
    );
  }

  // POST: /api/Checkout
  createCheckout(
    request: CreateCheckoutRequest
  ): Observable<CheckoutRecord> {
    return this.http.post<CheckoutRecord>(
      this.apiUrl,
      request
    );
  }
}
```

3. Replace your `checkout.ts` component

This version loads equipment when the page opens and automatically fetches the stored assignee whenever the selected equipment changes.

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

import { CheckoutService } from '../../services/checkout';
import {
  CreateCheckoutRequest,
  EquipmentOption
} from '../../models/checkout';

@Component({
  selector: 'app-checkout',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit {
  private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  saving = false;
  loadingEquipment = false;
```

```
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }
```

```
  // API 2: Get the assignee for the selected equipment ID.
  onEquipmentChange(): void {
    this.assignedTo = '';
    this.errorMessage = '';

    if (this.selectedEquipment === null) {
      return;
    }

    const equipmentId = this.selectedEquipment;

    this.loadingAssignee = true;

    this.checkoutService
      .getAssignedToByEquipmentId(equipmentId)
      .subscribe({
        next: (assignee) => {
          // Ignore an older response if the selection changed.
          if (this.selectedEquipment === equipmentId) {
            this.assignedTo = assignee;
          }

          this.loadingAssignee = false;
        },
        error: (error) => {
          console.error('Failed to load assignee:', error);

          this.loadingAssignee = false;

          if (this.selectedEquipment === equipmentId) {
            // A 404 may mean the equipment has no stored assignee.
            this.assignedTo = '';
            this.errorMessage =
              error?.status === 404
                ? 'No assignee is currently recorded for this equipment.'
                : 'Unable to load the equipment assignee.';
          }
        }
      });
  }

  // API 3: Submit checkout request.
  requestCheckOut(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';
```

```
  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
    this.errorMessage =
      'Please complete all required fields.';
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentId: this.selectedEquipment,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout request saved:', result);

      this.saving = false;
      this.successMessage =
        'Checkout request submitted successfully.';

      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout request failed:', error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message ||
            'Unable to submit checkout request.';
    }
  });
}

cancel(): void {
  this.router.navigate(['/dashboard']);
}
```

```
  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. Update the equipment dropdown in `checkout.html`

Replace your existing equipment dropdown with this:

```
<label for="equipment">Equipment</label>

<select
  id="equipment"
  name="equipment"
  [(ngModel)]="selectedEquipment"
  (ngModelChange)="onEquipmentChange()"
  required
  [disabled]="loadingEquipment || equipmentOptions.length === 0"
>
  <option [ngValue]="null" disabled>
    Select equipment
  </option>

  <option
    *ngFor="let item of equipmentOptions"
    [ngValue]="item.id"
  >
    {{ item.name }}
  </option>
</select>

<p *ngIf="loadingEquipment">
  Loading equipment...
</p>
```

The dropdown now uses each equipment record's database ID as its value, while showing the equipment name to the user.

5. Update the Assigned To field

If the assignee should be automatically fetched from the equipment record, use a read-only input:

```
<label for="assignedTo">Assigned To</label>

<input
```

```
  id="assignedTo"
  name="assignedTo"
  type="text"
  [(ngModel)]="assignedTo"
  readonly
  required
/>

<p *ngIf="loadingAssignee">
  Loading assignee...
</p>
```

Keep your existing expected return date and purpose fields. Ensure the form calls the method with `(ngSubmit)="requestCheckOut()"`, and that the submit button is disabled while saving or loading.

Important backend adjustment

Your earlier backend implementation used `EquipmentName`. To make this implementation work consistently, update the backend `Checkout` model and POST API to use:

```
public int EquipmentId { get; set; }
```

Then configure the checkout repository to use `EquipmentId` to find the corresponding equipment record. Prefer a foreign key relationship to `Equipment.Id`, and return a validation error if that equipment does not exist.

One business-rule detail: your current `assigned-to/{equipmentId}` API returns the assignee already stored on that equipment. It does not load a list of team members or choose a new person. If `Assigned To` should instead let the user select Ali Raza, Ahmad, Usman, or Muhammad Ahmed, the API and form need a different design.

24/10/2026, 21:51:46 · gpt-6

```
<div class="checkout-page">
  <div class="page-banner">4. Check Out Equipment</div>

  <main class="checkout-content">
    <div class="page-title">
      <button
        type="button"
        class="back-button"
        aria-label="Go back"
        (click)="goBack()"
      >
        ←
      </button>

      <h1>Check Out Equipment</h1>
    </div>

    <form
      #checkoutForm="ngForm"
      class="checkout-form"
      (ngSubmit)="requestCheckOut()"
    >
      <div class="form-grid">
        <!-- Select Equipment -->
        <div class="form-field">
          <label for="equipment">
            Select Equipment <span class="required">*</span>
          </label>

          <select
            id="equipment"
            name="equipment"
            [(ngModel)]="selectedEquipment"
            required
            #equipmentModel="ngModel"
            [class.invalid]="submitted &&
equipmentModel.invalid"
          >
            <option value="" disabled>Select
equipment</option>

            @for (item of equipmentOptions; track item.name) {
              <option
                [value]="item.name"
```

```
            [disabled]="item.availability !== 'Available'"
          >
            {{ item.name }} ({{ item.availability }})
          </option>
        }
      </select>

      @if (submitted && equipmentModel.invalid) {
        <small class="error-message">Please select
equipment.</small>
      }
    </div>

    <!-- Assign To -->
    <div class="form-field">
      <label for="assignedTo">
        Assign To <span class="required">*</span>
      </label>

      <select
        id="assignedTo"
        name="assignedTo"
        [(ngModel)]="assignedTo"
        required
        #assignedModel="ngModel"
        [class.invalid]="submitted && assignedModel.invalid"
      >
        <option value="" disabled>Select team
member</option>

        @for (member of teamMembers; track member) {
          <option [value]="member">{{ member }}</option>
        }
      </select>

      @if (submitted && assignedModel.invalid) {
        <small class="error-message">Please select a team
member.</small>
      }
    </div>

    <!-- Expected Return Date -->
    <div class="form-field">
      <label for="returnDate">
        Expected Return Date <span class="required">*
</span>
```

```
      </label>

      <input
        id="returnDate"
        name="returnDate"
        type="date"
        [(ngModel)]="expectedReturnDate"
        [min]="'2026-10-24'"
        required
        #dateModel="ngModel"
        [class.invalid]="submitted && dateModel.invalid"
      />

      @if (submitted && dateModel.invalid) {
        <small class="error-message">
          Please select a valid return date.
        </small>
      }
    </div>

    <!-- Purpose / Project -->
    <div class="form-field full-width">
      <label for="purpose">Purpose / Project</label>

      <textarea
        id="purpose"
        name="purpose"
        [(ngModel)]="purpose"
        rows="3"
        placeholder="Site survey - DHA Phase 6"
      ></textarea>
    </div>
  </div>

  <div class="form-actions">
    <button
      type="button"
      class="cancel-button"
      (click)="cancel()"
    >
      Cancel
    </button>

    <button type="submit" class="submit-button">
      Request Check Out
    </button>
```

```
      </div>
    </form>
  </main>
</div>
please provide complete code ??
```

24/10/2026, 21:57:12

Below is the **complete updated code** for your Angular Checkout page, keeping your existing layout and CSS classes while integrating all three APIs:

- `GET /api/Checkout/equipment` — load equipment from the database.
- `GET /api/Checkout/assigned-to/{equipmentId}` — load the stored assignee for the selected equipment.
- `POST /api/Checkout` — submit the checkout request.

The equipment dropdown will use the database ID internally and display the equipment name. The assignee field will be populated automatically from the selected equipment record.

1. `src/app/models/checkout.ts`

```ts
export interface EquipmentOption {
  id: number;
  name: string;
}

export interface CreateCheckoutRequest {
  equipmentId: number;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
}

export interface CheckoutRecord {
  id: number;
  equipmentId: number;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
  status: string;
  createdAt: string;
}
```

2. `src/app/services/checkout.ts`

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  CheckoutRecord,
  CreateCheckoutRequest,
  EquipmentOption
} from '../models/checkout';

@Injectable({
  providedIn: 'root'
})
export class CheckoutService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Checkout`;

  // GET: /api/Checkout/equipment
  getEquipmentOptions(): Observable<EquipmentOption[]> {
    return this.http.get<EquipmentOption[]>(
      `${this.apiUrl}/equipment`
    );
  }

  // GET: /api/Checkout/assigned-to/{equipmentId}
  getAssignedToByEquipmentId(
    equipmentId: number
  ): Observable<string> {
    return this.http.get<string>(
      `${this.apiUrl}/assigned-to/${equipmentId}`
    );
  }

  // POST: /api/Checkout
  createCheckout(
    request: CreateCheckoutRequest
  ): Observable<CheckoutRecord> {
    return this.http.post<CheckoutRecord>(
      this.apiUrl,
      request
    );
```

```
  }
}
```

3. `src/app/components/checkout/checkout.ts`

```ts
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

import { CheckoutService } from '../../services/checkout';
import {
  CreateCheckoutRequest,
  EquipmentOption
} from '../../models/checkout';

@Component({
  selector: 'app-checkout',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit {
  private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  loadingEquipment = false;
  loadingAssignee = false;
  saving = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  // Minimum return date is today.
  minReturnDate = this.getLocalDateString();

  ngOnInit(): void {
```

```
    this.loadEquipmentOptions();
  }

  private getLocalDateString(): string {
    const date = new Date();
    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const day = String(date.getDate()).padStart(2, '0');

    return `${year}-${month}-${day}`;
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }

  // API 2: Get the stored assignee for the selected equipment.
  onEquipmentChange(): void {
    this.assignedTo = '';
    this.errorMessage = '';

    if (this.selectedEquipment === null) {
```

```
      return;
    }

    const equipmentId = this.selectedEquipment;
    this.loadingAssignee = true;

    this.checkoutService
      .getAssignedToByEquipmentId(equipmentId)
      .subscribe({
        next: (assignee) => {
          if (this.selectedEquipment === equipmentId) {
            this.assignedTo = assignee;
          }

          this.loadingAssignee = false;
        },
        error: (error) => {
          console.error('Failed to load assignee:', error);

          this.loadingAssignee = false;

          if (this.selectedEquipment === equipmentId) {
            this.assignedTo = '';
            this.errorMessage =
              error?.status === 404
                ? 'No assignee is recorded for this equipment.'
                : 'Unable to load the equipment assignee.';
          }
        }
      });
  }

  // API 3: Submit checkout request.
  requestCheckOut(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';

    if (
      this.selectedEquipment === null ||
      !this.assignedTo.trim() ||
      !this.expectedReturnDate
    ) {
      this.errorMessage =
        'Please complete all required fields.';
      return;
```

```
    }

    if (this.expectedReturnDate < this.minReturnDate) {
      this.errorMessage =
        'Expected return date cannot be in the past.';
      return;
    }

    if (this.loadingEquipment || this.loadingAssignee) {
      this.errorMessage =
        'Please wait until equipment details finish loading.';
      return;
    }

    const request: CreateCheckoutRequest = {
      equipmentId: this.selectedEquipment,
      assignedTo: this.assignedTo.trim(),
      expectedReturnDate: this.expectedReturnDate,
      purpose: this.purpose.trim()
    };

    this.saving = true;

    this.checkoutService.createCheckout(request).subscribe({
      next: (result) => {
        console.log('Checkout request saved:', result);

        this.saving = false;
        this.successMessage =
          'Checkout request submitted successfully.';

        alert(this.successMessage);
      },
      error: (error) => {
        console.error('Checkout request failed:', error);

        this.saving = false;
        this.errorMessage =
          typeof error?.error === 'string'
            ? error.error
            : error?.error?.message ||
              'Unable to submit checkout request.';
      }
    });
  }
```

```
  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. `src/app/components/checkout/checkout.html`

This keeps your existing page banner, form layout, field labels, validation messages, and action buttons.

```
<div class="checkout-page">
  <div class="page-banner">4. Check Out Equipment</div>

  <main class="checkout-content">
    <div class="page-title">
      <button
        type="button"
        class="back-button"
        aria-label="Go back"
        (click)="goBack()"
      >
        ←
      </button>

      <h1>Check Out Equipment</h1>
    </div>

    @if (errorMessage) {
      <div class="error-message" role="alert">
        {{ errorMessage }}
      </div>
    }

    @if (successMessage) {
      <div class="success-message" role="status">
        {{ successMessage }}
      </div>
    }

    @if (loadingEquipment) {
      <p>Loading equipment...</p>
    }
```

```
<form
  #checkoutForm="ngForm"
  class="checkout-form"
  (ngSubmit)="requestCheckOut()"
>
  <div class="form-grid">

    <!-- Select Equipment -->
    <div class="form-field">
      <label for="equipment">
        Select Equipment <span class="required">*</span>
      </label>

      <select
        id="equipment"
        name="equipment"
        [(ngModel)]="selectedEquipment"
        (ngModelChange)="onEquipmentChange()"
        required
        #equipmentModel="ngModel"
        [class.invalid]="submitted && equipmentModel.invalid"
        [disabled]="loadingEquipment || equipmentOptions.length === 0"
      >
        <option [ngValue]="null" disabled>
          Select equipment
        </option>

        @for (item of equipmentOptions; track item.id) {
          <option [ngValue]="item.id">
            {{ item.name }}
          </option>
        }
      </select>

      @if (submitted && equipmentModel.invalid) {
        <small class="error-message">
          Please select equipment.
        </small>
      }
    </div>

    <!-- Assign To -->
    <div class="form-field">
      <label for="assignedTo">
        Assign To <span class="required">*</span>
```

```
      </label>

      <input
        id="assignedTo"
        name="assignedTo"
        type="text"
        [(ngModel)]="assignedTo"
        required
        readonly
        placeholder="Assignee will load automatically"
        #assignedModel="ngModel"
        [class.invalid]="submitted && assignedModel.invalid"
      />

      @if (loadingAssignee) {
        <small>Loading assignee...</small>
      }

      @if (submitted && assignedModel.invalid) {
        <small class="error-message">
          No assignee is available for this equipment.
        </small>
      }
    </div>

    <!-- Expected Return Date -->
    <div class="form-field">
      <label for="returnDate">
        Expected Return Date <span class="required">*</span>
      </label>

      <input
        id="returnDate"
        name="returnDate"
        type="date"
        [(ngModel)]="expectedReturnDate"
        [min]="minReturnDate"
        required
        #dateModel="ngModel"
        [class.invalid]="submitted && dateModel.invalid"
      />

      @if (submitted && dateModel.invalid) {
        <small class="error-message">
          Please select a valid return date.
        </small>
```

```
          }
        </div>

        <!-- Purpose / Project -->
        <div class="form-field full-width">
          <label for="purpose">Purpose / Project</label>

          <textarea
            id="purpose"
            name="purpose"
            [(ngModel)]="purpose"
            rows="3"
            placeholder="Site survey - DHA Phase 6"
          ></textarea>
        </div>
      </div>

      <div class="form-actions">
        <button
          type="button"
          class="cancel-button"
          (click)="cancel()"
        >
          Cancel
        </button>

        <button
          type="submit"
          class="submit-button"
          [disabled]="saving || loadingEquipment || loadingAssignee"
        >
          {{ saving ? 'Submitting...' : 'Request Check Out' }}
        </button>
      </div>
    </form>
  </main>
</div>
```

5. Backend compatibility check

Before testing, ensure your backend uses the same request property names.

The POST body sent by Angular will look like this:

```
{
  "equipmentId": 1,
  "assignedTo": "Ali Raza",
```

```
  "expectedReturnDate": "2026-11-10",
  "purpose": "Site survey - DHA Phase 6"
}
```

Your backend `Checkout` model and controller must accept `EquipmentId`, not just `EquipmentName`. Also ensure `GET /api/Checkout/assigned-to/{equipmentId}` returns a JSON string, such as `"Ali Raza"`, rather than a differently shaped object.

Important: this version automatically fills `Assign To` with the assignee already saved against the equipment. If the field should instead let the user choose a team member, you will need a team-member lookup API and a dropdown rather than this read-only input.

24/10/2026, 21:57:12 · gpt-6

X [ERROR] TS2339: Property 'minReturnDate' does not exist on type 'Checkout'. [plugin angular-compiler]

src/app/components/checkout/checkout.html:116:19:
116 | [min]="minReturnDate"
| ~~~~~~~~~~~~~~

Error occurs in the template of component Checkout.

src/app/components/checkout/checkout.ts:10:15:
10 | templateUrl: './checkout.html',

24/10/2026, 21:58:10

This error means `minReturnDate` is missing from your `Checkout` class in `checkout.ts`, but your HTML template references it.

Fix: Add this property to `checkout.ts`

Inside your `Checkout` class, add:

```
minReturnDate = this.getLocalDateString();

private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');

  return `${year}-${month}-${day}`;
}
```

For example, place it alongside your other properties:

```
loadingEquipment = false;
loadingAssignee = false;
saving = false;

errorMessage = '';
successMessage = '';
submitted = false;
```

```
minReturnDate = this.getLocalDateString();
```

Keep your existing HTML unchanged:

```
<input
  id="returnDate"
  name="returnDate"
  type="date"
  [(ngModel)]="expectedReturnDate"
  [min]="minReturnDate"
  required
  #dateModel="ngModel"
  [class.invalid]="submitted && dateModel.invalid"
/>
```

Why this works: `minReturnDate` provides today's date in `YYYY-MM-DD` format, preventing users from selecting a return date in the past.

Save the file and let Angular rebuild.

24/10/2026, 21:58:15 · gpt-6

in check out when i change the employee the assignee must have to load it and get it from the database and that assignee should appear ??

24/10/2026, 22:01:42

Yes, exactly. In your **Check Out Equipment** form, when you change the selected equipment or employee-related selection, Angular should call the backend, retrieve the corresponding assignee from SQL Server, and display that person's name automatically.

There is one important distinction in your current implementation: the existing API `GET /api/Checkout/assigned-to/{equipmentId}` retrieves the assignee using an **equipment ID**. It does not retrieve an assignee based on an employee ID. If you want the assignee to change based on the selected equipment, use the following implementation.

1. Update `checkout.ts`

Make sure your equipment-change method calls the API every time the selection changes:

```
onEquipmentChange(): void {
  this.assignedTo = '';
  this.errorMessage = '';

  const equipmentId = this.selectedEquipment;

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        // Only update the field if this equipment is still selected.
        if (this.selectedEquipment === equipmentId) {
          this.assignedTo = assignee ?? '';
        }

        this.loadingAssignee = false;
      },
      error: (error) => {
```

```
        console.error('Error fetching assignee:', error);

        if (this.selectedEquipment === equipmentId) {
          this.assignedTo = '';
          this.errorMessage =
            error.status === 404
              ? 'No assignee is assigned to this equipment.'
              : 'Failed to load assignee from the database.';
        }

        this.loadingAssignee = false;
      }
    });
}
```

2. Update the equipment dropdown in `checkout.html`

The key part is `(ngModelChange)="onEquipmentChange()"`.

```
<div class="form-field">
  <label for="equipment">
    Select Equipment <span class="required">*</span>
  </label>

  <select
    id="equipment"
    name="equipment"
    [(ngModel)]="selectedEquipment"
    (ngModelChange)="onEquipmentChange()"
    required
    [disabled]="loadingEquipment"
  >
    <option [ngValue]="null" disabled>
      Select equipment
    </option>

    @for (item of equipmentOptions; track item.id) {
      <option [ngValue]="item.id">
        {{ item.name }}
      </option>
    }
  </select>
</div>
```

3. Display the assignee retrieved from the database

```
<div class="form-field">
  <label for="assignedTo">
    Assign To <span class="required">*</span>
  </label>

  <input
    id="assignedTo"
    name="assignedTo"
    type="text"
    [ngModel]="assignedTo"
    readonly
    required
    placeholder="Select equipment first"
  />

  @if (loadingAssignee) {
    <small>Loading assignee from database...</small>
  }
</div>
```

4. Verify the backend API

Your repository should retrieve the assignee from the equipment table:

```
public async Task<string?> GetAssignedToByEquipmentIdAsync(
    int equipmentId)
{
    return await _context.Equipments
        .AsNoTracking()
        .Where(e => e.Id == equipmentId)
        .Select(e => e.AssignedTo)
        .FirstOrDefaultAsync();
}
```

Your controller should expose it:

```
[HttpGet("assigned-to/{equipmentId:int}")]
public async Task<IActionResult> GetAssignedTo(int equipmentId)
{
    var assignedTo =
        await _checkoutService.GetAssignedToByEquipmentIdAsync(
            equipmentId);

    if (assignedTo == null)
    {
        return NotFound("No assignee found for this equipment.");
```

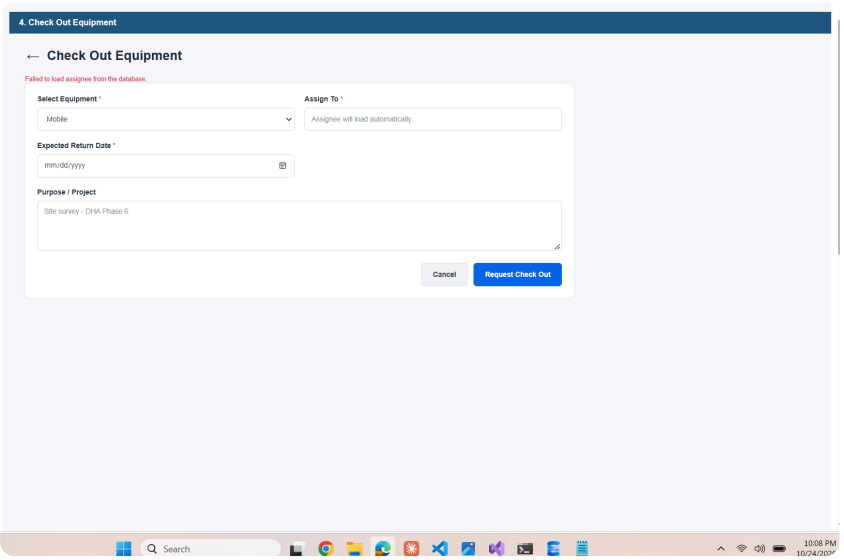
```
    }

    return Ok(assignedTo);
}
```

Expected behavior: Select equipment ID `1` → Angular requests `/api/Checkout/assigned-to/1` → the API queries SQL Server → the returned assignee appears in the **Assign To** field.

Important: This retrieves the assignee stored against the selected equipment. If you mean that selecting an employee should automatically retrieve their details, rather than selecting equipment, then the API needs to query by `employeeId` instead.

24/10/2026, 22:01:45 · gpt-6



assignee is not loading from the database ?? where is the problem

```
import { Component, inject, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest, EquipmentOption } from
'../../models/checkout';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit  {

 private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  minReturnDate = this.getLocalDateString();


private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');
```

```
  return `${year}-${month}-${day}`;
}

ngonInit(): void {
  this.loadEquipmentOptions();
}
  saving = false;
  loadingEquipment = false;
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
```

```
      console.error('Failed to load equipment:', error);

      this.loadingEquipment = false;
      this.errorMessage =
        'Unable to load equipment. Please try again.';
    }
  });
}

// API 2: Get the assignee for the selected equipment ID.
onEquipmentChange(): void {
this.assignedTo = '';
this.errorMessage = '';

const equipmentId = this.selectedEquipment;

if (equipmentId === null) {
  return;
}

this.loadingAssignee = true;

this.checkoutService
  .getAssignedToByEquipmentId(equipmentId)
  .subscribe({
    next: (assignee: string) => {
      // Only update the field if this equipment is still selected.
      if (this.selectedEquipment === equipmentId) {
        this.assignedTo = assignee ?? '';
      }

      this.loadingAssignee = false;
    },
    error: (error) => {
      console.error('Error fetching assignee:', error);

      if (this.selectedEquipment === equipmentId) {
        this.assignedTo = '';
        this.errorMessage =
          error.status === 404
            ? 'No assignee is assigned to this equipment.'
            : 'Failed to load assignee from the database.';
      }

      this.loadingAssignee = false;
    }
```

```
    });
  }

  // API 3: Submit checkout request.
  requestCheckOut(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';

    if (
      this.selectedEquipment === null ||
      !this.assignedTo.trim() ||
      !this.expectedReturnDate
    ) {
      this.errorMessage =
        'Please complete all required fields.';
      return;
    }

    const request: CreateCheckoutRequest = {
      equipmentId: this.selectedEquipment,
      assignedTo: this.assignedTo.trim(),
      expectedReturnDate: this.expectedReturnDate,
      purpose: this.purpose.trim()
    };

    this.saving = true;

    this.checkoutService.createCheckout(request).subscribe({
      next: (result) => {
        console.log('Checkout request saved:', result);

        this.saving = false;
        this.successMessage =
          'Checkout request submitted successfully.';

        alert(this.successMessage);
      },
      error: (error) => {
        console.error('Checkout request failed:', error);

        this.saving = false;
        this.errorMessage =
          typeof error?.error === 'string'
            ? error.error
            : error?.error?.message ||
```

```
            'Unable to submit checkout request.';
        }
      });
    }

    cancel(): void {
      this.router.navigate(['/dashboard']);
    }

    goBack(): void {
      this.router.navigate(['/dashboard']);
    }
  }

  <div class="checkout-page">
    <div class="page-banner">4. Check Out Equipment</div>

    <main class="checkout-content">
      <div class="page-title">
        <button
          type="button"
          class="back-button"
          aria-label="Go back"
          (click)="goBack()"
        >
          ←
        </button>

        <h1>Check Out Equipment</h1>
      </div>

      @if (errorMessage) {
        <div class="error-message" role="alert">
          {{ errorMessage }}
        </div>
      }

      @if (successMessage) {
        <div class="success-message" role="status">
          {{ successMessage }}
        </div>
      }

      @if (loadingEquipment) {
        <p>Loading equipment...</p>
      }
```

```
    <form
      #checkoutForm="ngForm"
      class="checkout-form"
      (ngSubmit)="requestCheckOut()"
    >
      <div class="form-grid">

        <!-- Select Equipment -->
        <div class="form-field">
          <label for="equipment">
            Select Equipment <span class="required">*</span>
          </label>

          <select
            id="equipment"
            name="equipment"
            [(ngModel)]="selectedEquipment"
            (ngModelChange)="onEquipmentChange()"
            required
            #equipmentModel="ngModel"
            [class.invalid]="submitted &&
equipmentModel.invalid"
            [disabled]="loadingEquipment ||
equipmentOptions.length === 0"
          >
            <option [ngValue]="null" disabled>
              Select equipment
            </option>

            @for (item of equipmentOptions; track item.id) {
              <option [ngValue]="item.id">
                {{ item.name }}
              </option>
            }
          </select>

          @if (submitted && equipmentModel.invalid) {
            <small class="error-message">
              Please select equipment.
            </small>
          }
        </div>

        <!-- Assign To -->
        <div class="form-field">
```

```
        <label for="assignedTo">
          Assign To <span class="required">*</span>
        </label>

        <input
          id="assignedTo"
          name="assignedTo"
          type="text"
          [(ngModel)]="assignedTo"
          required
          readonly
          placeholder="Assignee will load automatically"
          #assignedModel="ngModel"
          [class.invalid]="submitted && assignedModel.invalid"
        />

        @if (loadingAssignee) {
          <small>Loading assignee...</small>
        }

        @if (submitted && assignedModel.invalid) {
          <small class="error-message">
            No assignee is available for this equipment.
          </small>
        }
      </div>

      <!-- Expected Return Date -->
      <div class="form-field">
        <label for="returnDate">
          Expected Return Date <span class="required">*
</span>
        </label>

        <input
          id="returnDate"
          name="returnDate"
          type="date"
          [(ngModel)]="expectedReturnDate"
          [min]="minReturnDate"
          required
          #dateModel="ngModel"
          [class.invalid]="submitted && dateModel.invalid"
        />

        @if (submitted && dateModel.invalid) {
```

```
            <small class="error-message">
              Please select a valid return date.
            </small>
          }
        </div>

        <!-- Purpose / Project -->
        <div class="form-field full-width">
          <label for="purpose">Purpose / Project</label>

          <textarea
            id="purpose"
            name="purpose"
            [(ngModel)]="purpose"
            rows="3"
            placeholder="Site survey - DHA Phase 6"
          ></textarea>
        </div>
      </div>

      <div class="form-actions">
        <button
          type="button"
          class="cancel-button"
          (click)="cancel()"
        >
          Cancel
        </button>

        <button
          type="submit"
          class="submit-button"
          [disabled]="saving || loadingEquipment ||
loadingAssignee"
        >
          {{ saving ? 'Submitting...' : 'Request Check Out' }}
        </button>
      </div>
    </form>
  </main>
</div>
```

24/10/2026, 22:10:06

I can see the problem from your code. The page is successfully loading the equipment list, but the assignee API is failing. There is also a duplicate lifecycle method in your component.

1. Fix the duplicate `ngOnInit()`

Your component contains both:

```
ngonInit(): void {
  this.loadEquipmentOptions();
}
```

and:

```
ngOnInit(): void {
  this.loadEquipmentOptions();
}
```

Delete the first one (`ngonInit` with a lowercase `o`). Keep only the correctly spelled `ngOnInit()`.

However, this duplicate is not the main reason the assignee API fails.

2. Find the actual API error

Open your browser's Developer Tools with **F12 → Network**. Change the equipment selection and click the request named something like:

```
/api/Checkout/assigned-to/1
```

Check its **Status** and **Response**. The most likely causes are:

- **404:** The API route is incorrect, or the equipment has no assignee stored in the database.
- **500:** An exception occurred in the backend.
- **200, but the field is blank:** The response format or the database's `AssignedTo` value may not match what Angular expects.
- **CORS or connection error:** The browser cannot access the API successfully.

3. Add logging to identify the exact problem

Replace your `onEquipmentChange()` method with this temporary debugging version:

```
onEquipmentChange(): void {
  this.assignedTo = '';
  this.errorMessage = '';

  const equipmentId = this.selectedEquipment;
```

```
  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  console.log('Selected equipment ID:', equipmentId);

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (response: any) => {
        console.log('Assignee API response:', response);

        // Supports either a plain string or an object response.
        const assignee =
          typeof response === 'string'
            ? response
            : response?.assignedTo;

        this.assignedTo = assignee ?? '';
        this.loadingAssignee = false;

        if (!this.assignedTo.trim()) {
          this.errorMessage =
            'No assignee is saved for this equipment in the database.';
        }
      },
      error: (error) => {
        console.error('Assignee API URL:', error.url);
        console.error('HTTP status:', error.status);
        console.error('Backend error response:', error.error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage =
          `Unable to load assignee. HTTP status: ${error.status}.`;
      }
    });
}
```

4. Check the database value

Your repository currently queries:

```
_context.Equipments
    .Where(e => e.Id == equipmentId)
    .Select(e => e.AssignedTo)
    .FirstOrDefaultAsync();
```

This only works if the selected equipment's `AssignedTo` column actually contains a name.

For example, if your database contains:

Id	Name	AssignedTo
1	Mobile	Ali Raza
2	GPS Rover	Ahmad
3	Drone	Usman

Selecting **Mobile** should display **Ali Raza**.

But if `AssignedTo` is `NULL` or empty for Mobile, Angular cannot display a name that is not stored there.

Please do this next: open F12 → Network, select Mobile, click the `assigned-to/1` request, and share its **HTTP status code and Response body**. That will tell us whether the problem is the API route, the backend query, the response format, or missing database data.

24/10/2026, 22:10:22 · gpt-6

in network tab the api is getting the data perfectly but it is now showing it on the frontend ??

24/10/2026, 22:12:17

If the API returns the assignee correctly in the Network tab, your backend is working. The problem is most likely in **how Angular reads the API response or updates the input field**.

For example, the API might return either:

```
"Ali Raza"
```

or:

```
{
  "assignedTo": "Ali Raza"
}
```

Your Angular code must handle the actual response format.

1. Update `onEquipmentChange()` in `checkout.ts`

Replace your existing method with this version:

```
onEquipmentChange(): void {
  this.assignedTo = '';
  this.errorMessage = '';

  const equipmentId = this.selectedEquipment;

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (response: any) => {
        console.log('API response:', response);

        // Handle both string and object responses.
        const assignee =
```

```
          typeof response === 'string'
            ? response
            : response?.assignedTo;

        this.assignedTo = assignee ?? '';

        console.log('Assigned To field value:', this.assignedTo);

        this.loadingAssignee = false;

        if (!this.assignedTo.trim()) {
          this.errorMessage = 'The API returned no assignee name.';
        }
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.loadingAssignee = false;
        this.assignedTo = '';
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}
```

2. Update the Assign To input in `checkout.html`

Use `[value]` to display the value directly:

```
<div class="form-field">
  <label for="assignedTo">
    Assign To <span class="required">*</span>
  </label>

  <input
    id="assignedTo"
    name="assignedTo"
    type="text"
    [value]="assignedTo"
    readonly
    placeholder="Select equipment first"
  />

  @if (loadingAssignee) {
    <small>Loading assignee...</small>
  }
</div>
```

This removes `[(ngModel)]` from the read-only field, avoiding unnecessary two-way binding.

3. Verify the result

1. Save both files.
2. Refresh the Angular page.
3. Open the browser console.
4. Select an equipment item.
5. Check the values printed by `API response` and `Assigned To field value`.

If it still doesn't appear: copy the exact **Response** from the Network tab here. For example, tell me whether it is `"Ali Raza"`, `{"assignedTo":"Ali Raza"}`, or another structure. I can then match the Angular code precisely to your API response.

24/10/2026, 22:12:17 · gpt-6

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  CheckoutRecord,
  CreateCheckoutRequest,
  EquipmentOption
} from '../models/checkout';

@Injectable({
  providedIn: 'root'
})
export class CheckoutService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Checkout`;

  // GET: /api/Checkout/equipment
  getEquipmentOptions(): Observable<EquipmentOption[]> {
    return this.http.get<EquipmentOption[]>(
      `${this.apiUrl}/equipment`
    );
  }

  // GET: /api/Checkout/assigned-to/{equipmentId}
  getAssignedToByEquipmentId(
    equipmentId: number
  ): Observable<string> {
    return this.http.get<string>(
      `${this.apiUrl}/assigned-to/${equipmentId}`
    );
  }

  // POST: /api/Checkout
  createCheckout(
    request: CreateCheckoutRequest
  ): Observable<CheckoutRecord> {
    return this.http.post<CheckoutRecord>(
      this.apiUrl,
      request
    );
  }
```

```
}



import { Component, inject, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest, EquipmentOption } from
'../../models/checkout';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit  {

 private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  minReturnDate = this.getLocalDateString();


private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');

  return `${year}-${month}-${day}`;
}



  saving = false;
  loadingEquipment = false;
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
  selectedEquipment: number | null = null;
```

```
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }

  // API 2: Get the assignee for the selected equipment ID.

onEquipmentChange(): void {
  debugger;
  this.assignedTo = '';
  this.errorMessage = '';

  const equipmentId = this.selectedEquipment;
```

```
  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (response: any) => {
        console.log('API response:', response);

        // Handle both string and object responses.
        const assignee =
          typeof response === 'string'
            ? response
            : response?.assignedTo;

        this.assignedTo = assignee ?? '';

        console.log('Assigned To field value:', this.assignedTo);

        this.loadingAssignee = false;

        if (!this.assignedTo.trim()) {
          this.errorMessage = 'The API returned no assignee
name.';
        }
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.loadingAssignee = false;
        this.assignedTo = '';
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}


  // API 3: Submit checkout request.
  requestCheckOut(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';
```

```
  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
    this.errorMessage =
      'Please complete all required fields.';
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentId: this.selectedEquipment,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout request saved:', result);

      this.saving = false;
      this.successMessage =
        'Checkout request submitted successfully.';

      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout request failed:', error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message ||
            'Unable to submit checkout request.';
    }
  });
}

cancel(): void {
  this.router.navigate(['/dashboard']);
}
```

```
  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}



<div class="checkout-page">
  <div class="page-banner">4. Check Out Equipment</div>

  <main class="checkout-content">
    <div class="page-title">
      <button
        type="button"
        class="back-button"
        aria-label="Go back"
        (click)="goBack()"
      >
        ←
      </button>

      <h1>Check Out Equipment</h1>
    </div>

    @if (errorMessage) {
      <div class="error-message" role="alert">
        {{ errorMessage }}
      </div>
    }

    @if (successMessage) {
      <div class="success-message" role="status">
        {{ successMessage }}
      </div>
    }

    @if (loadingEquipment) {
      <p>Loading equipment...</p>
    }

    <form
      #checkoutForm="ngForm"
      class="checkout-form"
      (ngSubmit)="requestCheckOut()"
    >
```

```
      <div class="form-grid">

        <!-- Select Equipment -->
        <div class="form-field">
          <label for="equipment">
            Select Equipment <span class="required">*</span>
          </label>

          <select
            id="equipment"
            name="equipment"
            [(ngModel)]="selectedEquipment"
            (ngModelChange)="onEquipmentChange()"
            required
            #equipmentModel="ngModel"
            [class.invalid]="submitted &&
equipmentModel.invalid"
            [disabled]="loadingEquipment ||
equipmentOptions.length === 0"
          >
            <option [ngValue]="null" disabled>
              Select equipment
            </option>

            @for (item of equipmentOptions; track item.id) {
              <option [ngValue]="item.id">
                {{ item.name }}
              </option>
            }
          </select>

          @if (submitted && equipmentModel.invalid) {
            <small class="error-message">
              Please select equipment.
            </small>
          }
        </div>

        <!-- Assign To -->
        <div class="form-field">
          <label for="assignedTo">
            Assign To <span class="required">*</span>
          </label>

          <input
            id="assignedTo"
```

```
          name="assignedTo"
          type="text"
          [(ngModel)]="assignedTo"
          required
          readonly
          placeholder="Assignee will load automatically"
          #assignedModel="ngModel"
          [class.invalid]="submitted && assignedModel.invalid"
        />

        @if (loadingAssignee) {
          <small>Loading assignee...</small>
        }

        @if (submitted && assignedModel.invalid) {
          <small class="error-message">
            No assignee is available for this equipment.
          </small>
        }
      </div>

      <!-- Expected Return Date -->
      <div class="form-field">
        <label for="returnDate">
          Expected Return Date <span class="required">*
</span>
        </label>

        <input
          id="returnDate"
          name="returnDate"
          type="date"
          [(ngModel)]="expectedReturnDate"
          [min]="minReturnDate"
          required
          #dateModel="ngModel"
          [class.invalid]="submitted && dateModel.invalid"
        />

        @if (submitted && dateModel.invalid) {
          <small class="error-message">
            Please select a valid return date.
          </small>
        }
      </div>
```

```
        <!-- Purpose / Project -->
        <div class="form-field full-width">
          <label for="purpose">Purpose / Project</label>

          <textarea
            id="purpose"
            name="purpose"
            [(ngModel)]="purpose"
            rows="3"
            placeholder="Site survey - DHA Phase 6"
          ></textarea>
        </div>
      </div>

      <div class="form-actions">
        <button
          type="button"
          class="cancel-button"
          (click)="cancel()"
        >
          Cancel
        </button>

        <button
          type="submit"
          class="submit-button"
          [disabled]="saving || loadingEquipment ||
loadingAssignee"
        >
          {{ saving ? 'Submitting...' : 'Request Check Out' }}
        </button>
      </div>
    </form>
  </main>
</div>
```

the problem is still not resolved ??

24/10/2026, 22:18:43

I reviewed all three files you shared. The most important thing is that your Angular code currently assumes the API returns either a plain string or an object with an `assignedTo` property. If the API returns the data in a different format, the request can succeed while the input remains blank.

There is another issue worth fixing: your component uses `[(ngModel)]` for a field that is populated asynchronously. Let's simplify the binding and explicitly handle the API response.

1. First, check the exact response

In your browser, open **F12 → Network →** `assigned-to/{equipmentId}` **→ Response**.

For example, if the response is:

```
"Ali Raza"
```

the API returns a string. If it is:

```
{
  "assignedTo": "Ali Raza"
}
```

the API returns an object. The code below supports both formats, including common casing variations.

2. Update `getAssignedToByEquipmentId()` in `checkout.ts` service

Change its return type to `Observable<unknown>` so Angular can handle the actual response shape.

```
getAssignedToByEquipmentId(
  equipmentId: number
): Observable<unknown> {
  return this.http.get<unknown>(
    `${this.apiUrl}/assigned-to/${equipmentId}`
  );
}
```

3. Replace `onEquipmentChange()` in your component

```
onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;

  this.assignedTo = '';
  this.errorMessage = '';

  if (equipmentId === null || equipmentId === undefined) {
    this.loadingAssignee = false;
    return;
  }

  this.loadingAssignee = true;
```

```
  console.log('Fetching assignee for equipment ID:', equipmentId);

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (response: unknown) => {
        console.log('Raw assignee API response:', response);

        let assignee = '';

        if (typeof response === 'string') {
          assignee = response;
        } else if (response && typeof response === 'object') {
          const data = response as Record<string, unknown>;

          const value =
            data['assignedTo'] ??
            data['AssignedTo'] ??
            data['assignee'] ??
            data['Assignee'] ??
            data['name'] ??
            data['Name'];

          if (typeof value === 'string') {
            assignee = value;
          }
        }

        // Do not display a response for a previous selection.
        if (this.selectedEquipment === equipmentId) {
          this.assignedTo = assignee.trim();

          if (!this.assignedTo) {
            this.errorMessage =
              'The API succeeded, but no assignee name was found in its response.';
          }
        }

        this.loadingAssignee = false;

        console.log('Final assignedTo value:', this.assignedTo);
      },
      error: (error) => {
        console.error('Assignee API failed:', error.status, error.error);

        if (this.selectedEquipment === equipmentId) {
```

```
          this.assignedTo = '';
          this.errorMessage =
            `Unable to load assignee. HTTP ${error.status}.`;
        }

        this.loadingAssignee = false;
      }
    });
}
```

4. Replace only the Assign To input in `checkout.html`

Use one-way binding to display the value populated by the API:

```
<div class="form-field">
  <label for="assignedTo">
    Assign To <span class="required">*</span>
  </label>

  <input
    id="assignedTo"
    name="assignedTo"
    type="text"
    [value]="assignedTo"
    readonly
    placeholder="Assignee will load automatically"
  />

  @if (loadingAssignee) {
    <small>Loading assignee from database...</small>
  }
</div>
```

5. If the field is still blank

The next step is to identify the precise JSON structure rather than changing Angular code repeatedly.

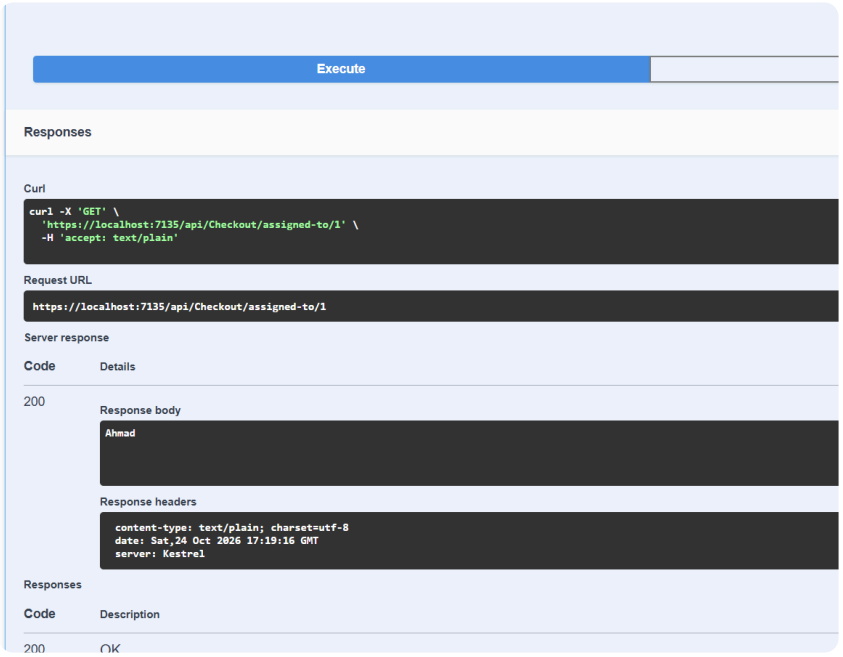
After selecting equipment, look at the browser console:

- `Raw assignee API response` shows what the server actually returned.
- `Final assignedTo value` shows what Angular extracted.

Please paste the exact `Raw assignee API response` from your console. That will let me match the code to your backend's actual response instead of guessing.

One final distinction: your current API looks up `Equipment.AssignedTo` using the selected equipment's ID. If the assignee is actually stored in a separate employee or checkout table, the backend repository query must be changed to read from that table instead.

24/10/2026, 22:18:45 · gpt-6



24/10/2026, 22:19:49

Now we have identified the issue more clearly. Your Swagger screenshot confirms that the API is working correctly.

- **HTTP status:** `200 OK`
- **Response body:** `Ahmad`
- **Content-Type:** `text/plain; charset=utf-8`

The API returns a **plain-text string**, not a JSON object. Therefore, Angular should assign that string directly to `assignedTo`.

1. Fix `checkout.ts` component

Replace your `onEquipmentChange()` method with this exact code:

```
onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;

  this.assignedTo = '';
  this.errorMessage = '';

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;
```

```
  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        console.log('Assignee received from API:', assignee);

        this.assignedTo = assignee.trim();
        this.loadingAssignee = false;

        console.log('Assigned To field:', this.assignedTo);
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}
```

2. Keep your service method like this

Your service is already correct for a plain-text response:

```
getAssignedToByEquipmentId(
  equipmentId: number
): Observable<string> {
  return this.http.get(
    `${this.apiUrl}/assigned-to/${equipmentId}`,
    { responseType: 'text' }
  );
}
```

The explicit `responseType: 'text'` is important because your backend returns `text/plain`.

3. Fix the Assign To input in `checkout.html`

Replace only your existing Assign To input with:

```
<input
  id="assignedTo"
  name="assignedTo"
  type="text"
  [ngModel]="assignedTo"
  readonly
```

```
  placeholder="Assignee will load automatically"
/>
```

4. Test it

1. Save both files.
2. Refresh the Angular application.
3. Select the equipment whose ID is `1`.
4. The Assign To field should display **Ahmad**.

If it still does not display, check the browser console for `Assignee received from API` and `Assigned To field`. Since Swagger already confirms the response is `Ahmad`, we can now focus specifically on the Angular binding rather than changing your backend repository or controller.

24/10/2026, 22:19:50 · gpt-6

i want to use

this.cdr.detectChanges()

how to apply it ??

24/10/2026, 22:25:51

You can use `this.cdr.detectChanges()` in your `Checkout` component by injecting Angular's `ChangeDetectorRef`. In your case, add it after assigning the assignee returned by the API.

1. Import `ChangeDetectorRef`

At the top of `checkout.ts`, update your imports:

```
import { Component, inject, OnInit, ChangeDetectorRef } from '@angular/core';
```

2. Inject `ChangeDetectorRef`

Inside your `Checkout` class, add:

```
private cdr = inject(ChangeDetectorRef);
```

For example:

```
export class Checkout implements OnInit {
  private router = inject(Router);
  private checkoutService = inject(CheckoutService);
  private cdr = inject(ChangeDetectorRef);

  // Your existing properties and methods...
}
```

3. Call `this.cdr.detectChanges()` inside the API response

Update your `onEquipmentChange()` method as follows:

```
onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;

  this.assignedTo = '';
  this.errorMessage = '';
```

```
  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;
  this.cdr.detectChanges();

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        console.log('Assignee received:', assignee);

        this.assignedTo = assignee.trim();
        this.loadingAssignee = false;

        // Update the view immediately.
        this.cdr.detectChanges();

        console.log('Assigned To:', this.assignedTo);
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage = 'Unable to load assignee.';

        this.cdr.detectChanges();
      }
    });
}
```

4. Check your HTML binding

Your Assign To input should bind to `assignedTo`:

```
<input
  id="assignedTo"
  name="assignedTo"
  type="text"
  [ngModel]="assignedTo"
  readonly
  placeholder="Assignee will load automatically"
/>
```

Also, ensure your service uses `responseType: 'text'`, because your API returns `Ahmad` as plain text:

```
getAssignedToByEquipmentId(
  equipmentId: number
): Observable<string> {
  return this.http.get(
    `${this.apiUrl}/assigned-to/${equipmentId}`,
    { responseType: 'text' }
  );
}
```

Important: Angular's `HttpClient` normally triggers change detection automatically. Use `detectChanges()` when you have a specific reason to force a view update. If the input remains blank even after this change, check the console output for `Assignee received` and confirm that the value is `Ahmad`.

24/10/2026, 22:25:51 · gpt-6

when i click on the submit checkout but it is not saving into the database ??

24/10/2026, 22:29:52

Let's trace the checkout submission from Angular to SQL Server. Since your assignee API is returning `Ahmad`, that confirms the GET endpoint works, but it does **not** confirm that the POST endpoint is saving successfully.

The most important things to check are:

1. Is Angular sending the correct POST request?
2. Does the request body match your backend `Checkout` model?
3. Is the controller calling the service and repository?
4. Is Entity Framework saving to the correct database?

1. Check your Angular submit method

Update `requestCheckOut()` to log the request and the server response. This will reveal whether the POST is failing.

Replace your current `requestCheckOut()` method in `checkout.ts` with this:

```
requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  if (this.saving || this.loadingAssignee) {
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentId: this.selectedEquipment,
```

```
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  console.log('Checkout POST request:', request);

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout saved successfully:', result);

      this.saving = false;
      this.successMessage = 'Checkout request saved successfully.';
    },
    error: (error) => {
      console.error('Checkout POST failed:', error.status);
      console.error('Error details:', error.error);

      this.saving = false;
      this.errorMessage =
        typeof error?.error === 'string'
          ? error.error
          : error?.error?.message ||
            `Unable to save checkout. HTTP status: ${error.status}.`;
    }
  });
}
```

2. Verify the Angular service

Your `createCheckout()` method should send the request to the correct endpoint:

```
createCheckout(
  request: CreateCheckoutRequest
): Observable<CheckoutRecord> {
  return this.http.post<CheckoutRecord>(
    this.apiUrl,
    request
  );
}
```

With your existing `apiUrl`, this calls:

```
POST https://localhost:7135/api/Checkout
```

The request body should resemble:

```
{
  "equipmentId": 1,
  "assignedTo": "Ahmad",
  "expectedReturnDate": "2026-11-10",
  "purpose": "Site survey - DHA Phase 6"
}
```

3. Verify the backend model

Your C# `Checkout` model must contain the same properties:

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models;

public class Checkout
{
    public int Id { get; set; }

    [Required]
    public int EquipmentId { get; set; }

    [Required]
    public string AssignedTo { get; set; } = string.Empty;

    [Required]
    public DateTime ExpectedReturnDate { get; set; }

    public string Purpose { get; set; } = string.Empty;

    public string Status { get; set; } = "Pending";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Your backend must also have the corresponding `DbSet`:

```
public DbSet<Checkout> Checkouts => Set<Checkout>();
```

And your repository must actually save the record:

```
public async Task<Checkout> InsertAsync(Checkout checkout)
{
    await _context.Checkouts.AddAsync(checkout);
```

```
    await _context.SaveChangesAsync();

    return checkout;
}
```

If you have changed the model from `EquipmentName` to `EquipmentId`, make sure you have created and applied the appropriate EF Core migration.

4. Check the POST request in your browser

Open **F12 → Network → Fetch/XHR**, then click **Request Check Out**.

Check the POST request to `/api/Checkout`.

Result	What it indicates
No POST request appears	Angular validation or form submission is preventing the request.
`400 Bad Request`	The request body or validation does not match the backend.
`404 Not Found`	The POST route is incorrect.
`500 Internal Server Error`	The backend encountered an exception; check the API console.
`200` or `201`	The API reports success; verify the database and connection string to confirm the row was saved.

Please share your backend `CheckoutController.cs`, `CheckoutService.cs`, and `CheckoutRepository.cs`, along with the POST request's status code and response body from Network. Those will let me pinpoint the exact saving problem instead of guessing.

24/10/2026, 22:29:53 · gpt-6

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "EquipmentName": [
      "The EquipmentName field is required."
    ]
  },
  "traceId": "00-6ce08903b67bd2870b792770a2baf3c5-1bf9e87b1091ceac-00"
}
```


24/10/2026, 22:32:58

The error confirms the problem: **your Angular frontend sends `EquipmentId`, but your backend `Checkout` model still requires `EquipmentName`.** ASP.NET Core rejects the request with `400 Bad Request` before saving it to the database.

You have two options. Since your frontend already uses `equipmentId`, I recommend updating the backend to use `EquipmentId` consistently.

1. Update `Models/Checkout.cs`

Replace `EquipmentName` with `EquipmentId`.

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
    public class Checkout
    {
        public int Id { get; set; }

        [Required]
        public int EquipmentId { get; set; }

        [Required]
        public string AssignedTo { get; set; } = string.Empty;

        [Required]
        public DateTime ExpectedReturnDate { get; set; }

        public string Purpose { get; set; } = string.Empty;
```

```
        public string Status { get; set; } = "Pending";

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    }
}
```

2. Update `CheckoutController.cs`

Make sure the POST endpoint accepts the `Checkout` model with `EquipmentId`:

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] Checkout checkout)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    var result = await _checkoutService.CreateAsync(checkout);

    return CreatedAtAction(
        nameof(Create),
        new { id = result.Id },
        result
    );
}
```

Keep your existing controller constructor and other endpoints. If your service returns a different type, adjust the result handling to match its existing signature.

3. Update `CheckoutService.cs`

Ensure the service no longer depends on `EquipmentName`:

```
public async Task<Checkout> CreateAsync(Checkout checkout)
{
    checkout.Id = 0;
    checkout.Status = "Pending";
    checkout.CreatedAt = DateTime.UtcNow;

    return await _checkoutRepository.InsertAsync(checkout);
}
```

Use your existing repository field and interface names if they differ.

4. Update `CheckoutRepository.cs`

```
public async Task<Checkout> InsertAsync(Checkout checkout)
{
    await _context.Checkouts.AddAsync(checkout);
    await _context.SaveChangesAsync();

    return checkout;
}
```

5. Update the SQL Server database

Because you changed the entity model, create and apply a migration.

In Visual Studio **Package Manager Console**, run:

```
Add-Migration UpdateCheckoutEquipmentId
Update-Database
```

If the previous migration has already created an `EquipmentName` column, inspect the generated migration to ensure it safely replaces that column with `EquipmentId`. If your existing checkout table contains data, preserve or migrate it as needed.

6. Test again

Your Angular request should continue sending:

```
{
  "equipmentId": 1,
  "assignedTo": "Ahmad",
  "expectedReturnDate": "2026-11-10",
  "purpose": "Site survey - DHA Phase 6"
}
```

After rebuilding and restarting the backend, submit the form again. You should receive a successful response, typically `201 Created`, provided the database schema and other validations are correct.

Important: For data integrity, the backend should also verify that `EquipmentId` exists in the `Equipments` table before saving the checkout record.

24/10/2026, 22:32:59 · gpt-6

i want to send equipment name not equipment ID ??

24/10/2026, 22:33:54

Understood. You want to send `equipmentName` **instead of** `equipmentId` from Angular to your backend and save the equipment name in the Checkout table.

You can keep using the equipment ID internally to fetch the assignee, while sending the equipment name in the POST request.

1. Update `src/app/models/checkout.ts`

Replace `equipmentId` with `equipmentName` in your checkout request model.

```
export interface CreateCheckoutRequest {
  equipmentName: string;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
}

export interface EquipmentOption {
  id: number;
  name: string;
}

export interface CheckoutRecord {
  id: number;
  equipmentName: string;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
  status: string;
  createdAt: string;
}
```

2. Update `requestCheckOut()` in `checkout.ts`

Replace your existing method with this version:

```
requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';
```

```
if (
  this.selectedEquipment === null ||
  !this.assignedTo.trim() ||
  !this.expectedReturnDate
) {
  this.errorMessage = 'Please complete all required fields.';
  return;
}

if (this.saving || this.loadingAssignee) {
  return;
}

const selectedItem = this.equipmentOptions.find(
  item => item.id === this.selectedEquipment
);

if (!selectedItem) {
  this.errorMessage = 'Please select valid equipment.';
  return;
}

const request: CreateCheckoutRequest = {
  equipmentName: selectedItem.name,
  assignedTo: this.assignedTo.trim(),
  expectedReturnDate: this.expectedReturnDate,
  purpose: this.purpose.trim()
};

console.log('Checkout request:', request);

this.saving = true;

this.checkoutService.createCheckout(request).subscribe({
  next: (result) => {
    console.log('Checkout saved successfully:', result);

    this.saving = false;
    this.successMessage = 'Checkout request saved successfully.';
    alert(this.successMessage);
  },
  error: (error) => {
    console.error('Checkout save failed:', error.status, error.error);

    this.saving = false;
```

```
      this.errorMessage =
        error?.error?.errors?.EquipmentName?.[0] ||
        error?.error?.title ||
        'Unable to save checkout request.';
    }
  });
}
```

3. Update your backend `Models/Checkout.cs`

Your backend currently requires `EquipmentName`, so retain that property.

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
    public class Checkout
    {
        public int Id { get; set; }

        [Required]
        public string EquipmentName { get; set; } = string.Empty;

        [Required]
        public string AssignedTo { get; set; } = string.Empty;

        [Required]
        public DateTime ExpectedReturnDate { get; set; }

        public string Purpose { get; set; } = string.Empty;

        public string Status { get; set; } = "Pending";

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    }
}
```

Your repository can continue saving the entity through Entity Framework:

```
public async Task<Checkout> InsertAsync(Checkout checkout)
{
    await _context.Checkouts.AddAsync(checkout);
    await _context.SaveChangesAsync();

    return checkout;
}
```

If `EquipmentName` already exists in your database table, you do not need a migration just to continue using that existing column.

4. Keep the equipment dropdown unchanged

Your dropdown should still bind to the equipment ID because that ID is used to retrieve the assignee:

```
<select
  id="equipment"
  name="equipment"
  [(ngModel)]="selectedEquipment"
  (ngModelChange)="onEquipmentChange()"
  required
>
  <option [ngValue]="null" disabled>Select equipment</option>

  @for (item of equipmentOptions; track item.id) {
    <option [ngValue]="item.id">
      {{ item.name }}
    </option>
  }
</select>
```

Expected POST request

For example, if the selected equipment is Mobile and the API returns Ahmad, Angular will send:

```
{
  "equipmentName": "Mobile",
  "assignedTo": "Ahmad",
  "expectedReturnDate": "2026-11-10",
  "purpose": "Site survey - DHA Phase 6"
}
```

Result: The assignee lookup continues using `equipmentId`, but the checkout request sends and saves `equipmentName`, as you requested.

24/10/2026, 22:33:53 · gpt-6

there is a problem with my code it is showing dashboard at the end on every page ??

24/10/2026, 22:37:46

This usually happens in Angular when the **Dashboard component or Dashboard layout is being rendered globally**, or when a navigation method redirects every route to `/dashboard`.

Let's check these common causes in your SolvefyTask project.

1. Check `app.component.html`

Open `src/app/app.html` (or `app.component.html`, depending on your project).

If you have something like this:

```
<app-dashboard></app-dashboard>
<router-outlet></router-outlet>
```

Remove the dashboard component from this file. Keep only:

```
<router-outlet></router-outlet>
```

The router outlet should display whichever page is active.

2. Check your routing file

Open `src/app/app.routes.ts`. Your routes should follow this pattern:

```
import { Routes } from '@angular/router';

import { Dashboard } from './components/dashboard/dashboard';
import { Equipment } from './components/equipment/equipment';
import { Checkout } from './components/checkout/checkout';

export const routes: Routes = [
  { path: 'dashboard', component: Dashboard },
  { path: 'equipment', component: Equipment },
  { path: 'checkout', component: Checkout },

  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', redirectTo: 'dashboard' }
];
```

Adjust the component import paths and class names to match your project.

The empty route redirects to Dashboard only when the URL is `/`. The wildcard route redirects there only when no route matches.

3. Check your navigation methods

In your Checkout component, you currently have:

```
cancel(): void {
  this.router.navigate(['/dashboard']);
}

goBack(): void {
  this.router.navigate(['/dashboard']);
}
```

These methods intentionally navigate to Dashboard when clicked. That is normal if this is what you want, but it should not happen automatically when a page loads.

4. Check the shared layout

If every page displays its own content and then Dashboard underneath it, the likely problem is that `<app-dashboard>` or another dashboard layout component is included in a shared template.

If the browser URL actually changes to `/dashboard` on every page, check your route configuration and any navigation code calling:

```
this.router.navigate(['/dashboard']);
```

Please share these two files so I can identify the exact issue:

- `src/app/app.html`
- `src/app/app.routes.ts`

Also tell me whether Dashboard appears **below every page** or whether the entire page **redirects to Dashboard**. These are different problems and require different fixes.

24/10/2026, 22:37:46 · gpt-6

```
<div class="dashboard-shell">

  <!-- Sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="selectMenu(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>

          @if (item.label === 'Approvals') {
            <span class="nav-badge">3</span>
          }
        </button>
      }
    </nav>

    <div class="sidebar-footer">
      <span class="status-dot"></span>
      Equipment Tracking System
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- Header -->
    <header class="topbar">
      <div class="mobile-brand">
        <div class="brand-mark">H</div>
        <span>Horizon</span>
      </div>

      <div class="topbar-spacer"></div>
```

```
  <button
    class="notification-button"
    type="button"
    aria-label="Notifications"
    title="Notifications"
  >
    ♧
    <span class="notification-dot"></span>
  </button>

  <div class="profile-container">
    <button
      type="button"
      class="profile-button"
      (click)="toggleProfile()"
    >
      <span class="avatar">AA</span>
      <span>Ahmad</span>
      <span class="chevron">⌄</span>
    </button>

    @if (showProfileMenu) {
      <div class="profile-menu">
        <div class="profile-menu-name">Ahmad</div>
        <button type="button" (click)="toggleProfile()">
          Close menu
        </button>
      </div>
    }
  </div>
</header>

<!-- Page Heading -->
<!-- <section class="page-heading">
  <div>
    <h1>{{ selectedMenu }}</h1>
    <p>Overview of equipment usage and status</p>
  </div>
</section> -->

<router-outlet></router-outlet>
<!-- Summary Cards -->
<section class="stats-grid">
  @for (stat of equipmentStats; track stat.title) {
    <article class="stat-card">
```

```
        <div class="stat-top">
          <span class="stat-icon" [class]=" 'stat-icon ' +
stat.color">
            {{ stat.icon }}
          </span>
          <span class="stat-title">{{ stat.title }}</span>
        </div>

        <div class="stat-value">{{ stat.value }}</div>
        <div class="stat-detail">{{ stat.detail }}</div>
      </article>
    }
  </section>

  <!-- Dashboard Panels -->
  <section class="dashboard-grid">

    <!-- Equipment Status -->
    <article class="panel status-panel">
      <div class="panel-heading">
        <h2>Equipment Status</h2>
        <button type="button" class="more-button"
title="Equipment status">
          ...
        </button>
      </div>

      <div class="status-content">
        <div class="donut-chart">
          <div class="donut-center">
            <strong>25</strong>
            <span>Total</span>
          </div>
        </div>

        <div class="legend">
          @for (status of equipmentStatus; track status.label) {
            <div class="legend-row">
              <span
                class="legend-dot"
                [style.background]="status.color"
              ></span>

              <span class="legend-label">{{ status.label }}
</span>
              <span class="legend-value">{{ status.value }}
```

```
</span>
          </div>
         }
        </div>
       </div>
     </article>

     <!-- Recent Activity -->
     <article class="panel activity-panel">
      <div class="panel-heading">
       <h2>Recent Activity</h2>
       <button
        type="button"
        class="view-all"
        (click)="toggleActivities()"
       >
        {{ showAllActivities ? 'Show less' : 'View All' }}
       </button>
      </div>

      <div class="activity-list">
       @for (activity of visibleActivities; track activity.title) {
        <div class="activity-item">
         <div
          class="activity-icon"
          [class.success]="activity.type === 'success'"
          [class.danger]="activity.type === 'danger'"
         >
          {{ activity.icon }}
         </div>

         <div class="activity-info">
          <p>{{ activity.title }}</p>
          <span>{{ activity.time }}</span>
         </div>
        </div>
       }
      </div>
     </article>

   </section>

   <footer class="dashboard-footer">
    Horizon Survey Services
    <span>Equipment Tracking System</span>
   </footer>
```

```
  </main>
</div>
```

24/10/2026, 22:38:49

I found the problem in your HTML. Your **Dashboard layout contains** `<router-outlet>` **followed by the Dashboard summary cards, panels, and footer**. Angular replaces only the router outlet's content, so the summary cards and panels remain visible on every page.

Fix: Update your dashboard template

You should keep the sidebar and topbar shared across pages, but move the Dashboard-specific content into its own component.

1. In your current dashboard HTML

Keep everything from the beginning through:

```
<router-outlet></router-outlet>
```

Then **remove the Summary Cards, Dashboard Panels, and Dashboard Footer from this shared layout**.

Your main content should look like this:

```
<!-- Main Content -->
<main class="main-content">

  <!-- Header -->
  <header class="topbar">
    <!-- Keep your existing topbar HTML here -->
  </header>

  <!-- Active page renders here -->
  <router-outlet></router-outlet>

</main>
```

Keep your existing sidebar and profile menu as they are.

2. Create a separate dashboard page

Create `src/app/components/dashboard/dashboard-home.html` and move these existing sections into it:

- Summary Cards (`stats-grid`)
- Dashboard Panels (`dashboard-grid`)
- Dashboard Footer (`dashboard-footer`)

Keep their existing HTML and bindings unchanged.

Create `dashboard-home.ts` for the component and move the properties those sections use—such as `equipmentStats`, `equipmentStatus`, `visibleActivities`, and `showAllActivities`—into that component, or reuse your existing Dashboard component if it already owns those properties.

3. Update your routes

Your `app.routes.ts` should distinguish between the shared layout and individual pages. For example:

```
import { Routes } from '@angular/router';

import { Dashboard } from './components/dashboard/dashboard';
import { Equipment } from './components/equipment/equipment';
import { Checkout } from './components/checkout/checkout';

export const routes: Routes = [
  {
    path: '',
    component: Dashboard,
    children: [
      {
        path: '',
        loadComponent: () =>
          import('./components/dashboard/dashboard-home')
            .then(m => m.DashboardHome)
      },
      {
        path: 'equipment',
        component: Equipment
      },
      {
        path: 'checkout',
        component: Checkout
      }
    ]
  }
];
```

This example assumes your `Dashboard` component is the shared layout containing the sidebar, topbar, and `<router-outlet>`. Adjust the imports to match your actual filenames and exported class names.

Expected result: The sidebar and topbar remain visible throughout the application, while the summary cards and recent activity appear only on the Dashboard page.

One more thing: your menu buttons currently call `selectMenu(item.label)`. Make sure that method navigates to the appropriate route; changing `selectedMenu` alone will not change the page displayed by `<router-outlet>`.

24/10/2026, 22:38:49 · gpt-6

```
<div class="dashboard-shell">

  <!-- Sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="selectMenu(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>

          @if (item.label === 'Approvals') {
            <span class="nav-badge">3</span>
          }
        </button>
      }
    </nav>

    <div class="sidebar-footer">
      <span class="status-dot"></span>
      Equipment Tracking System
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- Header -->
    <header class="topbar">
      <div class="mobile-brand">
        <div class="brand-mark">H</div>
        <span>Horizon</span>
      </div>

      <div class="topbar-spacer"></div>
```

```
  <button
    class="notification-button"
    type="button"
    aria-label="Notifications"
    title="Notifications"
  >
    ♧
    <span class="notification-dot"></span>
  </button>

  <div class="profile-container">
    <button
      type="button"
      class="profile-button"
      (click)="toggleProfile()"
    >
      <span class="avatar">AA</span>
      <span>Ahmad</span>
      <span class="chevron">⌄</span>
    </button>

    @if (showProfileMenu) {
      <div class="profile-menu">
        <div class="profile-menu-name">Ahmad</div>
        <button type="button" (click)="toggleProfile()">
          Close menu
        </button>
      </div>
    }
  </div>
</header>

<!-- Page Heading -->
<!-- <section class="page-heading">
  <div>
    <h1>{{ selectedMenu }}</h1>
    <p>Overview of equipment usage and status</p>
  </div>
</section> -->

<router-outlet></router-outlet>
<!-- Summary Cards -->
<section class="stats-grid">
  @for (stat of equipmentStats; track stat.title) {
    <article class="stat-card">
```

```
        <div class="stat-top">
          <span class="stat-icon" [class]=" 'stat-icon ' +
stat.color">
            {{ stat.icon }}
          </span>
          <span class="stat-title">{{ stat.title }}</span>
        </div>

        <div class="stat-value">{{ stat.value }}</div>
        <div class="stat-detail">{{ stat.detail }}</div>
      </article>
    }
  </section>

  <!-- Dashboard Panels -->
  <section class="dashboard-grid">

    <!-- Equipment Status -->
    <article class="panel status-panel">
      <div class="panel-heading">
        <h2>Equipment Status</h2>
        <button type="button" class="more-button"
title="Equipment status">
          ...
        </button>
      </div>

      <div class="status-content">
        <div class="donut-chart">
          <div class="donut-center">
            <strong>25</strong>
            <span>Total</span>
          </div>
        </div>

        <div class="legend">
          @for (status of equipmentStatus; track status.label) {
            <div class="legend-row">
              <span
                class="legend-dot"
                [style.background]="status.color"
              ></span>

              <span class="legend-label">{{ status.label }}
</span>
              <span class="legend-value">{{ status.value }}
```

```
</span>
          </div>
         }
        </div>
       </div>
     </article>

     <!-- Recent Activity -->
     <article class="panel activity-panel">
      <div class="panel-heading">
       <h2>Recent Activity</h2>
       <button
        type="button"
        class="view-all"
        (click)="toggleActivities()"
       >
        {{ showAllActivities ? 'Show less' : 'View All' }}
       </button>
      </div>

      <div class="activity-list">
       @for (activity of visibleActivities; track activity.title) {
        <div class="activity-item">
         <div
          class="activity-icon"
          [class.success]="activity.type === 'success'"
          [class.danger]="activity.type === 'danger'"
         >
          {{ activity.icon }}
         </div>

         <div class="activity-info">
          <p>{{ activity.title }}</p>
          <span>{{ activity.time }}</span>
         </div>
        </div>
       }
      </div>
     </article>

   </section>

   <footer class="dashboard-footer">
    Horizon Survey Services
    <span>Equipment Tracking System</span>
   </footer>
```

</main>

send me complete code ?

24/10/2026, 22:40:26

The issue is that your Dashboard HTML contains both the shared layout and the Dashboard-specific content after `<router-outlet>`. Below is a complete solution that keeps the **sidebar and topbar on every page**, while showing the summary cards, equipment status, and recent activity **only on the Dashboard page**.

I'll use your existing Angular standalone component naming convention.

1. `src/app/components/dashboard/dashboard.html`

Replace your existing Dashboard HTML with this complete shared layout:

```
<div class="dashboard-shell">

  <!-- Sidebar -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-mark">H</div>
      <span>Horizon</span>
    </div>

    <nav class="navigation">
      @for (item of menuItems; track item.label) {
        <button
          type="button"
          class="nav-item"
          [class.active]="selectedMenu === item.label"
          (click)="selectMenu(item.label)"
        >
          <span class="nav-icon">{{ item.icon }}</span>
          <span>{{ item.label }}</span>

          @if (item.label === 'Approvals') {
            <span class="nav-badge">3</span>
          }
        </button>
      }
    </nav>

    <div class="sidebar-footer">
```

```
      <span class="status-dot"></span>
      Equipment Tracking System
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- Header -->
    <header class="topbar">
      <div class="mobile-brand">
        <div class="brand-mark">H</div>
        <span>Horizon</span>
      </div>

      <div class="topbar-spacer"></div>

      <button
        class="notification-button"
        type="button"
        aria-label="Notifications"
        title="Notifications"
      >
        ♧
        <span class="notification-dot"></span>
      </button>

      <div class="profile-container">
        <button
          type="button"
          class="profile-button"
          (click)="toggleProfile()"
        >
          <span class="avatar">AA</span>
          <span>Ahmad</span>
          <span class="chevron">⌄</span>
        </button>

        @if (showProfileMenu) {
          <div class="profile-menu">
            <div class="profile-menu-name">Ahmad</div>

            <button
              type="button"
              (click)="toggleProfile()"
            >
```

```
              Close menu
            </button>
          </div>
        }
      </div>
    </header>

    <!-- Active page appears here -->
    <router-outlet></router-outlet>

  </main>
</div>
```

Important: Keep your existing `dashboard.css`. This change does not require removing your layout styles.

2. Create `src/app/components/dashboard/dashboard-home.ts`

This component owns the Dashboard-specific data and activity interactions.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

interface EquipmentStat {
  title: string;
  value: string;
  detail: string;
  icon: string;
  color: string;
}

interface EquipmentStatus {
  label: string;
  value: number;
  color: string;
}

interface Activity {
  title: string;
  time: string;
  icon: string;
  type: 'success' | 'danger' | 'default';
}

@Component({
  selector: 'app-dashboard-home',
  standalone: true,
```

```
  imports: [CommonModule],
  templateUrl: './dashboard-home.html',
  styleUrl: './dashboard-home.css'
})
export class DashboardHome {

  equipmentStats: EquipmentStat[] = [
    {
      title: 'Total Equipment',
      value: '25',
      detail: 'All registered equipment',
      icon: '▣',
      color: 'blue'
    },
    {
      title: 'Available',
      value: '15',
      detail: 'Ready for checkout',
      icon: '✓',
      color: 'green'
    },
    {
      title: 'Checked Out',
      value: '7',
      detail: 'Currently in use',
      icon: '↗',
      color: 'orange'
    },
    {
      title: 'Under Repair',
      value: '3',
      detail: 'Requires maintenance',
      icon: '⚙',
      color: 'red'
    }
  ];

  equipmentStatus: EquipmentStatus[] = [
    { label: 'Available', value: 15, color: '#22c55e' },
    { label: 'Checked Out', value: 7, color: '#f59e0b' },
    { label: 'Under Repair', value: 3, color: '#ef4444' }
  ];

  showAllActivities = false;

  activities: Activity[] = [
```

```
    {
      title: 'GPS Rover checked out by Ahmad',
      time: 'Today',
      icon: '↗',
      type: 'success'
    },
    {
      title: 'Total Station returned',
      time: 'Today',
      icon: '✓',
      type: 'success'
    },
    {
      title: 'Drone requires approval',
      time: 'Yesterday',
      icon: '!',
      type: 'danger'
    },
    {
      title: 'Field Tablet added',
      time: 'Yesterday',
      icon: '+',
      type: 'default'
    }
  ];

  get visibleActivities(): Activity[] {
    return this.showAllActivities
      ? this.activities
      : this.activities.slice(0, 3);
  }

  toggleActivities(): void {
    this.showAllActivities = !this.showAllActivities;
  }
}
```

These statistics and activities are example display data, not live database values. You can connect them to your backend later.

3. Create `src/app/components/dashboard/dashboard-home.html`

```
<div class="dashboard-home">

  <!-- Summary Cards -->
  <section class="stats-grid">
```

```
    @for (stat of equipmentStats; track stat.title) {
      <article class="stat-card">
        <div class="stat-top">
          <span
            class="stat-icon"
            [class]="'stat-icon ' + stat.color"
          >
            {{ stat.icon }}
          </span>

          <span class="stat-title">{{ stat.title }}</span>
        </div>

        <div class="stat-value">{{ stat.value }}</div>
        <div class="stat-detail">{{ stat.detail }}</div>
      </article>
    }
  </section>

  <!-- Dashboard Panels -->
  <section class="dashboard-grid">

    <!-- Equipment Status -->
    <article class="panel status-panel">
      <div class="panel-heading">
        <h2>Equipment Status</h2>

        <button
          type="button"
          class="more-button"
          title="Equipment status"
        >
          …
        </button>
      </div>

      <div class="status-content">
        <div class="donut-chart">
          <div class="donut-center">
            <strong>25</strong>
            <span>Total</span>
          </div>
        </div>

        <div class="legend">
          @for (status of equipmentStatus; track status.label) {
```

```
            <div class="legend-row">
              <span
                class="legend-dot"
                [style.background]="status.color"
              ></span>

              <span class="legend-label">
                {{ status.label }}
              </span>

              <span class="legend-value">
                {{ status.value }}
              </span>
            </div>
          }
        </div>
      </div>
    </article>

    <!-- Recent Activity -->
    <article class="panel activity-panel">
      <div class="panel-heading">
        <h2>Recent Activity</h2>

        <button
          type="button"
          class="view-all"
          (click)="toggleActivities()"
        >
          {{ showAllActivities ? 'Show less' : 'View All' }}
        </button>
      </div>

      <div class="activity-list">
        @for (activity of visibleActivities; track activity.title) {
          <div class="activity-item">
            <div
              class="activity-icon"
              [class.success]="activity.type === 'success'"
              [class.danger]="activity.type === 'danger'"
            >
              {{ activity.icon }}
            </div>

            <div class="activity-info">
              <p>{{ activity.title }}</p>
```

```
              <span>{{ activity.time }}</span>
            </div>
          </div>
        }
      </div>
    </article>

  </section>

  <!-- Dashboard Footer -->
  <footer class="dashboard-footer">
    Horizon Survey Services
    <span>Equipment Tracking System</span>
  </footer>

</div>
```

4. Create `src/app/components/dashboard/dashboard-home.css`

If your existing `dashboard.css` already contains styles for these sections, copy the corresponding rules into this file. Otherwise, you can start with this basic layout:

```
.dashboard-home {
  display: flex;
  flex-direction: column;
  gap: 24px;
  padding: 24px;
}

.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 18px;
}

.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 24px;
}

@media (max-width: 1000px) {
  .stats-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
```

```
@media (max-width: 650px) {
  .dashboard-home {
    padding: 16px;
  }

  .stats-grid,
  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}
```

To preserve the exact visual appearance, reuse your existing CSS rules for `.stat-card`, `.panel`, `.donut-chart`, `.legend`, `.activity-item`, and `.dashboard-footer` in `dashboard-home.css`.

5. Update `src/app/components/dashboard/dashboard.ts`

Your existing Dashboard component should now be responsible only for the shared layout and navigation.

Keep your existing `menuItems` data and `toggleProfile()` method. Make sure `selectMenu()` navigates to the appropriate route.

For example, if your menu labels are `Dashboard`, `Equipment`, `Check Out`, and `Approvals`:

```
import { Component } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

interface MenuItem {
  label: string;
  icon: string;
}

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css'
})
export class Dashboard {
  selectedMenu = 'Dashboard';
  showProfileMenu = false;

  menuItems: MenuItem[] = [
    { label: 'Dashboard', icon: '⌂' },
    { label: 'Equipment', icon: '▣' },
```

```
    { label: 'Check Out', icon: '↗' },
    { label: 'Approvals', icon: '✓' }
  ];

  constructor(private router: Router) {}

  selectMenu(label: string): void {
    this.selectedMenu = label;

    const routes: Record<string, string> = {
      'Dashboard': '/dashboard',
      'Equipment': '/equipment',
      'Check Out': '/checkout',
      'Approvals': '/approvals'
    };

    const route = routes[label];

    if (route) {
      this.router.navigateByUrl(route);
    }
  }

  toggleProfile(): void {
    this.showProfileMenu = !this.showProfileMenu;
  }
}
```

If your existing `dashboard.ts` has additional navigation logic, retain it and adapt only the routing behavior shown above.

6. Update `src/app/app.routes.ts`

Use the shared Dashboard component as the parent route, with the Dashboard home as its default child.

```
import { Routes } from '@angular/router';

import { Dashboard } from './components/dashboard/dashboard';
import { Equipment } from './components/equipment/equipment';
import { Checkout } from './components/checkout/checkout';

export const routes: Routes = [
  {
    path: '',
    component: Dashboard,
    children: [
```

```
      {
        path: 'dashboard',
        loadComponent: () =>
          import('./components/dashboard/dashboard-home')
            .then(m => m.DashboardHome)
      },
      {
        path: 'equipment',
        component: Equipment
      },
      {
        path: 'checkout',
        component: Checkout
      },
      {
        path: 'approvals',
        loadComponent: () =>
          import('./components/approvals/approvals')
            .then(m => m.Approvals)
      },
      {
        path: '',
        redirectTo: 'dashboard',
        pathMatch: 'full'
      }
    ]
  },
  {
    path: '**',
    redirectTo: 'dashboard'
  }
];
```

The `Approvals` import assumes you already have an `approvals.ts` component exporting `Approvals`. If you do not have that page yet, remove the `approvals` route and its menu navigation entry until you create it.

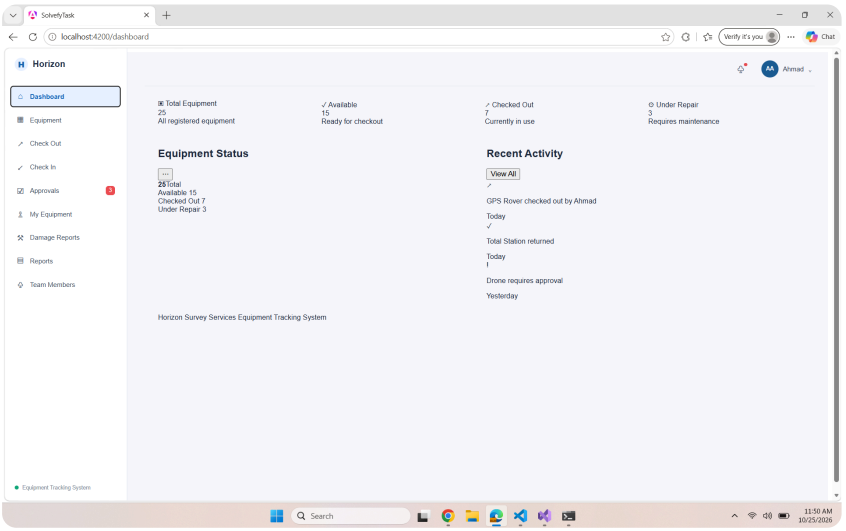
One final check: your `Equipment` and `Checkout` components must be routed as children of `Dashboard`, not placed directly beside the parent route. Otherwise, those pages will not share the sidebar and topbar.

After these changes:

- `/dashboard` shows the sidebar, topbar, summary cards, equipment status, and recent activity.
- `/equipment` shows the sidebar, topbar, and Equipment page only.

- `/checkout` shows the sidebar, topbar, and Checkout form only.

24/10/2026, 22:40:27 · gpt-6



the dashboard layout has been disturbed the rest of the pages are working fine but the dashboard home page is not working ??

25/10/2026, 11:51:50

25/10/2026, 11:51:52

> What does the dashboard home page need fixed first?
Restore the original dashboard layout and styling

25/10/2026, 11:51:58

The dashboard screenshot shows that the original styling has been lost: the summary cards no longer have proper card backgrounds, the equipment status chart is missing its styling, and the dashboard panels are displaying as plain text.

The fix should **restore your original dashboard CSS and layout without changing the other working pages**.

Please share these two current files:

- `src/app/components/dashboard/dashboard.css`
- `src/app/components/dashboard/dashboard-home.css` (if you created it)

I'll identify which styles were lost or moved and provide the corrected CSS. We should preserve the existing sidebar, topbar, card design, chart, spacing, and activity panel rather than redesigning them.

25/10/2026, 11:52:06 · gpt-6

\
.dashboard-home {\
  display: flex;\
  flex-direction: column;\
  gap: 24px;\
  padding: 24px;\
}\
\
.stats-grid {\
  display: grid;\
  grid-template-columns: repeat(4, minmax(0, 1fr));\
  gap: 18px;\
}\
\
.dashboard-grid {\
  display: grid;\
  grid-template-columns: repeat(2, minmax(0, 1fr));\
  gap: 24px;\
}\
\
@media (max-width: 1000px) {\
  .stats-grid {\
    grid-template-columns: repeat(2, minmax(0, 1fr));\
  }\
}\
\
@media (max-width: 650px) {\
  .dashboard-home {\
    padding: 16px;\
  }\
\
  .stats-grid,\
  .dashboard-grid {\
    grid-template-columns: 1fr;\
  }\
}\

the above is dashboard-home.css

:host {
  display: block;
  min-height: 100vh;

```
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 13px;
}

\* {
  box-sizing: border-box;
}

button {
  font: inherit;
}

.dashboard-shell {
  display: flex;
  min-height: 100vh;
  background: #f6f8fb;
}

/\* Sidebar \*/

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  bottom: 0;

  width: 225px;
  height: 100vh;

  display: flex;
  flex-direction: column;

  background: #fff;
  border-right: 1px solid #e5eaf1;

  padding: 18px 12px;
  overflow-y: auto;

  z-index: 1000;
}

.brand {
  display: flex;
  align-items: center;
  gap: 9px;
```

```
  padding: 0 7px 28px;
  color: #27364b;
  font-size: 16px;
  font-weight: 700;
}

.brand-mark {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 24px;
  height: 24px;
  border-radius: 50%;
  background: #e8f2ff;
  color: #1766b2;
  font-size: 14px;
  font-weight: 800;
}

.navigation {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  width: 100%;
  min-height: 38px;
  padding: 9px 10px;
  border: 0;
  border-radius: 5px;
  background: transparent;
  color: #435064;
  text-align: left;
  font-size: 12px;
  cursor: pointer;
  transition: *background* 0.2s, color 0.2s;
}

.nav-item:hover {
  background: #f0f5fc;
}
```

```
.nav-item.active {
  background: #eaf2ff;
  color: #1769c2;
  font-weight: 700;
}

.nav-icon {
  width: 15px;
  flex-shrink: 0;
  font-size: 15px;
  text-align: center;
}

.nav-badge {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 17px;
  height: 17px;
  margin-left: auto;
  border-radius: 5px;
  background: #ed4d54;
  color: white;
  font-size: 10px;
}

.sidebar-footer {
  display: flex;
  align-items: center;
  gap: 7px;
  margin-top: auto;
  padding: 18px 7px 0;
  color: #8792a3;
  font-size: 10px;
}

.status-dot,
.notification-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: #13a879;
}

/\* Main area \*/
```

```
.main-content {
  margin-left: 225px;
  width: calc(100% - 225px);
  min-width: 0;
  min-height: 100vh;
  padding: 0 24px 18px;
}

.topbar {
  position: relative;
  display: flex;
  align-items: center;
  gap: 15px;
  min-height: 62px;
  border-bottom: 1px solid #e9edf3;
}

.topbar-spacer {
  flex: 1;
}

.mobile-brand {
  display: none;
  align-items: center;
  gap: 7px;
  font-weight: 700;
}

.notification-button {
  position: relative;
  width: 34px;
  height: 34px;
  border: 0;
  background: transparent;
  color: #59677b;
  font-size: 19px;
  cursor: pointer;
}

.notification-dot {
  position: absolute;
  top: 6px;
  right: 5px;
  width: 6px;
  height: 6px;
  background: #e84b54;
```

```
}

.profile-container {
  position: relative;
}

.profile-button {
  display: flex;
  align-items: center;
  gap: 8px;
  border: 0;
  background: transparent;
  color: #344155;
  font-size: 12px;
  cursor: pointer;
}

.avatar {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 31px;
  height: 31px;
  border-radius: 50%;
  background: #1d5b94;
  color: white;
  font-size: 10px;
  font-weight: 700;
}

.chevron {
  color: #7b8798;
}

.profile-menu {
  position: absolute;
  z-index: 5;
  top: 42px;
  right: 0;
  min-width: 145px;
  padding: 12px;
  border: 1px solid #e5eaf1;
  border-radius: 7px;
  background: white;
  box-shadow: 0 6px 20px #1b35501a;
}
```

```
.profile-menu-name {
  margin-bottom: 10px;
  font-weight: 700;
}

.profile-menu button {
  border: 0;
  background: transparent;
  color: #1769c2;
  cursor: pointer;
}

.page-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 0 18px;
}

.page-heading h1 {
  margin: 0 0 6px;
  color: #202a39;
  font-size: 24px;
  font-weight: 700;
}

.page-heading p {
  margin: 0;
  color: #8994a4;
  font-size: 12px;
}

/\* Summary cards \*/

.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 20px;
}

.stat-card {
  min-width: 0;
  padding: 15px 14px 13px;
  border: 1px solid #e9edf3;
```

```
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.stat-top {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 10px;
}

.stat-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 6px;
  font-size: 15px;
  font-weight: 700;
}

.stat-icon.blue {
  background: #eaf2ff;
  color: #2675dd;
}

.stat-icon.red {
  background: #fff0ee;
  color: #e86661;
}

.stat-icon.green {
  background: #e6f8f1;
  color: #169c76;
}

.stat-icon.yellow {
  background: #fff6df;
  color: #d99c18;
}

.stat-title {
  color: #68758a;
```

```
  font-size: 11px;
  line-height: 1.4;
}

.stat-value {
  margin-bottom: 4px;
  color: #202a39;
  font-size: 27px;
  font-weight: 700;
  line-height: 1.1;
}

.stat-detail {
  color: #a0a9b7;
  font-size: 10px;
}

/\* Panels \*/

.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  align-items: stretch;
  gap: 16px;
}

.panel {
  min-width: 0;
  padding: 18px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.panel-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 23px;
}

.panel-heading h2 {
  margin: 0;
  color: #283447;
```

```
  font-size: 14px;
  font-weight: 700;
}

.more-button {
  border: 0;
  background: transparent;
  color: #8591a2;
  font-size: 20px;
  cursor: pointer;
}

.status-content {
  display: flex;
  align-items: center;
  justify-content: space-around;
  gap: 16px;
  padding: 5px 0 14px;
}

.donut-chart {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 155px;
  height: 155px;
  flex-shrink: 0;
  border-radius: 50%;
  background: conic-gradient(
    #119c78 0% 40%,
    #b98d61 40% 88%,
    #f4bd45 88% 96%,
    #f28b47 96% 100%
  );
  transform: rotate(-90deg);
}

.donut-center {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  width: 100px;
  height: 100px;
  border-radius: 50%;
  background: white;
```

```
  transform: rotate(90deg);
}

.donut-center strong {
  color: #263346;
  font-size: 27px;
  line-height: 1.2;
}

.donut-center span {
  margin-top: 3px;
  color: #929cac;
  font-size: 11px;
}

.legend {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 14px;
  min-width: 0;
}

.legend-row {
  display: grid;
  grid-template-columns: 8px minmax(0, 1fr) auto;
  align-items: center;
  gap: 7px;
  color: #5f6b7c;
  font-size: 11px;
}

.legend-dot {
  width: 8px;
  height: 8px;
  border-radius: 2px;
}

.legend-label {
  line-height: 1.4;
}

.legend-value {
  color: #748093;
}
```

```
/\* Recent activity \*/

.view-all {
  border: 0;
  background: transparent;
  color: #2675dd;
  font-size: 12px;
  font-weight: 600;
  white-space: nowrap;
  cursor: pointer;
}

.activity-list {
  display: flex;
  flex-direction: column;
}

.activity-item {
  display: flex;
  align-items: flex-start;
  gap: 11px;
  padding: 0 0 19px;
}

.activity-item:last-child {
  padding-bottom: 0;
}

.activity-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 7px;
  font-size: 14px;
  font-weight: 700;
}

.activity-icon.success {
  background: #e4f7ed;
  color: #139968;
}

.activity-icon.danger {
```

```
  background: #fff0ef;
  color: #e45454;
}

.activity-info {
  min-width: 0;
  padding-top: 1px;
}

.activity-info p {
  margin: 0 0 5px;
  color: #3b4657;
  font-size: 11px;
  line-height: 1.45;
  overflow-wrap: anywhere;
}

.activity-info span {
  color: #9aa4b2;
  font-size: 10px;
}

.dashboard-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  padding-top: 22px;
  color: #8a95a4;
  font-size: 10px;
}

.dashboard-footer span {
  text-align: right;
}

/\* Responsive layout \*/

@media (max-width: 1100px) {
  .sidebar {
    width: 190px;
  }

  .main-content {
    padding-right: 16px;
    padding-left: 16px;
  }
```

```
  .stats-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 650px) {
  .sidebar {
    display: none;
  }

  .main-content {
    padding: 0 13px 16px;
  }

  .topbar {
    min-height: 56px;
  }

  .mobile-brand {
    display: flex;
  }

  .page-heading {
    padding: 18px 0;
  }

  .page-heading h1 {
    font-size: 22px;
  }

  .stats-grid {
    gap: 10px;
  }

  .stat-card {
    padding: 12px 10px;
  }

  .stat-title {
    font-size: 10px;
  }
```

```
.stat-value {
  font-size: 24px;
}

.panel {
  padding: 14px;
}

.status-content {
  flex-wrap: wrap;
}

.donut-chart {
  width: 135px;
  height: 135px;
}

.donut-center {
  width: 88px;
  height: 88px;
}

.legend {
  min-width: 135px;
}
}
```

25/10/2026, 11:52:55

I found the cause. Your original `dashboard.css` contains the styling that makes the cards, donut chart, panels, and activity list look correct. But after moving the dashboard content into `dashboard-home.html`, those elements are now controlled by a different component. Angular's component-scoped CSS means the styles in `dashboard.css` do not automatically apply to the new `DashboardHome` component.

The simplest fix is to copy the Dashboard Home styles into `dashboard-home.css`. Keep your existing `dashboard.css` unchanged so the sidebar, topbar, and other pages continue working.

Replace `dashboard-home.css` with this complete code

This includes the original card and panel styles from your supplied CSS, along with responsive layout rules.

```scss
:host {
  display: block;
  min-width: 0;
  color: #202b3c;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 13px;
}

* {
  box-sizing: border-box;
}

button {
  font: inherit;
  cursor: pointer;
}

.dashboard-home {
  display: flex;
  flex-direction: column;
  gap: 20px;
  padding: 24px 0;
  min-width: 0;
}

/* Summary cards */

.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 0;
}

.stat-card {
  min-width: 0;
  padding: 15px 14px 13px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.stat-top {
  display: flex;
  align-items: center;
```

```
  gap: 8px;
  margin-bottom: 10px;
}

.stat-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 6px;
  font-size: 15px;
  font-weight: 700;
}

.stat-icon.blue {
  background: #eaf2ff;
  color: #2675dd;
}

.stat-icon.red {
  background: #fff0ee;
  color: #e86661;
}

.stat-icon.green {
  background: #e6f8f1;
  color: #169c76;
}

.stat-icon.yellow {
  background: #fff6df;
  color: #d99c18;
}

.stat-title {
  color: #68758a;
  font-size: 11px;
  line-height: 1.4;
}

.stat-value {
  margin-bottom: 4px;
  color: #202a39;
  font-size: 27px;
```

```
  font-weight: 700;
  line-height: 1.1;
}

.stat-detail {
  color: #a0a9b7;
  font-size: 10px;
}

/* Dashboard panels */

.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  align-items: stretch;
  gap: 16px;
}

.panel {
  min-width: 0;
  padding: 18px;
  border: 1px solid #e9edf3;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 5px #1b355005;
}

.panel-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 23px;
}

.panel-heading h2 {
  margin: 0;
  color: #283447;
  font-size: 14px;
  font-weight: 700;
}

.more-button {
  border: 0;
  background: transparent;
  color: #8591a2;
```

```
  font-size: 20px;
}

.status-content {
  display: flex;
  align-items: center;
  justify-content: space-around;
  gap: 16px;
  padding: 5px 0 14px;
}

/* Donut chart */

.donut-chart {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 155px;
  height: 155px;
  flex-shrink: 0;
  border-radius: 50%;
  background: conic-gradient(
    #119c78 0% 40%,
    #b98d61 40% 68%,
    #f4bd45 68% 88%,
    #f28b47 88% 100%
  );
  transform: rotate(-90deg);
}

.donut-center {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  width: 100px;
  height: 100px;
  border-radius: 50%;
  background: #fff;
  transform: rotate(90deg);
}

.donut-center strong {
  color: #263346;
  font-size: 27px;
  line-height: 1.2;
```

```
}

.donut-center span {
  margin-top: 3px;
  color: #929cac;
  font-size: 11px;
}

/* Equipment status legend */

.legend {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 14px;
  min-width: 0;
}

.legend-row {
  display: grid;
  grid-template-columns: 8px minmax(0, 1fr) auto;
  align-items: center;
  gap: 7px;
  color: #5f6b7c;
  font-size: 11px;
}

.legend-dot {
  width: 8px;
  height: 8px;
  border-radius: 2px;
}

.legend-label {
  line-height: 1.4;
}

.legend-value {
  color: #748093;
}

/* Recent activity */

.view-all {
  border: 0;
  background: transparent;
```

```
  color: #2675dd;
  font-size: 12px;
  font-weight: 600;
  white-space: nowrap;
}

.activity-list {
  display: flex;
  flex-direction: column;
}

.activity-item {
  display: flex;
  align-items: flex-start;
  gap: 11px;
  padding: 0 0 19px;
}

.activity-item:last-child {
  padding-bottom: 0;
}

.activity-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 7px;
  font-size: 14px;
  font-weight: 700;
}

.activity-icon.success {
  background: #e4f7ed;
  color: #139968;
}

.activity-icon.danger {
  background: #fff0ef;
  color: #e45454;
}

.activity-info {
  min-width: 0;
```

```
  padding-top: 1px;
}

.activity-info p {
  margin: 0 0 5px;
  color: #3b4657;
  font-size: 11px;
  line-height: 1.45;
  overflow-wrap: anywhere;
}

.activity-info span {
  color: #9aa4b2;
  font-size: 10px;
}

/* Footer */

.dashboard-footer {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  padding-top: 2px;
  color: #8a95a4;
  font-size: 10px;
}

.dashboard-footer span {
  text-align: right;
}

/* Responsive layout */

@media (max-width: 1100px) {
  .stats-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 650px) {
  .dashboard-home {
    gap: 16px;
```

```
    padding: 16px 0;
  }

  .stats-grid {
    gap: 10px;
  }

  .stat-card {
    padding: 12px 10px;
  }

  .stat-title {
    font-size: 10px;
  }

  .stat-value {
    font-size: 24px;
  }

  .panel {
    padding: 14px;
  }

  .status-content {
    flex-wrap: wrap;
  }

  .donut-chart {
    width: 135px;
    height: 135px;
  }

  .donut-center {
    width: 88px;
    height: 88px;
  }

  .legend {
    min-width: 135px;
  }
}
```

One additional check

In `dashboard-home.ts`, make sure the component points to this stylesheet:

```
@Component({
  selector: 'app-dashboard-home',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './dashboard-home.html',
  styleUrl: './dashboard-home.css'
})
```

Also, your HTML uses `stat.color` to set the icon class. Ensure the stat objects use the color names `blue`, `green`, `red`, or `yellow` so the correct icon styling is applied.

After saving, let Angular rebuild and refresh `/dashboard`. Your dashboard cards, chart, panels, and activity list should regain their styling without modifying the working Equipment and Checkout pages.

25/10/2026, 11:52:58 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-checkin',
  imports: [FormsModule],
  templateUrl: './checkin.html',
  styleUrl: './checkin.css',
})
export class Checkin {
  constructor(private router: Router) {}

  selectedEquipment = 'TS-001';
  returnDate = '2026-10-10';
  returnTime = '16:30';
  condition = 'good';
  notes = '';
  submitted = false;

  equipmentOptions = [
    {
      id: 'TS-001',
      name: 'Total Station - 01',
      serial: 'TS-001',
      type: 'Total Station'
    },
    {
      id: 'GPS-001',
      name: 'GPS Rover - 01',
      serial: 'GPS-001',
      type: 'GPS'
    },
    {
      id: 'DR-001',
      name: 'Drone - DJI Mavic',
      serial: 'DR-001',
      type: 'Drone'
    },
    {
      id: 'FT-001',
      name: 'Field Tablet',
      serial: 'FT-001',
      type: 'Tablet'
```

```
    }
  ];

  get selectedItem() {
    return this.equipmentOptions.find(
      item => item.id === this.selectedEquipment
    );
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  checkIn(): void {
    this.submitted = true;

    if (
      !this.selectedEquipment ||
      !this.returnDate ||
      !this.returnTime ||
      !this.condition
    ) {
      return;
    }

    // Demo only: connect your check-in API here.
    alert(
      `Check-in recorded for ${this.selectedItem?.name}.\n` +
      `Condition: ${this.condition}`
    );
  }
}
```

please implement the apis of this only dotnet backend and then implement the relavant apis ??

25/10/2026, 11:56:56

I'll implement the **Check In Equipment APIs in your existing .NET backend first**, following the repository → service → controller structure used by your Checkout APIs.

The Check In workflow will support:

1. **GET equipment** — retrieve equipment from SQL Server rather than using hardcoded Angular data.

2. **GET equipment details** — retrieve the selected equipment's name, serial number, and type.
3. **POST check-in** — save the return date, return time, condition, and notes in SQL Server.
4. **Update equipment status** — when a check-in is recorded, update the equipment's status to `Available` if the returned condition is good, or `Under Repair` if damaged.

One important detail: your current form uses `TS-001` as an equipment identifier. Your database model uses numeric equipment IDs, so we'll use the database ID for API lookups and keep the serial number as the displayed equipment identifier.

1. Create the Check In model

Create `Models/CheckIn.cs` in your .NET project.

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
    public class CheckIn
    {
        public int Id { get; set; }

        [Required]
        public int EquipmentId { get; set; }

        public DateTime ReturnDate { get; set; }

        public TimeSpan ReturnTime { get; set; }

        [Required]
        [MaxLength(30)]
        public string Condition { get; set; } = "Good";

        public string? Notes { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public Equipment? Equipment { get; set; }
    }
}
```

This stores the check-in as a separate record, preserving the history of returned equipment.

2. Update `ApplicationDbContext.cs`

Add the `CheckIns` table and its relationship to your existing `Equipment` model.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Models;

namespace SolvefyTask.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Equipment> Equipments => Set<Equipment>();
        public DbSet<Checkout> Checkouts => Set<Checkout>();
        public DbSet<CheckIn> CheckIns => Set<CheckIn>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<CheckIn>()
                .HasOne(c => c.Equipment)
                .WithMany()
                .HasForeignKey(c => c.EquipmentId)
                .OnDelete(DeleteBehavior.Restrict);
        }
    }
}
```

Important: If your existing `ApplicationDbContext` already configures other entities or relationships, keep those configurations and add only the `CheckIns` property and `CheckIn` relationship.

3. Create the Check In repository interface

Create `Repositories/ICheckInRepository.cs`.

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface ICheckInRepository
    {
        Task<List<Equipment>> GetEquipmentAsync();
```

```
        Task<Equipment?> GetEquipmentByIdAsync(int equipmentId);

        Task<CheckIn> CreateCheckInAsync(CheckIn checkIn);
    }
}
```

4. Implement the repository

Create `Repositories/CheckInRepository.cs`.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class CheckInRepository : ICheckInRepository
    {
        private readonly ApplicationDbContext _context;

        public CheckInRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<Equipment>> GetEquipmentAsync()
        {
            return await _context.Equipments
                .AsNoTracking()
                .OrderBy(e => e.Name)
                .ToListAsync();
        }

        public async Task<Equipment?> GetEquipmentByIdAsync(
            int equipmentId)
        {
            return await _context.Equipments
                .FirstOrDefaultAsync(e => e.Id == equipmentId);
        }

        public async Task<CheckIn> CreateCheckInAsync(CheckIn checkIn)
        {
            await using var transaction =
                await _context.Database.BeginTransactionAsync();

            try
```

```
            {
                var equipment = await _context.Equipments
                    .FirstOrDefaultAsync(
                        e => e.Id == checkIn.EquipmentId);

                if (equipment == null)
                {
                    throw new InvalidOperationException(
                        "Equipment was not found.");
                }

                // Record the return history.
                checkIn.Equipment = equipment;
                checkIn.CreatedAt = DateTime.UtcNow;

                await _context.CheckIns.AddAsync(checkIn);

                // Update the current equipment status.
                equipment.Status =
                    checkIn.Condition.Equals(
                        "Damaged",
                        StringComparison.OrdinalIgnoreCase)
                    || checkIn.Condition.Equals(
                        "Needs Repair",
                        StringComparison.OrdinalIgnoreCase)
                    || checkIn.Condition.Equals(
                        "Poor",
                        StringComparison.OrdinalIgnoreCase)
                        ? "Under Repair"
                        : "Available";

                await _context.SaveChangesAsync();
                await transaction.CommitAsync();

                return checkIn;
            }
            catch
            {
                await transaction.RollbackAsync();
                throw;
            }
        }
    }
}
```

This saves the check-in record and updates the equipment status in one database transaction.

5. Create the service interface

Create `Services/ICheckInService.cs`.

```
using SolvefyTask.Models;

namespace SolvefyTask.Services
{
    public interface ICheckInService
    {
        Task<List<Equipment>> GetEquipmentAsync();

        Task<Equipment?> GetEquipmentByIdAsync(int equipmentId);

        Task<CheckIn> CreateCheckInAsync(CheckIn checkIn);
    }
}
```

6. Implement the service

Create `Services/CheckInService.cs`.

```
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class CheckInService : ICheckInService
    {
        private readonly ICheckInRepository _repository;

        public CheckInService(ICheckInRepository repository)
        {
            _repository = repository;
        }

        public Task<List<Equipment>> GetEquipmentAsync()
        {
            return _repository.GetEquipmentAsync();
        }

        public Task<Equipment?> GetEquipmentByIdAsync(int equipmentId)
        {
            return _repository.GetEquipmentByIdAsync(equipmentId);
        }

        public async Task<CheckIn> CreateCheckInAsync(CheckIn checkIn)
```

```
        {
            checkIn.Id = 0;

            return await _repository.CreateCheckInAsync(checkIn);
        }
    }
}
```

7. Create `Controllers/CheckInController.cs`

This controller exposes the three endpoints your Check In screen needs.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Models;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CheckInController : ControllerBase
    {
        private readonly ICheckInService _service;

        public CheckInController(ICheckInService service)
        {
            _service = service;
        }

        // GET: /api/CheckIn/equipment
        [HttpGet("equipment")]
        public async Task<IActionResult> GetEquipment()
        {
            var equipment = await _service.GetEquipmentAsync();

            var result = equipment.Select(e => new
            {
                id = e.Id,
                name = e.Name,
                serial = e.SerialNumber,
                type = e.Type,
                status = e.Status
            });

            return Ok(result);
        }
```

```
// GET: /api/CheckIn/equipment/1
[HttpGet("equipment/{equipmentId:int}")]
public async Task<IActionResult> GetEquipmentById(
    int equipmentId)
{
    var equipment =
        await _service.GetEquipmentByIdAsync(equipmentId);

    if (equipment == null)
    {
        return NotFound("Equipment was not found.");
    }

    return Ok(new
    {
        id = equipment.Id,
        name = equipment.Name,
        serial = equipment.SerialNumber,
        type = equipment.Type,
        status = equipment.Status
    });
}

// POST: /api/CheckIn
[HttpPost]
public async Task<IActionResult> CreateCheckIn(
    [FromBody] CheckIn checkIn)
{
    if (checkIn.EquipmentId <= 0)
    {
        return BadRequest(new
        {
            message = "A valid equipment ID is required."
        });
    }

    var validConditions = new[]
    {
        "Good",
        "Fair",
        "Damaged",
        "Needs Repair",
        "Poor"
    };
```

```
            if (!validConditions.Contains(
                checkIn.Condition,
                StringComparer.OrdinalIgnoreCase))
            {
                return BadRequest(new
                {
                    message = "Invalid equipment condition."
                });
            }

            if (checkIn.ReturnDate.Date > DateTime.Today.AddYears(5))
            {
                return BadRequest(new
                {
                    message = "The return date is invalid."
                });
            }

            try
            {
                var result =
                    await _service.CreateCheckInAsync(checkIn);

                return CreatedAtAction(
                    nameof(GetEquipmentById),
                    new { equipmentId = result.EquipmentId },
                    new
                    {
                        id = result.Id,
                        equipmentId = result.EquipmentId,
                        returnDate = result.ReturnDate,
                        returnTime = result.ReturnTime.ToString(@"hh\:mm"),
                        condition = result.Condition,
                        notes = result.Notes,
                        status = result.Equipment?.Status,
                        createdAt = result.CreatedAt
                    });
            }
            catch (InvalidOperationException ex)
            {
                return NotFound(new { message = ex.Message });
            }
        }
    }
}
```

8. Register the repository and service in `Program.cs`

Add these registrations alongside your existing Checkout and Equipment registrations. Do not remove your existing registrations.

```
builder.Services.AddScoped<ICheckInRepository, CheckInRepository>();
builder.Services.AddScoped<ICheckInService, CheckInService>();
```

Add the required namespaces at the top if they are not already present:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

Your existing `app.MapControllers()` must remain in `Program.cs`.

9. Create the database migration

Open Visual Studio's **Package Manager Console** and run:

```
Add-Migration AddCheckInTable
Update-Database
```

This creates the `CheckIns` table and its relationship with `Equipments`.

If your project uses the .NET CLI instead, run these commands from the backend project directory:

```
dotnet ef migrations add AddCheckInTable
dotnet ef database update
```

---

10. Test the APIs in Swagger

Run your backend and test the following endpoints.

GET **Load equipment**
`/api/CheckIn/equipment`

GET **Get equipment details**
`/api/CheckIn/equipment/1`

POST **Save a check-in**
`/api/CheckIn`

Use this JSON body for the POST request. Make sure equipment ID `1` exists in your database.

```
{
  "equipmentId": 1,
  "returnDate": "2026-10-25",
  "returnTime": "16:30:00",
  "condition": "Good",
  "notes": "Equipment returned in good condition."
}
```

Expected result: `201 Created`, a new row in `CheckIns`, and the equipment status updated to `Available`. If the condition is `Damaged`, `Needs Repair`, or `Poor`, the status becomes `Under Repair`.

Important business-rule note: this implementation records the check-in and updates equipment status, but it does not yet verify that the equipment has an outstanding checkout. Before using it in production, add that validation to prevent someone from checking in equipment that was never checked out.

25/10/2026, 11:57:18 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-checkin',
  imports: [FormsModule],
  templateUrl: './checkin.html',
  styleUrl: './checkin.css',
})
export class Checkin {
  constructor(private router: Router) {}

  selectedEquipment = 'TS-001';
  returnDate = '2026-10-10';
  returnTime = '16:30';
  condition = 'good';
  notes = '';
  submitted = false;

  equipmentOptions = [
    {
      id: 'TS-001',
      name: 'Total Station - 01',
      serial: 'TS-001',
      type: 'Total Station'
    },
    {
      id: 'GPS-001',
      name: 'GPS Rover - 01',
      serial: 'GPS-001',
      type: 'GPS'
    },
    {
      id: 'DR-001',
      name: 'Drone - DJI Mavic',
      serial: 'DR-001',
      type: 'Drone'
    },
    {
      id: 'FT-001',
      name: 'Field Tablet',
      serial: 'FT-001',
      type: 'Tablet'
```

```
    }
  ];

  get selectedItem() {
    return this.equipmentOptions.find(
      item => item.id === this.selectedEquipment
    );
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  checkIn(): void {
    this.submitted = true;

    if (
      !this.selectedEquipment ||
      !this.returnDate ||
      !this.returnTime ||
      !this.condition
    ) {
      return;
    }

    // Demo only: connect your check-in API here.
    alert(
      `Check-in recorded for ${this.selectedItem?.name}.\n` +
      `Condition: ${this.condition}`
    );
  }
}
please implement the above 3 apis in this code ??
```

25/10/2026, 12:13:44

We'll connect your existing `Checkin` component to the three .NET APIs without changing your page design:

1. **GET** `/api/CheckIn/equipment` — load equipment from SQL Server.
2. **GET** `/api/CheckIn/equipment/{equipmentId}` — load the selected equipment's details.
3. **POST** `/api/CheckIn` — save the return date, time, condition, and notes.

Your current dropdown uses values such as `TS-001`, but the backend APIs use numeric database IDs. We'll use the numeric ID for API calls while displaying the equipment name and serial number in the UI.

1. Create `src/app/models/checkin.ts`

```
export interface CheckInEquipment {
  id: number;
  name: string;
  serial: string | null;
  type: string;
  status: string;
}

export interface CreateCheckInRequest {
  equipmentId: number;
  returnDate: string;
  returnTime: string;
  condition: string;
  notes: string;
}

export interface CheckInRecord {
  id: number;
  equipmentId: number;
  returnDate: string;
  returnTime: string;
  condition: string;
  notes: string | null;
  status: string;
  createdAt: string;
}
```

2. Create `src/app/services/checkin.ts`

This service calls all three backend endpoints.

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  CheckInEquipment,
  CheckInRecord,
  CreateCheckInRequest
```

```
} from '../models/checkin';

@Injectable({
  providedIn: 'root'
})
export class CheckInService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/CheckIn`;

  // API 1: GET /api/CheckIn/equipment
  getEquipment(): Observable<CheckInEquipment[]> {
    return this.http.get<CheckInEquipment[]>(
      `${this.apiUrl}/equipment`
    );
  }

  // API 2: GET /api/CheckIn/equipment/{equipmentId}
  getEquipmentById(
    equipmentId: number
  ): Observable<CheckInEquipment> {
    return this.http.get<CheckInEquipment>(
      `${this.apiUrl}/equipment/${equipmentId}`
    );
  }

  // API 3: POST /api/CheckIn
  createCheckIn(
    request: CreateCheckInRequest
  ): Observable<CheckInRecord> {
    return this.http.post<CheckInRecord>(
      this.apiUrl,
      request
    );
  }
}
```

3. Replace `src/app/components/checkin/checkin.ts`

This version loads equipment from the database, retrieves the selected item's details, and submits the check-in record.

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
```

```
import { CheckInService } from '../../services/checkin';
import {
  CheckInEquipment,
  CreateCheckInRequest
} from '../../models/checkin';

@Component({
  selector: 'app-checkin',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './checkin.html',
  styleUrl: './checkin.css',
})
export class Checkin implements OnInit {
  private router = inject(Router);
  private checkInService = inject(CheckInService);

  selectedEquipment: number | null = null;

  returnDate = this.getLocalDateString();
  returnTime = '16:30';
  condition = 'Good';
  notes = '';

  submitted = false;
  loadingEquipment = false;
  loadingDetails = false;
  saving = false;

  errorMessage = '';
  successMessage = '';

  equipmentOptions: CheckInEquipment[] = [];
  selectedItem: CheckInEquipment | null = null;

  ngOnInit(): void {
    this.loadEquipment();
  }

  private getLocalDateString(): string {
    const date = new Date();

    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const day = String(date.getDate()).padStart(2, '0');
```

```
  return `${year}-${month}-${day}`;
}

// API 1: Load equipment from SQL Server.
loadEquipment(): void {
  this.loadingEquipment = true;
  this.errorMessage = '';

  this.checkInService.getEquipment().subscribe({
    next: (equipment) => {
      this.equipmentOptions = equipment;
      this.loadingEquipment = false;

      if (equipment.length > 0) {
        this.selectedEquipment = equipment[0].id;
        this.onEquipmentChange();
      }
    },
    error: (error) => {
      console.error('Failed to load equipment:', error);

      this.loadingEquipment = false;
      this.errorMessage =
        'Unable to load equipment from the database.';
    }
  });
}

// API 2: Get the selected equipment details.
onEquipmentChange(): void {
  this.selectedItem = null;
  this.errorMessage = '';

  const equipmentId = this.selectedEquipment;

  if (equipmentId === null) {
    return;
  }

  this.loadingDetails = true;

  this.checkInService.getEquipmentById(equipmentId).subscribe({
    next: (equipment) => {
      if (this.selectedEquipment === equipmentId) {
        this.selectedItem = equipment;
```

```
        }

        this.loadingDetails = false;
      },
      error: (error) => {
        console.error('Failed to load equipment details:', error);

        if (this.selectedEquipment === equipmentId) {
          this.selectedItem = null;
          this.errorMessage =
            'Unable to load the selected equipment details.';
        }

        this.loadingDetails = false;
      }
    });
  }

  // API 3: Save the check-in record.
  checkIn(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';

    if (
      this.selectedEquipment === null ||
      !this.returnDate ||
      !this.returnTime ||
      !this.condition
    ) {
      this.errorMessage =
        'Please complete all required fields.';
      return;
    }

    if (this.loadingEquipment || this.loadingDetails || this.saving) {
      return;
    }

    if (!this.selectedItem) {
      this.errorMessage =
        'Please select valid equipment before checking it in.';
      return;
    }

    const request: CreateCheckInRequest = {
```

```
      equipmentId: this.selectedEquipment,
      returnDate: this.returnDate,
      returnTime: `${this.returnTime}:00`,
      condition: this.condition,
      notes: this.notes.trim()
    };

    console.log('Check-in request:', request);

    this.saving = true;

    this.checkInService.createCheckIn(request).subscribe({
      next: (result) => {
        console.log('Check-in saved:', result);

        this.saving = false;
        this.successMessage =
          'Equipment check-in saved successfully.';

        // Reflect the status returned by the backend.
        if (this.selectedItem && result.status) {
          this.selectedItem = {
            ...this.selectedItem,
            status: result.status
          };
        }

        alert(this.successMessage);
      },
      error: (error) => {
        console.error('Check-in failed:', error.status, error.error);

        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
          error?.error?.title ||
          'Unable to save the check-in record.';
      }
    });
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. Update the equipment dropdown in `checkin.html`

Your existing HTML must use the numeric database ID instead of the old hardcoded string IDs.

Replace the equipment dropdown with:

```
<div class="form-field">
  <label for="equipment">Select Equipment</label>

  <select
    id="equipment"
    name="equipment"
    [(ngModel)]="selectedEquipment"
    (ngModelChange)="onEquipmentChange()"
    required
    [disabled]="loadingEquipment || saving"
  >
    <option [ngValue]="null" disabled>
      Select equipment
    </option>

    @for (item of equipmentOptions; track item.id) {
      <option [ngValue]="item.id">
        {{ item.name }} ({{ item.serial || 'No serial' }})
      </option>
    }
  </select>
</div>

@if (loadingDetails) {
  <p>Loading equipment details...</p>
}

@if (selectedItem) {
  <div class="equipment-details">
    <p><strong>Name:</strong> {{ selectedItem.name }}</p>
    <p><strong>Serial:</strong> {{ selectedItem.serial || 'N/A' }}</p>
    <p><strong>Type:</strong> {{ selectedItem.type }}</p>
    <p><strong>Current Status:</strong> {{ selectedItem.status }}</p>
  </div>
}
```

Keep your existing return date, return time, condition, notes, and action buttons. Ensure your form still calls:

```
<form (ngSubmit)="checkIn()">
```

Disable the submit button while saving:

```
<button
  type="submit"
  [disabled]="saving || loadingEquipment || loadingDetails"
>
  {{ saving ? 'Saving...' : 'Check In Equipment' }}
</button>
```

Display API errors and success messages somewhere inside the page:

```
@if (errorMessage) {
  <p class="error-message">{{ errorMessage }}</p>
}

@if (successMessage) {
  <p class="success-message">{{ successMessage }}</p>
}
```

5. Verify the request body

When you submit the form, Angular should send a request like this:

```
{
  "equipmentId": 1,
  "returnDate": "2026-10-25",
  "returnTime": "16:30:00",
  "condition": "Good",
  "notes": "Equipment returned in good condition."
}
```

Your backend should then create a record in `CheckIns` and update the equipment status. The current backend implementation uses `Good`, `Fair`, `Damaged`, `Needs Repair`, or `Poor` as accepted conditions.

One important limitation: the backend implementation we created updates equipment status based on condition, but it does not yet verify that the equipment has an outstanding checkout. Add that check before using this workflow as a production equipment-tracking system.

25/10/2026, 12:13:55 · gpt-6

```
<div class="checkin-page">
  <section class="checkin-card">

    <!-- Header -->
    <header class="checkin-header">
      <div class="header-icon">
        <span>↓</span>
      </div>

      <div>
        <h1>Check In Equipment</h1>
        <p>Record the return of equipment to the office and
mark it as available again.</p>
      </div>
    </header>

    <form #checkInForm="ngForm" (ngSubmit)="checkIn()">

      <div class="checkin-body">

        <!-- Equipment -->
        <div class="form-field equipment-field">
          <label for="equipment">
            Equipment <span class="required">*</span>
          </label>

          <div class="equipment-select">
            <div class="equipment-thumbnail">📡</div>

            <div class="equipment-info">
              <strong>{{ selectedItem?.name }}</strong>
              <span>
                Serial No: {{ selectedItem?.serial }}
                  |  
                Type: {{ selectedItem?.type }}
              </span>
            </div>

            <select
              id="equipment"
              name="equipment"
              [(ngModel)]="selectedEquipment"
              required
              aria-label="Select returned equipment"
```

```
          >
            @for (item of equipmentOptions; track item.id) {
              <option [value]="item.id">{{ item.name }}
</option>
            }
          </select>

          <span class="select-chevron">⌄</span>
        </div>

        @if (submitted && !selectedEquipment) {
          <small class="error">Please select equipment.
</small>
        }
      </div>

      <!-- Return date and time -->
      <div class="date-time-grid">

        <div class="form-field">
          <label for="returnDate">
            Return Date <span class="required">*</span>
          </label>

          <input
            id="returnDate"
            name="returnDate"
            type="date"
            [(ngModel)]="returnDate"
            required
          />
        </div>

        <div class="form-field">
          <label for="returnTime">
            Return Time <span class="required">*</span>
          </label>

          <input
            id="returnTime"
            name="returnTime"
            type="time"
            [(ngModel)]="returnTime"
            required
          />
        </div>
```

```
      </div>

      <!-- Condition -->
      <fieldset class="condition-field">
        <legend>
          Condition on Return <span class="required">*
</span>
        </legend>

        <div class="condition-grid">

          <label class="condition-option good"
            [class.selected]="condition === 'good'">
            <input
              type="radio"
              name="condition"
              value="good"
              [(ngModel)]="condition"
              required
            />

            <span class="radio-visual"></span>

            <span class="condition-copy">
              <strong>Good</strong>
              <span>No issues</span>
            </span>
          </label>

          <label class="condition-option minor"
            [class.selected]="condition === 'minor'">
            <input
              type="radio"
              name="condition"
              value="minor"
              [(ngModel)]="condition"
              required
            />

            <span class="radio-visual"></span>

            <span class="condition-copy">
              <strong>Minor Issue</strong>
              <span>Small scratches, wear etc.</span>
            </span>
```

```
          </label>

          <label class="condition-option damaged"
           [class.selected]="condition === 'damaged'">
           <input
            type="radio"
            name="condition"
            value="damaged"
            [(ngModel)]="condition"
            required
           />

           <span class="radio-visual"></span>

           <span class="condition-copy">
            <strong>Damaged</strong>
            <span>Not working or needs repair</span>
           </span>
          </label>

         </div>
        </fieldset>

        <!-- Notes -->
        <div class="form-field notes-field">
         <label for="notes">Notes (Optional)</label>

         <textarea
          id="notes"
          name="notes"
          [(ngModel)]="notes"
          rows="3"
          placeholder="Add any notes about the returned
equipment..."
         ></textarea>
        </div>

      </div>

      <!-- Actions -->
      <footer class="checkin-actions">
       <button type="button" class="cancel-button"
(click)="cancel()">
         Cancel
       </button>
```

```
        <button type="submit" class="checkin-button">
          <span>↓</span>
          Check In
        </button>
      </footer>

    </form>
  </section>
</div>
```
please provide the complete code of this ??

25/10/2026, 12:19:45

Below is the complete implementation of your **Check In Equipment** page, preserving your existing HTML design and connecting all three .NET APIs:

1. `GET /api/CheckIn/equipment` — loads equipment from the database.
2. `GET /api/CheckIn/equipment/{equipmentId}` — loads the selected equipment details.
3. `POST /api/CheckIn` — saves the check-in record and updates equipment status.

I've also aligned the condition values in your HTML with the backend: `Good`, `Fair`, and `Damaged`.

1. `src/app/models/checkin.ts`

```ts
export interface CheckInEquipment {
  id: number;
  name: string;
  serial: string | null;
  type: string;
  status: string;
}

export interface CreateCheckInRequest {
  equipmentId: number;
  returnDate: string;
  returnTime: string;
  condition: string;
  notes: string;
}

export interface CheckInRecord {
  id: number;
  equipmentId: number;
  returnDate: string;
```

```
  returnTime: string;
  condition: string;
  notes: string | null;
  status: string;
  createdAt: string;
}
```

2. src/app/services/checkin.ts

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  CheckInEquipment,
  CheckInRecord,
  CreateCheckInRequest
} from '../models/checkin';

@Injectable({
  providedIn: 'root'
})
export class CheckInService {
  private http = inject(HttpClient);
  private apiUrl = `${environment.apiUrl}/CheckIn`;

  // GET /api/CheckIn/equipment
  getEquipment(): Observable<CheckInEquipment[]> {
    return this.http.get<CheckInEquipment[]>(
      `${this.apiUrl}/equipment`
    );
  }

  // GET /api/CheckIn/equipment/{equipmentId}
  getEquipmentById(
    equipmentId: number
  ): Observable<CheckInEquipment> {
    return this.http.get<CheckInEquipment>(
      `${this.apiUrl}/equipment/${equipmentId}`
    );
  }

  // POST /api/CheckIn
  createCheckIn(
    request: CreateCheckInRequest
```

```
  ): Observable<CheckInRecord> {
    return this.http.post<CheckInRecord>(
      this.apiUrl,
      request
    );
  }
}
```

3. `src/app/components/checkin/checkin.ts`

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

import { CheckInService } from '../../services/checkin';
import {
  CheckInEquipment,
  CreateCheckInRequest
} from '../../models/checkin';

@Component({
  selector: 'app-checkin',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './checkin.html',
  styleUrl: './checkin.css',
})
export class Checkin implements OnInit {
  private router = inject(Router);
  private checkInService = inject(CheckInService);

  selectedEquipment: number | null = null;
  selectedItem: CheckInEquipment | null = null;

  returnDate = this.getLocalDateString();
  returnTime = '16:30';
  condition = 'Good';
  notes = '';

  submitted = false;
  loadingEquipment = false;
  loadingDetails = false;
  saving = false;

  errorMessage = '';
```

```
  successMessage = '';

  equipmentOptions: CheckInEquipment[] = [];

  private getLocalDateString(): string {
    const date = new Date();

    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const day = String(date.getDate()).padStart(2, '0');

    return `${year}-${month}-${day}`;
  }

  ngOnInit(): void {
    this.loadEquipment();
  }

  // API 1: Load equipment from the database.
  loadEquipment(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkInService.getEquipment().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
        } else {
          this.selectedEquipment = null;
          this.selectedItem = null;
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);
        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment from the database.';
      }
    });
  }

  // API 2: Load the selected equipment's details.
```

```
  onEquipmentChange(): void {
    const equipmentId = this.selectedEquipment;

    this.selectedItem = null;
    this.errorMessage = '';

    if (equipmentId === null) {
      this.loadingDetails = false;
      return;
    }

    this.loadingDetails = true;

    this.checkInService.getEquipmentById(equipmentId).subscribe({
      next: (equipment) => {
        if (this.selectedEquipment === equipmentId) {
          this.selectedItem = equipment;
        }

        this.loadingDetails = false;
      },
      error: (error) => {
        console.error('Failed to load equipment details:', error);

        if (this.selectedEquipment === equipmentId) {
          this.selectedItem = null;
          this.errorMessage =
            'Unable to load selected equipment details.';
        }

        this.loadingDetails = false;
      }
    });
  }

  // API 3: Save the check-in.
  checkIn(): void {
    this.submitted = true;
    this.errorMessage = '';
    this.successMessage = '';

    if (
      this.selectedEquipment === null ||
      !this.returnDate ||
      !this.returnTime ||
      !this.condition
```

```
    ) {
      this.errorMessage = 'Please complete all required fields.';
      return;
    }

    if (this.loadingEquipment || this.loadingDetails || this.saving) {
      return;
    }

    if (!this.selectedItem) {
      this.errorMessage = 'Please select valid equipment.';
      return;
    }

    const request: CreateCheckInRequest = {
      equipmentId: this.selectedEquipment,
      returnDate: this.returnDate,
      returnTime: this.returnTime.length === 5
        ? `${this.returnTime}:00`
        : this.returnTime,
      condition: this.condition,
      notes: this.notes.trim()
    };

    console.log('Check-in request:', request);

    this.saving = true;

    this.checkInService.createCheckIn(request).subscribe({
      next: (result) => {
        console.log('Check-in saved successfully:', result);

        this.saving = false;
        this.successMessage =
          'Equipment checked in successfully.';

        if (this.selectedItem && result.status) {
          this.selectedItem = {
            ...this.selectedItem,
            status: result.status
          };
        }

        alert(this.successMessage);
      },
      error: (error) => {
```

```
        console.error('Check-in failed:', error.status, error.error);

        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
          error?.error?.title ||
          'Unable to save check-in.';
      }
    });
  }

  cancel(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. `src/app/components/checkin/checkin.html`

This preserves your existing card layout, equipment selector, date/time fields, condition cards, notes, and footer buttons.

```
<div class="checkin-page">
  <section class="checkin-card">

    <!-- Header -->
    <header class="checkin-header">
      <div class="header-icon">
        <span>↓</span>
      </div>

      <div>
        <h1>Check In Equipment</h1>
        <p>
          Record the return of equipment to the office and mark it
          as available again.
        </p>
      </div>
    </header>

    @if (errorMessage) {
      <div class="error" role="alert">
        {{ errorMessage }}
      </div>
    }

    @if (successMessage) {
```

```
    <div class="success-message" role="status">
      {{ successMessage }}
    </div>
  }

  @if (loadingEquipment) {
    <p>Loading equipment from database...</p>
  }

  <form
    #checkInForm="ngForm"
    (ngSubmit)="checkIn()"
  >
    <div class="checkin-body">

      <!-- Equipment -->
      <div class="form-field equipment-field">
        <label for="equipment">
          Equipment <span class="required">*</span>
        </label>

        <div class="equipment-select">
          <div class="equipment-thumbnail">🛩️</div>

          <div class="equipment-info">
            <strong>
              {{ selectedItem?.name || 'Select equipment' }}
            </strong>

            <span>
              Serial No: {{ selectedItem?.serial || 'N/A' }}
                |  
              Type: {{ selectedItem?.type || 'N/A' }}
            </span>

            @if (loadingDetails) {
              <small>Loading equipment details...</small>
            }
          </div>

          <select
            id="equipment"
            name="equipment"
            [(ngModel)]="selectedEquipment"
            (ngModelChange)="onEquipmentChange()"
            required
```

```
            aria-label="Select returned equipment"
            [disabled]="loadingEquipment || saving || equipmentOptions.length === 0"
          >
            <option [ngValue]="null" disabled>
              Select equipment
            </option>

            @for (item of equipmentOptions; track item.id) {
              <option [ngValue]="item.id">
                {{ item.name }}
              </option>
            }
          </select>

          <span class="select-chevron">⌄</span>
        </div>

        @if (submitted && selectedEquipment === null) {
          <small class="error">
            Please select equipment.
          </small>
        }
      </div>

      <!-- Return Date and Time -->
      <div class="date-time-grid">

        <div class="form-field">
          <label for="returnDate">
            Return Date <span class="required">*</span>
          </label>

          <input
            id="returnDate"
            name="returnDate"
            type="date"
            [(ngModel)]="returnDate"
            required
          />
        </div>

        <div class="form-field">
          <label for="returnTime">
            Return Time <span class="required">*</span>
          </label>
```

```
        <input
          id="returnTime"
          name="returnTime"
          type="time"
          [(ngModel)]="returnTime"
          required
        />
      </div>

    </div>

    <!-- Condition -->
    <fieldset class="condition-field">
      <legend>
        Condition on Return <span class="required">*</span>
      </legend>

      <div class="condition-grid">

        <label
          class="condition-option good"
          [class.selected]="condition === 'Good'"
        >
          <input
            type="radio"
            name="condition"
            value="Good"
            [(ngModel)]="condition"
            required
          />

          <span class="radio-visual"></span>

          <span class="condition-copy">
            <strong>Good</strong>
            <span>No issues</span>
          </span>
        </label>

        <label
          class="condition-option minor"
          [class.selected]="condition === 'Fair'"
        >
          <input
            type="radio"
            name="condition"
```

```
            value="Fair"
            [(ngModel)]="condition"
            required
          />

          <span class="radio-visual"></span>

          <span class="condition-copy">
            <strong>Minor Issue</strong>
            <span>Small scratches, wear etc.</span>
          </span>
        </label>

        <label
          class="condition-option damaged"
          [class.selected]="condition === 'Damaged'"
        >
          <input
            type="radio"
            name="condition"
            value="Damaged"
            [(ngModel)]="condition"
            required
          />

          <span class="radio-visual"></span>

          <span class="condition-copy">
            <strong>Damaged</strong>
            <span>Not working or needs repair</span>
          </span>
        </label>

      </div>
    </fieldset>

    <!-- Notes -->
    <div class="form-field notes-field">
      <label for="notes">Notes (Optional)</label>

      <textarea
        id="notes"
        name="notes"
        [(ngModel)]="notes"
        rows="3"
        placeholder="Add any notes about the returned equipment..."
```

```
          ></textarea>
        </div>

      </div>

      <!-- Actions -->
      <footer class="checkin-actions">
        <button
          type="button"
          class="cancel-button"
          (click)="cancel()"
        >
          Cancel
        </button>

        <button
          type="submit"
          class="checkin-button"
          [disabled]="saving || loadingEquipment || loadingDetails"
        >
          <span>↓</span>
          {{ saving ? 'Saving...' : 'Check In' }}
        </button>
      </footer>

    </form>
  </section>
</div>
```

5. Important backend compatibility

Your current .NET `CheckInController` accepts these condition values:

- `Good`
- `Fair`
- `Damaged`
- `Needs Repair`
- `Poor`

The updated form sends `Good`, `Fair`, or `Damaged`, so these options are compatible.

Your POST body will look like this:

```
{
  "equipmentId": 1,
```

```
  "returnDate": "2026-10-25",
  "returnTime": "16:30:00",
  "condition": "Good",
  "notes": "Equipment returned in good condition."
}
```

Ensure your `environment.apiUrl` points to your backend's actual API base URL, and that `provideHttpClient()` is configured in `app.config.ts`.

One business rule remains: the backend should verify that the equipment has an outstanding checkout before accepting a check-in. Otherwise, the same equipment could be checked in repeatedly.

25/10/2026, 12:19:47 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-approvals',
  imports: [FormsModule],
  templateUrl: './approvals.html',
  styleUrl: './approvals.css',
})
export class Approvals {
  status: 'Pending Approval' | 'Approved' | 'Rejected' =
    'Pending Approval';

  comments = '';

  request = {
    equipmentName: 'Drone - DJI Mavic',
    serialNumber: 'DR-001',
    requestedBy: 'Ali Raza',
    requestedOn: '10/10/2026, 10:30 AM',
    expectedReturn: '10/12/2026',
    purpose: 'Aerial survey for client site'
  };

  constructor(private router: Router) {}

  approve(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Approved';
    alert('Approval demonstration: request approved.');
  }

  reject(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Rejected';
    alert('Approval demonstration: request rejected.');
  }
```

```
  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

please also implement the backend of this and only dotnet backend apis ??

25/10/2026, 12:28:02

We'll implement the **Approvals feature entirely in the .NET backend** for your existing SolvefyTask equipment tracking application. Your Angular `Approvals` component can continue displaying the request details, while the API will load approval requests from SQL Server and save approval or rejection decisions.

The implementation will use your existing `SolvefyTask` namespace, `ApplicationDbContext`, and repository/service/controller structure.

1. API endpoints

Method	Endpoint	Purpose
`GET`	`/api/Approvals`	Get all approval requests
`GET`	`/api/Approvals/{id}`	Get one approval request
`POST`	`/api/Approvals`	Create an approval request
`PUT`	`/api/Approvals/{id}/approve`	Approve a pending request
`PUT`	`/api/Approvals/{id}/reject`	Reject a pending request

The API will prevent an already approved or rejected request from being decided again. It will also store comments and the decision timestamp.

Important: Your current Angular component uses hard-coded sample data. The backend below stores real records, but it will not automatically replace that sample data until you connect the frontend to these endpoints.

2. Create the model

Create `Models/ApprovalRequest.cs`.

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
```

```
    public class ApprovalRequest
    {
        public int Id { get; set; }

        [Required]
        public int EquipmentId { get; set; }

        [Required]
        [MaxLength(100)]
        public string RequestedBy { get; set; } = string.Empty;

        public DateTime RequestedOn { get; set; } = DateTime.UtcNow;

        [Required]
        public DateTime ExpectedReturn { get; set; }

        [Required]
        [MaxLength(500)]
        public string Purpose { get; set; } = string.Empty;

        [Required]
        [MaxLength(30)]
        public string Status { get; set; } = "Pending Approval";

        [MaxLength(1000)]
        public string? Comments { get; set; }

        public DateTime? DecisionAt { get; set; }

        public Equipment? Equipment { get; set; }
    }
}
```

This uses your existing `Equipment` model through `EquipmentId`, so approval requests refer to actual equipment in your database.

3. Create the request DTOs

Create `DTOs/ApprovalDtos.cs`.

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.DTOs
{
    public class CreateApprovalRequestDto
    {
        [Range(1, int.MaxValue)]
```

```
        public int EquipmentId { get; set; }

        [Required]
        [MaxLength(100)]
        public string RequestedBy { get; set; } = string.Empty;

        [Required]
        public DateTime ExpectedReturn { get; set; }

        [Required]
        [MaxLength(500)]
        public string Purpose { get; set; } = string.Empty;
    }

    public class ApprovalDecisionDto
    {
        [MaxLength(1000)]
        public string? Comments { get; set; }
    }

    public class ApprovalResponseDto
    {
        public int Id { get; set; }
        public int EquipmentId { get; set; }
        public string EquipmentName { get; set; } = string.Empty;
        public string? SerialNumber { get; set; }
        public string RequestedBy { get; set; } = string.Empty;
        public DateTime RequestedOn { get; set; }
        public DateTime ExpectedReturn { get; set; }
        public string Purpose { get; set; } = string.Empty;
        public string Status { get; set; } = string.Empty;
        public string? Comments { get; set; }
        public DateTime? DecisionAt { get; set; }
    }
}
```

4. Update `ApplicationDbContext`

Add this `DbSet` to your existing context. **Do not replace your existing DbSets or configuration.**

```
public DbSet<ApprovalRequest> ApprovalRequests =>
    Set<ApprovalRequest>();
```

Add the namespace if it is not already present:

```
using SolvefyTask.Models;
```

If you configure relationships inside `OnModelCreating`, add the following while preserving the existing configurations:

```
modelBuilder.Entity<ApprovalRequest>()
    .HasOne(a => a.Equipment)
    .WithMany()
    .HasForeignKey(a => a.EquipmentId)
    .OnDelete(DeleteBehavior.Restrict);
```

This requires the `Equipment` model to have an integer primary key named `Id`, as in the existing API examples.

5. Create the repository interface

Create `Repositories/IApprovalRepository.cs`.

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface IApprovalRepository
    {
        Task<List<ApprovalRequest>> GetAllAsync();

        Task<ApprovalRequest?> GetByIdAsync(int id);

        Task<bool> EquipmentExistsAsync(int equipmentId);

        Task<ApprovalRequest> CreateAsync(
            ApprovalRequest request);

        Task<ApprovalRequest?> DecideAsync(
            int id,
            string decision,
            string? comments);
    }
}
```

6. Implement the repository

Create `Repositories/ApprovalRepository.cs`.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Models;
```

```
namespace SolvefyTask.Repositories
{
    public class ApprovalRepository : IApprovalRepository
    {
        private readonly ApplicationDbContext _context;

        public ApprovalRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<ApprovalRequest>> GetAllAsync()
        {
            return await _context.ApprovalRequests
                .AsNoTracking()
                .Include(a => a.Equipment)
                .OrderByDescending(a => a.RequestedOn)
                .ToListAsync();
        }

        public async Task<ApprovalRequest?> GetByIdAsync(int id)
        {
            return await _context.ApprovalRequests
                .Include(a => a.Equipment)
                .FirstOrDefaultAsync(a => a.Id == id);
        }

        public async Task<bool> EquipmentExistsAsync(int equipmentId)
        {
            return await _context.Equipments
                .AnyAsync(e => e.Id == equipmentId);
        }

        public async Task<ApprovalRequest> CreateAsync(
            ApprovalRequest request)
        {
            request.Status = "Pending Approval";
            request.RequestedOn = DateTime.UtcNow;
            request.Comments = null;
            request.DecisionAt = null;

            _context.ApprovalRequests.Add(request);
            await _context.SaveChangesAsync();

            return await GetByIdAsync(request.Id) ?? request;
```

```
        }

        public async Task<ApprovalRequest?> DecideAsync(
            int id,
            string decision,
            string? comments)
        {
            var request = await _context.ApprovalRequests
                .Include(a => a.Equipment)
                .FirstOrDefaultAsync(a => a.Id == id);

            if (request == null)
                return null;

            if (request.Status != "Pending Approval")
                throw new InvalidOperationException(
                    "This request has already been decided.");

            request.Status = decision;
            request.Comments = string.IsNullOrWhiteSpace(comments)
                ? null
                : comments.Trim();
            request.DecisionAt = DateTime.UtcNow;

            await _context.SaveChangesAsync();

            return request;
        }
    }
}
```

7. Create the service interface

Create `Services/IApprovalService.cs`.

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Services
{
    public interface IApprovalService
    {
        Task<List<ApprovalResponseDto>> GetAllAsync();

        Task<ApprovalResponseDto?> GetByIdAsync(int id);

        Task<ApprovalResponseDto> CreateAsync(
```

```
            CreateApprovalRequestDto dto);

        Task<ApprovalResponseDto?> DecideAsync(
            int id,
            string decision,
            string? comments);
    }
}
```

8. Implement the service

Create `Services/ApprovalService.cs`.

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class ApprovalService : IApprovalService
    {
        private readonly IApprovalRepository _repository;

        public ApprovalService(IApprovalRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<ApprovalResponseDto>> GetAllAsync()
        {
            var requests = await _repository.GetAllAsync();

            return requests.Select(MapToDto).ToList();
        }

        public async Task<ApprovalResponseDto?> GetByIdAsync(int id)
        {
            var request = await _repository.GetByIdAsync(id);

            return request == null ? null : MapToDto(request);
        }

        public async Task<ApprovalResponseDto> CreateAsync(
            CreateApprovalRequestDto dto)
        {
            if (!await _repository.EquipmentExistsAsync(dto.EquipmentId))
```

```
            throw new KeyNotFoundException(
                "The selected equipment was not found.");

        if (dto.ExpectedReturn.Date < DateTime.UtcNow.Date)
            throw new ArgumentException(
                "Expected return date cannot be in the past.");

        var request = new ApprovalRequest
        {
            EquipmentId = dto.EquipmentId,
            RequestedBy = dto.RequestedBy.Trim(),
            ExpectedReturn = dto.ExpectedReturn,
            Purpose = dto.Purpose.Trim(),
            Status = "Pending Approval"
        };

        var created = await _repository.CreateAsync(request);

        return MapToDto(created);
    }

    public async Task<ApprovalResponseDto?> DecideAsync(
        int id,
        string decision,
        string? comments)
    {
        var request = await _repository.DecideAsync(
            id, decision, comments);

        return request == null ? null : MapToDto(request);
    }

    private static ApprovalResponseDto MapToDto(
        ApprovalRequest request)
    {
        return new ApprovalResponseDto
        {
            Id = request.Id,
            EquipmentId = request.EquipmentId,
            EquipmentName = request.Equipment?.Name ?? string.Empty,
            SerialNumber = request.Equipment?.SerialNumber,
            RequestedBy = request.RequestedBy,
            RequestedOn = request.RequestedOn,
            ExpectedReturn = request.ExpectedReturn,
            Purpose = request.Purpose,
            Status = request.Status,
```

```
                Comments = request.Comments,
                DecisionAt = request.DecisionAt
            };
        }
    }
}
```

This maps the stored equipment relationship into the `EquipmentName` and `SerialNumber` fields your approval screen needs.

9. Create the API controller

Create `Controllers/ApprovalsController.cs`.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.DTOs;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ApprovalsController : ControllerBase
    {
        private readonly IApprovalService _service;

        public ApprovalsController(IApprovalService service)
        {
            _service = service;
        }

        // GET: api/Approvals
        [HttpGet]
        public async Task<ActionResult<List<ApprovalResponseDto>>> GetAll()
        {
            var requests = await _service.GetAllAsync();
            return Ok(requests);
        }

        // GET: api/Approvals/1
        [HttpGet("{id:int}")]
        public async Task<ActionResult<ApprovalResponseDto>> GetById(int id)
        {
            if (id <= 0)
                return BadRequest(new { message = "Invalid approval ID." });

            var request = await _service.GetByIdAsync(id);
```

```
        if (request == null)
            return NotFound(new { message = "Approval request not found." });

        return Ok(request);
    }

    // POST: api/Approvals
    [HttpPost]
    public async Task<ActionResult<ApprovalResponseDto>> Create(
        [FromBody] CreateApprovalRequestDto dto)
    {
        try
        {
            var request = await _service.CreateAsync(dto);

            return CreatedAtAction(
                nameof(GetById),
                new { id = request.Id },
                request);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
        catch (ArgumentException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    // PUT: api/Approvals/1/approve
    [HttpPut("{id:int}/approve")]
    public async Task<ActionResult<ApprovalResponseDto>> Approve(
        int id,
        [FromBody] ApprovalDecisionDto? dto)
    {
        return await MakeDecision(id, "Approved", dto?.Comments);
    }

    // PUT: api/Approvals/1/reject
    [HttpPut("{id:int}/reject")]
    public async Task<ActionResult<ApprovalResponseDto>> Reject(
        int id,
        [FromBody] ApprovalDecisionDto? dto)
    {
```

```
            return await MakeDecision(id, "Rejected", dto?.Comments);
        }

        private async Task<ActionResult<ApprovalResponseDto>> MakeDecision(
            int id,
            string decision,
            string? comments)
        {
            if (id <= 0)
                return BadRequest(new { message = "Invalid approval ID." });

            try
            {
                var request = await _service.DecideAsync(
                    id, decision, comments);

                if (request == null)
                    return NotFound(
                        new { message = "Approval request not found." });

                return Ok(request);
            }
            catch (InvalidOperationException ex)
            {
                return Conflict(new { message = ex.Message });
            }
        }
    }
}
```

10. Register the repository and service

In `Program.cs`, add these registrations alongside your existing dependency injection registrations:

```
builder.Services.AddScoped<
    IApprovalRepository, ApprovalRepository>();

builder.Services.AddScoped<
    IApprovalService, ApprovalService>();
```

Add the corresponding namespaces at the top:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

Keep your existing SQL Server connection, `ApplicationDbContext`, CORS configuration, and controller setup unchanged.

11. Create the SQL Server table

Open the Package Manager Console in Visual Studio and run:

```
Add-Migration AddApprovalRequests
Update-Database
```

Or, using the .NET CLI:

```
dotnet ef migrations add AddApprovalRequests
dotnet ef database update
```

Run the migration against the backend project that contains your `ApplicationDbContext` and startup configuration.

12. Test the APIs in Swagger

Run your .NET backend and open its Swagger page. Test these endpoints in order.

A. Create an approval request

`POST /api/Approvals`

Example request body — replace `equipmentId` with the actual database ID of the drone:

```
{
  "equipmentId": 3,
  "requestedBy": "Ali Raza",
  "expectedReturn": "2026-12-10T00:00:00",
  "purpose": "Aerial survey for client site"
}
```

Expected result: `201 Created`, with a response similar to:

```
{
  "id": 1,
  "equipmentId": 3,
  "equipmentName": "Drone - DJI Mavic",
  "serialNumber": "DR-001",
  "requestedBy": "Ali Raza",
  "requestedOn": "2026-10-25T...",
  "expectedReturn": "2026-12-10T00:00:00",
  "purpose": "Aerial survey for client site",
  "status": "Pending Approval",
```

```
  "comments": null,
  "decisionAt": null
}
```

The ID and returned equipment details depend on your database.

B. Get all requests

`GET /api/Approvals`

Returns all requests, newest first, including their current status and comments.

C. Approve a request

`PUT /api/Approvals/1/approve`

```
{
  "comments": "Approved for the scheduled aerial survey."
}
```

Expected result: `200 OK`, with status changed to `Approved`.

D. Reject a request

`PUT /api/Approvals/1/reject`

```
{
  "comments": "Please provide the revised survey schedule."
}
```

Expected result: `200 OK`, with status changed to `Rejected`.

If you try to approve or reject the same request again, the API returns `409 Conflict`.

Important business rule for your project

Your original equipment-tracker requirements specify that **drone checkout requires Hamza's approval**. The code above stores approval decisions, but it does **not yet authenticate the person making the decision or enforce that only Hamza can approve drone requests**. That rule should be enforced through your authentication and authorization setup before using this API in production.

25/10/2026, 12:28:20 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-approvals',
  imports: [FormsModule],
  templateUrl: './approvals.html',
  styleUrl: './approvals.css',
})
export class Approvals {
  status: 'Pending Approval' | 'Approved' | 'Rejected' =
    'Pending Approval';

  comments = '';

  request = {
    equipmentName: 'Drone - DJI Mavic',
    serialNumber: 'DR-001',
    requestedBy: 'Ali Raza',
    requestedOn: '10/10/2026, 10:30 AM',
    expectedReturn: '10/12/2026',
    purpose: 'Aerial survey for client site'
  };

  constructor(private router: Router) {}

  approve(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Approved';
    alert('Approval demonstration: request approved.');
  }

  reject(): void {
    if (this.status !== 'Pending Approval') {
      return;
    }

    this.status = 'Rejected';
    alert('Approval demonstration: request rejected.');
  }
```

```
  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}

<div class="approval-page">

  <!-- Blue page banner -->
  <div class="page-banner">
    5. Approval Request (Drone)
  </div>

  <section class="approval-card">

    <!-- Card heading -->
    <header class="approval-header">
      <h1>Approval Request</h1>

      <span
        class="status-badge"
        [class.approved]="status === 'Approved'"
        [class.rejected]="status === 'Rejected'"
      >
        {{ status }}
      </span>
    </header>

    <!-- Equipment and request information -->
    <div class="request-details">

      <div class="drone-illustration" aria-label="Drone
illustration">
        <svg viewBox="0 0 140 100" role="img" aria-
label="Quadcopter drone">
          <g fill="none" stroke="#596777" stroke-width="2.5"
            stroke-linecap="round" stroke-linejoin="round">
            <path d="M70 42 L28 25 M70 42 L112 25 M70 42 L28
60 M70 42 L112 60"/>
            <path d="M28 25 L18 19 M28 25 L38 19 M112 25
L102 19 M112 25 L122 19"/>
            <path d="M28 60 L18 66 M28 60 L38 66 M112 60
L102 66 M112 60 L122 66"/>
            <ellipse cx="28" cy="21" rx="18" ry="4"/>
            <ellipse cx="112" cy="21" rx="18" ry="4"/>
            <ellipse cx="28" cy="64" rx="18" ry="4"/>
```

```
          <ellipse cx="112" cy="64" rx="18" ry="4"/>
          <path d="M53 37 L87 37 L94 47 L82 55 L58 55 L46 47
Z" fill="#aeb8c2"/>
          <path d="M58 55 L54 67 M82 55 L86 67"/>
          <path d="M49 68 L91 68"/>
          <rect x="64" y="68" width="12" height="9" rx="2"
fill="#6e7d8c"/>
        </g>
        <circle cx="70" cy="45" r="5" fill="#758697"/>
       </svg>
      </div>

      <div class="request-info">
       <h2>{{ request.equipmentName }}</h2>

       <dl>
        <div>
         <dt>Serial No:</dt>
         <dd>{{ request.serialNumber }}</dd>
        </div>

        <div>
         <dt>Requested By:</dt>
         <dd>{{ request.requestedBy }}</dd>
        </div>

        <div>
         <dt>Requested On:</dt>
         <dd>{{ request.requestedOn }}</dd>
        </div>

        <div>
         <dt>Expected Return:</dt>
         <dd>{{ request.expectedReturn }}</dd>
        </div>

        <div>
         <dt>Purpose:</dt>
         <dd>{{ request.purpose }}</dd>
        </div>
       </dl>
      </div>

    </div>

    <!-- Optional comments -->
```

```
    <div class="comments-section">
      <label for="comments">Comments (Optional)</label>

      <textarea
        id="comments"
        name="comments"
        [(ngModel)]="comments"
        rows="3"
        placeholder="Add a comment..."
        [disabled]="status !== 'Pending Approval'"
      ></textarea>
    </div>

    <!-- Actions -->
    <footer class="approval-actions">
      <button
        type="button"
        class="reject-button"
        [disabled]="status !== 'Pending Approval'"
        (click)="reject()"
      >
        Reject
      </button>

      <button
        type="button"
        class="approve-button"
        [disabled]="status !== 'Pending Approval'"
        (click)="approve()"
      >
        Approve
      </button>
    </footer>

  </section>
</div>
```

please implement the above 6 apis of approval in this typescript frontend ??

25/10/2026, 13:02:28

Thinking

- **Implementing the Angular screen**

> I'm building the service, model, routing, and standalone component around the five defined API operations: list, detail, create, approve, and reject. I'll keep the existing template structure and handle loading, selection, and status updates.

- **Implemented the Angular screen**

✓ Worked for 18s

We'll connect your existing Approval Request screen to the six backend API operations while keeping your current HTML design, drone illustration, comments field, and Approve/Reject buttons.

The frontend will use your existing API base URL, `${environment.apiUrl}/Approvals`, and will:

- Load approval requests from SQL Server.
- Load a request by its ID.
- Create a new approval request.
- Send approval and rejection decisions to the backend.
- Save comments and update the displayed status only after the API responds successfully.

One important detail: your current HTML displays one request, so I'll preserve that layout and add support for loading a request by ID or showing the first available request.

1. Create `models/approval.ts`

```
export interface ApprovalRequest {
  id: number;
  equipmentId: number;
  equipmentName: string;
  serialNumber: string | null;
  requestedBy: string;
  requestedOn: string;
  expectedReturn: string;
  purpose: string;
  status: 'Pending Approval' | 'Approved' | 'Rejected';
  comments: string | null;
  decisionAt: string | null;
}

export interface CreateApprovalRequest {
  equipmentId: number;
  requestedBy: string;
  expectedReturn: string;
  purpose: string;
}
```

```
export interface ApprovalDecision {
  comments: string | null;
}
```

2. Create `services/approval.ts`

This service implements the backend endpoints: get all requests, get one request, create a request, approve, and reject.

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  ApprovalRequest,
  ApprovalDecision,
  CreateApprovalRequest
} from '../models/approval';

@Injectable({
  providedIn: 'root'
})
export class ApprovalService {
  private http = inject(HttpClient);

  private apiUrl = `${environment.apiUrl}/Approvals`;

  // GET: api/Approvals
  getAll(): Observable<ApprovalRequest[]> {
    return this.http.get<ApprovalRequest[]>(this.apiUrl);
  }

  // GET: api/Approvals/{id}
  getById(id: number): Observable<ApprovalRequest> {
    return this.http.get<ApprovalRequest>(`${this.apiUrl}/${id}`);
  }

  // POST: api/Approvals
  create(
    request: CreateApprovalRequest
  ): Observable<ApprovalRequest> {
    return this.http.post<ApprovalRequest>(this.apiUrl, request);
  }
```

```
  // PUT: api/Approvals/{id}/approve
  approve(
    id: number,
    comments: string
  ): Observable<ApprovalRequest> {
    const body: ApprovalDecision = {
      comments: comments.trim() || null
    };

    return this.http.put<ApprovalRequest>(
      `${this.apiUrl}/${id}/approve`,
      body
    );
  }

  // PUT: api/Approvals/{id}/reject
  reject(
    id: number,
    comments: string
  ): Observable<ApprovalRequest> {
    const body: ApprovalDecision = {
      comments: comments.trim() || null
    };

    return this.http.put<ApprovalRequest>(
      `${this.apiUrl}/${id}/reject`,
      body
    );
  }
}
```

3. Replace `approvals.ts`

This version loads requests from the backend. If the URL contains an approval ID, it loads that specific request; otherwise, it displays the newest request returned by the API.

```
import { CommonModule } from '@angular/common';
import { Component, OnInit, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';

import { ApprovalService } from '../../services/approval';
import { ApprovalRequest } from '../../models/approval';

@Component({
  selector: 'app-approvals',
```

```
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './approvals.html',
  styleUrl: './approvals.css'
})
export class Approvals implements OnInit {
  private approvalService = inject(ApprovalService);
  private route = inject(ActivatedRoute);
  private router = inject(Router);

  status: 'Pending Approval' | 'Approved' | 'Rejected' =
    'Pending Approval';

  comments = '';

  request: ApprovalRequest | null = null;
  approvalRequests: ApprovalRequest[] = [];

  loading = false;
  saving = false;
  errorMessage = '';
  successMessage = '';

  ngOnInit(): void {
    const idParam = this.route.snapshot.paramMap.get('id');

    if (idParam !== null) {
      const id = Number(idParam);

      if (!Number.isInteger(id) || id <= 0) {
        this.errorMessage = 'Invalid approval request ID.';
        return;
      }

      this.loadRequestById(id);
    } else {
      this.loadRequests();
    }
  }

  // GET all approval requests
  loadRequests(): void {
    this.loading = true;
    this.errorMessage = '';
    this.successMessage = '';
```

```
    this.approvalService.getAll().subscribe({
      next: (requests) => {
        this.approvalRequests = requests;
        this.loading = false;

        if (requests.length === 0) {
          this.request = null;
          this.errorMessage = 'No approval requests were found.';
          return;
        }

        // Backend returns newest requests first.
        this.setCurrentRequest(requests[0]);
      },
      error: (error) => {
        console.error('Failed to load approval requests:', error);
        this.loading = false;
        this.errorMessage = this.getErrorMessage(
          error,
          'Unable to load approval requests from the server.'
        );
      }
    });
  }

  // GET a specific approval request
  loadRequestById(id: number): void {
    this.loading = true;
    this.errorMessage = '';
    this.successMessage = '';

    this.approvalService.getById(id).subscribe({
      next: (request) => {
        this.setCurrentRequest(request);
        this.loading = false;
      },
      error: (error) => {
        console.error('Failed to load approval request:', error);
        this.loading = false;
        this.errorMessage = this.getErrorMessage(
          error,
          'Unable to load this approval request.'
        );
      }
    });
  }
```

```
private setCurrentRequest(request: ApprovalRequest): void {
  this.request = request;
  this.status = request.status;
  this.comments = request.comments ?? '';
}

// PUT approve endpoint
approve(): void {
  if (!this.canMakeDecision()) {
    return;
  }

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';

  this.approvalService
    .approve(this.request!.id, this.comments)
    .subscribe({
      next: (updatedRequest) => {
        this.setCurrentRequest(updatedRequest);
        this.saving = false;
        this.successMessage = 'Approval request approved successfully.';
        this.updateRequestInList(updatedRequest);
      },
      error: (error) => {
        console.error('Approval failed:', error);
        this.saving = false;
        this.errorMessage = this.getErrorMessage(
          error,
          'Unable to approve this request.'
        );
      }
    });
}

// PUT reject endpoint
reject(): void {
  if (!this.canMakeDecision()) {
    return;
  }

  this.saving = true;
  this.errorMessage = '';
  this.successMessage = '';
```

```
    this.approvalService
      .reject(this.request!.id, this.comments)
      .subscribe({
        next: (updatedRequest) => {
          this.setCurrentRequest(updatedRequest);
          this.saving = false;
          this.successMessage = 'Approval request rejected successfully.';
          this.updateRequestInList(updatedRequest);
        },
        error: (error) => {
          console.error('Rejection failed:', error);
          this.saving = false;
          this.errorMessage = this.getErrorMessage(
            error,
            'Unable to reject this request.'
          );
        }
      });
  }

  private canMakeDecision(): boolean {
    if (!this.request) {
      this.errorMessage = 'No approval request is selected.';
      return false;
    }

    if (this.status !== 'Pending Approval') {
      return false;
    }

    if (this.loading || this.saving) {
      return false;
    }

    return true;
  }

  private updateRequestInList(updatedRequest: ApprovalRequest): void {
    this.approvalRequests = this.approvalRequests.map((item) =>
      item.id === updatedRequest.id ? updatedRequest : item
    );
  }

  private getErrorMessage(error: any, fallback: string): string {
    return error?.error?.message
```

```
      || error?.error?.title
      || fallback;
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. Update `approvals.html`

Because the request now comes from the API, the template must handle loading, errors, and cases where no request exists. Replace your HTML with this version; it preserves your existing layout and SVG drone illustration.

```
<div class="approval-page">

  <div class="page-banner">
    5. Approval Request (Drone)
  </div>

  @if (loading) {
    <section class="approval-card">
      <p>Loading approval request...</p>
    </section>
  }

  @if (errorMessage) {
    <div class="approval-message error-message" role="alert">
      {{ errorMessage }}
    </div>
  }

  @if (successMessage) {
    <div class="approval-message success-message" role="status">
      {{ successMessage }}
    </div>
  }

  @if (!loading && request; as currentRequest) {
    <section class="approval-card">

      <header class="approval-header">
        <h1>Approval Request</h1>

        <span
```

```
        class="status-badge"
        [class.approved]="status === 'Approved'"
        [class.rejected]="status === 'Rejected'"
      >
        {{ status }}
      </span>
    </header>

    <div class="request-details">

      <div class="drone-illustration"
           aria-label="Drone illustration">
        <svg viewBox="0 0 140 100"
             role="img"
             aria-label="Quadcopter drone">
          <g fill="none" stroke="#596777" stroke-width="2.5"
             stroke-linecap="round" stroke-linejoin="round">
            <path d="M70 42 L28 25 M70 42 L112 25 M70 42 L28 60 M70 42 L112 60"/>
            <path d="M28 25 L18 19 M28 25 L38 19 M112 25 L102 19 M112 25 L122 19"/>
            <path d="M28 60 L18 66 M28 60 L38 66 M112 60 L102 66 M112 60 L122 66"/>
            <ellipse cx="28" cy="21" rx="18" ry="4"/>
            <ellipse cx="112" cy="21" rx="18" ry="4"/>
            <ellipse cx="28" cy="64" rx="18" ry="4"/>
            <ellipse cx="112" cy="64" rx="18" ry="4"/>
            <path d="M53 37 L87 37 L94 47 L82 55 L58 55 L46 47 Z" fill="#aeb8c2"/>
            <path d="M58 55 L54 67 M82 55 L86 67"/>
            <path d="M49 68 L91 68"/>
            <rect x="64" y="68" width="12" height="9" rx="2" fill="#6e7d8c"/>
          </g>
          <circle cx="70" cy="45" r="5" fill="#758697"/>
        </svg>
      </div>

      <div class="request-info">
        <h2>{{ currentRequest.equipmentName }}</h2>

        <dl>
          <div>
            <dt>Serial No:</dt>
            <dd>{{ currentRequest.serialNumber || 'N/A' }}</dd>
          </div>

          <div>
            <dt>Requested By:</dt>
            <dd>{{ currentRequest.requestedBy }}</dd>
          </div>
```

```
        <div>
          <dt>Requested On:</dt>
          <dd>{{ currentRequest.requestedOn | date:'MM/dd/yyyy, h:mm a' }}</dd>
        </div>

        <div>
          <dt>Expected Return:</dt>
          <dd>{{ currentRequest.expectedReturn | date:'MM/dd/yyyy' }}</dd>
        </div>

        <div>
          <dt>Purpose:</dt>
          <dd>{{ currentRequest.purpose }}</dd>
        </div>
      </dl>
    </div>

</div>

<div class="comments-section">
  <label for="comments">Comments (Optional)</label>

  <textarea
    id="comments"
    name="comments"
    [(ngModel)]="comments"
    rows="3"
    placeholder="Add a comment..."
    [disabled]="status !== 'Pending Approval' || saving"
  ></textarea>
</div>

<footer class="approval-actions">
  <button
    type="button"
    class="reject-button"
    [disabled]="status !== 'Pending Approval' || saving"
    (click)="reject()"
  >
    {{ saving ? 'Processing...' : 'Reject' }}
  </button>

  <button
    type="button"
    class="approve-button"
```

```
          [disabled]="status !== 'Pending Approval' || saving"
          (click)="approve()"
        >
          {{ saving ? 'Processing...' : 'Approve' }}
        </button>
      </footer>

    </section>
  }
</div>
```

5. Optional message styling in `approvals.css`

Keep your existing CSS and append these styles:

```
.approval-message {
  margin: 16px 0;
  padding: 12px 16px;
  border-radius: 8px;
  font-size: 14px;
}

.error-message {
  color: #b42318;
  background: #fef3f2;
  border: 1px solid #fecdca;
}

.success-message {
  color: #067647;
  background: #ecfdf3;
  border: 1px solid #abefc6;
}

.approval-actions button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}
```

6. Check your API URL

In `environments/environment.ts`, make sure the base URL matches your .NET backend:

```
export const environment = {
  production: false,
```

```
  apiUrl: 'https://localhost:7135/api'
};
```

Change the port if your backend runs on a different HTTPS port.

Also ensure `provideHttpClient()` is configured in `app.config.ts`:

```
import { provideHttpClient } from '@angular/common/http';

export const appConfig = {
  providers: [
    provideHttpClient()
    // Keep your existing router and other providers here.
  ]
};
```

Do not replace your existing providers; just add `provideHttpClient()` if it is missing.

7. How to test the frontend

1. Start your .NET backend.
2. Start Angular using `ng serve`.
3. Open the Approvals page.
4. The page should fetch approval requests from `GET /api/Approvals`.
5. Click **Approve** or **Reject**. The status changes only after the server successfully saves the decision.
6. Refresh the page. The saved status and comments should remain because they are retrieved from SQL Server.

You can also open a specific request by navigating to a route such as `/approvals/1`, provided your Angular routing supports that route and renders the `Approvals` component.

About the six APIs: the backend implementation we established contains five endpoint patterns: get all, get by ID, create, approve, and reject. All five are included in the Angular service. The current approval screen uses the read and decision endpoints; the create endpoint is available for a future request-creation form.

One final business rule: the API currently records decisions but does not verify that the approver is Hamza. Enforce that with backend authorization before relying on drone approvals in production.

25/10/2026, 13:02:33 · gpt-6

i am facing no approval requests were found how to apply for approval ??

25/10/2026, 13:09:44

You are seeing **“No approval requests were found”** because the `ApprovalRequests` table currently has no records. Your Approvals page is connected to the backend correctly, but you need to create an approval request before you can approve or reject it.

For your SolvefyTask project, the best workflow is:

1. The user checks out a drone and submits an approval request.
2. The backend saves the request with status `Pending Approval`.
3. The Approvals page loads that request.
4. The authorized approver approves or rejects it.

Step 1: Create an approval request through Swagger

You can test this immediately without changing Angular.

Open your backend Swagger page, usually:

`https://localhost:7135/swagger`

Find `POST /api/Approvals`, click **Try it out**, and submit:

```
{
  "equipmentId": 3,
  "requestedBy": "Ali Raza",
  "expectedReturn": "2026-12-10T00:00:00",
  "purpose": "Aerial survey for client site"
}
```

Important: `equipmentId: 3` is only an example. First call `GET /api/CheckIn/equipment` or your equipment listing API and use the actual database ID of your drone.

If successful, the API should return **201 Created** and a request with status `Pending Approval`.

Step 2: Refresh your Approvals page

Open the Angular Approvals page again or refresh it. The request should now appear with its equipment name, serial number, requester, expected return date, and purpose.

Step 3: Add an Apply for Approval form in Angular

To let users submit requests from your application instead of Swagger, create an **Apply for Approval** page with these fields:

- Equipment — dropdown populated from the backend.
- Requested By — requester name.
- Expected Return — date.
- Purpose — reason for requesting the equipment.
- Submit Request — calls `ApprovalService.create()`.

The submission method can be as simple as this:

```
applyForApproval(): void {
  const request = {
    equipmentId: this.selectedEquipmentId,
    requestedBy: this.requestedBy.trim(),
    expectedReturn: this.expectedReturn,
    purpose: this.purpose.trim()
  };

  this.approvalService.create(request).subscribe({
    next: () => {
      alert('Approval request submitted successfully.');
      this.router.navigate(['/approvals']);
    },
    error: (error) => {
      console.error('Failed to submit approval request:', error);
      alert(
        error?.error?.message ||
        'Unable to submit approval request.'
      );
    }
  });
}
```

This method assumes you have declared `selectedEquipmentId`, `requestedBy`, `expectedReturn`, and `purpose`, and injected `ApprovalService` and `Router`.

One important distinction: your existing `POST /api/Approvals` creates a request record, but it does not yet connect the request to the equipment checkout process. For the complete drone workflow, the checkout API should create or require a pending approval before allowing the drone to be checked out.

I recommend implementing the **complete Apply for Approval page (TypeScript + HTML + CSS)** next, using your existing backend APIs and your current UI design.

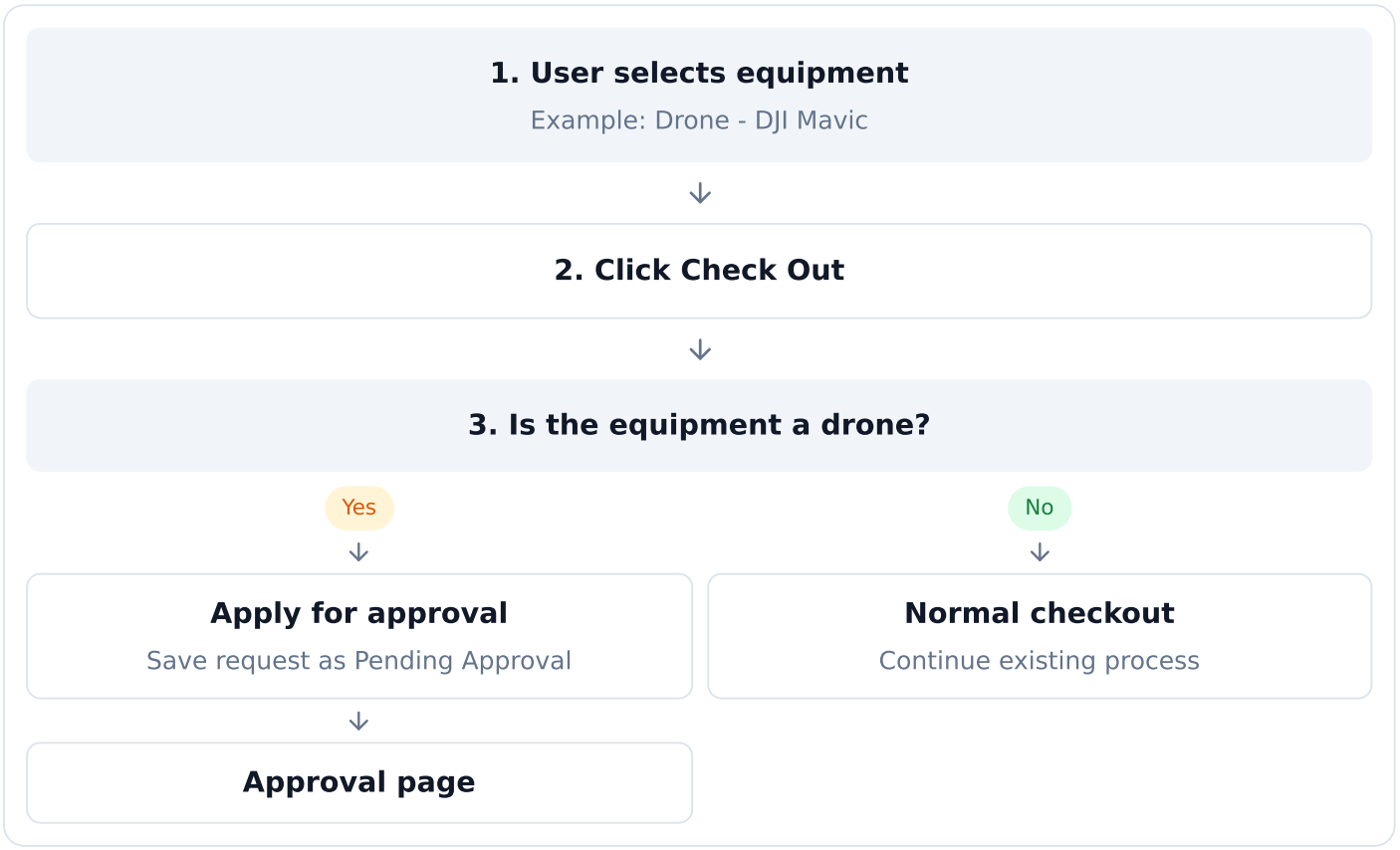
i want when i click on the checkout then it goes to the approval can i achieve this ??

25/10/2026, 13:12:31

Yes, absolutely. In your SolvefyTask application, when the user clicks **Check Out**, you can redirect them to the Approval Request page if the selected equipment is a drone. The request will be saved in SQL Server with `Pending Approval` status.

For other equipment, you can continue with your normal checkout workflow.

Recommended workflow



1. Update your Checkout TypeScript

In your existing `checkout.ts`, add the approval service and router navigation. Keep your current checkout fields and equipment-loading code.

```
import { ApprovalService } from '../../services/approval';
import { CreateApprovalRequest } from '../../models/approval';
```

Inside your existing component class, inject the service if you use Angular's `inject()` pattern:

```
private approvalService = inject(ApprovalService);
```

Also make sure `inject` is imported from `@angular/core`.

Then, in your existing checkout submission method, check whether the selected equipment is a drone **before submitting the normal checkout**:

```
if (selectedItem.type.toLowerCase().includes('drone')) {
  const approvalRequest: CreateApprovalRequest = {
    equipmentId: selectedItem.id,
    requestedBy: this.assignedTo.trim(),
    expectedReturn: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  this.approvalService.create(approvalRequest).subscribe({
    next: (request) => {
      this.router.navigate(['/approvals', request.id]);
    },
    error: (error) => {
      console.error('Approval request failed:', error);
      this.errorMessage =
        error?.error?.message ||
        'Unable to submit the approval request.';
    }
  });

  return;
}

// For non-drone equipment, continue your existing checkout API call here.
```

Important: This snippet assumes your existing `checkout.ts` has `selectedItem`, `assignedTo`, `expectedReturnDate`, `purpose`, `router`, and `errorMessage`. Adapt the names if your component uses different property names. The `return` prevents the normal checkout API from being called for a drone.

2. Make sure your approval route accepts an ID

In your Angular routes, configure the approval component to support `/approvals/:id`:

```
{
  path: 'approvals/:id',
  component: Approvals
}
```

Keep your existing general approvals route too, if you use it:

```
{
  path: 'approvals',
  component: Approvals
}
```

Your `Approvals` component already checks the route ID and calls `getById(id)`, so it can display the request just created.

3. One backend rule is still necessary

Redirecting to the approval page is not enough to enforce approval. Your .NET checkout endpoint must also reject a drone checkout while its approval is pending or rejected. Otherwise, a user could bypass the Angular screen and call the checkout API directly.

The correct sequence is:

- Submit the approval request.
- Wait for approval.
- Allow the drone checkout only after its status is `Approved`.

This also means the checkout API should verify the approval request's status on the server before saving the checkout.

To give you a complete, ready-to-paste solution, share your current `checkout.ts` and I can integrate this into your exact component without disturbing your existing checkout logic.

25/10/2026, 13:12:36 · gpt-6

```
import { ChangeDetectorRef, Component, inject, OnInit } from
'@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest, EquipmentOption } from
'../../models/checkout';
import { ApprovalService } from '../../services/approval';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit  {

  private approvalService = inject(ApprovalService);
 private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  minReturnDate = this.getLocalDateString();

private cdr = inject(ChangeDetectorRef);

private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');

  return `${year}-${month}-${day}`;
}


  saving = false;
  loadingEquipment = false;
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
```

```
  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
          this.cdr.detectChanges();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }

  // API 2: Get the assignee for the selected equipment ID.


onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;
```

```
  this.assignedTo = '';
  this.errorMessage = '';

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        console.log('Assignee received from API:', assignee);

        this.assignedTo = assignee.trim();
        this.loadingAssignee = false;

        console.log('Assigned To field:', this.assignedTo);
        this.cdr.detectChanges();
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}



  // API 3: Submit checkout request.

requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
    this.errorMessage = 'Please complete all required fields.';
```

```
    return;
  }

  if (this.saving || this.loadingAssignee) {
    return;
  }

  const selectedItem = this.equipmentOptions.find(
    item => item.id === this.selectedEquipment
  );

  if (!selectedItem) {
    this.errorMessage = 'Please select valid equipment.';
    return;
  }

  const request: CreateCheckoutRequest = {
    equipmentName: selectedItem.name,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  console.log('Checkout request:', request);

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout saved successfully:', result);

      this.saving = false;
      this.successMessage = 'Checkout request saved
successfully.';
      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout save failed:', error.status,
error.error);

      this.saving = false;
      this.errorMessage =
        error?.error?.errors?.EquipmentName?.[0] ||
        error?.error?.title ||
        'Unable to save checkout request.';
    }
```

```
  });
}


  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

can you tell me in this code where to write the drone part ??

25/10/2026, 13:21:46

You need to write the **drone approval logic inside your existing `requestCheckOut()` method**, immediately after the `selectedItem` validation and **before** creating the normal checkout request.

I'll show you exactly where to put it in your current code.

1. Replace your existing `requestCheckOut()` method

Keep the rest of your `checkout.ts` unchanged. Replace only `requestCheckOut()` with this code:

```
requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  if (this.saving || this.loadingAssignee) {
    return;
  }

  const selectedItem = this.equipmentOptions.find(
    item => item.id === this.selectedEquipment
```

```
  );

  if (!selectedItem) {
    this.errorMessage = 'Please select valid equipment.';
    return;
  }

  // STEP 1: If the selected equipment is a drone,
  // create an approval request instead of checking it out.
  if (
    selectedItem.name.toLowerCase().includes('drone') ||
    selectedItem.type?.toLowerCase().includes('drone')
  ) {
    const approvalRequest = {
      equipmentId: selectedItem.id,
      requestedBy: this.assignedTo.trim(),
      expectedReturn: this.expectedReturnDate,
      purpose: this.purpose.trim()
    };

    this.saving = true;

    this.approvalService.create(approvalRequest).subscribe({
      next: (result) => {
        this.saving = false;
        this.successMessage = 'Approval request submitted successfully.';

        // Open the approval page for the newly created request.
        this.router.navigate(['/approvals', result.id]);
      },
      error: (error) => {
        console.error('Approval request failed:', error);

        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
          error?.error?.title ||
          'Unable to submit the approval request.';
      }
    });

    // IMPORTANT: Stop here. Do not call the normal checkout API.
    return;
  }

  // STEP 2: Non-drone equipment follows your existing checkout process.
```

```
  const request: CreateCheckoutRequest = {
    equipmentName: selectedItem.name,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  console.log('Checkout request:', request);

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout saved successfully:', result);

      this.saving = false;
      this.successMessage = 'Checkout request saved successfully.';
      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout save failed:', error.status, error.error);

      this.saving = false;
      this.errorMessage =
        error?.error?.errors?.EquipmentName?.[0] ||
        error?.error?.title ||
        'Unable to save checkout request.';
    }
  });
}
```

2. Where exactly does the drone part go?

First: Validate the selected equipment.

Next: Check whether the equipment is a drone.

If drone: Call `this.approvalService.create()` and navigate to `/approvals/{id}`.

If not a drone: Continue with `this.checkoutService.createCheckout()`.

3. Check these two things

- **Equipment type:** Your `EquipmentOption` must contain `type` for the type-based check. If it only contains `id` and `name`, remove the `selectedItem.type` condition and rely on the equipment name.

- **Angular route:** Your routes must include `approvals/:id`, and your `Approvals` component must load the request using that ID.

Important: This code creates the approval request and redirects to the approval page; it does not perform a drone checkout. To finish the workflow securely, your .NET checkout API must also prevent a drone from being checked out until its approval status is `Approved`.

25/10/2026, 13:21:49 · gpt-6

```
import { ChangeDetectorRef, Component, inject, OnInit } from
'@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest, EquipmentOption } from
'../../models/checkout';
import { ApprovalService } from '../../services/approval';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit  {

  private approvalService = inject(ApprovalService);
 private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  minReturnDate = this.getLocalDateString();

private cdr = inject(ChangeDetectorRef);

private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');

  return `${year}-${month}-${day}`;
}


  saving = false;
  loadingEquipment = false;
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
```

```
  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
          this.cdr.detectChanges();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }

  // API 2: Get the assignee for the selected equipment ID.


onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;
```

```
  this.assignedTo = '';
  this.errorMessage = '';

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        console.log('Assignee received from API:', assignee);

        this.assignedTo = assignee.trim();
        this.loadingAssignee = false;

        console.log('Assigned To field:', this.assignedTo);
        this.cdr.detectChanges();
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}



  // API 3: Submit checkout request.


requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
```

```
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  if (this.saving || this.loadingAssignee) {
    return;
  }

  const selectedItem = this.equipmentOptions.find(
    item => item.id === this.selectedEquipment
  );

  if (!selectedItem) {
    this.errorMessage = 'Please select valid equipment.';
    return;
  }

  // STEP 1: If the selected equipment is a drone,
  // create an approval request instead of checking it out.
  if (
    selectedItem.name.toLowerCase().includes('drone')
  ) {
    const approvalRequest = {
      equipmentId: selectedItem.id,
      requestedBy: this.assignedTo.trim(),
      expectedReturn: this.expectedReturnDate,
      purpose: this.purpose.trim()
    };

    this.saving = true;

    this.approvalService.create(approvalRequest).subscribe({
      next: (result) => {
        this.saving = false;
        this.successMessage = 'Approval request submitted
successfully.';

        // Open the approval page for the newly created request.
        this.router.navigate(['/approvals', result.id]);
      },
      error: (error) => {
        console.error('Approval request failed:', error);

        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
```

```
          error?.error?.title ||
          'Unable to submit the approval request.';
      }
    });

    // IMPORTANT: Stop here. Do not call the normal checkout
API.
    return;
  }

  // STEP 2: Non-drone equipment follows your existing
checkout process.
  const request: CreateCheckoutRequest = {
    equipmentName: selectedItem.name,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  console.log('Checkout request:', request);

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout saved successfully:', result);

      this.saving = false;
      this.successMessage = 'Checkout request saved
successfully.';
      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout save failed:', error.status,
error.error);

      this.saving = false;
      this.errorMessage =
        error?.error?.errors?.EquipmentName?.[0] ||
        error?.error?.title ||
        'Unable to save checkout request.';
    }
  });
}
```

```
  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}




export interface CreateCheckoutRequest {
  equipmentName: string;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
}

export interface EquipmentOption {
  id: number;
  name: string;
}

export interface CheckoutRecord {
  id: number;
  equipmentName: string;
  assignedTo: string;
  expectedReturnDate: string;
  purpose: string;
  status: string;
  createdAt: string;
}
```

i have selected drone in the equipment category and i have also checkkout it but the problem is it is not showing in the approval request because the requirement was if it was drone then it needs approval ??

25/10/2026, 13:26:41

The problem is likely in the way your checkout and approval workflows are connected. In your current TypeScript code, a drone should **create an approval request instead of calling the**

normal checkout API. However, selecting a drone in the equipment category does not guarantee that this condition is met.

I can see two important things to check:

1. **Your equipment model only has `id` and `name`.** Your code detects a drone using `selectedItem.name.toLowerCase().includes('drone')`. If the database name is something like `DJI Mavic` without the word “drone,” the approval branch will never run.
2. **Your Approvals page loads requests from the `ApprovalRequests` table.** If the POST request fails, or if the page is looking at a different API/database, the request will not appear there.

Also, if you previously checked out the drone using the normal checkout API, that existing checkout record does not automatically create an approval request.

Fix 1: Detect the equipment by its actual category

Your current `EquipmentOption` is:

```
export interface EquipmentOption {
  id: number;
  name: string;
}
```

It does not contain a category or type. If the actual database name is `DJI Mavic`, the condition `includes('drone')` will be false even if the equipment category is Drone.

The better fix is to add the equipment category to the backend's equipment dropdown API and frontend model, then check that category.

For a quick test, if the name is consistently `Drone - DJI Mavic`, your existing name check should work. Add this log immediately after finding `selectedItem`:

```
console.log('Selected equipment:', selectedItem);
console.log(
  'Is drone:',
  selectedItem.name.toLowerCase().includes('drone')
);
```

Click Check Out and inspect the browser console.

Fix 2: Recommended permanent solution — include the category

A. Update `EquipmentOption` in `models/checkout.ts`

```
export interface EquipmentOption {
  id: number;
```

```
  name: string;
  type: string;
}
```

B. Update your .NET checkout equipment endpoint

Find the endpoint that handles `GET /api/Checkout/equipment`. Make sure it returns the equipment type along with the ID and name:

```
[HttpGet("equipment")]
public async Task<IActionResult> GetEquipment()
{
    var equipment = await _context.Equipments
        .AsNoTracking()
        .Select(e => new
        {
            id = e.Id,
            name = e.Name,
            type = e.Type
        })
        .ToListAsync();

    return Ok(equipment);
}
```

Add `using Microsoft.EntityFrameworkCore;` if needed. This example assumes your equipment model has a `Type` property. If your category is stored in a property called `Category`, use `e.Category` instead. If your controller already uses a service/repository, preserve that architecture and update its DTO and projection rather than accessing the context directly.

C. Replace the drone condition in `requestCheckOut()`

Replace:

```
if (selectedItem.name.toLowerCase().includes('drone'))
```

with:

```
const isDrone =
  selectedItem.type?.trim().toLowerCase() === 'drone' ||
  selectedItem.name.toLowerCase().includes('drone');

if (isDrone) {
```

```
    // Keep your existing approvalService.create(...) logic here.
}
```

Now the decision uses the actual equipment type as well as the name.

Fix 3: Confirm that the request is being saved

After clicking Check Out on the drone, open your browser's **Developer Tools → Network** and inspect the request:

- `POST /api/Approvals` should return **201 Created**.
- Its response should contain an `id` and `"status": "Pending Approval"`.
- Then `GET /api/Approvals` should include the newly created request.

If the POST fails, inspect its response body. If it succeeds but the GET returns an empty list, verify that both endpoints use the same SQL Server database and connection string.

One final distinction: if you already checked out the drone before adding this approval logic, that old checkout will not automatically appear in approvals. You must submit a new approval request or create one for the existing checkout.

Start by checking the console log from Fix 1. It will tell us immediately whether the drone condition is being triggered or whether we need to investigate the API/database.

25/10/2026, 13:26:47 · gpt-6

how to delete from tables with referenced by foreign key and the id becomes 1 in the next insert ??

25/10/2026, 13:32:37

In your SQL Server database, you can delete records from tables that have foreign-key relationships, but you must delete them in the correct order. To make the next inserted ID become **1**, you also need to reset the table's identity counter.

Important: The following commands permanently delete data. Back up your database first if you need to preserve anything.

1. Delete records from tables with foreign keys

For your SolvefyTask project, suppose `ApprovalRequests` references `Equipments`, and `CheckIns` also references `Equipments`.

Delete the dependent records first, then the parent records:

```
DELETE FROM ApprovalRequests;
DELETE FROM CheckIns;
DELETE FROM Checkouts;
DELETE FROM Equipments;
```

Use your actual table names. If another table references one of these tables, that dependent table must also be handled first.

If SQL Server reports a foreign-key constraint error, another referencing table still contains records that prevent the deletion. Do not disable constraints blindly.

2. Reset the next ID to 1

After deleting all records from a table with an `IDENTITY` column, run `DBCC CHECKIDENT`:

```
DBCC CHECKIDENT ('ApprovalRequests', RESEED, 0);
DBCC CHECKIDENT ('CheckIns', RESEED, 0);
DBCC CHECKIDENT ('Checkouts', RESEED, 0);
DBCC CHECKIDENT ('Equipments', RESEED, 0);
```

For an empty table, reseeding to `0` makes the next generated identity value **1**.

3. Complete example with a transaction

Run this in SQL Server Management Studio after confirming the table names and dependencies:

```
BEGIN TRY
    BEGIN TRANSACTION;

    DELETE FROM ApprovalRequests;
    DELETE FROM CheckIns;
    DELETE FROM Checkouts;
    DELETE FROM Equipments;

    DBCC CHECKIDENT ('ApprovalRequests', RESEED, 0);
    DBCC CHECKIDENT ('CheckIns', RESEED, 0);
    DBCC CHECKIDENT ('Checkouts', RESEED, 0);
    DBCC CHECKIDENT ('Equipments', RESEED, 0);

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
```

This transaction rolls back the data changes if an error occurs. However, identity-counter changes are not necessarily restored by a rollback, so verify the counters if the script fails partway through.

4. Alternative: `TRUNCATE TABLE`

`TRUNCATE TABLE` automatically resets the identity counter, but **SQL Server will not allow it on a table referenced by a foreign key**, even if the referencing table is empty.

For your linked tables, `DELETE` followed by `DBCC CHECKIDENT` is generally the more flexible approach.

5. Verify the result

After running the script, check that the tables are empty:

```
SELECT * FROM ApprovalRequests;
SELECT * FROM CheckIns;
SELECT * FROM Checkouts;
SELECT * FROM Equipments;
```

The next inserted record in each correctly reseeded table should receive ID `1`.

One question: Do you want to clear **all tables in your SolvefyTask database**, including equipment, checkout, check-in, and approval records, or only selected tables? I can give you the

exact deletion order for your foreign-key relationships.

25/10/2026, 13:32:39 · gpt-6

```
import { ChangeDetectorRef, Component, inject, OnInit } from
'@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CheckoutService } from '../../services/checkout';
import { CreateCheckoutRequest, EquipmentOption } from
'../../models/checkout';
import { ApprovalService } from '../../services/approval';

@Component({
  selector: 'app-checkout',
  imports: [FormsModule],
  templateUrl: './checkout.html',
  styleUrl: './checkout.css',
})
export class Checkout implements OnInit  {

  private approvalService = inject(ApprovalService);
 private router = inject(Router);
  private checkoutService = inject(CheckoutService);

  minReturnDate = this.getLocalDateString();

private cdr = inject(ChangeDetectorRef);

private getLocalDateString(): string {
  const date = new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');

  return `${year}-${month}-${day}`;
}


  saving = false;
  loadingEquipment = false;
  loadingAssignee = false;

  errorMessage = '';
  successMessage = '';
  submitted = false;

  // Store the equipment ID, not its name.
```

```
  selectedEquipment: number | null = null;
  assignedTo = '';
  expectedReturnDate = '';
  purpose = '';

  equipmentOptions: EquipmentOption[] = [];

  ngOnInit(): void {
    this.loadEquipmentOptions();
  }

  // API 1: Load equipment from the database.
  loadEquipmentOptions(): void {
    this.loadingEquipment = true;
    this.errorMessage = '';

    this.checkoutService.getEquipmentOptions().subscribe({
      next: (equipment) => {
        this.equipmentOptions = equipment;
        this.loadingEquipment = false;

        if (equipment.length > 0) {
          this.selectedEquipment = equipment[0].id;
          this.onEquipmentChange();
          this.cdr.detectChanges();
        } else {
          this.selectedEquipment = null;
          this.assignedTo = '';
        }
      },
      error: (error) => {
        console.error('Failed to load equipment:', error);

        this.loadingEquipment = false;
        this.errorMessage =
          'Unable to load equipment. Please try again.';
      }
    });
  }

  // API 2: Get the assignee for the selected equipment ID.


onEquipmentChange(): void {
  const equipmentId = this.selectedEquipment;
```

```
  this.assignedTo = '';
  this.errorMessage = '';

  if (equipmentId === null) {
    return;
  }

  this.loadingAssignee = true;

  this.checkoutService
    .getAssignedToByEquipmentId(equipmentId)
    .subscribe({
      next: (assignee: string) => {
        console.log('Assignee received from API:', assignee);

        this.assignedTo = assignee.trim();
        this.loadingAssignee = false;

        console.log('Assigned To field:', this.assignedTo);
        this.cdr.detectChanges();
      },
      error: (error) => {
        console.error('Assignee API error:', error);

        this.assignedTo = '';
        this.loadingAssignee = false;
        this.errorMessage = 'Unable to load assignee.';
      }
    });
}



  // API 3: Submit checkout request.


requestCheckOut(): void {
  this.submitted = true;
  this.errorMessage = '';
  this.successMessage = '';

  if (
    this.selectedEquipment === null ||
    !this.assignedTo.trim() ||
    !this.expectedReturnDate
  ) {
```

```
    this.errorMessage = 'Please complete all required fields.';
    return;
  }

  if (this.saving || this.loadingAssignee) {
    return;
  }

  const selectedItem = this.equipmentOptions.find(
    item => item.id === this.selectedEquipment
  );

  if (!selectedItem) {
    this.errorMessage = 'Please select valid equipment.';
    return;
  }

  // STEP 1: If the selected equipment is a drone,
  // create an approval request instead of checking it out.
  if (
    selectedItem.category.toLowerCase().includes('drone')
  ) {
    const approvalRequest = {
      equipmentId: selectedItem.id,
      requestedBy: this.assignedTo.trim(),
      expectedReturn: this.expectedReturnDate,
      purpose: this.purpose.trim()
    };

    this.saving = true;

    this.approvalService.create(approvalRequest).subscribe({
      next: (result) => {
        this.saving = false;
        this.successMessage = 'Approval request submitted
successfully.';

        // Open the approval page for the newly created request.
        this.router.navigate(['/approvals', result.id]);
      },
      error: (error) => {
        console.error('Approval request failed:', error);

        this.saving = false;
        this.errorMessage =
          error?.error?.message ||
```

```
          error?.error?.title ||
          'Unable to submit the approval request.';
      }
    });

    // IMPORTANT: Stop here. Do not call the normal checkout
API.
    return;
  }

  // STEP 2: Non-drone equipment follows your existing
checkout process.
  const request: CreateCheckoutRequest = {
    equipmentName: selectedItem.name,
    assignedTo: this.assignedTo.trim(),
    expectedReturnDate: this.expectedReturnDate,
    purpose: this.purpose.trim()
  };

  console.log('Checkout request:', request);

  this.saving = true;

  this.checkoutService.createCheckout(request).subscribe({
    next: (result) => {
      console.log('Checkout saved successfully:', result);

      this.saving = false;
      this.successMessage = 'Checkout request saved
successfully.';
      alert(this.successMessage);
    },
    error: (error) => {
      console.error('Checkout save failed:', error.status,
error.error);

      this.saving = false;
      this.errorMessage =
        error?.error?.errors?.EquipmentName?.[0] ||
        error?.error?.title ||
        'Unable to save checkout request.';
    }
  });
}
```

```
  cancel(): void {
    this.router.navigate(['/dashboard']);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

the problem is here that when i click on the checkout request button then it takes to the login page resolve it ??

25/10/2026, 13:48:10

The issue is probably not in the drone condition itself. In your `requestCheckOut()` method, the normal checkout API is called for non-drone equipment, while the approval API is called for drones. **Neither of these calls should automatically send you to the login page.**

The login redirect is more likely caused by an Angular route guard, an HTTP interceptor, or an authentication error returned by the backend.

1. First, identify what is redirecting you

Open your browser's Developer Tools with `F12`, then:

1. Open the **Network** tab.
2. Click the Checkout Request button.
3. Look for the request that fails or returns `401` or `403`.
4. Check whether the browser navigates to `/login` immediately after that request.

Also check your browser console for errors.

2. Fix the most likely issue in your checkout code

Your code currently checks:

```
if (selectedItem.category.toLowerCase().includes('drone'))
```

Make sure `category` actually exists in your `EquipmentOption` interface and is returned by your backend. Otherwise, this may throw a runtime error.

For safer handling, use:

```
const isDrone =
  selectedItem.category?.trim().toLowerCase() === 'drone' ||
```

```
  selectedItem.name.toLowerCase().includes('drone');

if (isDrone) {
  // Your existing approvalService.create(...) logic
}
```

Your model should include the category:

```
export interface EquipmentOption {
  id: number;
  name: string;
  category: string;
}
```

And your .NET equipment endpoint must return `category` using the actual category property from your database.

3. Check your Angular route guard

Look in your route configuration (`app.routes.ts`) for something like:

```
{
  path: 'checkout',
  component: Checkout,
  canActivate: [authGuard]
}
```

Then inspect `authGuard`. If it redirects to `/login` when authentication is missing or invalid, that explains the behavior.

Also inspect your HTTP interceptor for logic such as:

```
if (error.status === 401) {
  this.router.navigate(['/login']);
}
```

A `401` response from the approval API could trigger that redirect.

Don't remove the guard or interceptor blindly. First confirm whether the API is returning an authentication error.

4. I need one specific piece of information

Please share your `app.routes.ts` **and HTTP interceptor or auth guard code**. If you have a screenshot of the Network tab showing the failed request, share that too.

With those, I can pinpoint why clicking Checkout sends you to Login and give you the exact corrected code without breaking your authentication.

Request URL\
https\://localhost:7135/api/Approvals\
Request method\
POST\
Status code\
201 Created\
Remote address\
[::1]:7135\
Referrer policy\
strict-origin-when-cross-origin\
access-control-allow-origin\
http\://localhost:4200\
content-type\
application/json; charset=utf-8\
date\
Sun, 25 Oct 2026 08:49:07 GMT\
location\
https\://localhost:7135/api/Approvals/3\
server\
Kestrel\
:authority\
localhost:7135\
:method\
POST\
:path\
/api/Approvals\
:scheme\
https\
accept\
application/json, text/plain, */*\
accept-encoding\
gzip, deflate, br, zstd\
accept-language\
en-US,en;q=0.9\
content-length\
91\
content-type\
application/json\
origin\
http\://localhost:4200\
priority\
u=1, i\
referer\
http\://localhost:4200/\

sec-ch-ua\
"Not:A-Brand";v="8", "Chromium";v="156", "Microsoft Edge";v="156"\
sec-ch-ua-mobile\
?0\
sec-ch-ua-platform\
"Windows"\
sec-fetch-dest\
empty\
sec-fetch-mode\
cors\
sec-fetch-site\
cross-site\
user-agent\
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36 Edg/156.0.0.0

{\
   "id": 3,\
   "equipmentId": 1,\
   "equipmentName": "Ahmad",\
   "serialNumber": "drone-1102",\
   "requestedBy": "Ahmad",\
   "requestedOn": "2026-10-25T08:49:08.0932793Z",\
   "expectedReturn": "2026-10-30T00:00:00",\
   "purpose": "sadfsfsdf",\
   "status": "Pending Approval",\
   "comments": null,\
   "decisionAt": null\
}

1. {id: 3, equipmentId: 1, equipmentName: "Ahmad", serialNumber: "drone-1102", requestedBy: "Ahmad",…}
   1. comments: null
   2. decisionAt: null

3. equipmentId: 1
4. equipmentName: "Ahmad"
5. expectedReturn: "2026-10-30T00:00:00"
6. id: 3
7. purpose: "sadfsfsdf"
8. requestedBy: "Ahmad"
9. requestedOn: "2026-10-25T08:49:08.0932793Z"
10. serialNumber: "drone-1102"
11. status: "Pending Approval"

25/10/2026, 13:50:23

Your Network details confirm that **the approval request is being saved successfully**. The backend is not returning an authentication error for this request.

- `POST /api/Approvals` → **201 Created**
- Approval ID → `3`
- Status → `Pending Approval`
- Created request URL → `https://localhost:7135/api/Approvals/3`

So the issue is likely happening **after the POST succeeds**, when Angular navigates to the approval page.

1. Fix the navigation after approval creation

In your `checkout.ts`, find this code inside `approvalService.create()`:

```
next: (result) => {
  this.saving = false;
  this.successMessage = 'Approval request submitted successfully.';

  this.router.navigate(['/approvals', result.id]);
},
```

Replace it with:

```
next: (result) => {
  this.saving = false;
  this.successMessage = 'Approval request submitted successfully.';

  console.log('Approval created:', result);
  console.log('Navigating to approval ID:', result.id);

  this.router.navigate(['/approvals', result.id]).then(
```

```
    (navigated) => {
      console.log('Navigation successful:', navigated);

      if (!navigated) {
        console.error('Angular could not navigate to the approval page.');
      }
    }
  );
},
```

2. Check `app.routes.ts`

Make sure you have a route that accepts the approval ID. For example:

```
{
  path: 'approvals/:id',
  component: Approvals
}
```

Keep your existing general approvals route too, if you use it:

```
{
  path: 'approvals',
  component: Approvals
}
```

If the `approvals/:id` route is missing, Angular may redirect to another route, potentially your login page.

3. Verify the request details

Your response contains:

```
{
  "id": 3,
  "equipmentId": 1,
  "equipmentName": "Ahmad",
  "serialNumber": "drone-1102",
  "status": "Pending Approval"
}
```

Notice that `equipmentName` is `"Ahmad"`, while the serial number is `"drone-1102"`. This suggests your database equipment name may not be what you expected. The approval request is being saved, but you should verify the equipment record and its category separately.

4. One thing I need you to check

After adding the navigation logs, click Checkout Request again and look at the browser console.

Does it show:

- `Navigation successful: true`, and then the login page appears?
- `Navigation successful: false`?
- Or does it remain on the checkout page?

That will tell us whether the redirect comes from Angular routing or from another part of your application.

25/10/2026, 13:50:25 · gpt-6

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
interface AssignedEquipment {
  id: number;
  name: string;
  serialNumber: string;
  type: string;
  checkedOutDate: string;
  expectedReturnDate: string;
  status: 'Checked Out' | 'Overdue' | 'Returned';
  icon: string;
}

@Component({
  selector: 'app-myequipment',
  imports: [],
  templateUrl: './myequipment.html',
  styleUrl: './myequipment.css',
})
export class Myequipment {
  constructor(private router: Router) {}

  equipment: AssignedEquipment[] = [
    {
      id: 1,
      name: 'GPS Rover - 01',
      serialNumber: 'GPS-001',
      type: 'GPS',
      checkedOutDate: '08/10/2026',
      expectedReturnDate: '09/10/2026',
      status: 'Overdue',
      icon: 'gps'
    },
    {
      id: 2,
      name: 'Field Tablet - 02',
      serialNumber: 'FT-002',
      type: 'Tablet',
      checkedOutDate: '10/10/2026',
      expectedReturnDate: '12/10/2026',
      status: 'Checked Out',
      icon: 'tablet'
    }
  ];
```

```
  returnEquipment(item: AssignedEquipment): void {
    if (item.status === 'Returned') {
      return;
    }

    const confirmed = window.confirm(
      `Do you want to return ${item.name}?`
    );

    if (!confirmed) {
      return;
    }

    // Demonstration only: connect your return API here.
    item.status = 'Returned';
    alert(`${item.name} has been marked as returned in this
demo.`);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

please implement the apis of these only the backend dotnet apis ??

25/10/2026, 14:15:15

Yes. We'll implement **only the .NET backend APIs** for your `My Equipment` page, using your existing ASP.NET Core + EF Core + SQL Server structure. No Angular changes are needed for this step.

The backend should support:

1. **GET** — retrieve equipment assigned to a user, including serial number, type, checkout date, expected return date, and status.
2. **PUT** — return a checked-out piece of equipment and update its status.
3. **GET** — optionally retrieve one assigned equipment record by ID.

Your current Angular array is hardcoded, so the APIs will replace that demo data once you connect the frontend. We should reuse your existing `Equipment` and checkout records rather than create a duplicate equipment table.

1. API endpoints

Method	Endpoint	Purpose
GET	`/api/MyEquipment?assignedTo=Ahmad`	Get equipment assigned to a user
GET	`/api/MyEquipment/{id}`	Get one equipment record
PUT	`/api/MyEquipment/{id}/return`	Mark equipment as returned

I'll use the existing checkout records as the source of checkout and return status. The implementation below assumes your EF Core checkout entity is named `Checkout` and is available through `ApplicationDbContext`. If your actual entity or property names differ, those references must match your project.

2. Create the response DTO

Create `DTOs/MyEquipmentResponseDto.cs`.

```
namespace SolvefyTask.DTOs
{
    public class MyEquipmentResponseDto
    {
        public int Id { get; set; }
        public string EquipmentName { get; set; } = string.Empty;
        public string? SerialNumber { get; set; }
        public string? Type { get; set; }
        public DateTime CheckedOutDate { get; set; }
        public DateTime ExpectedReturnDate { get; set; }
        public string Status { get; set; } = string.Empty;
    }
}
```

3. Create the My Equipment controller

Create `Controllers/MyEquipmentController.cs`.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SolvefyTask.DTOs;
using SolvefyTask.Data;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class MyEquipmentController : ControllerBase
```

```
    {
        private readonly ApplicationDbContext _context;

        public MyEquipmentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/MyEquipment?assignedTo=Ahmad
        [HttpGet]
        public async Task<ActionResult<IEnumerable<MyEquipmentResponseDto>>>
GetMyEquipment(
            [FromQuery] string assignedTo)
        {
            if (string.IsNullOrWhiteSpace(assignedTo))
            {
                return BadRequest(new
                {
                    message = "The assignedTo parameter is required."
                });
            }

            var records = await _context.Checkouts
                .AsNoTracking()
                .Where(x =>
                    x.AssignedTo == assignedTo &&
                    x.Status != "Returned")
                .OrderByDescending(x => x.CreatedAt)
                .Select(x => new MyEquipmentResponseDto
                {
                    Id = x.Id,
                    EquipmentName = x.EquipmentName,
                    CheckedOutDate = x.CreatedAt,
                    ExpectedReturnDate = x.ExpectedReturnDate,
                    Status = x.ExpectedReturnDate.Date < DateTime.UtcNow.Date
                        ? "Overdue"
                        : "Checked Out"
                })
                .ToListAsync();

            return Ok(records);
        }

        // GET: api/MyEquipment/1
        [HttpGet("{id:int}")]
        public async Task<ActionResult<MyEquipmentResponseDto>> GetEquipment(int id)
```

```
    {
        var record = await _context.Checkouts
            .AsNoTracking()
            .Where(x => x.Id == id)
            .Select(x => new MyEquipmentResponseDto
            {
                Id = x.Id,
                EquipmentName = x.EquipmentName,
                CheckedOutDate = x.CreatedAt,
                ExpectedReturnDate = x.ExpectedReturnDate,
                Status = x.Status == "Returned"
                    ? "Returned"
                    : x.ExpectedReturnDate.Date < DateTime.UtcNow.Date
                        ? "Overdue"
                        : "Checked Out"
            })
            .FirstOrDefaultAsync();

        if (record == null)
        {
            return NotFound(new
            {
                message = "Equipment checkout record not found."
            });
        }

        return Ok(record);
    }

    // PUT: api/MyEquipment/1/return
    [HttpPut("{id:int}/return")]
    public async Task<IActionResult> ReturnEquipment(int id)
    {
        var record = await _context.Checkouts
            .FirstOrDefaultAsync(x => x.Id == id);

        if (record == null)
        {
            return NotFound(new
            {
                message = "Equipment checkout record not found."
            });
        }

        if (record.Status == "Returned")
        {
```

```
                return BadRequest(new
                {
                    message = "This equipment has already been returned."
                });
            }

            record.Status = "Returned";

            await _context.SaveChangesAsync();

            return Ok(new
            {
                message = "Equipment returned successfully.",
                id = record.Id,
                equipmentName = record.EquipmentName,
                status = record.Status
            });
        }
    }
}
```

Important adjustments before running

- **DbSet name:** I used `_context.Checkouts`. Replace it with your actual `DbSet` name if different.
- **Equipment details:** The code currently returns the checkout record's equipment name. To return the actual serial number and type, we need to join or navigate to your existing `Equipment` entity.
- **Checkout date:** I used `CreatedAt` as the checkout date because that property appears in your existing checkout model.
- **Return behavior:** This updates the checkout record to `Returned`. If your system also maintains a separate equipment availability status, that status should be updated in the same transaction.

4. Test the APIs

Run your backend and test these requests in Swagger.

Get a user's equipment

```
GET /api/MyEquipment?assignedTo=Ahmad
```

Expected response structure:

```
[
  {
    "id": 1,
    "equipmentName": "GPS Rover - 01",
    "serialNumber": null,
    "type": null,
    "checkedOutDate": "2026-10-08T00:00:00",
    "expectedReturnDate": "2026-10-09T00:00:00",
    "status": "Overdue"
  }
]
```


Return equipment

```
PUT /api/MyEquipment/1/return
```

Expected response:


```
{
  "message": "Equipment returned successfully.",
  "id": 1,
  "equipmentName": "GPS Rover - 01",
  "status": "Returned"
}
```


These are illustrative responses; the returned data will depend on your database.

5. One thing I need to make this compile exactly in your project

Please share your existing `Checkout` **entity/model class** and the relevant `Equipment` entity/model class. I'll then align the queries with your actual navigation properties and add the real serial number and equipment type, without creating duplicate tables or changing your existing architecture.

25/10/2026, 14:15:22 · gpt-6

please send the repositories and service layers like you have send for the previuous apis ??

25/10/2026, 14:17:37

Absolutely. We'll follow the same **Repository → Service → Controller** architecture used for your previous .NET APIs, keeping the `My Equipment` functionality separate and maintainable.

We'll implement these three endpoints:

- `GET /api/MyEquipment?assignedTo=Ahmad` — retrieve equipment assigned to a user.
- `GET /api/MyEquipment/{id}` — retrieve a checkout record.
- `PUT /api/MyEquipment/{id}/return` — mark equipment as returned.

The repository will handle database operations, the service will handle business rules such as overdue status and duplicate returns, and the controller will expose the HTTP endpoints.

1. DTO — `DTOs/MyEquipmentResponseDto.cs`

```
namespace SolvefyTask.DTOs
{
    public class MyEquipmentResponseDto
    {
        public int Id { get; set; }
        public string EquipmentName { get; set; } = string.Empty;
        public string? SerialNumber { get; set; }
        public string? Type { get; set; }
        public DateTime CheckedOutDate { get; set; }
        public DateTime ExpectedReturnDate { get; set; }
        public string Status { get; set; } = string.Empty;
    }
}
```

2. Repository layer

`Repositories/IMyEquipmentRepository.cs`

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Repositories
{
    public interface IMyEquipmentRepository
```

```
    {
        Task<IEnumerable<MyEquipmentResponseDto>> GetMyEquipmentAsync(
            string assignedTo);

        Task<MyEquipmentResponseDto?> GetByIdAsync(int id);

        Task<bool> ReturnEquipmentAsync(int id);
    }
}
```

`Repositories/MyEquipmentRepository.cs`

This layer communicates directly with EF Core and your database.

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.DTOs;

namespace SolvefyTask.Repositories
{
    public class MyEquipmentRepository : IMyEquipmentRepository
    {
        private readonly ApplicationDbContext _context;

        public MyEquipmentRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<MyEquipmentResponseDto>>
            GetMyEquipmentAsync(string assignedTo)
        {
            var today = DateTime.UtcNow.Date;

            return await _context.Checkouts
                .AsNoTracking()
                .Where(x =>
                    x.AssignedTo == assignedTo &&
                    x.Status != "Returned")
                .OrderByDescending(x => x.CreatedAt)
                .Select(x => new MyEquipmentResponseDto
                {
                    Id = x.Id,
                    EquipmentName = x.EquipmentName,
                    CheckedOutDate = x.CreatedAt,
                    ExpectedReturnDate = x.ExpectedReturnDate,
```

```
                Status = x.ExpectedReturnDate.Date < today
                    ? "Overdue"
                    : "Checked Out"
            })
            .ToListAsync();
    }

    public async Task<MyEquipmentResponseDto?> GetByIdAsync(int id)
    {
        var today = DateTime.UtcNow.Date;

        return await _context.Checkouts
            .AsNoTracking()
            .Where(x => x.Id == id)
            .Select(x => new MyEquipmentResponseDto
            {
                Id = x.Id,
                EquipmentName = x.EquipmentName,
                CheckedOutDate = x.CreatedAt,
                ExpectedReturnDate = x.ExpectedReturnDate,
                Status = x.Status == "Returned"
                    ? "Returned"
                    : x.ExpectedReturnDate.Date < today
                        ? "Overdue"
                        : "Checked Out"
            })
            .FirstOrDefaultAsync();
    }

    public async Task<bool> ReturnEquipmentAsync(int id)
    {
        var checkout = await _context.Checkouts
            .FirstOrDefaultAsync(x => x.Id == id);

        if (checkout == null)
        {
            return false;
        }

        checkout.Status = "Returned";

        await _context.SaveChangesAsync();

        return true;
    }
```

```
    }
}
```

Note: This assumes your existing context exposes `_context.Checkouts` and the checkout entity has `AssignedTo`, `EquipmentName`, `CreatedAt`, `ExpectedReturnDate`, and `Status` properties. Serial number and type are left unset until we map your actual equipment relationship.

3. Service layer

`Services/IMyEquipmentService.cs`

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Services
{
    public interface IMyEquipmentService
    {
        Task<IEnumerable<MyEquipmentResponseDto>> GetMyEquipmentAsync(
            string assignedTo);

        Task<MyEquipmentResponseDto?> GetByIdAsync(int id);

        Task<(bool Success, string Message)> ReturnEquipmentAsync(int id);
    }
}
```

`Services/MyEquipmentService.cs`

This layer contains the business rules.

```
using SolvefyTask.DTOs;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class MyEquipmentService : IMyEquipmentService
    {
        private readonly IMyEquipmentRepository _repository;

        public MyEquipmentService(IMyEquipmentRepository repository)
        {
            _repository = repository;
        }

        public async Task<IEnumerable<MyEquipmentResponseDto>>
            GetMyEquipmentAsync(string assignedTo)
```

```
        {
            if (string.IsNullOrWhiteSpace(assignedTo))
            {
                throw new ArgumentException(
                    "Assigned employee is required.");
            }

            return await _repository.GetMyEquipmentAsync(
                assignedTo.Trim());
        }

        public async Task<MyEquipmentResponseDto?> GetByIdAsync(int id)
        {
            if (id <= 0)
            {
                throw new ArgumentException(
                    "A valid equipment checkout ID is required.");
            }

            return await _repository.GetByIdAsync(id);
        }

        public async Task<(bool Success, string Message)>
            ReturnEquipmentAsync(int id)
        {
            if (id <= 0)
            {
                return (false, "A valid checkout ID is required.");
            }

            var equipment = await _repository.GetByIdAsync(id);

            if (equipment == null)
            {
                return (false, "Equipment checkout record not found.");
            }

            if (equipment.Status == "Returned")
            {
                return (false, "This equipment has already been returned.");
            }

            var returned = await _repository.ReturnEquipmentAsync(id);

            if (!returned)
            {
```

```
                return (false, "Unable to return the equipment.");
            }

            return (true, "Equipment returned successfully.");
        }
    }
}
```

4. Controller layer

`Controllers/MyEquipmentController.cs`

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class MyEquipmentController : ControllerBase
    {
        private readonly IMyEquipmentService _service;

        public MyEquipmentController(IMyEquipmentService service)
        {
            _service = service;
        }

        // GET: api/MyEquipment?assignedTo=Ahmad
        [HttpGet]
        public async Task<IActionResult> GetMyEquipment(
            [FromQuery] string assignedTo)
        {
            if (string.IsNullOrWhiteSpace(assignedTo))
            {
                return BadRequest(new
                {
                    message = "The assignedTo parameter is required."
                });
            }

            var equipment = await _service.GetMyEquipmentAsync(assignedTo);

            return Ok(equipment);
        }
```

```
        // GET: api/MyEquipment/1
        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var equipment = await _service.GetByIdAsync(id);

            if (equipment == null)
            {
                return NotFound(new
                {
                    message = "Equipment checkout record not found."
                });
            }

            return Ok(equipment);
        }

        // PUT: api/MyEquipment/1/return
        [HttpPut("{id:int}/return")]
        public async Task<IActionResult> ReturnEquipment(int id)
        {
            var result = await _service.ReturnEquipmentAsync(id);

            if (!result.Success)
            {
                if (result.Message == "Equipment checkout record not found.")
                {
                    return NotFound(new { message = result.Message });
                }

                return BadRequest(new { message = result.Message });
            }

            return Ok(new
            {
                message = result.Message,
                id,
                status = "Returned"
            });
        }
    }
}
```

5. Register the services in `Program.cs`

Add these registrations alongside your existing repository and service registrations:

```
builder.Services.AddScoped<IMyEquipmentRepository, MyEquipmentRepository>();
builder.Services.AddScoped<IMyEquipmentService, MyEquipmentService>();
```

Add these namespaces if needed:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

No new migration is needed for these classes because they use your existing checkout table.

6. Test in Swagger

GET **Get assigned equipment**

`/api/MyEquipment?assignedTo=Ahmad`

GET **Get a checkout record**

`/api/MyEquipment/1`

PUT **Return equipment**

`/api/MyEquipment/1/return`

Important: The implementation above follows the same layered architecture, but one detail needs checking against your existing code: the exact `Checkout` entity and how it relates to `Equipment`. Once that relationship is mapped, we can return the real `SerialNumber` and `Type` fields and ensure a return also updates equipment availability where your existing system stores it.

25/10/2026, 14:17:41 · gpt-6

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
interface AssignedEquipment {
  id: number;
  name: string;
  serialNumber: string;
  type: string;
  checkedOutDate: string;
  expectedReturnDate: string;
  status: 'Checked Out' | 'Overdue' | 'Returned';
  icon: string;
}

@Component({
  selector: 'app-myequipment',
  imports: [],
  templateUrl: './myequipment.html',
  styleUrl: './myequipment.css',
})
export class Myequipment {
  constructor(private router: Router) {}

  equipment: AssignedEquipment[] = [
    {
      id: 1,
      name: 'GPS Rover - 01',
      serialNumber: 'GPS-001',
      type: 'GPS',
      checkedOutDate: '08/10/2026',
      expectedReturnDate: '09/10/2026',
      status: 'Overdue',
      icon: 'gps'
    },
    {
      id: 2,
      name: 'Field Tablet - 02',
      serialNumber: 'FT-002',
      type: 'Tablet',
      checkedOutDate: '10/10/2026',
      expectedReturnDate: '12/10/2026',
      status: 'Checked Out',
      icon: 'tablet'
    }
  ];
```

```
  returnEquipment(item: AssignedEquipment): void {
    if (item.status === 'Returned') {
      return;
    }

    const confirmed = window.confirm(
      `Do you want to return ${item.name}?`
    );

    if (!confirmed) {
      return;
    }

    // Demonstration only: connect your return API here.
    item.status = 'Returned';
    alert(`${item.name} has been marked as returned in this
demo.`);
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}




<div class="my-equipment-page">

  <div class="page-banner">
    6. My Equipment
  </div>

  <main class="page-content">

    <header class="page-heading">
      <div>
        <h1>My Equipment</h1>
        <p>Equipment currently assigned to you</p>
      </div>
    </header>

    <section class="equipment-list">

      @for (item of equipment; track item.id) {
```

```
      <article class="equipment-card">

        <!-- Equipment image/icon -->
        <div class="equipment-image" [class]=" 'equipment-
image ' + item.icon">

          @if (item.icon === 'gps') {
            <svg viewBox="0 0 64 80" role="img" aria-
label="GPS rover">
              <ellipse cx="32" cy="14" rx="18" ry="10"
fill="#e0a52c"/>
              <rect x="17" y="12" width="30" height="13"
rx="4" fill="#687786"/>
              <rect x="23" y="23" width="18" height="26"
rx="4" fill="#d9dfe5"/>
              <rect x="28" y="45" width="8" height="23" rx="2"
fill="#667786"/>
              <path d="M20 30 L12 39 M44 30 L52 39"
stroke="#697887" stroke-width="3"/>
              <circle cx="32" cy="17" r="5" fill="#f8d35c"/>
            </svg>
          } @else {
            <svg viewBox="0 0 64 80" role="img" aria-
label="Field tablet">
              <rect x="7" y="3" width="50" height="72" rx="6"
fill="#253d52"/>
              <rect x="11" y="9" width="42" height="58" rx="3"
fill="#397ca0"/>
              <path d="M13 45 L27 27 L37 39 L46 22 L52 31 L52
65 L13 65 Z"
                fill="#70a8c0"/>
              <circle cx="32" cy="70" r="2" fill="#c7d2dc"/>
              <circle cx="32" cy="6" r="1" fill="#bfcbd7"/>
            </svg>
          }

        </div>

        <!-- Equipment details -->
        <div class="equipment-details">

          <div class="equipment-title-row">
            <h2>{{ item.name }}</h2>

            @if (item.status === 'Overdue') {
              <span class="status-badge
```

```
overdue">Overdue</span>
            } @else if (item.status === 'Returned') {
              <span class="status-badge
returned">Returned</span>
            }
          </div>

          <div class="equipment-meta">
            <span>Checked Out:</span>
            <strong>{{ item.checkedOutDate }}</strong>
          </div>

          <div class="equipment-meta">
            <span>Expected Return:</span>
            <strong>{{ item.expectedReturnDate }}</strong>
          </div>

          <div class="equipment-meta">
            <span>Status:</span>
            <strong
              class="status-text"
              [class.overdue-text]="item.status === 'Overdue'"
              [class.returned-text]="item.status === 'Returned'"
            >
              {{ item.status === 'Returned' ? 'Returned' :
'Checked Out' }}
            </strong>
          </div>

        </div>

        <!-- Return action -->
        <div class="equipment-actions">
          <button
            type="button"
            class="return-button"
            [disabled]="item.status === 'Returned'"
            (click)="returnEquipment(item)"
          >
            {{ item.status === 'Returned' ? 'Returned' : 'Return'
}}
          </button>
        </div>

      </article>
    } @empty {
```

```
      <div class="empty-state">
        <span class="empty-icon">✓</span>
        <h2>No equipment assigned</h2>
        <p>You currently have no equipment checked out.</p>
      </div>
    }

  </section>

  <footer class="page-footer">
    <span>{{ equipment.length }} equipment record(s)
</span>
    <span>Equipment Tracking System</span>
  </footer>

 </main>
</div>
```
please implement the above apis of myequipment in this typescript code ??

25/10/2026, 14:35:51

We'll connect your existing `Myequipment` component to the backend APIs while keeping your current HTML design, SVG icons, and styling intact.

The updated TypeScript will:

- Load equipment from `GET /api/MyEquipment?assignedTo=...`.
- Display the real checkout dates and calculate overdue status from the API response.
- Call `PUT /api/MyEquipment/{id}/return` when the user confirms a return.
- Show loading, success, and error messages.
- Remove the hardcoded demonstration equipment.

1. Create the model

Create `src/app/models/my-equipment.ts`:

```typescript
export interface MyEquipment {
  id: number;
  equipmentName: string;
  serialNumber: string | null;
  type: string | null;
  checkedOutDate: string;
  expectedReturnDate: string;
```

```
  status: 'Checked Out' | 'Overdue' | 'Returned';
}

export interface ReturnEquipmentResponse {
  message: string;
  id: number;
  equipmentName: string;
  status: string;
}
```

2. Create the API service

Create `src/app/services/my-equipment.ts`. Adjust the relative import of `environment` if your service folder structure differs.

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import {
  MyEquipment,
  ReturnEquipmentResponse
} from '../models/my-equipment';

@Injectable({
  providedIn: 'root'
})
export class MyEquipmentService {
  private http = inject(HttpClient);
  private apiUrl = `${environment.apiUrl}/MyEquipment`;

  getMyEquipment(assignedTo: string): Observable<MyEquipment[]> {
    return this.http.get<MyEquipment[]>(this.apiUrl, {
      params: { assignedTo }
    });
  }

  getById(id: number): Observable<MyEquipment> {
    return this.http.get<MyEquipment>(`${this.apiUrl}/${id}`);
  }

  returnEquipment(id: number): Observable<ReturnEquipmentResponse> {
    return this.http.put<ReturnEquipmentResponse>(
      `${this.apiUrl}/${id}/return`,
      {}
    );
```

```
  }
}
```

3. Replace your `myequipment.ts`

This version uses your existing HTML property names such as `item.name`, `item.icon`, and `item.status`. Therefore, **you do not need to redesign your HTML**.

```
import {
  ChangeDetectorRef,
  Component,
  inject,
  OnInit
} from '@angular/core';
import { Router } from '@angular/router';
import { finalize } from 'rxjs';

import { MyEquipmentService } from '../../services/my-equipment';
import { MyEquipment } from '../../models/my-equipment';

interface AssignedEquipment {
  id: number;
  name: string;
  serialNumber: string;
  type: string;
  checkedOutDate: string;
  expectedReturnDate: string;
  status: 'Checked Out' | 'Overdue' | 'Returned';
  icon: string;
}

@Component({
  selector: 'app-myequipment',
  standalone: true,
  imports: [],
  templateUrl: './myequipment.html',
  styleUrl: './myequipment.css',
})
export class Myequipment implements OnInit {
  private router = inject(Router);
  private myEquipmentService = inject(MyEquipmentService);
  private cdr = inject(ChangeDetectorRef);

  // Replace with the currently logged-in employee's name.
  assignedTo = 'Ahmad';
```

```
  equipment: AssignedEquipment[] = [];

  loading = false;
  returningId: number | null = null;
  errorMessage = '';
  successMessage = '';

  ngOnInit(): void {
    this.loadMyEquipment();
  }

  loadMyEquipment(): void {
    this.loading = true;
    this.errorMessage = '';

    this.myEquipmentService
      .getMyEquipment(this.assignedTo)
      .pipe(
        finalize(() => {
          this.loading = false;
          this.cdr.detectChanges();
        })
      )
      .subscribe({
        next: (records: MyEquipment[]) => {
          this.equipment = records.map(record => ({
            id: record.id,
            name: record.equipmentName,
            serialNumber: record.serialNumber ?? '',
            type: record.type ?? '',
            checkedOutDate: this.formatDate(record.checkedOutDate),
            expectedReturnDate: this.formatDate(
              record.expectedReturnDate
            ),
            status: this.getStatus(record),
            icon: this.getIcon(
              record.type,
              record.equipmentName
            )
          }));
        },
        error: error => {
          console.error('Failed to load my equipment:', error);
          this.errorMessage =
            error?.error?.message ??
            'Unable to load your equipment. Please try again.';
```

```
      }
    });
  }

  returnEquipment(item: AssignedEquipment): void {
    if (
      item.status === 'Returned' ||
      this.returningId !== null
    ) {
      return;
    }

    const confirmed = window.confirm(
      `Do you want to return ${item.name}?`
    );

    if (!confirmed) {
      return;
    }

    this.returningId = item.id;
    this.errorMessage = '';
    this.successMessage = '';

    this.myEquipmentService
      .returnEquipment(item.id)
      .pipe(
        finalize(() => {
          this.returningId = null;
          this.cdr.detectChanges();
        })
      )
      .subscribe({
        next: response => {
          item.status = 'Returned';
          this.successMessage =
            response.message ||
            `${item.name} has been returned successfully.`;
        },
        error: error => {
          console.error('Failed to return equipment:', error);

          this.errorMessage =
            error?.error?.message ??
            'Unable to return equipment. Please try again.';
        }
```

```
      });
  }

  private getStatus(
    record: MyEquipment
  ): AssignedEquipment['status'] {
    if (record.status === 'Returned') {
      return 'Returned';
    }

    const expectedDate = new Date(record.expectedReturnDate);
    const today = new Date();

    expectedDate.setHours(0, 0, 0, 0);
    today.setHours(0, 0, 0, 0);

    return expectedDate < today ? 'Overdue' : 'Checked Out';
  }

  private formatDate(value: string): string {
    if (!value) {
      return '';
    }

    const date = new Date(value);

    if (Number.isNaN(date.getTime())) {
      return value;
    }

    const day = String(date.getDate()).padStart(2, '0');
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const year = date.getFullYear();

    return `${day}/${month}/${year}`;
  }

  private getIcon(type: string | null, name: string): string {
    const value = `${type ?? ''} ${name}`.toLowerCase();

    if (value.includes('gps') || value.includes('rover')) {
      return 'gps';
    }

    if (value.includes('tablet')) {
      return 'tablet';
```

```
    }

    if (value.includes('drone')) {
      return 'drone';
    }

    if (value.includes('tripod')) {
      return 'tripod';
    }

    return 'tablet';
  }

  goBack(): void {
    this.router.navigate(['/dashboard']);
  }
}
```

4. Make two small additions to `myequipment.html`

Your existing HTML can remain as it is, with the following additions.

Place these messages after your page heading and before the equipment list:

```
@if (loading) {
  <div class="loading-message">Loading your equipment...</div>
}

@if (errorMessage) {
  <div class="error-message">{{ errorMessage }}</div>
}

@if (successMessage) {
  <div class="success-message">{{ successMessage }}</div>
}
```

Update your existing Return button to show when a return is in progress:

```
<button
  type="button"
  class="return-button"
  [disabled]="item.status === 'Returned' || returningId === item.id"
  (click)="returnEquipment(item)"
>
  @if (returningId === item.id) {
    Returning...
  } @else {
```

```
    {{ item.status === 'Returned' ? 'Returned' : 'Return' }}
  }
</button>
```

For the empty state, you can retain your existing `@empty` block. If you want to avoid showing “No equipment assigned” while the API is loading or has failed, change the equipment-list condition to:

```
@if (!loading && !errorMessage) {
  <section class="equipment-list">
    <!-- Keep your existing @for and @empty blocks here. -->
  </section>
}
```

If you use that wrapper, remove the original outer `<section class="equipment-list">` so the sections are not duplicated.

5. Important checks

1. Your `environment.apiUrl` should be `https://localhost:7135/api`.
2. `provideHttpClient()` must already be configured in `app.config.ts`.
3. The backend must return `serialNumber` and `type` if you want those values populated; the earlier backend implementation left them unset.
4. **Employee filtering:** `assignedTo = 'Ahmad'` is a temporary example. Replace it with the authenticated employee's actual identity. The backend should also authorize the request and prevent users from viewing or returning another employee's equipment; a query-string name alone is not secure.
5. The backend return endpoint must update the existing checkout record successfully before the UI marks it as returned.

One final backend detail: the repository implementation we discussed filters out returned records, so returned items will disappear after a page refresh. If you want returned equipment to remain visible in this page with a `Returned` badge, we should adjust the GET API to return the user's full checkout history instead.

25/10/2026, 14:35:54 · gpt-6

can you tell me how to assign equipment because it is showing No Equipment Assigned ??

25/10/2026, 14:43:04

Your **My Equipment** page is showing “No equipment assigned” because the API is returning an empty array (`[]`). In the backend we implemented, equipment appears only when a checkout record exists with a matching `AssignedTo` value and its status is not `Returned` .

Let's fix this step by step.

Step 1: Check your existing checkout records

Open Swagger at:

`https://localhost:7135/swagger`

Find the endpoint that retrieves checkout records, if you already have one. Check whether the database contains a record with:

- `EquipmentName` : the name of the equipment
- `AssignedTo` : `Ahmad` (or the employee name used in your Angular component)
- `Status` : `Checked Out`
- `CreatedAt` : checkout date
- `ExpectedReturnDate` : expected return date

Important: Your Angular component currently has `assignedTo = 'Ahmad'` . If the database stores `Ahmad Abdullah` , `ahmad` , or another employee name, the exact-match query won't find it.

Step 2: Create a checkout record

If no checkout record exists, use your existing checkout creation API.

In Swagger, find:

`POST /api/Checkout`

Use a request body similar to this, adjusting the equipment name and dates to match your database:

```
{
  "equipmentName": "GPS Rover - 01",
  "assignedTo": "Ahmad",
  "expectedReturnDate": "2026-10-30",
```

```
  "purpose": "Field survey"
}
```

If the API successfully saves this record, it should become available on the My Equipment page, provided the backend GET query uses the same employee name.

Step 3: Confirm the API response

Open this URL in Swagger or your browser:

`https://localhost:7135/api/MyEquipment?assignedTo=Ahmad`

- If it returns an equipment array, the backend is finding records.
- If it returns `[]`, check the saved checkout record's `AssignedTo` and `Status` values.
- If it returns an error, check the backend console and API response.

Step 4: Make sure Angular uses the correct employee name

In `myequipment.ts`, you currently have:

```
assignedTo = 'Ahmad';
```

Replace `'Ahmad'` with the exact employee name saved in the checkout record, for example:

```
assignedTo = 'Ahmad Abdullah';
```

Use the actual value from your database, not necessarily this example.

The important design distinction

Your current `POST /api/Checkout` creates a checkout record, while the My Equipment API reads those records. There isn't a separate assignment process in the implementation we've discussed.

Please tell me one thing: When you create a checkout request, does it save directly as `Checked Out`, or does it first require approval?

That determines how we should implement equipment assignment correctly, particularly for your drone approval workflow.

25/10/2026, 14:43:06 · gpt-6

please make a myequipment table you have not made previous and then migrations ?

25/10/2026, 15:05:05

Yes. Since your `My Equipment` page needs its own database records, we can create a `MyEquipment` **table** in SQL Server and add the EF Core migration.

We'll keep your existing `Equipment`, `Checkout`, and `ApprovalRequest` tables intact. The new table will store which employee has which equipment, when it was checked out, its expected return date, and whether it has been returned. This will allow your page to load real assigned equipment instead of relying on hardcoded TypeScript data.

1. Create the entity

Create `Models/MyEquipment.cs`:

```
namespace SolvefyTask.Models
{
    public class MyEquipment
    {
        public int Id { get; set; }

        public int EquipmentId { get; set; }

        public string AssignedTo { get; set; } = string.Empty;

        public DateTime CheckedOutDate { get; set; }

        public DateTime ExpectedReturnDate { get; set; }

        public string Status { get; set; } = "Checked Out";

        // Navigation property
        public Equipment Equipment { get; set; } = null!;
    }
}
```

This design uses `EquipmentId` as a foreign key, so the equipment name, serial number, and type can come from your existing `Equipment` table rather than being duplicated.

2. Add the table to `ApplicationDbContext`

Open `Data/ApplicationDbContext.cs` and add the `DbSet`:

```
public DbSet<MyEquipment> MyEquipments { get; set; }
```

Add the namespace at the top if necessary:

```
using SolvefyTask.Models;
```

Inside `OnModelCreating`, add the relationship if you already override that method:

```
modelBuilder.Entity<MyEquipment>()
    .HasOne(x => x.Equipment)
    .WithMany()
    .HasForeignKey(x => x.EquipmentId)
    .OnDelete(DeleteBehavior.Restrict);

modelBuilder.Entity<MyEquipment>()
    .Property(x => x.AssignedTo)
    .HasMaxLength(150)
    .IsRequired();

modelBuilder.Entity<MyEquipment>()
    .Property(x => x.Status)
    .HasMaxLength(30)
    .IsRequired();
```

If `OnModelCreating` does not exist yet, add this configuration inside a new override and call `base.OnModelCreating(modelBuilder)`.

3. Create the migration

Open the terminal in the backend project directory—the directory containing your `.csproj` file.

If you use Visual Studio's Package Manager Console, run:

```
Add-Migration CreateMyEquipmentTable
Update-Database
```

If you use the .NET CLI, run:

```
dotnet ef migrations add CreateMyEquipmentTable
dotnet ef database update
```

If your solution has separate API and data projects, the command may need `--project` and `--startup-project` arguments pointing to those projects.

EF Core will generate the migration and create the `MyEquipments` table in the database configured by your application's connection string.

4. What the new table will contain

Column	Purpose
`Id`	Unique assignment record
`EquipmentId`	Foreign key to `Equipment`
`AssignedTo`	Employee assigned the equipment
`CheckedOutDate`	Date equipment was checked out
`ExpectedReturnDate`	Expected return date
`Status`	`Checked Out` or `Returned`

The `Overdue` status can be calculated from `ExpectedReturnDate` when retrieving records.

5. Important: update the repository to use the new table

Creating the table alone will not populate your My Equipment page. Your repository must query `_context.MyEquipments`, not `_context.Checkouts`, and your assignment workflow must insert records into the new table.

For example, the query should start like this:

```
var records = await _context.MyEquipments
    .AsNoTracking()
    .Include(x => x.Equipment)
    .Where(x => x.AssignedTo == assignedTo)
    .OrderByDescending(x => x.CheckedOutDate)
    .ToListAsync();
```

You can then map `x.Equipment.Name`, `x.Equipment.SerialNumber`, and the actual equipment category/type property to your response DTO. When returning equipment, update the corresponding `MyEquipment` record.

One important distinction: your existing checkout creation API must also create a `MyEquipment` record when a checkout is actually approved or completed. For drones, that should happen only after approval—not when the approval request is first submitted.

After running the migration, the table will exist, but it will initially be empty. Existing checkout records will not automatically move into it. If you want, we can next update your repository, service, and controller so the new table is fully connected to the assignment and return workflow.

25/10/2026, 15:05:12 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
interface DamageReport {
  id: number;
  date: string;
  equipment: string;
  serial: string;
  description: string;
  reportedBy: string;
  status: 'Pending' | 'Resolved';
  icon: string;
}

@Component({
  selector: 'app-damagereports',
  imports: [FormsModule],
  templateUrl: './damagereports.html',
  styleUrl: './damagereports.css',
})
export class Damagereports {
  searchText = '';
  selectedStatus = 'All Status';
  startDate = '';
  endDate = '';
  showReportForm = false;
  currentPage = 1;
  pageSize = 4;
  selectedReport: DamageReport | null = null;

  newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };

  reports: DamageReport[] = [
    {
      id: 1,
      date: '20/10/2026',
      equipment: 'GPS Rover - 01',
      serial: 'GPS-001',
      description: 'Antenna damaged',
      reportedBy: 'Ali Raza',
      status: 'Pending',
```

```
    icon: 'gps'
  },
  {
    id: 2,
    date: '18/10/2026',
    equipment: 'Drone - DJI Mavic',
    serial: 'DR-001',
    description: 'Propeller broken',
    reportedBy: 'Hamza',
    status: 'Resolved',
    icon: 'drone'
  },
  {
    id: 3,
    date: '15/10/2026',
    equipment: 'Field Tablet - 02',
    serial: 'FT-002',
    description: 'Screen cracked',
    reportedBy: 'Bilal',
    status: 'Pending',
    icon: 'tablet'
  },
  {
    id: 4,
    date: '12/10/2026',
    equipment: 'Tripod',
    serial: 'TR-001',
    description: 'Lock not working',
    reportedBy: 'Usman',
    status: 'Resolved',
    icon: 'tripod'
  }
];

get filteredReports(): DamageReport[] {
  const search = this.searchText.trim().toLowerCase();

  return this.reports.filter(report => {
    const matchesSearch =
      !search ||
      report.equipment.toLowerCase().includes(search) ||
      report.description.toLowerCase().includes(search) ||
      report.reportedBy.toLowerCase().includes(search) ||
      report.serial.toLowerCase().includes(search);

    const matchesStatus =
```

```
        this.selectedStatus === 'All Status' ||
        report.status === this.selectedStatus;

      const reportDate = this.toDate(report.date);

      const matchesStart =
        !this.startDate || reportDate >= this.startDate;

      const matchesEnd =
        !this.endDate || reportDate <= this.endDate;

      return matchesSearch && matchesStatus &&
        matchesStart && matchesEnd;
    });
  }

  get paginatedReports(): DamageReport[] {
    const start = (this.currentPage - 1) * this.pageSize;

    return this.filteredReports.slice(start, start + this.pageSize);
  }

  get totalPages(): number {
    return Math.max(1, Math.ceil(this.filteredReports.length /
this.pageSize));
  }

  get firstVisible(): number {
    return this.filteredReports.length === 0
      ? 0
      : (this.currentPage - 1) * this.pageSize + 1;
  }

  get lastVisible(): number {
    return Math.min(
      this.currentPage * this.pageSize,
      this.filteredReports.length
    );
  }

  private toDate(date: string): string {
    const [day, month, year] = date.split('/');
    return `${year}-${month}-${day}`;
  }

  applyFilters(): void {
```

```
  this.currentPage = 1;
}

resetFilters(): void {
  this.searchText = '';
  this.selectedStatus = 'All Status';
  this.startDate = '';
  this.endDate = '';
  this.currentPage = 1;
}

changePage(page: number): void {
  if (page >= 1 && page <= this.totalPages) {
    this.currentPage = page;
  }
}

openReportForm(): void {
  this.newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };
  this.showReportForm = true;
}

saveReport(): void {
  if (!this.newReport.equipment.trim() ||
      !this.newReport.description.trim()) {
    return;
  }

  const today = new Date();
  const formattedDate = [
    String(today.getDate()).padStart(2, '0'),
    String(today.getMonth() + 1).padStart(2, '0'),
    today.getFullYear()
  ].join('/');

  this.reports.unshift({
    id: Math.max(0, ...this.reports.map(r => r.id)) + 1,
    date: formattedDate,
    equipment: this.newReport.equipment.trim(),
    serial: 'N/A',
    description: this.newReport.description.trim(),
    reportedBy: this.newReport.reportedBy.trim() || 'Ahmad',
```

```
      status: 'Pending',
      icon: 'equipment'
    });

    this.showReportForm = false;
    this.resetFilters();
  }

  viewReport(report: DamageReport): void {
    this.selectedReport = report;
  }

  closeDetails(): void {
    this.selectedReport = null;
  }
}

<div class="damage-page">

  <!-- Page heading -->
  <header class="damage-header">
    <div class="heading-icon">!</div>

    <div class="heading-text">
      <h1>Damaged Reports</h1>
      <p>View and track reports of damaged equipment.</p>
    </div>

    <button
      type="button"
      class="report-button"
      (click)="openReportForm()"
    >
      <span class="plus">+</span>
      Report Damage
    </button>
  </header>

  <!-- Search and filters -->
  <section class="filter-bar">
    <div class="search-box">
      <span class="search-icon">⌕</span>
      <input
        type="search"
        name="search"
        placeholder="Search equipment..."
```

```
    [(ngModel)]="searchText"
    (ngModelChange)="applyFilters()"
  />
</div>

<select
  name="status"
  class="status-filter"
  [(ngModel)]="selectedStatus"
  (ngModelChange)="applyFilters()"
  aria-label="Filter by status"
>
  <option>All Status</option>
  <option>Pending</option>
  <option>Resolved</option>
</select>

<div class="date-range">
  <span class="calendar-icon">▦</span>
  <input
    type="date"
    aria-label="Start date"
    title="Start date"
    [(ngModel)]="startDate"
    (ngModelChange)="applyFilters()"
  />
  <span class="date-separator">–</span>
  <input
    type="date"
    aria-label="End date"
    title="End date"
    [(ngModel)]="endDate"
    (ngModelChange)="applyFilters()"
  />
</div>

<button
  type="button"
  class="filter-button"
  (click)="applyFilters()"
>
  Filter
</button>

<button
  type="button"
```

```
    class="reset-button"
    (click)="resetFilters()"
  >
    Reset
  </button>
</section>

<!-- Reports table -->
<section class="table-card">
  <div class="table-scroll">
    <table>
      <thead>
        <tr>
          <th>Date</th>
          <th>Equipment</th>
          <th>Description</th>
          <th>Reported By</th>
          <th>Status</th>
          <th class="actions-heading">Actions</th>
        </tr>
      </thead>

      <tbody>
        @for (report of paginatedReports; track report.id) {
          <tr>
            <td class="date-cell">{{ report.date }}</td>

            <td>
              <div class="equipment-cell">
                <div class="equipment-icon" [class]=" 'equipment-
icon ' + report.icon">

                  @if (report.icon === 'gps') {
                    <svg viewBox="0 0 48 48" aria-label="GPS
rover">
                      <ellipse cx="24" cy="10" rx="11" ry="5"
fill="#e5a72f"/>
                      <rect x="14" y="10" width="20" height="8"
rx="3" fill="#586a7a"/>
                      <rect x="19" y="18" width="10" height="15"
rx="3" fill="#c4ccd3"/>
                      <path d="M22 32 L22 43 M26 32 L26 43"
stroke="#687887" stroke-width="2"/>
                      <circle cx="24" cy="12" r="3" fill="#ffda67"/>
                    </svg>
                  } @else if (report.icon === 'drone') {
```

```
                <svg viewBox="0 0 48 48" aria-label="Drone">
                  <g stroke="#647383" stroke-width="2.5"
stroke-linecap="round" fill="none">
                    <path d="M24 23 L10 13 M24 23 L38 13 M24
23 L10 34 M24 23 L38 34"/>
                    <ellipse cx="10" cy="11" rx="8" ry="2.5"/>
                    <ellipse cx="38" cy="11" rx="8" ry="2.5"/>
                    <ellipse cx="10" cy="36" rx="8" ry="2.5"/>
                    <ellipse cx="38" cy="36" rx="8" ry="2.5"/>
                  </g>
                  <rect x="17" y="18" width="14" height="11"
rx="4" fill="#aeb9c5"/>
                  <circle cx="24" cy="23" r="3"
fill="#647383"/>
                </svg>
              } @else if (report.icon === 'tablet') {
                <svg viewBox="0 0 48 48" aria-label="Tablet">
                  <rect x="9" y="2" width="30" height="44"
rx="4" fill="#233c51"/>
                  <rect x="12" y="6" width="24" height="34"
rx="2" fill="#327ca5"/>
                  <path d="M13 28 L22 17 L28 24 L35 14 L35 39
L13 39 Z" fill="#70b2d0"/>
                  <circle cx="24" cy="43" r="1" fill="#fff"/>
                </svg>
              } @else if (report.icon === 'tripod') {
                <svg viewBox="0 0 48 48" aria-label="Tripod">
                  <rect x="19" y="4" width="10" height="6"
rx="1" fill="#273c4e"/>
                  <path d="M21 10 L10 44 M25 10 L25 44 M28
10 L39 44" stroke="#ce7a25" stroke-width="3" stroke-
linecap="round"/>
                  <path d="M18 17 L30 17" stroke="#4c5966"
stroke-width="2"/>
                </svg>
              } @else {
                <span>⚒</span>
              }

            </div>

            <span>{{ report.equipment }}</span>
          </div>
        </td>

        <td>{{ report.description }}</td>
```

```
            <td>{{ report.reportedBy }}</td>

            <td>
              <span
                class="status-badge"
                [class.pending]="report.status === 'Pending'"
                [class.resolved]="report.status === 'Resolved'"
              >
                {{ report.status }}
              </span>
            </td>

            <td class="actions-cell">
              <button
                type="button"
                class="view-button"
                title="View report details"
                aria-label="View report details"
                (click)="viewReport(report)"
              >
                <svg viewBox="0 0 24 24" width="19"
height="19"
                    fill="none" stroke="currentColor" stroke-
width="2">
                  <path d="M2 12s3.5-7 10-7 10 7 10 7-3.5 7-10
7S2 12 2 12Z"/>
                  <circle cx="12" cy="12" r="3"/>
                </svg>
              </button>
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="6" class="empty-cell">
              No damage reports match your filters.
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <!-- Pagination -->
  <footer class="table-footer">
    <span class="results-text">
      Showing {{ firstVisible }} to {{ lastVisible }}
```

```
        of {{ filteredReports.length }} reports
      </span>

      <div class="pagination">
        <button
          type="button"
          class="page-arrow"
          aria-label="Previous page"
          [disabled]="currentPage === 1"
          (click)="changePage(currentPage - 1)"
        >
          ‹
        </button>

        @for (page of [].constructor(totalPages); track $index) {
          <button
            type="button"
            class="page-number"
            [class.active]="currentPage === $index + 1"
            (click)="changePage($index + 1)"
          >
            {{ $index + 1 }}
          </button>
        }

        <button
          type="button"
          class="page-arrow"
          aria-label="Next page"
          [disabled]="currentPage === totalPages"
          (click)="changePage(currentPage + 1)"
        >
          ›
        </button>
      </div>
    </footer>
  </section>

  <!-- Report damage form -->
  @if (showReportForm) {
    <div class="modal-backdrop" (click)="showReportForm =
false">
      <section class="modal-card"
(click)="$event.stopPropagation()">
        <h2>Report Damage</h2>
        <p>Enter the details of the damaged equipment.</p>
```

```
      <form #damageForm="ngForm"
(ngSubmit)="saveReport()">
        <label for="equipmentName">Equipment name *
</label>
        <input
          id="equipmentName"
          name="equipmentName"
          [(ngModel)]="newReport.equipment"
          required
          placeholder="e.g. GPS Rover - 01"
        />

        <label for="damageDescription">Description *</label>
        <textarea
          id="damageDescription"
          name="damageDescription"
          [(ngModel)]="newReport.description"
          required
          rows="3"
          placeholder="Describe the damage..."
        ></textarea>

        <label for="reportedBy">Reported by</label>
        <input
          id="reportedBy"
          name="reportedBy"
          [(ngModel)]="newReport.reportedBy"
        />

        <div class="modal-actions">
          <button
            type="button"
            class="cancel-button"
            (click)="showReportForm = false"
          >
            Cancel
          </button>

          <button
            type="submit"
            class="report-button"
            [disabled]="damageForm.invalid"
          >
            Submit Report
          </button>
```

```
        </div>
      </form>
    </section>
  </div>
 }

 <!-- Report details -->
 @if (selectedReport; as report) {
  <div class="modal-backdrop" (click)="closeDetails()">
   <section class="modal-card details-card"
(click)="$event.stopPropagation()">
     <div class="details-heading">
      <h2>Damage Report Details</h2>
      <button type="button" class="close-button"
(click)="closeDetails()">×</button>
     </div>

     <p><strong>Equipment:</strong> {{ report.equipment
}}</p>
     <p><strong>Serial number:</strong> {{ report.serial
}}</p>
     <p><strong>Date:</strong> {{ report.date }}</p>
     <p><strong>Description:</strong> {{
report.description }}</p>
     <p><strong>Reported by:</strong> {{
report.reportedBy }}</p>
     <p><strong>Status:</strong> {{ report.status }}</p>
   </section>
  </div>
 }

</div>
```

pleaese implement the apis of the above code only dotnet backend ??

25/10/2026, 15:26:22

For your **Damage Reports** page, we'll implement only the .NET backend, following the same **Entity → DTO → Repository → Service → Controller** architecture used for your previous APIs.

The backend will support:

- `GET /api/DamageReports` — retrieve reports, with optional search, status, and date filters.
- `GET /api/DamageReports/{id}` — retrieve a report's details.

- `POST /api/DamageReports` — submit a new damage report.
- `PUT /api/DamageReports/{id}/resolve` — mark a report as resolved.

We'll store reports in SQL Server using EF Core. Your Angular filtering, pagination, modals, and UI can remain unchanged for now.

1. Create the entity

Create `Models/DamageReport.cs`.

```
namespace SolvefyTask.Models
{
    public class DamageReport
    {
        public int Id { get; set; }

        public DateTime Date { get; set; } = DateTime.UtcNow;

        public string Equipment { get; set; } = string.Empty;

        public string Serial { get; set; } = "N/A";

        public string Description { get; set; } = string.Empty;

        public string ReportedBy { get; set; } = string.Empty;

        public string Status { get; set; } = "Pending";
    }
}
```

This entity stores the fields shown in your Angular page. We'll use the equipment name and serial number directly in the damage report, so the API can support your current form without requiring changes to the existing equipment table.

2. Create the DTOs

Create `DTOs/DamageReportDto.cs`.

```
namespace SolvefyTask.DTOs
{
    public class DamageReportDto
    {
        public int Id { get; set; }
        public DateTime Date { get; set; }
        public string Equipment { get; set; } = string.Empty;
        public string Serial { get; set; } = "N/A";
```

```
        public string Description { get; set; } = string.Empty;
        public string ReportedBy { get; set; } = string.Empty;
        public string Status { get; set; } = "Pending";
    }

    public class CreateDamageReportDto
    {
        public string Equipment { get; set; } = string.Empty;
        public string? Serial { get; set; }
        public string Description { get; set; } = string.Empty;
        public string ReportedBy { get; set; } = string.Empty;
    }
}
```

The creation DTO matches the information submitted by your form. The backend will automatically set the report date, default serial number, and initial `Pending` status.

3. Add the DbSet

In `Data/ApplicationDbContext.cs`, add:

```
public DbSet<DamageReport> DamageReports { get; set; }
```

Add this using statement if needed:

```
using SolvefyTask.Models;
```

4. Create the repository layer

`Repositories/IDamageReportRepository.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface IDamageReportRepository
    {
        Task<List<DamageReport>> GetAllAsync(
            string? search,
            string? status,
            DateTime? startDate,
            DateTime? endDate);

        Task<DamageReport?> GetByIdAsync(int id);

        Task<DamageReport> CreateAsync(DamageReport report);
```

```
        Task<bool> ResolveAsync(int id);
    }
}
```

`Repositories/DamageReportRepository.cs`

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class DamageReportRepository : IDamageReportRepository
    {
        private readonly ApplicationDbContext _context;

        public DamageReportRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<DamageReport>> GetAllAsync(
            string? search,
            string? status,
            DateTime? startDate,
            DateTime? endDate)
        {
            IQueryable<DamageReport> query =
                _context.DamageReports.AsNoTracking();

            if (!string.IsNullOrWhiteSpace(search))
            {
                var term = search.Trim().ToLower();

                query = query.Where(x =>
                    x.Equipment.ToLower().Contains(term) ||
                    x.Serial.ToLower().Contains(term) ||
                    x.Description.ToLower().Contains(term) ||
                    x.ReportedBy.ToLower().Contains(term));
            }

            if (!string.IsNullOrWhiteSpace(status) &&
                !status.Equals("All Status",
                    StringComparison.OrdinalIgnoreCase))
            {
```

```
            query = query.Where(x => x.Status == status);
        }

        if (startDate.HasValue)
        {
            var start = startDate.Value.Date;
            query = query.Where(x => x.Date >= start);
        }

        if (endDate.HasValue)
        {
            var endExclusive = endDate.Value.Date.AddDays(1);
            query = query.Where(x => x.Date < endExclusive);
        }

        return await query
            .OrderByDescending(x => x.Date)
            .ToListAsync();
    }

    public async Task<DamageReport?> GetByIdAsync(int id)
    {
        return await _context.DamageReports
            .AsNoTracking()
            .FirstOrDefaultAsync(x => x.Id == id);
    }

    public async Task<DamageReport> CreateAsync(DamageReport report)
    {
        _context.DamageReports.Add(report);
        await _context.SaveChangesAsync();

        return report;
    }

    public async Task<bool> ResolveAsync(int id)
    {
        var report = await _context.DamageReports
            .FirstOrDefaultAsync(x => x.Id == id);

        if (report == null)
        {
            return false;
        }

        report.Status = "Resolved";
```

```
            await _context.SaveChangesAsync();

            return true;
        }
    }
}
```

The repository applies search, status, and date filters in SQL Server and returns the newest reports first.

5. Create the service layer

`Services/IDamageReportService.cs`

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Services
{
    public interface IDamageReportService
    {
        Task<List<DamageReportDto>> GetAllAsync(
            string? search,
            string? status,
            DateTime? startDate,
            DateTime? endDate);

        Task<DamageReportDto?> GetByIdAsync(int id);

        Task<DamageReportDto> CreateAsync(
            CreateDamageReportDto request);

        Task<(bool Success, string Message)> ResolveAsync(int id);
    }
}
```

`Services/DamageReportService.cs`

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class DamageReportService : IDamageReportService
    {
```

```
    private readonly IDamageReportRepository _repository;

    public DamageReportService(IDamageReportRepository repository)
    {
        _repository = repository;
    }

    public async Task<List<DamageReportDto>> GetAllAsync(
        string? search,
        string? status,
        DateTime? startDate,
        DateTime? endDate)
    {
        if (startDate.HasValue &&
            endDate.HasValue &&
            startDate.Value.Date > endDate.Value.Date)
        {
            throw new ArgumentException(
                "Start date cannot be after end date.");
        }

        var reports = await _repository.GetAllAsync(
            search, status, startDate, endDate);

        return reports.Select(MapToDto).ToList();
    }

    public async Task<DamageReportDto?> GetByIdAsync(int id)
    {
        if (id <= 0)
        {
            throw new ArgumentException(
                "A valid damage report ID is required.");
        }

        var report = await _repository.GetByIdAsync(id);

        return report == null ? null : MapToDto(report);
    }

    public async Task<DamageReportDto> CreateAsync(
        CreateDamageReportDto request)
    {
        if (string.IsNullOrWhiteSpace(request.Equipment))
        {
            throw new ArgumentException(
```

```
                "Equipment name is required.");
        }

        if (string.IsNullOrWhiteSpace(request.Description))
        {
            throw new ArgumentException(
                "Damage description is required.");
        }

        if (string.IsNullOrWhiteSpace(request.ReportedBy))
        {
            throw new ArgumentException(
                "Reported by is required.");
        }

        var report = new DamageReport
        {
            Equipment = request.Equipment.Trim(),
            Serial = string.IsNullOrWhiteSpace(request.Serial)
                ? "N/A"
                : request.Serial.Trim(),
            Description = request.Description.Trim(),
            ReportedBy = request.ReportedBy.Trim(),
            Date = DateTime.UtcNow,
            Status = "Pending"
        };

        var created = await _repository.CreateAsync(report);

        return MapToDto(created);
    }

    public async Task<(bool Success, string Message)> ResolveAsync(
        int id)
    {
        if (id <= 0)
        {
            return (false, "A valid damage report ID is required.");
        }

        var report = await _repository.GetByIdAsync(id);

        if (report == null)
        {
            return (false, "Damage report not found.");
        }
```

```
            if (report.Status == "Resolved")
            {
                return (false, "This damage report is already resolved.");
            }

            var resolved = await _repository.ResolveAsync(id);

            return resolved
                ? (true, "Damage report resolved successfully.")
                : (false, "Unable to resolve the damage report.");
        }

        private static DamageReportDto MapToDto(DamageReport report)
        {
            return new DamageReportDto
            {
                Id = report.Id,
                Date = report.Date,
                Equipment = report.Equipment,
                Serial = report.Serial,
                Description = report.Description,
                ReportedBy = report.ReportedBy,
                Status = report.Status
            };
        }
    }
}
```

6. Create the controller

Create `Controllers/DamageReportsController.cs`.

This exposes the API endpoints for listing, filtering, creating, viewing, and resolving damage reports.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.DTOs;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class DamageReportsController : ControllerBase
    {
        private readonly IDamageReportService _service;
```

```
public DamageReportsController(IDamageReportService service)
{
    _service = service;
}

// GET: api/DamageReports
// Optional filters: search, status, startDate, endDate
[HttpGet]
public async Task<IActionResult> GetAll(
    [FromQuery] string? search,
    [FromQuery] string? status,
    [FromQuery] DateTime? startDate,
    [FromQuery] DateTime? endDate)
{
    try
    {
        var reports = await _service.GetAllAsync(
            search, status, startDate, endDate);

        return Ok(reports);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// GET: api/DamageReports/1
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(int id)
{
    try
    {
        var report = await _service.GetByIdAsync(id);

        if (report == null)
        {
            return NotFound(new
            {
                message = "Damage report not found."
            });
        }

        return Ok(report);
    }
```

```
        catch (ArgumentException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    // POST: api/DamageReports
    [HttpPost]
    public async Task<IActionResult> Create(
        [FromBody] CreateDamageReportDto request)
    {
        try
        {
            var report = await _service.CreateAsync(request);

            return CreatedAtAction(
                nameof(GetById),
                new { id = report.Id },
                report);
        }
        catch (ArgumentException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    // PUT: api/DamageReports/1/resolve
    [HttpPut("{id:int}/resolve")]
    public async Task<IActionResult> Resolve(int id)
    {
        var result = await _service.ResolveAsync(id);

        if (!result.Success)
        {
            if (result.Message == "Damage report not found.")
            {
                return NotFound(new { message = result.Message });
            }

            return BadRequest(new { message = result.Message });
        }

        return Ok(new
        {
            id,
            status = "Resolved",
```

```
                message = result.Message
            });
        }
    }
}
```

7. Register the repository and service in `Program.cs`

Add these registrations alongside your existing ones:

```
builder.Services.AddScoped<
    IDamageReportRepository, DamageReportRepository>();

builder.Services.AddScoped<
    IDamageReportService, DamageReportService>();
```

Add the namespaces if necessary:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

8. Create the SQL Server table with EF Core migrations

Run these commands from your backend project directory.

Package Manager Console:

```
Add-Migration CreateDamageReportsTable
Update-Database
```

Or .NET CLI:

```
dotnet ef migrations add CreateDamageReportsTable
dotnet ef database update
```

This creates the `DamageReports` table through EF Core. You don't need a separate SQL `CREATE TABLE` statement.

9. Test the APIs in Swagger

GET **Get all damage reports**

`/api/DamageReports`

GET **Filter reports**

`/api/DamageReports?status=Pending&search=GPS&startDate=2026-10-01&endDate=2026-10-31`

POST **Create a report**

`/api/DamageReports`

GET **View report details**

`/api/DamageReports/1`

PUT **Resolve a report**

`/api/DamageReports/1/resolve`

Example POST request:

```
{
  "equipment": "GPS Rover - 01",
  "serial": "GPS-001",
  "description": "Antenna damaged",
  "reportedBy": "Ahmad"
}
```

The API automatically records the date and sets the status to `Pending`.

One important detail: this backend is ready for your existing form's data structure, but it does not yet verify that the reported equipment exists in the `Equipment` table. Also, your current Angular code will continue showing its hardcoded reports until you connect it to these endpoints.

25/10/2026, 15:26:47 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
interface DamageReport {
  id: number;
  date: string;
  equipment: string;
  serial: string;
  description: string;
  reportedBy: string;
  status: 'Pending' | 'Resolved';
  icon: string;
}

@Component({
  selector: 'app-damagereports',
  imports: [FormsModule],
  templateUrl: './damagereports.html',
  styleUrl: './damagereports.css',
})
export class Damagereports {
  searchText = '';
  selectedStatus = 'All Status';
  startDate = '';
  endDate = '';
  showReportForm = false;
  currentPage = 1;
  pageSize = 4;
  selectedReport: DamageReport | null = null;

  newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };

  reports: DamageReport[] = [
    {
      id: 1,
      date: '20/10/2026',
      equipment: 'GPS Rover - 01',
      serial: 'GPS-001',
      description: 'Antenna damaged',
      reportedBy: 'Ali Raza',
      status: 'Pending',
```

```
    icon: 'gps'
  },
  {
    id: 2,
    date: '18/10/2026',
    equipment: 'Drone - DJI Mavic',
    serial: 'DR-001',
    description: 'Propeller broken',
    reportedBy: 'Hamza',
    status: 'Resolved',
    icon: 'drone'
  },
  {
    id: 3,
    date: '15/10/2026',
    equipment: 'Field Tablet - 02',
    serial: 'FT-002',
    description: 'Screen cracked',
    reportedBy: 'Bilal',
    status: 'Pending',
    icon: 'tablet'
  },
  {
    id: 4,
    date: '12/10/2026',
    equipment: 'Tripod',
    serial: 'TR-001',
    description: 'Lock not working',
    reportedBy: 'Usman',
    status: 'Resolved',
    icon: 'tripod'
  }
];

get filteredReports(): DamageReport[] {
  const search = this.searchText.trim().toLowerCase();

  return this.reports.filter(report => {
    const matchesSearch =
      !search ||
      report.equipment.toLowerCase().includes(search) ||
      report.description.toLowerCase().includes(search) ||
      report.reportedBy.toLowerCase().includes(search) ||
      report.serial.toLowerCase().includes(search);

    const matchesStatus =
```

```
        this.selectedStatus === 'All Status' ||
        report.status === this.selectedStatus;

      const reportDate = this.toDate(report.date);

      const matchesStart =
        !this.startDate || reportDate >= this.startDate;

      const matchesEnd =
        !this.endDate || reportDate <= this.endDate;

      return matchesSearch && matchesStatus &&
        matchesStart && matchesEnd;
    });
  }

  get paginatedReports(): DamageReport[] {
    const start = (this.currentPage - 1) * this.pageSize;

    return this.filteredReports.slice(start, start + this.pageSize);
  }

  get totalPages(): number {
    return Math.max(1, Math.ceil(this.filteredReports.length /
this.pageSize));
  }

  get firstVisible(): number {
    return this.filteredReports.length === 0
      ? 0
      : (this.currentPage - 1) * this.pageSize + 1;
  }

  get lastVisible(): number {
    return Math.min(
      this.currentPage * this.pageSize,
      this.filteredReports.length
    );
  }

  private toDate(date: string): string {
    const [day, month, year] = date.split('/');
    return `${year}-${month}-${day}`;
  }

  applyFilters(): void {
```

```
  this.currentPage = 1;
}

resetFilters(): void {
  this.searchText = '';
  this.selectedStatus = 'All Status';
  this.startDate = '';
  this.endDate = '';
  this.currentPage = 1;
}

changePage(page: number): void {
  if (page >= 1 && page <= this.totalPages) {
    this.currentPage = page;
  }
}

openReportForm(): void {
  this.newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };
  this.showReportForm = true;
}

saveReport(): void {
  if (!this.newReport.equipment.trim() ||
      !this.newReport.description.trim()) {
    return;
  }

  const today = new Date();
  const formattedDate = [
    String(today.getDate()).padStart(2, '0'),
    String(today.getMonth() + 1).padStart(2, '0'),
    today.getFullYear()
  ].join('/');

  this.reports.unshift({
    id: Math.max(0, ...this.reports.map(r => r.id)) + 1,
    date: formattedDate,
    equipment: this.newReport.equipment.trim(),
    serial: 'N/A',
    description: this.newReport.description.trim(),
    reportedBy: this.newReport.reportedBy.trim() || 'Ahmad',
```

```
      status: 'Pending',
      icon: 'equipment'
    });

    this.showReportForm = false;
    this.resetFilters();
  }

  viewReport(report: DamageReport): void {
    this.selectedReport = report;
  }

  closeDetails(): void {
    this.selectedReport = null;
  }
}
```

please implement the above apis of damagereport into this typescript code ??

25/10/2026, 15:39:24

We'll now connect your existing `Damagereports` TypeScript component to the .NET `DamageReports` APIs we created earlier. Your HTML and CSS can stay the same.

The updated component will load reports from SQL Server, apply search/status/date filters through the API, submit new reports, and retrieve report details. It will also preserve your existing pagination, equipment icons, and modal behavior.

1. Create the Angular model

Create `src/app/models/damage-report.ts`:

```ts
export interface DamageReport {
  id: number;
  date: string;
  equipment: string;
  serial: string;
  description: string;
  reportedBy: string;
  status: 'Pending' | 'Resolved';
  icon: string;
}

export interface CreateDamageReportRequest {
  equipment: string;
  serial?: string;
```

```
  description: string;
  reportedBy: string;
}
```

2. Create the API service

Create `src/app/services/damage-report.ts`:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import {
  DamageReport,
  CreateDamageReportRequest
} from '../models/damage-report';

@Injectable({
  providedIn: 'root'
})
export class DamageReportService {
  private http = inject(HttpClient);
  private apiUrl = `${environment.apiUrl}/DamageReports`;

  getAll(filters?: {
    search?: string;
    status?: string;
    startDate?: string;
    endDate?: string;
  }): Observable<DamageReport[]> {
    let params = new HttpParams();

    if (filters?.search?.trim()) {
      params = params.set('search', filters.search.trim());
    }

    if (filters?.status && filters.status !== 'All Status') {
      params = params.set('status', filters.status);
    }

    if (filters?.startDate) {
      params = params.set('startDate', filters.startDate);
    }

    if (filters?.endDate) {
```

```
      params = params.set('endDate', filters.endDate);
    }

    return this.http.get<DamageReport[]>(this.apiUrl, { params });
  }

  getById(id: number): Observable<DamageReport> {
    return this.http.get<DamageReport>(`${this.apiUrl}/${id}`);
  }

  create(
    request: CreateDamageReportRequest
  ): Observable<DamageReport> {
    return this.http.post<DamageReport>(this.apiUrl, request);
  }

  resolve(id: number): Observable<unknown> {
    return this.http.put(`${this.apiUrl}/${id}/resolve`, {});
  }
}
```

3. Replace `damagereports.ts`

This version keeps your existing property names and template bindings. The API response is mapped into your current UI model, including the equipment icon.

```
import {
  Component,
  OnInit,
  inject,
  ChangeDetectorRef
} from '@angular/core';
import { FormsModule } from '@angular/forms';
import { finalize } from 'rxjs';

import { DamageReportService } from '../../services/damage-report';
import {
  DamageReport as DamageReportApi,
  CreateDamageReportRequest
} from '../../models/damage-report';

interface DamageReport {
  id: number;
  date: string;
  equipment: string;
  serial: string;
```

```
  description: string;
  reportedBy: string;
  status: 'Pending' | 'Resolved';
  icon: string;
}

@Component({
  selector: 'app-damagereports',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './damagereports.html',
  styleUrl: './damagereports.css',
})
export class Damagereports implements OnInit {
  private damageReportService = inject(DamageReportService);
  private cdr = inject(ChangeDetectorRef);

  searchText = '';
  selectedStatus = 'All Status';
  startDate = '';
  endDate = '';

  showReportForm = false;
  currentPage = 1;
  pageSize = 4;

  selectedReport: DamageReport | null = null;

  loading = false;
  saving = false;
  errorMessage = '';
  successMessage = '';

  newReport = {
    equipment: '',
    description: '',
    reportedBy: 'Ahmad'
  };

  reports: DamageReport[] = [];

  ngOnInit(): void {
    this.loadReports();
  }

  // GET: api/DamageReports
```

```
  loadReports(): void {
    this.loading = true;
    this.errorMessage = '';

    this.damageReportService.getAll({
      search: this.searchText,
      status: this.selectedStatus,
      startDate: this.startDate,
      endDate: this.endDate
    })
    .pipe(
      finalize(() => {
        this.loading = false;
        this.cdr.detectChanges();
      })
    )
    .subscribe({
      next: records => {
        this.reports = records.map(record => this.mapReport(record));

        this.currentPage = Math.min(
          this.currentPage,
          this.totalPages
        );
      },
      error: error => {
        console.error('Failed to load damage reports:', error);
        this.errorMessage =
          error?.error?.message ??
          'Unable to load damage reports. Please try again.';
      }
    });
  }

  // API filtering is server-side.
  get filteredReports(): DamageReport[] {
    return this.reports;
  }

  get paginatedReports(): DamageReport[] {
    const start = (this.currentPage - 1) * this.pageSize;
    return this.filteredReports.slice(start, start + this.pageSize);
  }

  get totalPages(): number {
    return Math.max(
```

```
      1,
      Math.ceil(this.filteredReports.length / this.pageSize)
    );
  }

  get firstVisible(): number {
    return this.filteredReports.length === 0
      ? 0
      : (this.currentPage - 1) * this.pageSize + 1;
  }

  get lastVisible(): number {
    return Math.min(
      this.currentPage * this.pageSize,
      this.filteredReports.length
    );
  }

  applyFilters(): void {
    this.currentPage = 1;
    this.loadReports();
  }

  resetFilters(): void {
    this.searchText = '';
    this.selectedStatus = 'All Status';
    this.startDate = '';
    this.endDate = '';
    this.currentPage = 1;

    this.loadReports();
  }

  changePage(page: number): void {
    if (page >= 1 && page <= this.totalPages) {
      this.currentPage = page;
    }
  }

  openReportForm(): void {
    this.newReport = {
      equipment: '',
      description: '',
      reportedBy: 'Ahmad'
    };
```

```
    this.errorMessage = '';
    this.successMessage = '';
    this.showReportForm = true;
  }

  // POST: api/DamageReports
  saveReport(): void {
    if (
      !this.newReport.equipment.trim() ||
      !this.newReport.description.trim() ||
      !this.newReport.reportedBy.trim() ||
      this.saving
    ) {
      return;
    }

    const request: CreateDamageReportRequest = {
      equipment: this.newReport.equipment.trim(),
      description: this.newReport.description.trim(),
      reportedBy: this.newReport.reportedBy.trim(),
      serial: 'N/A'
    };

    this.saving = true;
    this.errorMessage = '';
    this.successMessage = '';

    this.damageReportService.create(request)
      .pipe(
        finalize(() => {
          this.saving = false;
          this.cdr.detectChanges();
        })
      )
      .subscribe({
        next: created => {
          this.showReportForm = false;
          this.successMessage = 'Damage report submitted successfully.';

          // Clear filters so the newly created report is visible.
          this.searchText = '';
          this.selectedStatus = 'All Status';
          this.startDate = '';
          this.endDate = '';
          this.currentPage = 1;
```

```
        // Use the server response, then refresh the list.
        const mapped = this.mapReport(created);

        this.reports = [
          mapped,
          ...this.reports.filter(r => r.id !== mapped.id)
        ];

        this.loadReports();
      },
      error: error => {
        console.error('Failed to create damage report:', error);
        this.errorMessage =
          error?.error?.message ??
          'Unable to submit the damage report. Please try again.';
      }
    });
}

// GET: api/DamageReports/{id}
viewReport(report: DamageReport): void {
  this.errorMessage = '';

  this.damageReportService.getById(report.id).subscribe({
    next: record => {
      this.selectedReport = this.mapReport(record);
    },
    error: error => {
      console.error('Failed to load damage report details:', error);
      this.errorMessage =
        error?.error?.message ??
        'Unable to load report details.';
    }
  });
}

closeDetails(): void {
  this.selectedReport = null;
}

private mapReport(record: DamageReportApi): DamageReport {
  return {
    id: record.id,
    date: this.formatDate(record.date),
    equipment: record.equipment,
    serial: record.serial || 'N/A',
```

```
      description: record.description,
      reportedBy: record.reportedBy,
      status: record.status === 'Resolved' ? 'Resolved' : 'Pending',
      icon: this.getIcon(record.equipment)
    };
  }

  private formatDate(value: string): string {
    if (!value) {
      return '';
    }

    const date = new Date(value);

    if (Number.isNaN(date.getTime())) {
      return value;
    }

    const day = String(date.getDate()).padStart(2, '0');
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const year = date.getFullYear();

    return `${day}/${month}/${year}`;
  }

  private getIcon(equipment: string): string {
    const value = equipment.toLowerCase();

    if (value.includes('gps') || value.includes('rover')) {
      return 'gps';
    }

    if (value.includes('drone')) {
      return 'drone';
    }

    if (value.includes('tablet')) {
      return 'tablet';
    }

    if (value.includes('tripod')) {
      return 'tripod';
    }

    return 'equipment';
```

```
  }
}
```

4. Two small checks

A. Confirm the service import paths.

The code assumes your component is in `src/app/components/damagereports/`, with services and models in `src/app/services/` and `src/app/models/`. If your folders differ, adjust the relative imports.

B. Confirm the API base URL.

Your `environment.ts` should contain:

```
export const environment = {
  apiUrl: 'https://localhost:7135/api'
};
```

Also ensure `provideHttpClient()` is configured in `app.config.ts`.

5. How the component now works

User action	API called
Open Damage Reports	`GET /api/DamageReports`
Search equipment or description	`GET /api/DamageReports?search=...`
Filter by status	`GET /api/DamageReports?status=Pending`
Select date range	`GET /api/DamageReports?startDate=...&endDate=...`
Submit a report	`POST /api/DamageReports`
View report details	`GET /api/DamageReports/{id}`

Your existing pagination remains client-side, while filtering happens in SQL Server through the backend.

Important: Your current form sends the equipment name as text and uses `"N/A"` for the serial number. The backend we created accepts that structure. If you want users to select actual equipment from your existing equipment table, we can later change the form and backend to use `EquipmentId` instead.

25/10/2026, 15:39:38 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
interface EquipmentHistoryRecord {
  id: number;
  date: string;
  equipment: string;
  action: 'Returned' | 'Checked Out';
  person: string;
  remarks: string;
}
@Component({
  selector: 'app-reports',
  imports: [FormsModule],
  templateUrl: './reports.html',
  styleUrl: './reports.css',
})
export class Reports {

  startDate = '2026-09-01';
  endDate = '2026-09-30';

  appliedStartDate = this.startDate;
  appliedEndDate = this.endDate;

  history: EquipmentHistoryRecord[] = [
    {
      id: 1,
      date: '28/09/2026',
      equipment: 'Total Station - 02',
      action: 'Returned',
      person: 'Usman',
      remarks: ''
    },
    {
      id: 2,
      date: '25/09/2026',
      equipment: 'GPS Rover - 01',
      action: 'Checked Out',
      person: 'Ali Raza',
      remarks: 'Site work'
    },
    {
      id: 3,
      date: '20/09/2026',
```

```
    equipment: 'Field Tablet',
    action: 'Returned',
    person: 'Bilal',
    remarks: ''
  },
  {
    id: 4,
    date: '18/09/2026',
    equipment: 'Drone - DJI Mavic',
    action: 'Checked Out',
    person: 'Hamza',
    remarks: 'Aerial survey'
  },
  {
    id: 5,
    date: '15/09/2026',
    equipment: 'GPS Rover - 02',
    action: 'Returned',
    person: 'Asif',
    remarks: ''
  }
];

get filteredHistory(): EquipmentHistoryRecord[] {
  return this.history.filter(record => {
    const recordDate = this.toISODate(record.date);

    const matchesStart =
      !this.appliedStartDate ||
      recordDate >= this.appliedStartDate;

    const matchesEnd =
      !this.appliedEndDate ||
      recordDate <= this.appliedEndDate;

    return matchesStart && matchesEnd;
  });
}

private toISODate(date: string): string {
  const [day, month, year] = date.split('/');
  return `${year}-${month}-${day}`;
}

applyFilter(): void {
  if (
```

```
      this.startDate &&
      this.endDate &&
      this.startDate > this.endDate
    ) {
      alert('Start date cannot be later than end date.');
      return;
    }

    this.appliedStartDate = this.startDate;
    this.appliedEndDate = this.endDate;
  }

  resetFilter(): void {
    this.startDate = '';
    this.endDate = '';
    this.appliedStartDate = '';
    this.appliedEndDate = '';
  }
}



<div class="history-page">

  <!-- Page banner -->
  <div class="page-banner">
    7. Reports / History
  </div>

  <section class="history-card">

    <!-- Heading -->
    <header class="history-header">
      <div>
        <h1>Equipment History</h1>
        <p>View past check-outs, returns and activity</p>
      </div>
    </header>

    <!-- Date filters -->
    <form class="filter-bar" (ngSubmit)="applyFilter()">

      <div class="date-control">
        <label for="startDate" class="visually-hidden">
          Start date
        </label>
```

```
    <input
      id="startDate"
      type="date"
      name="startDate"
      [(ngModel)]="startDate"
    />
  </div>

  <span class="date-separator">to</span>

  <div class="date-control">
    <label for="endDate" class="visually-hidden">
      End date
    </label>

    <input
      id="endDate"
      type="date"
      name="endDate"
      [(ngModel)]="endDate"
    />
  </div>

  <button type="submit" class="filter-button">
    Filter
  </button>

  <button
    type="button"
    class="reset-button"
    (click)="resetFilter()"
  >
    Reset
  </button>

</form>

<!-- History table -->
<div class="table-scroll">
  <table>
    <thead>
      <tr>
        <th>Date</th>
        <th>Equipment</th>
        <th>Action</th>
```

```
          <th>Person</th>
          <th>Remarks</th>
        </tr>
      </thead>

      <tbody>
        @for (record of filteredHistory; track record.id) {
          <tr>
            <td class="date-cell">{{ record.date }}</td>

            <td class="equipment-cell">
              {{ record.equipment }}
            </td>

            <td>
              <span
                class="action-badge"
                [class.returned]="record.action === 'Returned'"
                [class.checked-out]="record.action === 'Checked
Out'"
              >
                {{ record.action }}
              </span>
            </td>

            <td>{{ record.person }}</td>

            <td class="remarks-cell">
              {{ record.remarks || '—' }}
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="5" class="empty-state">
              No equipment history found for the selected dates.
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <!-- Footer -->
  <footer class="history-footer">
    <span>
      Showing {{ filteredHistory.length }}
```

```
        of {{ history.length }} records
      </span>

      <button
        type="button"
        class="reset-link"
        (click)="resetFilter()"
      >
        Clear filters
      </button>
    </footer>

  </section>
</div>
```

please implement apis of these only backend dotnet ??

25/10/2026, 16:21:07

For your **Reports / History** page, we'll implement only the .NET backend using your existing **Entity → DTO → Repository → Service → Controller** architecture.

The API will retrieve equipment activity records, including checkouts and returns, and filter them by a date range. The history will come from actual database records rather than the five hardcoded records in your Angular component.

We'll use these endpoints:

Method	Endpoint	Purpose
`GET`	`/api/EquipmentHistory`	Retrieve all history records
`GET`	`/api/EquipmentHistory?startDate=2026-09-01&endDate=2026-09-30`	Filter history by dates
`GET`	`/api/EquipmentHistory/{id}`	Retrieve one history record

Because history should preserve past actions, the backend should record a checkout or return event when that action occurs, rather than reconstructing every event from the equipment's current status.

1. Create the entity

Create `Models/EquipmentHistory.cs`.

```
namespace SolvefyTask.Models
{
    public class EquipmentHistory
    {
        public int Id { get; set; }

        public DateTime Date { get; set; } = DateTime.UtcNow;

        public string Equipment { get; set; } = string.Empty;

        // Allowed values: Checked Out, Returned
        public string Action { get; set; } = string.Empty;

        public string Person { get; set; } = string.Empty;

        public string Remarks { get; set; } = string.Empty;
    }
}
```

This stores a snapshot of each activity, so the history remains available even after equipment is returned.

2. Create the DTO

Create `DTOs/EquipmentHistoryDto.cs`.

```
namespace SolvefyTask.DTOs
{
    public class EquipmentHistoryDto
    {
        public int Id { get; set; }
        public DateTime Date { get; set; }
        public string Equipment { get; set; } = string.Empty;
        public string Action { get; set; } = string.Empty;
        public string Person { get; set; } = string.Empty;
        public string Remarks { get; set; } = string.Empty;
    }
}
```

3. Add the DbSet to `ApplicationDbContext`

Open `Data/ApplicationDbContext.cs` and add:

```
public DbSet<EquipmentHistory> EquipmentHistories { get; set; }
```

Add the namespace if necessary:

```
using SolvefyTask.Models;
```

4. Create the repository layer

`Repositories/IEquipmentHistoryRepository.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface IEquipmentHistoryRepository
    {
        Task<List<EquipmentHistory>> GetAllAsync(
            DateTime? startDate,
            DateTime? endDate);

        Task<EquipmentHistory?> GetByIdAsync(int id);

        Task<EquipmentHistory> CreateAsync(
            EquipmentHistory history);
    }
}
```

`Repositories/EquipmentHistoryRepository.cs`

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class EquipmentHistoryRepository
        : IEquipmentHistoryRepository
    {
        private readonly ApplicationDbContext _context;

        public EquipmentHistoryRepository(
            ApplicationDbContext context)
        {
            _context = context;
        }
```

```
        public async Task<List<EquipmentHistory>> GetAllAsync(
            DateTime? startDate,
            DateTime? endDate)
        {
            IQueryable<EquipmentHistory> query =
                _context.EquipmentHistories.AsNoTracking();

            if (startDate.HasValue)
            {
                var start = startDate.Value.Date;
                query = query.Where(x => x.Date >= start);
            }

            if (endDate.HasValue)
            {
                var endExclusive = endDate.Value.Date.AddDays(1);
                query = query.Where(x => x.Date < endExclusive);
            }

            return await query
                .OrderByDescending(x => x.Date)
                .ToListAsync();
        }

        public async Task<EquipmentHistory?> GetByIdAsync(int id)
        {
            return await _context.EquipmentHistories
                .AsNoTracking()
                .FirstOrDefaultAsync(x => x.Id == id);
        }

        public async Task<EquipmentHistory> CreateAsync(
            EquipmentHistory history)
        {
            _context.EquipmentHistories.Add(history);

            await _context.SaveChangesAsync();

            return history;
        }
    }
}
```

The `CreateAsync` method lets the checkout and return workflows record history events when those actions happen.

5. Create the service layer

`Services/IEquipmentHistoryService.cs`

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Services
{
    public interface IEquipmentHistoryService
    {
        Task<List<EquipmentHistoryDto>> GetAllAsync(
            DateTime? startDate,
            DateTime? endDate);

        Task<EquipmentHistoryDto?> GetByIdAsync(int id);
    }
}
```

`Services/EquipmentHistoryService.cs`

```
using SolvefyTask.DTOs;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class EquipmentHistoryService
        : IEquipmentHistoryService
    {
        private readonly IEquipmentHistoryRepository _repository;

        public EquipmentHistoryService(
            IEquipmentHistoryRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<EquipmentHistoryDto>> GetAllAsync(
            DateTime? startDate,
            DateTime? endDate)
        {
            if (startDate.HasValue &&
                endDate.HasValue &&
                startDate.Value.Date > endDate.Value.Date)
            {
                throw new ArgumentException(
                    "Start date cannot be later than end date.");
```

```
            }

            var records = await _repository.GetAllAsync(
                startDate, endDate);

            return records.Select(x => new EquipmentHistoryDto
            {
                Id = x.Id,
                Date = x.Date,
                Equipment = x.Equipment,
                Action = x.Action,
                Person = x.Person,
                Remarks = x.Remarks
            }).ToList();
        }

        public async Task<EquipmentHistoryDto?> GetByIdAsync(int id)
        {
            if (id <= 0)
            {
                throw new ArgumentException(
                    "A valid history record ID is required.");
            }

            var record = await _repository.GetByIdAsync(id);

            if (record == null)
            {
                return null;
            }

            return new EquipmentHistoryDto
            {
                Id = record.Id,
                Date = record.Date,
                Equipment = record.Equipment,
                Action = record.Action,
                Person = record.Person,
                Remarks = record.Remarks
            };
        }
    }
}
```

6. Create the controller

Create `Controllers/EquipmentHistoryController.cs`.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EquipmentHistoryController : ControllerBase
    {
        private readonly IEquipmentHistoryService _service;

        public EquipmentHistoryController(
            IEquipmentHistoryService service)
        {
            _service = service;
        }

        // GET: api/EquipmentHistory
        // Optional: ?startDate=2026-09-01&endDate=2026-09-30
        [HttpGet]
        public async Task<IActionResult> GetAll(
            [FromQuery] DateTime? startDate,
            [FromQuery] DateTime? endDate)
        {
            try
            {
                var history = await _service.GetAllAsync(
                    startDate, endDate);

                return Ok(history);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }

        // GET: api/EquipmentHistory/1
        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            try
            {
                var record = await _service.GetByIdAsync(id);
```

```
                if (record == null)
                {
                    return NotFound(new
                    {
                        message = "Equipment history record not found."
                    });
                }

                return Ok(record);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }
    }
}
```

7. Register the repository and service

Add these registrations in `Program.cs` alongside your other dependency-injection registrations:

```
builder.Services.AddScoped<
    IEquipmentHistoryRepository,
    EquipmentHistoryRepository>();

builder.Services.AddScoped<
    IEquipmentHistoryService,
    EquipmentHistoryService>();
```

Add the relevant namespaces if needed:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

8. Create the database migration

Run the commands from the backend project directory.

Package Manager Console:

```
Add-Migration CreateEquipmentHistoryTable
Update-Database
```

Or .NET CLI:

```
dotnet ef migrations add CreateEquipmentHistoryTable
dotnet ef database update
```

This creates the `EquipmentHistories` table in the database configured by your application.

9. Test the API in Swagger

GET **All equipment history**

`/api/EquipmentHistory`

GET **Filter by date range**

`/api/EquipmentHistory?startDate=2026-09-01&endDate=2026-09-30`

GET **Get one history record**

`/api/EquipmentHistory/1`

A response will have the structure your Angular table expects:

```
[
  {
    "id": 1,
    "date": "2026-09-28T10:00:00Z",
    "equipment": "Total Station - 02",
    "action": "Returned",
    "person": "Usman",
    "remarks": ""
  }
]
```

This is an example response; the new table will be empty until activity records are inserted.

10. Important: record actual checkout and return events

The GET endpoints and migration establish the history API, but they do **not** automatically populate history. To make the report accurate, your existing checkout service should insert a `Checked Out` history record when a checkout is completed, and your return service should insert a `Returned` record when equipment is returned. Drone checkout history should be recorded after approval and completion, not merely when an approval is requested.

Once those write operations are connected, the Reports / History page can show real activity rather than sample records. The date filter will then operate against the stored event dates.

25/10/2026, 16:21:24 · gpt-6

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
interface EquipmentHistoryRecord {
  id: number;
  date: string;
  equipment: string;
  action: 'Returned' | 'Checked Out';
  person: string;
  remarks: string;
}
@Component({
  selector: 'app-reports',
  imports: [FormsModule],
  templateUrl: './reports.html',
  styleUrl: './reports.css',
})
export class Reports {

  startDate = '2026-09-01';
  endDate = '2026-09-30';

  appliedStartDate = this.startDate;
  appliedEndDate = this.endDate;

  history: EquipmentHistoryRecord[] = [
    {
      id: 1,
      date: '28/09/2026',
      equipment: 'Total Station - 02',
      action: 'Returned',
      person: 'Usman',
      remarks: ''
    },
    {
      id: 2,
      date: '25/09/2026',
      equipment: 'GPS Rover - 01',
      action: 'Checked Out',
      person: 'Ali Raza',
      remarks: 'Site work'
    },
    {
      id: 3,
      date: '20/09/2026',
```

```
    equipment: 'Field Tablet',
    action: 'Returned',
    person: 'Bilal',
    remarks: ''
  },
  {
    id: 4,
    date: '18/09/2026',
    equipment: 'Drone - DJI Mavic',
    action: 'Checked Out',
    person: 'Hamza',
    remarks: 'Aerial survey'
  },
  {
    id: 5,
    date: '15/09/2026',
    equipment: 'GPS Rover - 02',
    action: 'Returned',
    person: 'Asif',
    remarks: ''
  }
];

get filteredHistory(): EquipmentHistoryRecord[] {
  return this.history.filter(record => {
    const recordDate = this.toISODate(record.date);

    const matchesStart =
      !this.appliedStartDate ||
      recordDate >= this.appliedStartDate;

    const matchesEnd =
      !this.appliedEndDate ||
      recordDate <= this.appliedEndDate;

    return matchesStart && matchesEnd;
  });
}

private toISODate(date: string): string {
  const [day, month, year] = date.split('/');
  return `${year}-${month}-${day}`;
}

applyFilter(): void {
  if (
```

```
      this.startDate &&
      this.endDate &&
      this.startDate > this.endDate
    ) {
      alert('Start date cannot be later than end date.');
      return;
    }

    this.appliedStartDate = this.startDate;
    this.appliedEndDate = this.endDate;
  }

  resetFilter(): void {
    this.startDate = '';
    this.endDate = '';
    this.appliedStartDate = '';
    this.appliedEndDate = '';
  }
}

<div class="history-page">

  <!-- Page banner -->
  <div class="page-banner">
    7. Reports / History
  </div>

  <section class="history-card">

    <!-- Heading -->
    <header class="history-header">
      <div>
        <h1>Equipment History</h1>
        <p>View past check-outs, returns and activity</p>
      </div>
    </header>

    <!-- Date filters -->
    <form class="filter-bar" (ngSubmit)="applyFilter()">

      <div class="date-control">
        <label for="startDate" class="visually-hidden">
          Start date
        </label>

        <input
```

```
        id="startDate"
        type="date"
        name="startDate"
        [(ngModel)]="startDate"
      />
    </div>

    <span class="date-separator">to</span>

    <div class="date-control">
      <label for="endDate" class="visually-hidden">
        End date
      </label>

      <input
        id="endDate"
        type="date"
        name="endDate"
        [(ngModel)]="endDate"
      />
    </div>

    <button type="submit" class="filter-button">
      Filter
    </button>

    <button
      type="button"
      class="reset-button"
      (click)="resetFilter()"
    >
      Reset
    </button>

  </form>

  <!-- History table -->
  <div class="table-scroll">
    <table>
      <thead>
        <tr>
          <th>Date</th>
          <th>Equipment</th>
          <th>Action</th>
          <th>Person</th>
          <th>Remarks</th>
```

```
        </tr>
      </thead>

      <tbody>
        @for (record of filteredHistory; track record.id) {
          <tr>
            <td class="date-cell">{{ record.date }}</td>

            <td class="equipment-cell">
              {{ record.equipment }}
            </td>

            <td>
              <span
                class="action-badge"
                [class.returned]="record.action === 'Returned'"
                [class.checked-out]="record.action === 'Checked
Out'"
              >
                {{ record.action }}
              </span>
            </td>

            <td>{{ record.person }}</td>

            <td class="remarks-cell">
              {{ record.remarks || '—' }}
            </td>
          </tr>
        } @empty {
          <tr>
            <td colspan="5" class="empty-state">
              No equipment history found for the selected dates.
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>

  <!-- Footer -->
  <footer class="history-footer">
    <span>
      Showing {{ filteredHistory.length }}
      of {{ history.length }} records
    </span>
```

```
      <button
        type="button"
        class="reset-link"
        (click)="resetFilter()"
      >
        Clear filters
      </button>
    </footer>

  </section>
</div>
```
implement the above apis in this typescript code ??

25/10/2026, 16:59:59

We'll connect your existing `Reports` TypeScript component to the .NET `EquipmentHistory` APIs created earlier, without changing your HTML or CSS design.

The updated component will:

- Load history records from `GET /api/EquipmentHistory`.
- Apply the start and end date filters through the backend API.
- Display real checkout and return events from SQL Server.
- Keep your existing table, action badges, and date-filter controls.
- Remove the hardcoded five sample records.

1. Create the Angular model

Create `src/app/models/equipment-history.ts`:

```ts
export interface EquipmentHistoryRecord {
  id: number;
  date: string;
  equipment: string;
  action: 'Returned' | 'Checked Out';
  person: string;
  remarks: string;
}
```

2. Create the API service

Create `src/app/services/equipment-history.ts`:

```ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';
import { EquipmentHistoryRecord } from '../models/equipment-history';

@Injectable({
  providedIn: 'root'
})
export class EquipmentHistoryService {
  private http = inject(HttpClient);
  private apiUrl = `${environment.apiUrl}/EquipmentHistory`;

  getAll(
    startDate?: string,
    endDate?: string
  ): Observable<EquipmentHistoryRecord[]> {
    let params = new HttpParams();

    if (startDate) {
      params = params.set('startDate', startDate);
    }

    if (endDate) {
      params = params.set('endDate', endDate);
    }

    return this.http.get<EquipmentHistoryRecord[]>(
      this.apiUrl,
      { params }
    );
  }

  getById(id: number): Observable<EquipmentHistoryRecord> {
    return this.http.get<EquipmentHistoryRecord>(
      `${this.apiUrl}/${id}`
    );
  }
}
```

3. Replace your `reports.ts`

This version preserves your current property names and template bindings, so your existing `reports.html` should continue to work.

```ts
import {
  Component,
  OnInit,
  inject,
  ChangeDetectorRef
} from '@angular/core';
import { FormsModule } from '@angular/forms';
import { finalize } from 'rxjs';

import { EquipmentHistoryService } from '../../services/equipment-history';
import { EquipmentHistoryRecord } from '../../models/equipment-history';

@Component({
  selector: 'app-reports',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './reports.html',
  styleUrl: './reports.css',
})
export class Reports implements OnInit {
  private historyService = inject(EquipmentHistoryService);
  private cdr = inject(ChangeDetectorRef);

  startDate = '2026-09-01';
  endDate = '2026-09-30';

  appliedStartDate = this.startDate;
  appliedEndDate = this.endDate;

  history: EquipmentHistoryRecord[] = [];

  loading = false;
  errorMessage = '';

  ngOnInit(): void {
    this.loadHistory();
  }

  // GET: api/EquipmentHistory
  loadHistory(): void {
    this.loading = true;
    this.errorMessage = '';

    this.historyService
      .getAll(this.appliedStartDate, this.appliedEndDate)
      .pipe(
```

```
      finalize(() => {
        this.loading = false;
        this.cdr.detectChanges();
      })
    )
    .subscribe({
      next: records => {
        this.history = records.map(record => ({
          ...record,
          date: this.formatDate(record.date),
          remarks: record.remarks ?? ''
        }));
      },
      error: error => {
        console.error('Failed to load equipment history:', error);

        this.errorMessage =
          error?.error?.message ??
          'Unable to load equipment history. Please try again.';
      }
    });
}

get filteredHistory(): EquipmentHistoryRecord[] {
  // Date filtering is performed by the backend.
  return this.history;
}

private formatDate(value: string): string {
  if (!value) {
    return '';
  }

  const date = new Date(value);

  if (Number.isNaN(date.getTime())) {
    return value;
  }

  const day = String(date.getDate()).padStart(2, '0');
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const year = date.getFullYear();

  return `${day}/${month}/${year}`;
}
```

```
  applyFilter(): void {
    if (
      this.startDate &&
      this.endDate &&
      this.startDate > this.endDate
    ) {
      alert('Start date cannot be later than end date.');
      return;
    }

    this.appliedStartDate = this.startDate;
    this.appliedEndDate = this.endDate;

    this.loadHistory();
  }

  resetFilter(): void {
    this.startDate = '';
    this.endDate = '';
    this.appliedStartDate = '';
    this.appliedEndDate = '';

    this.loadHistory();
  }
}
```

4. Add loading and error messages to your HTML

Place this below the date filter form and above the history table:

```
@if (loading) {
  <p class="loading-message">Loading equipment history...</p>
}

@if (errorMessage) {
  <p class="error-message">{{ errorMessage }}</p>
}
```

Your existing table and footer can remain unchanged.

5. Verify the API configuration

Make sure `environment.ts` contains:

```
export const environment = {
  apiUrl: 'https://localhost:7135/api'
```

```
};
```

Also ensure `provideHttpClient()` is configured in `app.config.ts`.

6. Test the result

Open Swagger and test:

`GET /api/EquipmentHistory?startDate=2026-09-01&endDate=2026-09-30`

The API should return records with fields matching your Angular model:

```
[
  {
    "id": 1,
    "date": "2026-09-28T10:00:00Z",
    "equipment": "Total Station - 02",
    "action": "Returned",
    "person": "Usman",
    "remarks": ""
  }
]
```

This is an illustrative response; your actual records will come from SQL Server.

Important: The history table will initially be empty if you have not inserted history records. Your checkout and return services must save a corresponding `EquipmentHistory` record whenever an actual checkout or return occurs. Otherwise, this page cannot display those past activities automatically.

25/10/2026, 17:00:01 · gpt-6

can u guide me how to populate the equipment history means which functionality also saves data into equipmenthistory table ??

25/10/2026, 17:03:46

Yes. In your equipment tracking system, the **Equipment History** table should be populated automatically when users perform actions elsewhere in the application. Users should not need to enter history records manually.

1. Which functionality should save data into `EquipmentHistories`?

1. Checkout Equipment

When a checkout is successfully completed, insert a history record with `Action = "Checked Out"`.

Example: GPS Rover - 01 · Ahmad · Site work

2. Return Equipment

When the return API successfully marks equipment as returned, insert a history record with `Action = "Returned"`.

Example: GPS Rover - 01 · Ahmad · Returned after fieldwork

3. Drone Approval

Do not record a completed checkout when the drone approval is merely requested. Record `Checked Out` when the request is approved and the equipment is actually issued.

4. Damage Reports

These should populate your separate `DamageReports` table. They should not create checkout or return history events unless you explicitly decide to track damage reports as a separate type of history event.

2. Add a method to your Equipment History service

You already have `EquipmentHistoryRepository.CreateAsync()`. We need to expose a method in the service layer so your checkout and return workflows can create history records.

Update `Services/IEquipmentHistoryService.cs`

Add this method to your existing interface:

```
Task RecordActivityAsync(
    string equipment,
    string action,
    string person,
    string? remarks);
```

Update `Services/EquipmentHistoryService.cs`

Add this method to your existing service class:

```
public async Task RecordActivityAsync(
    string equipment,
    string action,
    string person,
    string? remarks)
{
    if (string.IsNullOrWhiteSpace(equipment))
        throw new ArgumentException("Equipment name is required.");

    if (action != "Checked Out" && action != "Returned")
        throw new ArgumentException("Invalid equipment history action.");

    if (string.IsNullOrWhiteSpace(person))
        throw new ArgumentException("Person is required.");

    var history = new SolvefyTask.Models.EquipmentHistory
    {
        Date = DateTime.UtcNow,
        Equipment = equipment.Trim(),
        Action = action,
        Person = person.Trim(),
        Remarks = remarks?.Trim() ?? string.Empty
    };

    await _repository.CreateAsync(history);
}
```

This uses your existing `IEquipmentHistoryRepository.CreateAsync()` method, so you don't need to create another repository.

3. Connect it to the checkout functionality

Find the service method that handles your existing `POST /api/Checkout` endpoint. After the checkout has been successfully saved, record the event.

For example, in your checkout service, inject `IEquipmentHistoryService`:

```
private readonly IEquipmentHistoryService _historyService;

public CheckoutService(
    /* your existing dependencies */,
    IEquipmentHistoryService historyService)
{
    // Keep your existing dependency assignments.
    _historyService = historyService;
}
```

Then, **after the checkout succeeds**, call:

```
await _historyService.RecordActivityAsync(
    equipment: checkout.EquipmentName,
    action: "Checked Out",
    person: checkout.AssignedTo,
    remarks: checkout.Purpose
);
```

Here, `checkout` represents your saved checkout entity. Replace it with the actual variable name in your service.

This will add a record like:

```
{
  "equipment": "GPS Rover - 01",
  "action": "Checked Out",
  "person": "Ahmad",
  "remarks": "Site work"
}
```

The backend automatically generates the history ID and date.

For drone checkouts: put this call in the workflow that actually issues the equipment after approval, not in the method that creates the pending approval request.

4. Connect it to the return functionality

In your existing return service, first ensure that the checkout is successfully marked as `Returned`. Then record the return event:

```
await _historyService.RecordActivityAsync(
    equipment: checkout.EquipmentName,
    action: "Returned",
    person: checkout.AssignedTo,
```

```
    remarks: "Equipment returned"
);
```

Use your actual checkout entity variable and return remarks if your workflow stores them.

This creates a separate history entry. It should not overwrite the original `Checked Out` event.

5. Important: keep the database changes consistent

The examples above show where to add history recording, but the checkout/return operations and history inserts should ideally be committed in **one database transaction**. Otherwise, the checkout could succeed while the history insert fails.

If your existing service and history repository use the same scoped `ApplicationDbContext`, you can add the `EquipmentHistory` entity to that context and call `SaveChangesAsync()` once after both changes. Alternatively, use an explicit EF Core transaction.

Also, your current `MyEquipment` table is separate from `Checkouts`. If your application uses `MyEquipments` as the authoritative assignment record, place the history calls in the service that actually creates or returns those assignments.

6. Test the complete workflow

1. Create or complete a checkout through your existing checkout API.
2. Check SQL Server's `EquipmentHistories` table; a `Checked Out` record should appear.
3. Return the equipment through your return API.
4. Check the table again; a second `Returned` record should appear.
5. Open `GET /api/EquipmentHistory` in Swagger. Both events should be returned and displayed on your Reports / History page.

To integrate this accurately into your project: please share your existing `CheckoutService.cs` and the service/controller method that handles equipment returns. I can show you exactly where to insert the history-recording logic using your actual class names and variables, without disturbing your existing APIs.

25/10/2026, 17:03:58 · gpt-6

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MyEquipmentController : ControllerBase
    {
        private readonly IMyEquipmentService _service;

        public MyEquipmentController(IMyEquipmentService
service)
        {
            _service = service;
        }

        // GET: api/MyEquipment?assignedTo=Ahmad
        [HttpGet]
        public async Task<IActionResult> GetMyEquipment(
            [FromQuery] string assignedTo)
        {
            if (string.IsNullOrWhiteSpace(assignedTo))
            {
                return BadRequest(new
                {
                    message = "The assignedTo parameter is
required."
                });
            }

            var equipment = await
_service.GetMyEquipmentAsync(assignedTo);

            return Ok(equipment);
        }

        // GET: api/MyEquipment/1
        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var equipment = await _service.GetByIdAsync(id);
```

```
            if (equipment == null)
            {
                return NotFound(new
                {
                    message = "Equipment checkout record not
found."
                });
            }

            return Ok(equipment);
        }

        // PUT: api/MyEquipment/1/return
        [HttpPut("{id:int}/return")]
        public async Task<IActionResult> ReturnEquipment(int
id)
        {
            var result = await _service.ReturnEquipmentAsync(id);

            if (!result.Success)
            {
                if (result.Message == "Equipment checkout record
not found.")
                {
                    return NotFound(new { message =
result.Message });
                }

                return BadRequest(new { message = result.Message
});
            }

            return Ok(new
            {
                message = result.Message,
                id,
                status = "Returned"
            });
        }
    }
}
```

how to implement in this i have already implemented in checkout now how to in this ??

25/10/2026, 17:14:17

Your `MyEquipmentController` is already calling `_service.ReturnEquipmentAsync(id)`, so **you do not need to change the controller significantly**. The best place to save the `Returned` history record is inside `MyEquipmentService`, immediately after the return operation succeeds.

Since you have already implemented history recording in Checkout, we'll add the same integration to the return workflow.

1. Update `MyEquipmentService.cs`

Inject `IEquipmentHistoryService` into your existing service. Then update `ReturnEquipmentAsync()` as shown below.

```
using SolvefyTask.Repositories;
// Keep your other existing using statements.

namespace SolvefyTask.Services
{
    public class MyEquipmentService : IMyEquipmentService
    {
        private readonly IMyEquipmentRepository _repository;
        private readonly IEquipmentHistoryService _historyService;

        public MyEquipmentService(
            IMyEquipmentRepository repository,
            IEquipmentHistoryService historyService)
        {
            _repository = repository;
            _historyService = historyService;
        }

        // Keep your existing GetMyEquipmentAsync() and GetByIdAsync()
        // methods unchanged.

        public async Task<(bool Success, string Message)>
            ReturnEquipmentAsync(int id)
        {
            if (id <= 0)
            {
                return (false, "A valid checkout ID is required.");
            }

            var equipment = await _repository.GetByIdAsync(id);

            if (equipment == null)
            {
```

```
                return (false, "Equipment checkout record not found.");
            }

            if (equipment.Status == "Returned")
            {
                return (false, "This equipment has already been returned.");
            }

            // Update the assignment/checkout status first.
            var returned = await _repository.ReturnEquipmentAsync(id);

            if (!returned)
            {
                return (false, "Unable to return the equipment.");
            }

            // Record the successful return in EquipmentHistories.
            await _historyService.RecordActivityAsync(
                equipment: equipment.EquipmentName,
                action: "Returned",
                person: equipment.AssignedTo,
                remarks: "Equipment returned"
            );

            return (true, "Equipment returned successfully.");
        }
    }
}
```

Important property-name check

This example assumes your `MyEquipmentResponseDto` has `EquipmentName`, `AssignedTo`, and `Status` properties. In the earlier DTO, the employee property may not have been included. If your actual DTO differs, use the corresponding fields from your `MyEquipment` entity or repository result.

Also, the return operation and history insert should ideally share one database transaction. Otherwise, the status may change successfully while the history insert fails.

2. Keep your controller as it is

Your existing controller method already calls the service and returns the result:

```
[HttpPut("{id:int}/return")]
public async Task<IActionResult> ReturnEquipment(int id)
{
```

```
    var result = await _service.ReturnEquipmentAsync(id);

    if (!result.Success)
    {
        if (result.Message == "Equipment checkout record not found.")
        {
            return NotFound(new { message = result.Message });
        }

        return BadRequest(new { message = result.Message });
    }

    return Ok(new
    {
        message = result.Message,
        id,
        status = "Returned"
    });
}
```

3. Ensure dependency injection is registered

In `Program.cs`, make sure these registrations exist:

```
builder.Services.AddScoped<
    IEquipmentHistoryRepository,
    EquipmentHistoryRepository>();

builder.Services.AddScoped<
    IEquipmentHistoryService,
    EquipmentHistoryService>();
```

Your existing `MyEquipmentService` registration can remain unchanged, provided its constructor dependencies are registered too.

4. Test the return workflow

1. Call `PUT /api/MyEquipment/1/return` in Swagger.
2. Confirm the response says `Equipment returned successfully.`
3. Check the SQL Server `EquipmentHistories` table.
4. Confirm a new row appears with `Action = "Returned"` and the correct equipment and employee.

One important distinction: your current `MyEquipmentController` returns records from the `MyEquipments` table, while your earlier Checkout API may use the separate `Checkouts` table.

Make sure `MyEquipmentRepository.ReturnEquipmentAsync()` updates the same record that `GetByIdAsync()` retrieves. Otherwise, the status and history could become inconsistent.

25/10/2026, 17:14:19 · gpt-6

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.DTOs;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ApprovalsController : ControllerBase
    {
        private readonly IApprovalService _service;

        public ApprovalsController(IApprovalService service)
        {
            _service = service;
        }

        // GET: api/Approvals
        [HttpGet]
        public async
Task<ActionResult<List<ApprovalResponseDto>>> GetAll()
        {
            var requests = await _service.GetAllAsync();
            return Ok(requests);
        }

        // GET: api/Approvals/1
        [HttpGet("{id:int}")]
        public async
Task<ActionResult<ApprovalResponseDto>> GetById(int id)
        {
            if (id <= 0)
                return BadRequest(new { message = "Invalid
approval ID." });

            var request = await _service.GetByIdAsync(id);

            if (request == null)
                return NotFound(new { message = "Approval
request not found." });

            return Ok(request);
        }
```

```
        // POST: api/Approvals
        [HttpPost]
        public async
Task<ActionResult<ApprovalResponseDto>> Create(
            [FromBody] CreateApprovalRequestDto dto)
        {
            try
            {
                var request = await _service.CreateAsync(dto);

                return CreatedAtAction(
                    nameof(GetById),
                    new { id = request.Id },
                    request);
            }
            catch (KeyNotFoundException ex)
            {
                return NotFound(new { message = ex.Message });
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }

        // PUT: api/Approvals/1/approve
        [HttpPut("{id:int}/approve")]
        public async
Task<ActionResult<ApprovalResponseDto>> Approve(
            int id,
            [FromBody] ApprovalDecisionDto? dto)
        {
            return await MakeDecision(id, "Approved",
dto?.Comments);
        }

        // PUT: api/Approvals/1/reject
        [HttpPut("{id:int}/reject")]
        public async
Task<ActionResult<ApprovalResponseDto>> Reject(
            int id,
            [FromBody] ApprovalDecisionDto? dto)
        {
            return await MakeDecision(id, "Rejected",
dto?.Comments);
```

```
        }

        private async
Task<ActionResult<ApprovalResponseDto>> MakeDecision(
            int id,
            string decision,
            string? comments)
        {
            if (id <= 0)
                return BadRequest(new { message = "Invalid
approval ID." });

            try
            {
                var request = await _service.DecideAsync(
                    id, decision, comments);

                if (request == null)
                    return NotFound(
                        new { message = "Approval request not
found." });

                return Ok(request);
            }
            catch (InvalidOperationException ex)
            {
                return Conflict(new { message = ex.Message });
            }
        }
    }
}
```

how to insert in this equipmenthistory ??

25/10/2026, 17:20:17

آپ کے `ApprovalsController` میں `EquipmentHistory` کی entry ڈالنے کے لیے بہتر طریقہ یہ ہے کہ **history کریں۔ `ApprovalService` میں **record** نہ کریں، بلکہ **insert** میں براہِ راست **controller** کو**

چونکہ drone کے لیے approval ضروری ہے، اس لیے history میں `"Checked Out"` کی entry **صرف اس وقت** بننی چاہیے جب **approval** منظور ہونے کے بعد **equipment** واقعی **issue** ہو۔ صرف approval request کے approve بنانے یا checkout کرنے پر system میں approval مکمل نہیں سمجھا جائے گا، جب تک آپ کے equipment issue ساتھ ہی کرنے کا عمل نہ ہو۔

آپ کے موجودہ controller میں تبدیلی کی ضرورت نہیں؛ اصل تبدیلی `ApprovalService` میں ہوگی۔

1. `ApprovalService` میں history service inject کریں

اپنی `ApprovalService.cs` میں یہ dependency add کریں:

```
private readonly IEquipmentHistoryService _historyService;

public ApprovalService(
    IApprovalRepository repository,
    IEquipmentHistoryService historyService)
{
    _repository = repository;
    _historyService = historyService;
}
```

نوٹ: `_repository` اور constructor کے باقی parameters کے موجودہ نام آپ کے code کے مطابق رکھیں۔ اگر آپ کے constructor میں دوسری dependencies بھی ہیں تو انہیں remove نہ کریں۔

2. `DecideAsync()` میں history کب insert کرنی ہے؟

آپ کے موجودہ `ApprovalService` میں `DecideAsync()` method کے اندر یہ logic ہونا چاہیے:

- Request کو fetch کریں۔
- اگر request نہ ملے تو `null` return کریں۔
- اگر فیصلہ `Approved` ہے تو approval status update کریں۔
- اگر equipment واقعی issue ہو گیا ہے، تب history insert کریں۔
- اگر request `Rejected` ہے تو `"Checked Out"` history insert نہ کریں۔

مثال کے طور پر، history record کرنے کے لیے یہ call استعمال ہوگی:

```
await _historyService.RecordActivityAsync(
    equipment: equipmentName,
    action: "Checked Out",
    person: assignedTo,
    remarks: "Drone checkout approved and issued");
```

یہاں `equipmentName` اور `assignedTo` کو آپ کی actual approval request اور checkout logic سے لینا ہوگا۔ properties مختلف ہو سکتی ہیں۔ model میں نہ کریں، کیونکہ آپ کے copy صرف مثال کے ناموں کو

3. اہم بات: duplicate history سے بچیں

آپ نے بتایا ہے کہ checkout میں history پہلے سے implement ہے۔ اس لیے اگر approval approve ہونے کے بعد آپ کا code پہلے ہی checkout والا method call کرتا ہے، اور وہ method history insert کرتا ہے، تو entries کی دو checkout بن سکتی ہیں۔ **نہ کریں۔** ورنہ ایک ہی history insert میں دوبارہ `ApprovalService`

مجھے یہ code بھیج دیں

کی یہ دو چیزیں `ApprovalService.cs` دے سکوں، اپنی code ہونے والا compile، تاکہ میں آپ کو بالکل درست share کریں:

1. Constructor اور اس کی dependencies۔
2. مکمل `DecideAsync()` method۔

پھر میں آپ کے موجودہ code میں ہی درست جگہ `EquipmentHistory` insert کرنے کی تبدیلی دوں گا، بغیر آپ کو خراب کیے۔ approval logic کے باقی

25/10/2026, 17:20:21 · gpt-6

in english ??

25/10/2026, 17:20:44

In your `ApprovalsController`, you do not need to insert `EquipmentHistory` directly. The recommended approach is to record history in your `ApprovalService`.

Since drone checkout requires approval, add a **"Checked Out"** history entry only when the approval is granted and the equipment is actually issued.

1. Inject `IEquipmentHistoryService` into `ApprovalService`

Add this dependency to your existing service:

```
private readonly IEquipmentHistoryService _historyService;

public ApprovalService(
    IApprovalRepository repository,
    IEquipmentHistoryService historyService)
{
    _repository = repository;
    _historyService = historyService;
}
```

Keep your existing constructor dependencies if you have additional ones.

2. Record history in `DecideAsync()`

When your `DecideAsync()` method approves a request and the equipment is actually issued, call:

```
await _historyService.RecordActivityAsync(
    equipment: equipmentName,
    action: "Checked Out",
    person: assignedTo,
    remarks: "Drone checkout approved and issued");
```

Here, `equipmentName` and `assignedTo` are example variable names. Replace them with the actual values from your approval and checkout logic.

- **Approved and issued:** Record `"Checked Out"`.
- **Rejected:** Do not record a checkout.
- **Approved but not yet issued:** Do not record a checkout yet.

3. Avoid duplicate history records

You mentioned that you have already implemented history recording in checkout. If your approval process calls that existing checkout method, and it already inserts the history record, **do not insert the same record again in** `ApprovalService`.

Share your `ApprovalService.cs`

Please send your existing `ApprovalService.cs`, especially its constructor and complete `DecideAsync()` method. I will modify your actual code to insert history at the correct point without duplicating your existing checkout logic.

25/10/2026, 17:20:47 · gpt-6

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class ApprovalService : IApprovalService
    {
        private readonly IApprovalRepository _repository;

        public ApprovalService(IApprovalRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<ApprovalResponseDto>>
GetAllAsync()
        {
            var requests = await _repository.GetAllAsync();

            return requests.Select(MapToDto).ToList();
        }

        public async Task<ApprovalResponseDto?>
GetByIdAsync(int id)
        {
            var request = await _repository.GetByIdAsync(id);

            return request == null ? null : MapToDto(request);
        }

        public async Task<ApprovalResponseDto> CreateAsync(
            CreateApprovalRequestDto dto)
        {
            if (!await
_repository.EquipmentExistsAsync(dto.EquipmentId))
                throw new KeyNotFoundException(
                    "The selected equipment was not found.");

            if (dto.ExpectedReturn.Date < DateTime.UtcNow.Date)
                throw new ArgumentException(
                    "Expected return date cannot be in the past.");

            var request = new ApprovalRequest
```

```
            {
                EquipmentId = dto.EquipmentId,
                RequestedBy = dto.RequestedBy.Trim(),
                ExpectedReturn = dto.ExpectedReturn,
                Purpose = dto.Purpose.Trim(),
                Status = "Pending Approval"
            };

            var created = await _repository.CreateAsync(request);

            return MapToDto(created);
        }

        public async Task<ApprovalResponseDto?> DecideAsync(
            int id,
            string decision,
            string? comments)
        {
            var request = await _repository.DecideAsync(
                id, decision, comments);

            return request == null ? null : MapToDto(request);
        }

        private static ApprovalResponseDto MapToDto(
            ApprovalRequest request)
        {

            return new ApprovalResponseDto
            {
                Id = request.Id,
                EquipmentId = request.EquipmentId,
                EquipmentName = request.Equipment?.Name ??
string.Empty,
                SerialNumber = request.Equipment?.SerialNumber,
                RequestedBy = request.RequestedBy,
                RequestedOn = request.RequestedOn,
                ExpectedReturn = request.ExpectedReturn,
                Purpose = request.Purpose,
                Status = request.Status,
                Comments = request.Comments,
                DecisionAt = request.DecisionAt
            };
        }
    }
```

}
now insert how to ??

25/10/2026, 17:22:29

In your current `ApprovalService`, the `DecideAsync()` method delegates the approval decision to `_repository.DecideAsync()`. To insert a record into `EquipmentHistories`, we need to connect `IEquipmentHistoryService` and record the checkout **only when the drone has actually been issued after approval**.

One important detail: your current code does not show whether `_repository.DecideAsync()` also issues the equipment or only changes the approval status. The implementation below assumes the repository updates the approval status and loads the related `Equipment`.

1. Update `ApprovalService.cs`

First, add the history service dependency to your existing class.

Replace your constructor and `DecideAsync()` method with the following code. Keep your other methods unchanged.

```
private readonly IApprovalRepository _repository;
private readonly IEquipmentHistoryService _historyService;

public ApprovalService(
    IApprovalRepository repository,
    IEquipmentHistoryService historyService)
{
    _repository = repository;
    _historyService = historyService;
}
```

Replace `DecideAsync()` with:

```
public async Task<ApprovalResponseDto?> DecideAsync(
    int id,
    string decision,
    string? comments)
{
    var request = await _repository.DecideAsync(
        id, decision, comments);

    if (request == null)
        return null;
```

```
    // Record history only when approval also completes checkout.
    if (request.Status == "Approved")
    {
        await _historyService.RecordActivityAsync(
            equipment: request.Equipment?.Name
                ?? throw new InvalidOperationException(
                    "Equipment information was not loaded."),
            action: "Checked Out",
            person: request.RequestedBy,
            remarks: string.IsNullOrWhiteSpace(comments)
                ? "Approved drone checkout"
                : comments);
    }

    return MapToDto(request);
}
```

2. Important: verify your checkout workflow

The code above assumes that approving the request means the drone is issued immediately. If approval and physical issue are separate steps, record the history in the actual issue/checkout method instead.

Also, because you already implemented history recording in checkout, **do not add this call here if the approval workflow already invokes that checkout method**. Otherwise, you may get duplicate history records.

3. Register the service in `Program.cs`

Make sure these registrations already exist:

```
builder.Services.AddScoped<
    IEquipmentHistoryService,
    EquipmentHistoryService>();

builder.Services.AddScoped<
    IEquipmentHistoryRepository,
    EquipmentHistoryRepository>();
```

One final check: your `IApprovalRepository.DecideAsync()` must return the associated `Equipment` navigation property, including its `Name`. Otherwise, the history insertion will fail with the exception shown above.

Please share your `ApprovalRepository.cs`, especially its `DecideAsync()` method. I can then verify whether approval actually issues the drone and show you the safest exact implementation without duplicate history entries.

25/10/2026, 17:22:34 · gpt-6

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class DamageReportService : IDamageReportService
    {
        private readonly IDamageReportRepository _repository;

        public DamageReportService(IDamageReportRepository
repository)
        {
            _repository = repository;
        }

        public async Task<List<DamageReportDto>>
GetAllAsync(
            string? search,
            string? status,
            DateTime? startDate,
            DateTime? endDate)
        {
            if (startDate.HasValue &&
                endDate.HasValue &&
                startDate.Value.Date > endDate.Value.Date)
            {
                throw new ArgumentException(
                    "Start date cannot be after end date.");
            }

            var reports = await _repository.GetAllAsync(
                search, status, startDate, endDate);

            return reports.Select(MapToDto).ToList();
        }

        public async Task<DamageReportDto?> GetByIdAsync(int
id)
        {
            if (id <= 0)
            {
                throw new ArgumentException(
                    "A valid damage report ID is required.");
```

```
        }

        var report = await _repository.GetByIdAsync(id);

        return report == null ? null : MapToDto(report);
    }

    public async Task<DamageReportDto> CreateAsync(
        CreateDamageReportDto request)
    {
        if (string.IsNullOrWhiteSpace(request.Equipment))
        {
            throw new ArgumentException(
                "Equipment name is required.");
        }

        if (string.IsNullOrWhiteSpace(request.Description))
        {
            throw new ArgumentException(
                "Damage description is required.");
        }

        if (string.IsNullOrWhiteSpace(request.ReportedBy))
        {
            throw new ArgumentException(
                "Reported by is required.");
        }

        var report = new DamageReport
        {
            Equipment = request.Equipment.Trim(),
            Serial = string.IsNullOrWhiteSpace(request.Serial)
                ? "N/A"
                : request.Serial.Trim(),
            Description = request.Description.Trim(),
            ReportedBy = request.ReportedBy.Trim(),
            Date = DateTime.UtcNow,
            Status = "Pending"
        };

        var created = await _repository.CreateAsync(report);

        return MapToDto(created);
    }

    public async Task<(bool Success, string Message)>
```

```
ResolveAsync(
            int id)
        {
            if (id <= 0)
            {
                return (false, "A valid damage report ID is
required.");
            }

            var report = await _repository.GetByIdAsync(id);

            if (report == null)
            {
                return (false, "Damage report not found.");
            }

            if (report.Status == "Resolved")
            {
                return (false, "This damage report is already
resolved.");
            }

            var resolved = await _repository.ResolveAsync(id);

            return resolved
                ? (true, "Damage report resolved successfully.")
                : (false, "Unable to resolve the damage report.");
        }

        private static DamageReportDto
MapToDto(DamageReport report)
        {
            return new DamageReportDto
            {
                Id = report.Id,
                Date = report.Date,
                Equipment = report.Equipment,
                Serial = report.Serial,
                Description = report.Description,
                ReportedBy = report.ReportedBy,
                Status = report.Status
            };
        }
    }
}
```

how to implement in this ??

Your `DamageReportService` is already structured correctly. If you want to automatically insert a record into `EquipmentHistories` when a damage report is created or resolved, you can inject `IEquipmentHistoryService` and record the activity at the appropriate point.

Recommendation: Keep damage reports separate from checkout/return history unless you specifically want damage events to appear in the History/Reports screen. If you do, use actions such as `"Damage Reported"` and `"Damage Resolved"`.

1. Update the constructor

Add the history service to your existing `DamageReportService`:

```
private readonly IDamageReportRepository _repository;
private readonly IEquipmentHistoryService _historyService;

public DamageReportService(
    IDamageReportRepository repository,
    IEquipmentHistoryService historyService)
{
    _repository = repository;
    _historyService = historyService;
}
```

2. Update `CreateAsync()`

After the damage report is successfully created, record the activity in `EquipmentHistories`.

Replace your existing `CreateAsync()` with:

```
public async Task<DamageReportDto> CreateAsync(
    CreateDamageReportDto request)
{
    if (string.IsNullOrWhiteSpace(request.Equipment))
        throw new ArgumentException("Equipment name is required.");

    if (string.IsNullOrWhiteSpace(request.Description))
        throw new ArgumentException("Damage description is required.");

    if (string.IsNullOrWhiteSpace(request.ReportedBy))
        throw new ArgumentException("Reported by is required.");

    var report = new DamageReport
    {
        Equipment = request.Equipment.Trim(),
```

```
        Serial = string.IsNullOrWhiteSpace(request.Serial)
            ? "N/A"
            : request.Serial.Trim(),
        Description = request.Description.Trim(),
        ReportedBy = request.ReportedBy.Trim(),
        Date = DateTime.UtcNow,
        Status = "Pending"
    };

    var created = await _repository.CreateAsync(report);

    await _historyService.RecordActivityAsync(
        equipment: created.Equipment,
        action: "Damage Reported",
        person: created.ReportedBy,
        remarks: created.Description
    );

    return MapToDto(created);
}
```

3. Update `ResolveAsync()`

If you also want to record when a damage report is resolved, replace your existing method with:

```
public async Task<(bool Success, string Message)> ResolveAsync(
    int id)
{
    if (id <= 0)
        return (false, "A valid damage report ID is required.");

    var report = await _repository.GetByIdAsync(id);

    if (report == null)
        return (false, "Damage report not found.");

    if (report.Status == "Resolved")
        return (false, "This damage report is already resolved.");

    var resolved = await _repository.ResolveAsync(id);

    if (!resolved)
        return (false, "Unable to resolve the damage report.");

    await _historyService.RecordActivityAsync(
        equipment: report.Equipment,
```

```
        action: "Damage Resolved",
        person: report.ReportedBy,
        remarks: report.Description
    );

    return (true, "Damage report resolved successfully.");
}
```

4. Important: update history validation

Your `RecordActivityAsync()` method must allow the new actions `"Damage Reported"` and `"Damage Resolved"`. If it currently accepts only `"Checked Out"` and `"Returned"`, update its validation accordingly:

```
var allowedActions = new[]
{
    "Checked Out",
    "Returned",
    "Damage Reported",
    "Damage Resolved"
};

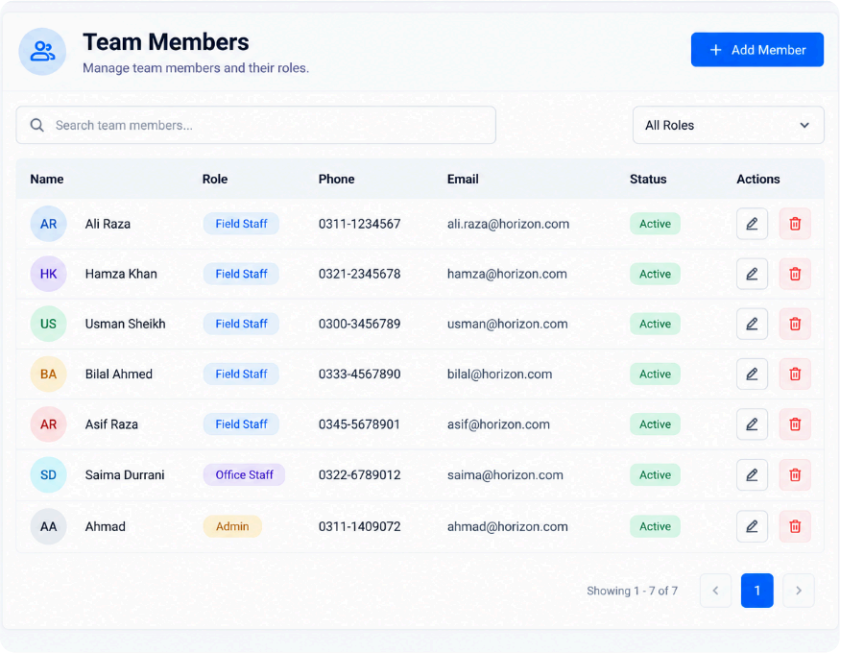
if (!allowedActions.Contains(action))
{
    throw new ArgumentException("Invalid history action.");
}
```

Expected result

Event	`EquipmentHistories.Action`
Equipment checked out	`Checked Out`
Equipment returned	`Returned`
Damage report created	`Damage Reported`
Damage report resolved	`Damage Resolved`

One technical consideration: These operations currently save the damage report and history separately. If history insertion fails, the damage report may still be saved. For production reliability, consider using a shared database transaction so both changes succeed or fail together.

25/10/2026, 17:24:52 · gpt-6



please generate the typescript and html and css of this image ??

25/10/2026, 17:54:01

Below is the Angular implementation of your **Team Members** UI, based on the screenshot. It includes TypeScript, HTML, and CSS with search, role filtering, Add Member, Edit, Delete, status badges, and pagination.

Create or update these three files in your Angular component.

1. `team-members.ts`

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface TeamMember {
  id: number;
  name: string;
  role: 'Field Staff' | 'Office Staff' | 'Admin';
  phone: string;
  email: string;
  status: 'Active' | 'Inactive';
}

@Component({
  selector: 'app-team-members',
```

```
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './team-members.html',
  styleUrl: './team-members.css'
})
export class TeamMembersComponent {
  searchText = '';
  selectedRole = 'All Roles';

  currentPage = 1;
  pageSize = 7;

  showForm = false;
  isEditing = false;
  editingId: number | null = null;

  roles = ['All Roles', 'Field Staff', 'Office Staff', 'Admin'];

  memberForm: TeamMember = this.emptyMember();

  members: TeamMember[] = [
    {
      id: 1,
      name: 'Ali Raza',
      role: 'Field Staff',
      phone: '0311-1234567',
      email: 'ali.raza@horizon.com',
      status: 'Active'
    },
    {
      id: 2,
      name: 'Hamza Khan',
      role: 'Field Staff',
      phone: '0321-2345678',
      email: 'hamza@horizon.com',
      status: 'Active'
    },
    {
      id: 3,
      name: 'Usman Sheikh',
      role: 'Field Staff',
      phone: '0300-3456789',
      email: 'usman@horizon.com',
      status: 'Active'
    },
    {
```

```
      id: 4,
      name: 'Bilal Ahmed',
      role: 'Field Staff',
      phone: '0333-4567890',
      email: 'bilal@horizon.com',
      status: 'Active'
    },
    {
      id: 5,
      name: 'Asif Raza',
      role: 'Field Staff',
      phone: '0345-5678901',
      email: 'asif@horizon.com',
      status: 'Active'
    },
    {
      id: 6,
      name: 'Saima Durrani',
      role: 'Office Staff',
      phone: '0322-6789012',
      email: 'saima@horizon.com',
      status: 'Active'
    },
    {
      id: 7,
      name: 'Ahmad',
      role: 'Admin',
      phone: '0311-1409072',
      email: 'ahmad@horizon.com',
      status: 'Active'
    }
  ];

  private emptyMember(): TeamMember {
    return {
      id: 0,
      name: '',
      role: 'Field Staff',
      phone: '',
      email: '',
      status: 'Active'
    };
  }

  get filteredMembers(): TeamMember[] {
    const search = this.searchText.trim().toLowerCase();
```

```
    return this.members.filter(member => {
      const matchesSearch =
        member.name.toLowerCase().includes(search) ||
        member.phone.toLowerCase().includes(search) ||
        member.email.toLowerCase().includes(search);

      const matchesRole =
        this.selectedRole === 'All Roles' ||
        member.role === this.selectedRole;

      return matchesSearch && matchesRole;
    });
  }

  get totalPages(): number {
    return Math.max(1, Math.ceil(this.filteredMembers.length / this.pageSize));
  }

  get paginatedMembers(): TeamMember[] {
    const start = (this.currentPage - 1) * this.pageSize;
    return this.filteredMembers.slice(start, start + this.pageSize);
  }

  get firstVisible(): number {
    return this.filteredMembers.length === 0
      ? 0
      : (this.currentPage - 1) * this.pageSize + 1;
  }

  get lastVisible(): number {
    return Math.min(
      this.currentPage * this.pageSize,
      this.filteredMembers.length
    );
  }

  onFilterChange(): void {
    this.currentPage = 1;
  }

  changePage(page: number): void {
    if (page >= 1 && page <= this.totalPages) {
      this.currentPage = page;
    }
  }
```

```
openAddForm(): void {
  this.isEditing = false;
  this.editingId = null;
  this.memberForm = this.emptyMember();
  this.showForm = true;
}

editMember(member: TeamMember): void {
  this.isEditing = true;
  this.editingId = member.id;
  this.memberForm = { ...member };
  this.showForm = true;
}

saveMember(): void {
  const name = this.memberForm.name.trim();
  const phone = this.memberForm.phone.trim();
  const email = this.memberForm.email.trim();

  if (!name || !phone || !email) {
    alert('Please fill in all required fields.');
    return;
  }

  if (this.isEditing && this.editingId !== null) {
    this.members = this.members.map(member =>
      member.id === this.editingId
        ? { ...this.memberForm, id: this.editingId, name, phone, email }
        : member
    );
  } else {
    const newId = Math.max(0, ...this.members.map(m => m.id)) + 1;

    this.members = [
      ...this.members,
      {
        ...this.memberForm,
        id: newId,
        name,
        phone,
        email
      }
    ];
  }
```

```
    this.showForm = false;
    this.currentPage = this.totalPages;
  }

  deleteMember(member: TeamMember): void {
    if (confirm(`Are you sure you want to delete ${member.name}?`)) {
      this.members = this.members.filter(m => m.id !== member.id);
      this.currentPage = Math.min(this.currentPage, this.totalPages);
    }
  }

  cancelForm(): void {
    this.showForm = false;
  }

  getInitials(name: string): string {
    return name
      .trim()
      .split(/\s+/)
      .slice(0, 2)
      .map(part => part.charAt(0))
      .join('')
      .toUpperCase();
  }

  trackById(_index: number, member: TeamMember): number {
    return member.id;
  }
}
```

2. `team-members.html`

```
<div class="team-page">
  <div class="team-card">

    <!-- Header -->
    <header class="page-header">
      <div class="header-left">
        <div class="header-icon">
          <svg viewBox="0 0 24 24" fill="none"
               stroke="currentColor" stroke-width="2"
               stroke-linecap="round" stroke-linejoin="round">
            <path d="M16 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"/>
            <circle cx="10" cy="7" r="4"/>
            <path d="M20 8v6M23 11h-6"/>
            <path d="M16 21v-2a4 4 0 0 0-3-3.87"/>
```

```
        </svg>
      </div>

      <div>
        <h1>Team Members</h1>
        <p>Manage team members and their roles.</p>
      </div>
    </div>

    <button class="btn-primary" type="button" (click)="openAddForm()">
      <span class="plus-icon">+</span>
      Add Member
    </button>
  </header>

  <!-- Search and Filter -->
  <div class="toolbar">
    <div class="search-box">
      <svg viewBox="0 0 24 24" fill="none"
           stroke="currentColor" stroke-width="2">
        <circle cx="11" cy="11" r="7"/>
        <path d="m20 20-4-4"/>
      </svg>

      <input
        type="text"
        placeholder="Search team members..."
        [(ngModel)]="searchText"
        (ngModelChange)="onFilterChange()"
      />
    </div>

    <select
      class="role-filter"
      [(ngModel)]="selectedRole"
      (ngModelChange)="onFilterChange()"
    >
      <option *ngFor="let role of roles" [value]="role">
        {{ role }}
      </option>
    </select>
  </div>

  <!-- Table -->
  <div class="table-wrapper">
    <table class="members-table">
```

```
        <thead>
          <tr>
            <th>Name</th>
            <th>Role</th>
            <th>Phone</th>
            <th>Email</th>
            <th>Status</th>
            <th class="actions-heading">Actions</th>
          </tr>
        </thead>

        <tbody>
          <tr *ngFor="let member of paginatedMembers; trackBy: trackById">
            <td>
              <div class="member-info">
                <div
                  class="avatar"
                  [ngClass]="'avatar-' + ((member.id - 1) % 7)"
                >
                  {{ getInitials(member.name) }}
                </div>

                <span class="member-name">{{ member.name }}</span>
              </div>
            </td>

            <td>
              <span
                class="role-badge"
                [ngClass]="{
                  'role-field': member.role === 'Field Staff',
                  'role-office': member.role === 'Office Staff',
                  'role-admin': member.role === 'Admin'
                }"
              >
                {{ member.role }}
              </span>
            </td>

            <td class="phone-cell">{{ member.phone }}</td>
            <td class="email-cell">{{ member.email }}</td>

            <td>
              <span
                class="status-badge"
                [ngClass]="{
```

```
            'status-active': member.status === 'Active',
            'status-inactive': member.status === 'Inactive'
          }"
        >
          {{ member.status }}
        </span>
      </td>

      <td>
        <div class="action-buttons">
          <button
            class="icon-button edit-button"
            type="button"
            title="Edit member"
            aria-label="Edit member"
            (click)="editMember(member)"
          >
            <svg viewBox="0 0 24 24" fill="none"
                 stroke="currentColor" stroke-width="2"
                 stroke-linecap="round" stroke-linejoin="round">
              <path d="M12 20h9"/>
              <path d="M16.5 3.5a2.12 2.12 0 0 1 3 3L9 17l-4 1 1-4Z"/>
            </svg>
          </button>

          <button
            class="icon-button delete-button"
            type="button"
            title="Delete member"
            aria-label="Delete member"
            (click)="deleteMember(member)"
          >
            <svg viewBox="0 0 24 24" fill="none"
                 stroke="currentColor" stroke-width="2"
                 stroke-linecap="round" stroke-linejoin="round">
              <path d="M3 6h18"/>
              <path d="M8 6V4h8v2"/>
              <path d="m19 6-1 14H6L5 6"/>
              <path d="M10 11v5M14 11v5"/>
            </svg>
          </button>
        </div>
      </td>
    </tr>

    <tr *ngIf="filteredMembers.length === 0">
```

```
            <td colspan="6" class="empty-state">
              No team members found.
            </td>
          </tr>
        </tbody>
      </table>
    </div>

    <!-- Pagination -->
    <footer class="pagination-footer">
      <div class="pagination-summary">
        Showing {{ firstVisible }} - {{ lastVisible }} of
        {{ filteredMembers.length }}
      </div>

      <div class="pagination-controls">
        <button
          type="button"
          class="page-arrow"
          aria-label="Previous page"
          [disabled]="currentPage === 1"
          (click)="changePage(currentPage - 1)"
        >
          ‹
        </button>

        <button
          type="button"
          class="page-number"
          aria-current="page"
        >
          {{ currentPage }}
        </button>

        <button
          type="button"
          class="page-arrow"
          aria-label="Next page"
          [disabled]="currentPage === totalPages"
          (click)="changePage(currentPage + 1)"
        >
          ›
        </button>
      </div>
    </footer>
  </div>
```

```
</div>

<!-- Add/Edit Member Modal -->
<div class="modal-overlay" *ngIf="showForm" (click)="cancelForm()">
  <div
    class="member-modal"
    role="dialog"
    aria-modal="true"
    aria-labelledby="modal-title"
    (click)="$event.stopPropagation()"
  >
    <div class="modal-header">
      <h2 id="modal-title">
        {{ isEditing ? 'Edit Member' : 'Add Member' }}
      </h2>

      <button
        type="button"
        class="modal-close"
        aria-label="Close"
        (click)="cancelForm()"
      >
        ×
      </button>
    </div>

    <form (ngSubmit)="saveMember()" #memberNgForm="ngForm">
      <label for="memberName">Name</label>
      <input
        id="memberName"
        name="name"
        [(ngModel)]="memberForm.name"
        required
        placeholder="Enter full name"
      />

      <label for="memberRole">Role</label>
      <select
        id="memberRole"
        name="role"
        [(ngModel)]="memberForm.role"
        required
      >
        <option value="Field Staff">Field Staff</option>
        <option value="Office Staff">Office Staff</option>
        <option value="Admin">Admin</option>
```

```
      </select>

      <label for="memberPhone">Phone</label>
      <input
        id="memberPhone"
        name="phone"
        [(ngModel)]="memberForm.phone"
        required
        placeholder="03XX-XXXXXXX"
      />

      <label for="memberEmail">Email</label>
      <input
        id="memberEmail"
        name="email"
        type="email"
        [(ngModel)]="memberForm.email"
        required
        email
        placeholder="name@horizon.com"
      />

      <label for="memberStatus">Status</label>
      <select
        id="memberStatus"
        name="status"
        [(ngModel)]="memberForm.status"
      >
        <option value="Active">Active</option>
        <option value="Inactive">Inactive</option>
      </select>

      <div class="modal-actions">
        <button
          type="button"
          class="btn-secondary"
          (click)="cancelForm()"
        >
          Cancel
        </button>

        <button
          type="submit"
          class="btn-primary"
          [disabled]="memberNgForm.invalid"
        >
```

```
          {{ isEditing ? 'Save Changes' : 'Add Member' }}
        </button>
      </div>
    </form>
  </div>
</div>
```

3. `team-members.css`

```css
:host {
  display: block;
  min-width: 0;
  color: #17233d;
  font-family: Inter, "Segoe UI", Arial, sans-serif;
}

* {
  box-sizing: border-box;
}

.team-page {
  min-height: 100%;
  padding: 12px;
  background: #f5f8fc;
}

.team-card {
  width: 100%;
  min-width: 0;
  overflow: hidden;
  background: #fff;
  border: 1px solid #e8edf5;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(30, 50, 80, 0.035);
}

/* Header */

.page-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 16px;
  border-bottom: 1px solid #edf1f7;
}
```

```css
.header-left {
  display: flex;
  align-items: center;
  gap: 16px;
  min-width: 0;
}

.header-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  flex: 0 0 50px;
  width: 50px;
  height: 50px;
  color: #0875ff;
  background: #e3efff;
  border-radius: 50%;
}

.header-icon svg {
  width: 26px;
  height: 26px;
}

.page-header h1 {
  margin: 0;
  color: #111d36;
  font-size: 24px;
  font-weight: 700;
  letter-spacing: -0.6px;
}

.page-header p {
  margin: 4px 0 0;
  color: #657394;
  font-size: 14px;
}

.btn-primary {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 38px;
  padding: 9px 16px;
```

```
  border: 1px solid #0867f9;
  border-radius: 6px;
  background: #0867f9;
  color: #fff;
  font: inherit;
  font-size: 13px;
  font-weight: 500;
  white-space: nowrap;
  cursor: pointer;
  transition: background 0.2s, box-shadow 0.2s;
}

.btn-primary:hover:not(:disabled) {
  background: #0058e5;
  box-shadow: 0 3px 8px rgba(8, 103, 249, 0.18);
}

.btn-primary:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

.plus-icon {
  font-size: 21px;
  line-height: 15px;
  font-weight: 300;
}

/* Search and role filter */

.toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 14px;
}

.search-box {
  display: flex;
  align-items: center;
  gap: 10px;
  width: min(500px, 65%);
  height: 41px;
  padding: 0 12px;
  border: 1px solid #dce4ef;
```

```
  border-radius: 6px;
  background: #fff;
  box-shadow: 0 1px 2px rgba(15, 35, 65, 0.02);
}

.search-box:focus-within,
.role-filter:focus {
  border-color: #80b4ff;
  outline: 2px solid rgba(8, 103, 249, 0.08);
}

.search-box svg {
  flex: 0 0 17px;
  width: 17px;
  height: 17px;
  color: #6d7d98;
}

.search-box input {
  width: 100%;
  min-width: 0;
  padding: 0;
  border: 0;
  outline: none;
  background: transparent;
  color: #263550;
  font: inherit;
  font-size: 13px;
}

.search-box input::placeholder {
  color: #8190a9;
}

.role-filter {
  width: 200px;
  max-width: 42%;
  height: 41px;
  padding: 0 12px;
  border: 1px solid #dce4ef;
  border-radius: 6px;
  background: #fff;
  color: #1b2942;
  font: inherit;
  font-size: 13px;
  cursor: pointer;
```

```
}

/* Table */

.table-wrapper {
  width: calc(100% - 28px);
  margin: 0 14px;
  overflow-x: auto;
  border: 1px solid #edf1f7;
  border-radius: 7px;
}

.members-table {
  width: 100%;
  min-width: 750px;
  border-collapse: collapse;
  text-align: left;
}

.members-table thead {
  background: #f4f7fb;
}

.members-table th {
  height: 41px;
  padding: 0 14px;
  color: #18243b;
  font-size: 12px;
  font-weight: 650;
  white-space: nowrap;
}

.members-table td {
  height: 52px;
  padding: 6px 14px;
  border-top: 1px solid #edf1f7;
  color: #283750;
  font-size: 12.5px;
  white-space: nowrap;
}

.members-table tbody tr {
  transition: background 0.15s;
}

.members-table tbody tr:hover {
```

```
  background: #fafcff;
}

.member-info {
  display: flex;
  align-items: center;
  gap: 12px;
}

.avatar {
  display: flex;
  align-items: center;
  justify-content: center;
  flex: 0 0 38px;
  width: 38px;
  height: 38px;
  border-radius: 50%;
  font-size: 12px;
  font-weight: 500;
}

.avatar-0 {
  color: #0759df;
  background: #dcecff;
}

.avatar-1 {
  color: #6535ee;
  background: #e9e0ff;
}

.avatar-2 {
  color: #008b69;
  background: #d9f7e9;
}

.avatar-3 {
  color: #c66a00;
  background: #fff0d4;
}

.avatar-4 {
  color: #d82d36;
  background: #ffe3e4;
}
```

```css
.avatar-5 {
  color: #0878bf;
  background: #d9f4ff;
}

.avatar-6 {
  color: #26354d;
  background: #e9edf2;
}

.member-name {
  color: #18243b;
  font-weight: 500;
}

.role-badge,
.status-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 24px;
  padding: 4px 12px;
  border-radius: 9px;
  font-size: 11.5px;
  font-weight: 500;
}

.role-field {
  color: #0064e9;
  background: #e6f1ff;
}

.role-office {
  color: #6336e9;
  background: #eee7ff;
}

.role-admin {
  color: #b85a00;
  background: #fff0d5;
}

.status-active {
  color: #00875e;
  background: #d9f8e9;
}
```

```css
.status-inactive {
  color: #b42332;
  background: #ffe4e6;
}

.phone-cell,
.email-cell {
  color: #40516e !important;
}

.actions-heading {
  text-align: center;
}

.action-buttons {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
}

.icon-button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 34px;
  height: 35px;
  padding: 7px;
  border: 1px solid #dce4ef;
  border-radius: 7px;
  background: #fff;
  cursor: pointer;
  transition: background 0.15s, border-color 0.15s;
}

.icon-button svg {
  width: 16px;
  height: 16px;
}

.edit-button {
  color: #52637d;
}

.edit-button:hover {
```

```
  border-color: #a8caff;
  background: #eef5ff;
  color: #0867f9;
}

.delete-button {
  border-color: #ffe3e6;
  background: #fff0f1;
  color: #f04452;
}

.delete-button:hover {
  border-color: #ffc5cb;
  background: #ffe0e3;
}

.empty-state {
  height: 80px !important;
  color: #7b8aa2 !important;
  text-align: center;
}

/* Pagination */

.pagination-footer {
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 16px;
  padding: 20px 24px;
}

.pagination-summary {
  color: #75839d;
  font-size: 12px;
}

.pagination-controls {
  display: flex;
  align-items: center;
  gap: 8px;
}

.page-arrow,
.page-number {
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  width: 35px;
  height: 36px;
  border: 1px solid #e2e9f2;
  border-radius: 6px;
  background: #fff;
  color: #536681;
  font: inherit;
  font-size: 18px;
  cursor: pointer;
}

.page-arrow:disabled {
  background: #f9fbfd;
  color: #b2bfd0;
  cursor: not-allowed;
}

.page-number {
  border-color: #0867f9;
  background: #0867f9;
  color: #fff;
  font-size: 13px;
  font-weight: 600;
}

/* Add/Edit modal */

.modal-overlay {
  position: fixed;
  z-index: 1000;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  overflow-y: auto;
  background: rgba(15, 29, 52, 0.42);
}

.member-modal {
  width: 100%;
  max-width: 460px;
  max-height: calc(100vh - 40px);
  overflow-y: auto;
```

```
  padding: 24px;
  border: 1px solid #e7edf5;
  border-radius: 12px;
  background: #fff;
  box-shadow: 0 18px 55px rgba(20, 38, 65, 0.18);
}

.modal-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 22px;
}

.modal-header h2 {
  margin: 0;
  color: #15223b;
  font-size: 20px;
}

.modal-close {
  width: 32px;
  height: 32px;
  border: 0;
  border-radius: 6px;
  background: #f2f5f9;
  color: #53627a;
  font-size: 23px;
  cursor: pointer;
}

.member-modal form {
  display: flex;
  flex-direction: column;
}

.member-modal label {
  margin: 0 0 7px;
  color: #34445d;
  font-size: 13px;
  font-weight: 600;
}

.member-modal input,
.member-modal select {
```

```
  width: 100%;
  height: 40px;
  margin-bottom: 16px;
  padding: 0 11px;
  border: 1px solid #dce4ef;
  border-radius: 6px;
  outline: none;
  background: #fff;
  color: #263550;
  font: inherit;
  font-size: 13px;
}

.member-modal input:focus,
.member-modal select:focus {
  border-color: #80b4ff;
  box-shadow: 0 0 0 3px rgba(8, 103, 249, 0.08);
}

.modal-actions {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  margin-top: 8px;
}

.btn-secondary {
  min-height: 38px;
  padding: 8px 16px;
  border: 1px solid #dce4ef;
  border-radius: 6px;
  background: #fff;
  color: #34445d;
  font: inherit;
  font-size: 13px;
  cursor: pointer;
}

.btn-secondary:hover {
  background: #f7f9fc;
}

/* Responsive layout */

@media (max-width: 650px) {
  .team-page {
```

```
  padding: 6px;
}

.page-header {
  align-items: flex-start;
  flex-wrap: wrap;
  padding: 14px;
}

.header-left {
  gap: 10px;
}

.header-icon {
  flex-basis: 42px;
  width: 42px;
  height: 42px;
}

.page-header h1 {
  font-size: 20px;
}

.page-header p {
  font-size: 12px;
}

.page-header > .btn-primary {
  margin-left: auto;
}

.toolbar {
  align-items: stretch;
  flex-direction: column;
  gap: 10px;
}

.search-box,
.role-filter {
  width: 100%;
  max-width: none;
}

.pagination-footer {
  flex-wrap: wrap;
  gap: 12px;
```

```
    padding: 16px;
  }

  .pagination-summary {
    margin-right: auto;
  }
}
```

4. Add the component to your Angular app

If you are using standalone routing, add a route in `app.routes.ts`:

```
import { Routes } from '@angular/router';
import { TeamMembersComponent } from './team-members/team-members';

export const routes: Routes = [
  {
    path: 'team-members',
    component: TeamMembersComponent
  }
];
```

Adjust the import path if your component is in a different folder.

Then navigate to:

```
http://localhost:4200/team-members
```

Important: This version uses in-memory sample data, matching the seven members in your screenshot. Add an Angular service connected to your ASP.NET Core API to persist additions, edits, and deletions in SQL Server.

25/10/2026, 17:54:06 · gpt-6

_debug_node-chunk.mjs:11046 Angular is running in development mode.
installHook.js:1 JIT compilation failed for component class TeamMembersComponent2 {
  searchText = "";
  selectedRole = "All Roles";
  currentPage = 1;
  pageSize = 7;
  showForm = false;
  isEditing = false;
  editingId = null;
  roles = ["All Roles", "Field…
overrideMethod @ installHook.js:1
(anonymous) @ _debug_node-chunk.mjs:131
(anonymous) @ _debug_node-chunk.mjs:17749
getComponentDef @ _effect-chunk2.mjs:489
(anonymous) @ _debug_node-chunk.mjs:9003
activateWith @ _router-chunk.mjs:1741
activateRoutes @ _router-chunk.mjs:2131
(anonymous) @ _router-chunk.mjs:2095
activateChildRoutes @ _router-chunk.mjs:2094
activateRoutes @ _router-chunk.mjs:2109
(anonymous) @ _router-chunk.mjs:2095
activateChildRoutes @ _router-chunk.mjs:2094
activate @ _router-chunk.mjs:2021
(anonymous) @ _router-chunk.mjs:3728
(anonymous) @ tap.js:17
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ take.js:12
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14

(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ map.js:7
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ map.js:6
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ map.js:7
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ map.js:6
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14

(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ tap.js:18
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ mergeInternals.js:25
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ mergeInternals.js:25
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ mergeInternals.js:19
(anonymous) @ mergeInternals.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ takeLast.js:18
(anonymous) @ OperatorSubscriber.js:38
(anonymous) @ Subscriber.js:52
(anonymous) @ tap.js:23
(anonymous) @ OperatorSubscriber.js:38
(anonymous) @ Subscriber.js:52
(anonymous) @ mergeInternals.js:11
(anonymous) @ mergeInternals.js:55
(anonymous) @ OperatorSubscriber.js:38
(anonymous) @ Subscriber.js:52
(anonymous) @ innerFrom.js:87
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10

(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ takeLast.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ mergeInternals.js:19
(anonymous) @ mergeInternals.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ mergeInternals.js:25
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ map.js:7
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ mergeInternals.js:25
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34

```
(anonymous) @ defaultIfEmpty.js:11
(anonymous) @ OperatorSubscriber.js:38
(anonymous) @ Subscriber.js:52
(anonymous) @ Subscriber.js:75
(anonymous) @ Subscriber.js:52
(anonymous) @ Subscriber.js:75
(anonymous) @ Subscriber.js:52
(anonymous) @ mergeInternals.js:11
(anonymous) @ mergeInternals.js:55
(anonymous) @ OperatorSubscriber.js:38
(anonymous) @ Subscriber.js:52
(anonymous) @ innerFrom.js:53
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ filter.js:6
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ take.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
Promise.then
(anonymous) @ innerFrom.js:59
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ mergeInternals.js:19
(anonymous) @ mergeInternals.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:61
Promise.then
(anonymous) @ innerFrom.js:59
```

(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ innerFrom.js:51
(anonymous) @ Observable.js:38
(anonymous) @ Observable.js:32
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23

```
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ map.js:6
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
mergeInternals @ mergeInternals.js:53
(anonymous) @ mergeMap.js:14
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ take.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:10
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
```

```
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ takeUntil.js:8
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ tap.js:15
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ takeUntil.js:8
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ finalize.js:5
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ catchError.js:9
(anonymous) @ lift.js:10
(anonymous) @ Observable.js:27
errorContext @ errorContext.js:19
(anonymous) @ Observable.js:23
(anonymous) @ switchMap.js:14
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ filter.js:6
(anonymous) @ OperatorSubscriber.js:15
(anonymous) @ Subscriber.js:34
(anonymous) @ Subject.js:41
errorContext @ errorContext.js:19
(anonymous) @ Subject.js:31
(anonymous) @ BehaviorSubject.js:31
(anonymous) @ _router-chunk.mjs:3540
untracked @ _untracked-chunk.mjs:95
(anonymous) @ _resource-chunk.mjs:65
handleNavigationRequest @ _router-chunk.mjs:3539
scheduleNavigation @ _router-chunk.mjs:4443
navigateByUrl @ _router-chunk.mjs:4373
```

navigate @ _router-chunk.mjs:4379
selectMenu @ dashboard.ts:116
Dashboard_For_9_Template_button_click_0_listener @ dashboard.html:17
(anonymous) @ _debug_node-chunk.mjs:8247
(anonymous) @ _debug_node-chunk.mjs:8234
(anonymous) @ _dom_renderer-chunk.mjs:566
installHook.js:1 ERROR Error: The component 'TeamMembersComponent2' needs to be compiled using the JIT compiler, but '@angular/compiler' is not available.

JIT compilation is discouraged for production use-cases! Consider using AOT mode instead.
Alternatively, the JIT compiler should be loaded by bootstrapping using '@angular/platform-browser-dynamic' or '@angular/platform-server',
or manually provide the compiler with 'import "@angular/compiler";' before bootstrapping.
    at getCompilerFacade (_debug_node-chunk.mjs:143:11)
    at TeamMembersComponent2.get (_debug_node-chunk.mjs:17749:26)
    at getComponentDef (_effect-chunk2.mjs:489:10)
    at _R3ViewContainerRef.createComponent (_debug_node-chunk.mjs:9003:23)
    at _RouterOutlet.activateWith (_router-chunk.mjs:1741:31)
    at ActivateRoutes.activateRoutes (_router-chunk.mjs:2131:28)
    at anonymousCallbackTo: futureNode.children.forEach (_router-chunk.mjs:2095:12)
    at Array.forEach (<anonymous>)
    at ActivateRoutes.activateChildRoutes (_router-chunk.mjs:2094:25)
    at ActivateRoutes.activateRoutes (_router-chunk.mjs:2109:14)
overrideMethod @ installHook.js:1
ErrorHandler.handleError @ _effect-chunk2.mjs:2643
anonymousCallbackTo: zone.runOutsideAngular @ _effect-chunk2.mjs:2659
NoopNgZone.runOutsideAngular @ _effect-chunk2.mjs:2618
anonymousFunction @ _effect-chunk2.mjs:2652
rejectionListener @ _effect-chunk2.mjs:2688
[NEW] Explain Console errors by using Copilot in Edge: click to explain an error Learn moreDon’t show again

25/10/2026, 18:02:30

The error is related to Angular component compilation, not your CSS or the team-members table logic.

The key message is:

> *The component `TeamMembersComponent2` needs to be compiled using the JIT compiler, but `@angular/compiler` is not available.*

Since your project uses Angular 21 with standalone components, **the first thing to check is how `TeamMembersComponent` is imported and registered in your routing or dashboard navigation**.

1. Check `team-members.ts`

Make sure the component has the correct standalone configuration and file paths:

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-team-members',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './team-members.html',
  styleUrl: './team-members.css'
})
export class TeamMembersComponent {
  // Keep your existing component code here.
}
```

Do not create a second component class named `TeamMembersComponent2`. Keep one exported `TeamMembersComponent` class.

2. Check `app.routes.ts`

If you are using Angular standalone routing, register the component directly:

```
import { Routes } from '@angular/router';
import { TeamMembersComponent } from './team-members/team-members';

export const routes: Routes = [
  {
    path: 'team-members',
    component: TeamMembersComponent
```

```
  }
];
```

Adjust the import path to match your actual folder structure.

If you already have other routes, **add this route to your existing array instead of replacing the other routes**.

3. Check `dashboard.ts`

Your stack trace shows that navigation starts from:

```
selectMenu @ dashboard.ts:116
```

If your dashboard dynamically creates components, check whether you are using `ViewContainerRef.createComponent()` or loading a component through a menu configuration.

For a routed dashboard, prefer Angular Router navigation:

```
import { Router } from '@angular/router';
import { inject } from '@angular/core';

export class DashboardComponent {
  private router = inject(Router);

  selectMenu(menu: string): void {
    if (menu === 'Team Members') {
      this.router.navigate(['/team-members']);
    }
  }
}
```

Adapt this example to your existing `selectMenu()` logic; do not replace unrelated menu behavior.

4. Restart Angular

After correcting the component registration:

1. Stop the Angular development server with `Ctrl + C`.
2. Run:

```
ng serve
```

3. Refresh the browser and open the Team Members page again.

You should not need to install `@angular/compiler` just to fix this error in a correctly configured standalone application.

If the error continues

Please share these two files so I can identify the exact cause in your project:

- `dashboard.ts` — especially `selectMenu()` around line 116.
- `app.routes.ts` — or whichever file registers your routes.

Your stack trace suggests the issue may be in the way the dashboard activates the component, rather than in the Team Members component itself.

25/10/2026, 18:02:36 · gpt-6

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-dashboard',
   imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
export class Dashboard {
private router = inject(Router);

  equipmentStats = [
    {
      title: 'Total Equipment',
      value: 25,
      icon: '▣',
      color: 'blue',
      detail: 'Registered assets'
    },
    {
      title: 'Checked Out',
      value: 12,
      icon: '▤',
      color: 'red',
      detail: 'Currently in use'
    },
    {
      title: 'Available',
      value: 10,
      icon: '▦',
      color: 'green',
      detail: 'Ready to use'
    },
    {
      title: 'Pending Approval',
      value: 2,
      icon: '♧',
      color: 'yellow',
      detail: 'Awaiting review'
    }
  ];

  equipmentStatus = [
```

```
  { label: 'Available', value: 10, color: '#119c78' },
  { label: 'Checked Out', value: 12, color: '#b98d61' },
  { label: 'Pending Approval', value: 2, color: '#f4bd45' },
  { label: 'Under Repair', value: 1, color: '#f28b47' }
];

activities = [
  {
    type: 'success',
    title: 'Ali Raza took GPS Rover-01',
    time: '2 hours ago',
    icon: '↗'
  },
  {
    type: 'success',
    title: 'Usman returned Total Station-02',
    time: '5 hours ago',
    icon: '↙'
  },
  {
    type: 'danger',
    title: 'Damage reported for Field Tablet',
    time: '1 day ago',
    icon: '!'
  },
  {
    type: 'danger',
    title: 'Drone request pending approval',
    time: '1 day ago',
    icon: '!'
  }
];

menuItems = [
  { label: 'Dashboard', icon: '⌂', active: true },
  { label: 'Equipment', icon: '▦', active: false },
  { label: 'Check Out', icon: '↗', active: false },
  { label: 'Check In', icon: '↙', active: false },
  { label: 'Approvals', icon: '☑', active: false },
  { label: 'My Equipment', icon: '♙', active: false },
  { label: 'Damage Reports', icon: '⚒', active: false },
  { label: 'Reports', icon: '▤', active: false },
  { label: 'Team Members', icon: '♧', active: false }
];

selectedMenu = 'Dashboard';
```

```
  showAllActivities = false;
  showProfileMenu = false;

  get visibleActivities() {
    return this.showAllActivities
      ? this.activities
      : this.activities.slice(0, 4);
  }

 selectMenu(label: string): void {
    const routes: Record<string, string> = {
      Dashboard: '/dashboard',
      Equipment: '/equipment',
      'Check Out': '/check-out',
      'Check In': '/check-in',
      Approvals: '/approvals',
      'My Equipment': '/my-equipment',
      'Damage Reports': '/damage-reports',
      Reports: '/reports',
      'Team Members': '/team-members'
    };

    const path = routes[label];

    if (path) {
      this.router.navigate([path]);
    }

    if (label === 'Check Out') {
      this.router.navigate(['/check-out']);
    }
  }

  toggleActivities(): void {
    this.showAllActivities = !this.showAllActivities;
  }

  toggleProfile(): void {
    this.showProfileMenu = !this.showProfileMenu;
  }
}
import { Routes } from '@angular/router';
import { Login } from './components/login/login';
import { Dashboard } from
'./components/dashboard/dashboard';
import { Equipment } from
```

```
'./components/equipment/equipment';
import { DashboardHome } from './components/dashboard-
home/dashboard-home';
import { Checkout } from './components/checkout/checkout';
import { Checkin } from './components/checkin/checkin';
import { Approvals } from './components/approvals/approvals';
import { Myequipment } from
'./components/myequipment/myequipment';
import { Damagereports } from
'./components/damagereports/damagereports';
import { Reports } from './components/reports/reports';
import { TeamMembersComponent } from
'./components/teamsmembers/teamsmembers';


export const routes: Routes = [
   { path: 'login', component: Login },
  {
    path: '',
    component: Dashboard,
    children: [
      { path: 'dashboard', component: DashboardHome },
      { path: 'equipment', component: Equipment },
      { path: 'check-out', component: Checkout },
      { path: 'check-in', component: Checkin },
      { path: 'approvals', component: Approvals },
      { path: 'approvals/:id',component: Approvals},
      { path: 'my-equipment', component: Myequipment },
      { path: 'damage-reports', component: Damagereports },
      { path: 'reports', component: Reports },
       {path: 'team-members',component:
TeamMembersComponent},
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },
  { path: '**', redirectTo: 'login' }
];
```

25/10/2026, 18:06:10

Your `app.routes.ts` looks correct: `TeamMembersComponent` is registered under `/team-members`, and your dashboard navigates to that route correctly.

The likely issue is that **your `Dashboard` component is missing `standalone: true`**, or `TeamMembersComponent` is not configured as a standalone component. Your other components may

have the same configuration issue.

1. Fix `dashboard.ts`

Update the component decorator to include `standalone: true`:

```
import { Component, inject } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './dashboard.html',
  styleUrl: './dashboard.css',
})
export class Dashboard {
  private router = inject(Router);

  // Keep all your existing properties and methods here.
}
```

You do not need to change your existing `selectMenu()` method.

2. Check `teamsmembers.ts`

Open:

`src/app/components/teamsmembers/teamsmembers.ts`

Make sure the component is configured like this:

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-team-members',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './teamsmembers.html',
  styleUrl: './teamsmembers.css'
})
export class TeamMembersComponent {
  // Keep your existing Team Members component code here.
}
```

Important: Your route imports the component from `./components/teamsmembers/teamsmembers`, so use the actual filenames in that folder. The filenames in the example above match that import pattern.

3. Keep `app.routes.ts` as it is

This route is already correct:

```
{
  path: 'team-members',
  component: TeamMembersComponent
}
```

You don't need to import `TeamMembersComponent` inside `Dashboard`, because Angular Router handles that navigation.

4. Restart the development server

Stop the server with `Ctrl + C`, then run:

```
ng serve
```

Open the Team Members page again through your dashboard.

If the same JIT error persists, send me the actual `teamsmembers.ts` file. The stack trace alone does not confirm the exact cause, and I can check its component metadata and imports directly.

25/10/2026, 18:06:13 · gpt-6

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface TeamMember {
  id: number;
  name: string;
  role: 'Field Staff' | 'Office Staff' | 'Admin';
  phone: string;
  email: string;
  status: 'Active' | 'Inactive';
}

@Component({
  selector: 'app-team-members',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './teamsmembers.html',
  styleUrl: './teamsmembers.css'
})
export class TeamMembers {
  searchText = '';
  selectedRole = 'All Roles';

  currentPage = 1;
  pageSize = 7;

  showForm = false;
  isEditing = false;
  editingId: number | null = null;

  roles = ['All Roles', 'Field Staff', 'Office Staff', 'Admin'];

  memberForm: TeamMember = this.emptyMember();

  members: TeamMember[] = [
    {
      id: 1,
      name: 'Ali Raza',
      role: 'Field Staff',
      phone: '0311-1234567',
      email: 'ali.raza@horizon.com',
      status: 'Active'
    },
```

```
  {
    id: 2,
    name: 'Hamza Khan',
    role: 'Field Staff',
    phone: '0321-2345678',
    email: 'hamza@horizon.com',
    status: 'Active'
  },
  {
    id: 3,
    name: 'Usman Sheikh',
    role: 'Field Staff',
    phone: '0300-3456789',
    email: 'usman@horizon.com',
    status: 'Active'
  },
  {
    id: 4,
    name: 'Bilal Ahmed',
    role: 'Field Staff',
    phone: '0333-4567890',
    email: 'bilal@horizon.com',
    status: 'Active'
  },
  {
    id: 5,
    name: 'Asif Raza',
    role: 'Field Staff',
    phone: '0345-5678901',
    email: 'asif@horizon.com',
    status: 'Active'
  },
  {
    id: 6,
    name: 'Saima Durrani',
    role: 'Office Staff',
    phone: '0322-6789012',
    email: 'saima@horizon.com',
    status: 'Active'
  },
  {
    id: 7,
    name: 'Ahmad',
    role: 'Admin',
    phone: '0311-1409072',
    email: 'ahmad@horizon.com',
```

```
      status: 'Active'
    }
  ];

  private emptyMember(): TeamMember {
    return {
      id: 0,
      name: '',
      role: 'Field Staff',
      phone: '',
      email: '',
      status: 'Active'
    };
  }

  get filteredMembers(): TeamMember[] {
    const search = this.searchText.trim().toLowerCase();

    return this.members.filter(member => {
      const matchesSearch =
        member.name.toLowerCase().includes(search) ||
        member.phone.toLowerCase().includes(search) ||
        member.email.toLowerCase().includes(search);

      const matchesRole =
        this.selectedRole === 'All Roles' ||
        member.role === this.selectedRole;

      return matchesSearch && matchesRole;
    });
  }

  get totalPages(): number {
    return Math.max(1, Math.ceil(this.filteredMembers.length /
this.pageSize));
  }

  get paginatedMembers(): TeamMember[] {
    const start = (this.currentPage - 1) * this.pageSize;
    return this.filteredMembers.slice(start, start +
this.pageSize);
  }

  get firstVisible(): number {
    return this.filteredMembers.length === 0
      ? 0
```

```
    : (this.currentPage - 1) * this.pageSize + 1;
}

get lastVisible(): number {
  return Math.min(
    this.currentPage * this.pageSize,
    this.filteredMembers.length
  );
}

onFilterChange(): void {
  this.currentPage = 1;
}

changePage(page: number): void {
  if (page >= 1 && page <= this.totalPages) {
    this.currentPage = page;
  }
}

openAddForm(): void {
  this.isEditing = false;
  this.editingId = null;
  this.memberForm = this.emptyMember();
  this.showForm = true;
}

editMember(member: TeamMember): void {
  this.isEditing = true;
  this.editingId = member.id;
  this.memberForm = { ...member };
  this.showForm = true;
}

saveMember(): void {
  const name = this.memberForm.name.trim();
  const phone = this.memberForm.phone.trim();
  const email = this.memberForm.email.trim();

  if (!name || !phone || !email) {
    alert('Please fill in all required fields.');
    return;
  }

  if (this.isEditing && this.editingId !== null) {
    this.members = this.members.map(member =>
```

```
        member.id === this.editingId
          ? { ...this.memberForm, id: this.editingId, name, phone,
email }
          : member
      );
    } else {
      const newId = Math.max(0, ...this.members.map(m =>
m.id)) + 1;

      this.members = [
        ...this.members,
        {
          ...this.memberForm,
          id: newId,
          name,
          phone,
          email
        }
      ];
    }

    this.showForm = false;
    this.currentPage = this.totalPages;
  }

  deleteMember(member: TeamMember): void {
    if (confirm(`Are you sure you want to delete
${member.name}?`)) {
      this.members = this.members.filter(m => m.id !==
member.id);
      this.currentPage = Math.min(this.currentPage,
this.totalPages);
    }
  }

  cancelForm(): void {
    this.showForm = false;
  }

  getInitials(name: string): string {
    return name
      .trim()
      .split(/\s+/)
      .slice(0, 2)
      .map(part => part.charAt(0))
      .join('')
```

```
      .toUpperCase();
  }

  trackById(_index: number, member: TeamMember): number {
    return member.id;
  }
}
```

please implement the apis of these ??

25/10/2026, 18:10:30

We can connect your **Team Members** Angular component to your existing ASP.NET Core API and SQL Server database, replacing the hardcoded sample data with real API calls.

The implementation will support:

- **GET** — Load team members from the database.
- **POST** — Add a new team member.
- **PUT** — Update an existing team member.
- **DELETE** — Delete a team member.
- Search, role filtering, pagination, and the existing UI.
- Loading and error handling.

Your current component uses `TeamMember` fields `id`, `name`, `role`, `phone`, `email`, and `status`. We'll retain those fields in the Angular UI. The backend entity and API DTOs must map to the same data.

1. Create `team-member.ts` model

Path: `src/app/models/team-member.ts`

```ts
export type TeamRole = 'Field Staff' | 'Office Staff' | 'Admin';
export type TeamStatus = 'Active' | 'Inactive';

export interface TeamMember {
  id: number;
  name: string;
  role: TeamRole;
  phone: string;
  email: string;
  status: TeamStatus;
}
```

```ts
export interface CreateTeamMemberDto {
  name: string;
  role: TeamRole;
  phone: string;
  email: string;
  status: TeamStatus;
}

export type UpdateTeamMemberDto = CreateTeamMemberDto;
```

2. Create `team-members.service.ts`

Path: `src/app/services/team-members.service.ts`

This service calls your ASP.NET Core API. It assumes your controller route is `api/TeamMembers`.

```ts
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';

import {
  TeamMember,
  CreateTeamMemberDto,
  UpdateTeamMemberDto
} from '../models/team-member';

@Injectable({
  providedIn: 'root'
})
export class TeamMembersService {
  private readonly http = inject(HttpClient);
  private readonly apiUrl = `${environment.apiUrl}/TeamMembers`;

  getAll(): Observable<TeamMember[]> {
    return this.http.get<TeamMember[]>(this.apiUrl);
  }

  getById(id: number): Observable<TeamMember> {
    return this.http.get<TeamMember>(`${this.apiUrl}/${id}`);
  }

  create(dto: CreateTeamMemberDto): Observable<TeamMember> {
    return this.http.post<TeamMember>(this.apiUrl, dto);
  }
```

```
  update(id: number, dto: UpdateTeamMemberDto): Observable<TeamMember> {
    return this.http.put<TeamMember>(`${this.apiUrl}/${id}`, dto);
  }

  delete(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Your existing environment should have:

```
export const environment = {
  production: false,
  apiUrl: 'https://localhost:7135/api'
};
```

Keep your actual API URL if it differs.

3. Update `teamsmembers.ts`

Keep your existing HTML and CSS. Replace the TypeScript with this API-connected version:

```
import { CommonModule } from '@angular/common';
import { Component, OnInit, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';

import { TeamMembersService } from '../../services/team-members.service';
import {
  TeamMember,
  TeamRole,
  TeamStatus,
  CreateTeamMemberDto
} from '../../models/team-member';

@Component({
  selector: 'app-team-members',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './teamsmembers.html',
  styleUrl: './teamsmembers.css'
})
export class TeamMembers implements OnInit {
  private readonly teamService = inject(TeamMembersService);

  searchText = '';
  selectedRole = 'All Roles';
```

```
  currentPage = 1;
  pageSize = 7;

  showForm = false;
  isEditing = false;
  editingId: number | null = null;

  isLoading = false;
  isSaving = false;
  errorMessage = '';

  roles = ['All Roles', 'Field Staff', 'Office Staff', 'Admin'];

  memberForm: CreateTeamMemberDto = this.emptyMember();

  members: TeamMember[] = [];

  ngOnInit(): void {
    this.loadMembers();
  }

  private emptyMember(): CreateTeamMemberDto {
    return {
      name: '',
      role: 'Field Staff',
      phone: '',
      email: '',
      status: 'Active'
    };
  }

  loadMembers(): void {
    this.isLoading = true;
    this.errorMessage = '';

    this.teamService.getAll().subscribe({
      next: (members) => {
        this.members = members;
        this.currentPage = Math.min(this.currentPage, this.totalPages);
        this.isLoading = false;
      },
      error: (error) => {
        console.error('Error loading team members:', error);
        this.errorMessage =
          error.error?.message ?? 'Unable to load team members.';
        this.isLoading = false;
```

```
    }
  });
}

get filteredMembers(): TeamMember[] {
  const search = this.searchText.trim().toLowerCase();

  return this.members.filter(member => {
    const matchesSearch =
      member.name.toLowerCase().includes(search) ||
      member.phone.toLowerCase().includes(search) ||
      member.email.toLowerCase().includes(search);

    const matchesRole =
      this.selectedRole === 'All Roles' ||
      member.role === this.selectedRole;

    return matchesSearch && matchesRole;
  });
}

get totalPages(): number {
  return Math.max(1, Math.ceil(this.filteredMembers.length / this.pageSize));
}

get paginatedMembers(): TeamMember[] {
  const start = (this.currentPage - 1) * this.pageSize;
  return this.filteredMembers.slice(start, start + this.pageSize);
}

get firstVisible(): number {
  return this.filteredMembers.length === 0
    ? 0
    : (this.currentPage - 1) * this.pageSize + 1;
}

get lastVisible(): number {
  return Math.min(
    this.currentPage * this.pageSize,
    this.filteredMembers.length
  );
}

onFilterChange(): void {
  this.currentPage = 1;
}
```

```
changePage(page: number): void {
  if (page >= 1 && page <= this.totalPages) {
    this.currentPage = page;
  }
}

openAddForm(): void {
  this.isEditing = false;
  this.editingId = null;
  this.memberForm = this.emptyMember();
  this.errorMessage = '';
  this.showForm = true;
}

editMember(member: TeamMember): void {
  this.isEditing = true;
  this.editingId = member.id;

  this.memberForm = {
    name: member.name,
    role: member.role,
    phone: member.phone,
    email: member.email,
    status: member.status
  };

  this.errorMessage = '';
  this.showForm = true;
}

saveMember(): void {
  const name = this.memberForm.name.trim();
  const phone = this.memberForm.phone.trim();
  const email = this.memberForm.email.trim();

  if (!name || !phone || !email) {
    this.errorMessage = 'Please fill in all required fields.';
    return;
  }

  if (this.isSaving) {
    return;
  }

  const dto: CreateTeamMemberDto = {
```

```
      name,
      role: this.memberForm.role,
      phone,
      email,
      status: this.memberForm.status
    };

    this.isSaving = true;
    this.errorMessage = '';

    const request$ =
      this.isEditing && this.editingId !== null
        ? this.teamService.update(this.editingId, dto)
        : this.teamService.create(dto);

    request$.subscribe({
      next: (savedMember) => {
        if (this.isEditing) {
          this.members = this.members.map(member =>
            member.id === savedMember.id ? savedMember : member
          );
        } else {
          this.members = [...this.members, savedMember];
        }

        this.showForm = false;
        this.isSaving = false;
        this.currentPage = this.totalPages;
      },
      error: (error) => {
        console.error('Error saving team member:', error);
        this.errorMessage =
          error.error?.message ?? 'Unable to save team member.';
        this.isSaving = false;
      }
    });
  }

  deleteMember(member: TeamMember): void {
    if (!confirm(`Are you sure you want to delete ${member.name}?`)) {
      return;
    }

    this.errorMessage = '';

    this.teamService.delete(member.id).subscribe({
```

```
      next: () => {
        this.members = this.members.filter(m => m.id !== member.id);
        this.currentPage = Math.min(this.currentPage, this.totalPages);
      },
      error: (error) => {
        console.error('Error deleting team member:', error);
        this.errorMessage =
          error.error?.message ?? 'Unable to delete team member.';
      }
    });
  }

  cancelForm(): void {
    this.showForm = false;
    this.errorMessage = '';
  }

  getInitials(name: string): string {
    return name
      .trim()
      .split(/\s+/)
      .slice(0, 2)
      .map(part => part.charAt(0))
      .join('')
      .toUpperCase();
  }

  trackById(_index: number, member: TeamMember): number {
    return member.id;
  }
}
```

The `TeamRole` and `TeamStatus` imports above can be omitted if your TypeScript compiler reports that they are unused.

4. Make two small HTML updates

Your existing table and modal HTML can remain unchanged. Add loading and error messages just below the toolbar in `teamsmembers.html`:

```
<div *ngIf="isLoading" class="empty-state">
  Loading team members...
</div>

<div *ngIf="errorMessage" class="error-message" role="alert">
```

```
  {{ errorMessage }}
</div>
```

Update the form's submit button to prevent duplicate requests:

```
<button
  type="submit"
  class="btn-primary"
  [disabled]="memberNgForm.invalid || isSaving"
>
  {{
    isSaving
      ? 'Saving...'
      : isEditing
        ? 'Save Changes'
        : 'Add Member'
  }}
</button>
```

Add this to the end of `teamsmembers.css`:

```
.error-message {
  margin: 0 14px 12px;
  padding: 10px 12px;
  border: 1px solid #fecaca;
  border-radius: 6px;
  background: #fff1f2;
  color: #b42332;
  font-size: 13px;
}
```

5. Backend API requirements

The Angular code expects these endpoints:

Method	Endpoint	Purpose
GET	`/api/TeamMembers`	Return all team members
GET	`/api/TeamMembers/{id}`	Return one member
POST	`/api/TeamMembers`	Create a member
PUT	`/api/TeamMembers/{id}`	Update a member
DELETE	`/api/TeamMembers/{id}`	Delete a member

The request body for creating or updating a member should look like:

```
{
  "name": "Ali Raza",
  "role": "Field Staff",
  "phone": "0311-1234567",
  "email": "ali.raza@horizon.com",
  "status": "Active"
}
```

Ensure `provideHttpClient()` is configured in `app.config.ts`, as in your existing Angular application.

One important point: I have not seen your existing team-member entity or backend controller, so `/api/TeamMembers` is an assumed route. If you share your current `Models` folder structure and `ApplicationDbContext.cs`, I can provide the matching ASP.NET Core model, DTOs, repository, service, controller, and migration for your actual database.

25/10/2026, 18:10:33 · gpt-6

i want to implement the backend dotnet first please give me the code of apis etc ??

25/10/2026, 18:11:28

We'll implement the **Team Members backend first** using your existing ASP.NET Core Web API, repository/service architecture, Entity Framework Core, and SQL Server.

The API will support creating, viewing, updating, and deleting team members, matching the fields in your Angular screen.

Backend structure

Add these files to your existing `SolvefyTask` project:

Models

`TeamMember.cs`

DTOs

`CreateTeamMemberDto.cs` · `UpdateTeamMemberDto.cs` · `TeamMemberResponseDto.cs`

Repositories

`ITeamMemberRepository.cs` · `TeamMemberRepository.cs`

Services

`ITeamMemberService.cs` · `TeamMemberService.cs`

Controllers

`TeamMembersController.cs`

We'll use the route `/api/TeamMembers`. The examples below assume your existing namespace is `SolvefyTask`, your EF Core context is `ApplicationDbContext`, and your context is in `SolvefyTask.Data`.

1. Create the model

File: `Models/TeamMember.cs`

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.Models
{
    public class TeamMember
```

```
    {
        public int Id { get; set; }

        [Required]
        [MaxLength(150)]
        public string Name { get; set; } = string.Empty;

        [Required]
        [MaxLength(50)]
        public string Role { get; set; } = "Field Staff";

        [Required]
        [MaxLength(20)]
        public string Phone { get; set; } = string.Empty;

        [Required]
        [MaxLength(150)]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [MaxLength(20)]
        public string Status { get; set; } = "Active";
    }
}
```

This creates a database entity for the five fields shown in your UI, with `Id` as the primary key.

2. Create the DTOs

DTOs define the data the API accepts and returns, so clients don't need to send the database ID when creating a member.

File: `DTOs/CreateTeamMemberDto.cs`

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.DTOs
{
    public class CreateTeamMemberDto
    {
        [Required]
        [MaxLength(150)]
        public string Name { get; set; } = string.Empty;

        [Required]
        [MaxLength(50)]
```

```
        public string Role { get; set; } = "Field Staff";

        [Required]
        [MaxLength(20)]
        public string Phone { get; set; } = string.Empty;

        [Required]
        [MaxLength(150)]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [MaxLength(20)]
        public string Status { get; set; } = "Active";
    }
}
```

File: `DTOs/UpdateTeamMemberDto.cs`

```
using System.ComponentModel.DataAnnotations;

namespace SolvefyTask.DTOs
{
    public class UpdateTeamMemberDto
    {
        [Required]
        [MaxLength(150)]
        public string Name { get; set; } = string.Empty;

        [Required]
        [MaxLength(50)]
        public string Role { get; set; } = "Field Staff";

        [Required]
        [MaxLength(20)]
        public string Phone { get; set; } = string.Empty;

        [Required]
        [MaxLength(150)]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [MaxLength(20)]
        public string Status { get; set; } = "Active";
```

```
    }
}
```

File: `DTOs/TeamMemberResponseDto.cs`

```
namespace SolvefyTask.DTOs
{
    public class TeamMemberResponseDto
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;

        public string Role { get; set; } = string.Empty;

        public string Phone { get; set; } = string.Empty;

        public string Email { get; set; } = string.Empty;

        public string Status { get; set; } = string.Empty;
    }
}
```

3. Update `ApplicationDbContext.cs`

Open `Data/ApplicationDbContext.cs` and add the `TeamMember` DbSet to your existing context.
Keep all your existing DbSets and configuration.

Add this using statement if it is not already present:

```
using SolvefyTask.Models;
```

Inside the `ApplicationDbContext` class, add:

```
public DbSet<TeamMember> TeamMembers { get; set; }
```

For example, your context will contain something like:

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(
        DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
```

```
    // Keep your existing DbSets here.

    public DbSet<TeamMember> TeamMembers { get; set; }
}
```

4. Create the repository interface

File: `Repositories/ITeamMemberRepository.cs`

```
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public interface ITeamMemberRepository
    {
        Task<List<TeamMember>> GetAllAsync();

        Task<TeamMember?> GetByIdAsync(int id);

        Task<TeamMember> CreateAsync(TeamMember member);

        Task<TeamMember?> UpdateAsync(
            int id,
            TeamMember updatedMember);

        Task<bool> DeleteAsync(int id);
    }
}
```

5. Implement the repository

File: `Repositories/TeamMemberRepository.cs`

```
using Microsoft.EntityFrameworkCore;
using SolvefyTask.Data;
using SolvefyTask.Models;

namespace SolvefyTask.Repositories
{
    public class TeamMemberRepository : ITeamMemberRepository
    {
        private readonly ApplicationDbContext _context;

        public TeamMemberRepository(ApplicationDbContext context)
        {
            _context = context;
```

```
    }

    public async Task<List<TeamMember>> GetAllAsync()
    {
        return await _context.TeamMembers
            .AsNoTracking()
            .OrderBy(x => x.Name)
            .ToListAsync();
    }

    public async Task<TeamMember?> GetByIdAsync(int id)
    {
        return await _context.TeamMembers
            .AsNoTracking()
            .FirstOrDefaultAsync(x => x.Id == id);
    }

    public async Task<TeamMember> CreateAsync(TeamMember member)
    {
        _context.TeamMembers.Add(member);

        await _context.SaveChangesAsync();

        return member;
    }

    public async Task<TeamMember?> UpdateAsync(
        int id,
        TeamMember updatedMember)
    {
        var existingMember = await _context.TeamMembers
            .FirstOrDefaultAsync(x => x.Id == id);

        if (existingMember == null)
            return null;

        existingMember.Name = updatedMember.Name;
        existingMember.Role = updatedMember.Role;
        existingMember.Phone = updatedMember.Phone;
        existingMember.Email = updatedMember.Email;
        existingMember.Status = updatedMember.Status;

        await _context.SaveChangesAsync();

        return existingMember;
    }
```

```
        public async Task<bool> DeleteAsync(int id)
        {
            var member = await _context.TeamMembers
                .FirstOrDefaultAsync(x => x.Id == id);

            if (member == null)
                return false;

            _context.TeamMembers.Remove(member);

            await _context.SaveChangesAsync();

            return true;
        }
    }
}
```

This repository handles database operations only; validation and API responses will be handled in the service and controller layers.

6. Create the service interface

File: `Services/ITeamMemberService.cs`

```
using SolvefyTask.DTOs;

namespace SolvefyTask.Services
{
    public interface ITeamMemberService
    {
        Task<List<TeamMemberResponseDto>> GetAllAsync();

        Task<TeamMemberResponseDto?> GetByIdAsync(int id);

        Task<TeamMemberResponseDto> CreateAsync(
            CreateTeamMemberDto dto);

        Task<TeamMemberResponseDto?> UpdateAsync(
            int id,
            UpdateTeamMemberDto dto);

        Task<bool> DeleteAsync(int id);
    }
}
```

7. Implement the service

File: `Services/TeamMemberService.cs`

This layer validates the role and status, trims input values, and maps entities to DTOs.

```
using SolvefyTask.DTOs;
using SolvefyTask.Models;
using SolvefyTask.Repositories;

namespace SolvefyTask.Services
{
    public class TeamMemberService : ITeamMemberService
    {
        private readonly ITeamMemberRepository _repository;

        private static readonly string[] AllowedRoles =
        {
            "Field Staff",
            "Office Staff",
            "Admin"
        };

        private static readonly string[] AllowedStatuses =
        {
            "Active",
            "Inactive"
        };

        public TeamMemberService(ITeamMemberRepository repository)
        {
            _repository = repository;
        }

        public async Task<List<TeamMemberResponseDto>> GetAllAsync()
        {
            var members = await _repository.GetAllAsync();

            return members.Select(MapToDto).ToList();
        }

        public async Task<TeamMemberResponseDto?> GetByIdAsync(int id)
        {
            if (id <= 0)
                throw new ArgumentException("Invalid team member ID.");

            var member = await _repository.GetByIdAsync(id);
```

```
        return member == null ? null : MapToDto(member);
    }

    public async Task<TeamMemberResponseDto> CreateAsync(
        CreateTeamMemberDto dto)
    {
        ValidateFields(
            dto.Name,
            dto.Role,
            dto.Phone,
            dto.Email,
            dto.Status);

        var member = new TeamMember
        {
            Name = dto.Name.Trim(),
            Role = NormalizeRole(dto.Role),
            Phone = dto.Phone.Trim(),
            Email = dto.Email.Trim(),
            Status = NormalizeStatus(dto.Status)
        };

        var created = await _repository.CreateAsync(member);

        return MapToDto(created);
    }

    public async Task<TeamMemberResponseDto?> UpdateAsync(
        int id,
        UpdateTeamMemberDto dto)
    {
        if (id <= 0)
            throw new ArgumentException("Invalid team member ID.");

        ValidateFields(
            dto.Name,
            dto.Role,
            dto.Phone,
            dto.Email,
            dto.Status);

        var updatedMember = new TeamMember
        {
            Name = dto.Name.Trim(),
            Role = NormalizeRole(dto.Role),
```

```
            Phone = dto.Phone.Trim(),
            Email = dto.Email.Trim(),
            Status = NormalizeStatus(dto.Status)
        };

        var updated = await _repository.UpdateAsync(
            id,
            updatedMember);

        return updated == null ? null : MapToDto(updated);
    }

    public async Task<bool> DeleteAsync(int id)
    {
        if (id <= 0)
            throw new ArgumentException("Invalid team member ID.");

        return await _repository.DeleteAsync(id);
    }

    private static void ValidateFields(
        string name,
        string role,
        string phone,
        string email,
        string status)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new ArgumentException("Name is required.");

        if (name.Trim().Length > 150)
            throw new ArgumentException(
                "Name cannot exceed 150 characters.");

        if (string.IsNullOrWhiteSpace(role) ||
            !AllowedRoles.Contains(
                role.Trim(),
                StringComparer.OrdinalIgnoreCase))
        {
            throw new ArgumentException(
                "Role must be Field Staff, Office Staff, or Admin.");
        }

        if (string.IsNullOrWhiteSpace(phone))
            throw new ArgumentException("Phone is required.");
```

```
        if (phone.Trim().Length > 20)
            throw new ArgumentException(
                "Phone cannot exceed 20 characters.");

        if (string.IsNullOrWhiteSpace(email) ||
            !new System.ComponentModel.DataAnnotations
                .EmailAddressAttribute()
                .IsValid(email.Trim()))
        {
            throw new ArgumentException(
                "A valid email address is required.");
        }

        if (email.Trim().Length > 150)
            throw new ArgumentException(
                "Email cannot exceed 150 characters.");

        if (string.IsNullOrWhiteSpace(status) ||
            !AllowedStatuses.Contains(
                status.Trim(),
                StringComparer.OrdinalIgnoreCase))
        {
            throw new ArgumentException(
                "Status must be Active or Inactive.");
        }
    }

    private static string NormalizeRole(string role)
    {
        return AllowedRoles.First(
            x => x.Equals(
                role.Trim(),
                StringComparison.OrdinalIgnoreCase));
    }

    private static string NormalizeStatus(string status)
    {
        return AllowedStatuses.First(
            x => x.Equals(
                status.Trim(),
                StringComparison.OrdinalIgnoreCase));
    }

    private static TeamMemberResponseDto MapToDto(
        TeamMember member)
    {
```

```
            return new TeamMemberResponseDto
            {
                Id = member.Id,
                Name = member.Name,
                Role = member.Role,
                Phone = member.Phone,
                Email = member.Email,
                Status = member.Status
            };
        }
    }
}
```

8. Create the API controller

File: `Controllers/TeamMembersController.cs`

This controller exposes all five endpoints needed by your Angular component.

```
using Microsoft.AspNetCore.Mvc;
using SolvefyTask.DTOs;
using SolvefyTask.Services;

namespace SolvefyTask.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TeamMembersController : ControllerBase
    {
        private readonly ITeamMemberService _service;

        public TeamMembersController(ITeamMemberService service)
        {
            _service = service;
        }

        // GET: api/TeamMembers
        [HttpGet]
        public async Task<ActionResult<List<TeamMemberResponseDto>>>
            GetAll()
        {
            var members = await _service.GetAllAsync();

            return Ok(members);
        }

        // GET: api/TeamMembers/1
```

```
        [HttpGet("{id:int}")]
        public async Task<ActionResult<TeamMemberResponseDto>>
            GetById(int id)
        {
            try
            {
                var member = await _service.GetByIdAsync(id);

                if (member == null)
                {
                    return NotFound(new
                    {
                        message = "Team member not found."
                    });
                }

                return Ok(member);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }

        // POST: api/TeamMembers
        [HttpPost]
        public async Task<ActionResult<TeamMemberResponseDto>>
            Create([FromBody] CreateTeamMemberDto dto)
        {
            try
            {
                var member = await _service.CreateAsync(dto);

                return CreatedAtAction(
                    nameof(GetById),
                    new { id = member.Id },
                    member);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }

        // PUT: api/TeamMembers/1
        [HttpPut("{id:int}")]
```

```
        public async Task<ActionResult<TeamMemberResponseDto>>
            Update(int id, [FromBody] UpdateTeamMemberDto dto)
        {
            try
            {
                var member = await _service.UpdateAsync(id, dto);

                if (member == null)
                {
                    return NotFound(new
                    {
                        message = "Team member not found."
                    });
                }

                return Ok(member);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }

        // DELETE: api/TeamMembers/1
        [HttpDelete("{id:int}")]
        public async Task<IActionResult> Delete(int id)
        {
            try
            {
                var deleted = await _service.DeleteAsync(id);

                if (!deleted)
                {
                    return NotFound(new
                    {
                        message = "Team member not found."
                    });
                }

                return Ok(new
                {
                    message = "Team member deleted successfully."
                });
            }
            catch (ArgumentException ex)
            {
```

```
                return BadRequest(new { message = ex.Message });
            }
        }
    }
}
```

Because `[ApiController]` is enabled, ASP.NET Core automatically returns a `400 Bad Request` when DTO validation fails.

9. Register the repository and service in `Program.cs`

Add these two registrations alongside your existing dependency registrations:

```
builder.Services.AddScoped<
    ITeamMemberRepository,
    TeamMemberRepository>();

builder.Services.AddScoped<
    ITeamMemberService,
    TeamMemberService>();
```

Make sure `Program.cs` has these namespaces:

```
using SolvefyTask.Repositories;
using SolvefyTask.Services;
```

Do not remove your existing registrations for equipment, checkout, approvals, damage reports, or history.

10. Create the SQL Server table using EF Core migration

Open the terminal in your ASP.NET Core project directory, where the `.csproj` file is located.

Run:

```
dotnet ef migrations add CreateTeamMembersTable
```

Then:

```
dotnet ef database update
```

If `dotnet ef` is unavailable, check that your project has the EF Core Design package and that the EF Core CLI tool is installed. Use versions compatible with your existing EF Core packages.

This will create the `TeamMembers` table in the database configured by your existing `ApplicationDbContext`.

11. Test the APIs before connecting Angular

Run your ASP.NET Core API and open Swagger, if it is enabled.

GET **Get all team members**

`/api/TeamMembers`

Returns an array of team members from SQL Server.

GET **Get a member by ID**

`/api/TeamMembers/1`

POST **Create a member**

`/api/TeamMembers`

Use the JSON request body below.

PUT **Update a member**

`/api/TeamMembers/1`

Sends the updated member fields.

DELETE **Delete a member**

`/api/TeamMembers/1`

Example POST body:

```
{
  "name": "Ali Raza",
  "role": "Field Staff",
  "phone": "0311-1234567",
  "email": "ali.raza@horizon.com",
  "status": "Active"
}
```

Expected response: `201 Created`, including the generated `id`.

12. Important considerations

- **Existing sample members:** Your seven Angular sample members will not automatically appear in SQL Server. Add them through POST requests or seed them into the database.
- **Deletion:** If team members are referenced by equipment assignments or other records through foreign keys, you may need to prevent deletion or implement a soft-delete approach.
- **API URL:** The Angular service should use the same API base URL configured in your project, such as `https://localhost:7135/api`, if that is still your backend address.

Once these backend files are added and the migration succeeds, test the CRUD endpoints in Swagger. Then we can connect your `teamsmembers.ts` component to these APIs.

25/10/2026, 18:11:51 · gpt-6